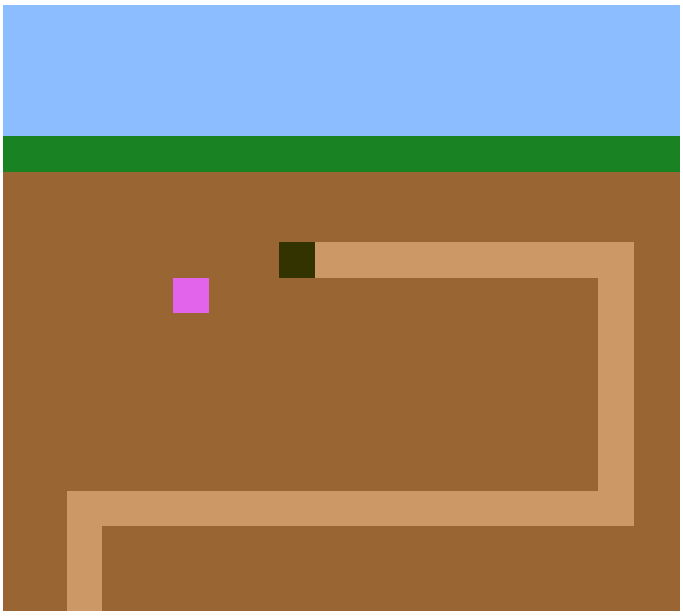
Introduction to programming with Scala

Study Guide 1

First study period: Module 1 – 7



Björn Regnell, Hans Middelkoop (translation)

H-ICT-SE-2-25 2026
Object Oriented Software Development
The Hague University of Applied Science

Compilation date: March 28, 2026

Editor: Björn Regnell

Contributors in alphabetical order: Anders Buhl, André Philipsson Eriksson, Anna Axelsson, Anna Palmqvist Sjövall, Annie Predel, Anton Andersson, Benjamin Lindberg, Björn Regnell, Casper Schreiter, Cecilia Lindskog, Dag Hemberg, Elias Åradsson, Elliot Bräck, Elsa Cervetti Ogestad, Emelie Engström, Emil Wihlander, Erik Bjäreholt, Erik Grampp, Evelyn Beck, Felix Ohrgren, Fredrik Danebjer, Fritjof Bengtsson, Gustav Cedersjö, Henrik Olsson, Hjalmar Rutberg, Hussein Taher, Jakob Hök, Jakob Sinclair, Johan Ravnborg, Johannes Nydahl, Jonas Danebjer, Jos Rosenqvist, Lovisa Löfgren, Maj Stenmark, Maria Kulesh, Melker Widén, Måns Magnusson, Nicholas Boyd Isacsson, Niklas Sandén, Oliver Levay, Oliver Persson, Oscar Sigurdsson, Oskar Berg, Oskar Widmark, Patrik Persson, Per Holm, Philip Sadrian, Sandra Nilsson, Sebastian Hegardt, Simon Persson, Stefan Jonsson, Theodor Lundqvist, Tim Borglund, Tom Postema, Valthor Halldorsson, Viktor Claesson, Wilhelm Wanecek, William Karlsson.

Home: https://lunduniversity.github.io/pgk

Repo: https://github.com/lunduniversity/introprog

This compendium is on-going work.
Contributions are welcome!
Contact: bjorn.regnell@cs.lth.se

Versions:
Scala 3.7.2
JDK 21
introprog-scalalib 1.4.0

You can use this work if you respect this **LICENSE**: CC BY-SA 4.0
http://creativecommons.org/licenses/by-sa/4.0/
Do *not* distribute solutions to lab assignments and projects.

Copyright © 2015-2026.
Björn Regnell, Dept. of Computer Science, LTH, Lund University.

Progress Protocol

Completed Exercises

Each lab comes with an exercise containing tasks that serve as preparation for the lab. You need to master at least the basic tasks in order to complete the lab within a reasonable time. If you feel you need more practice on the basics, then also do the extra tasks. If you want to deepen your understanding, do the advanced tasks which are at a more advanced level. Check off below which exercises you have done, so it will be easier for your supervisor to tailor the discussion to the knowledge you have acquired so far.

Exercise	Basic	Extra	Advanced

Approved Mandatory Components

To pass the lab assignments and the project assignment you must solve the sub-tasks and discuss your solutions with a supervisor. This discussion is your opportunity to get feedback on your solutions. Take advantage of it! Make sure the supervisor notes below when you have been approved on the respective mandatory components. Keep this sheet until you have received your final grade in the course.

Name: .....

Signature: .....

Lab	compl+date, pass+date	Supv. signature + printed name
`kojo`	.....	.....
`irritext`	.....	.....
`blockmole`	.....	.....
`blockbattle0`	.....	.....
`blockbattle1`	.....	.....
`shuffle`	.....	.....
`life`	.....	.....
`words`	.....	.....
`snake0`	.....	.....
`snake1`	.....	.....
Project Assignment	.....	.....

☐ `bank` *If custom defined, provide brief description here:*

☐ `music`

☐ `photo`

☐ *custom defined*

Oral Exam

☐ approved	.....	.....

Preface

Programming is not only a way to gain control over the man-made systems that are the foundation for our modern society and its continued digitalization. Programming is also a powerful tool for thought. With knowledge of the basics of programming, you can begin the lifelong learning journey that it entails to be a system developer and an abstraction artist. Programming languages and development tools come and go, but the fundamental concepts behind *all* software remain: sequence, alternatives, repetition, and abstraction.

This compendium constitutes course material for an introductory programming course, aiming to provide a solid foundation for engineering students and others who want to develop systems with software. The material covers one semester of studies at one-quarter pace and assumes knowledge equivalent to high school level in Swedish, mathematics, and English.

The compendium is distributed as open source. It may be used freely as long as credit is given and any modifications are published under the same license as the original material.

In the course repository github.com/lunduniversity/introprog there are instructions on how you can contribute to the course material.

The learning material focuses on learning through practical programming work and contains exercises and lab assignments organized into modules. Each module has a theme and a theory section that is covered in lectures.

In the course, we use the programming language Scala, which has simple syntax and enables several fundamentally different ways of programming within the same language. We use Scala to illustrate the basics of imperative and object-oriented programming, along with elementary functional programming.

Perhaps the most important success factor when studying programming is that you embrace your own curiosity and experimental spirit. The great thing about programming is that your own intellectual constructions actually *do* something that *you* have decided! Take advantage of that and explore by coding your own ideas—it is fun when it works, but at least as educational is debugging, fixing bugs, and all the failed attempts that, sometimes after hard work, turn into successful solutions and lasting lessons.

Welcome to the fascinating world of computer science and best of luck with your studies!

Contents

Progress Protocol iii

Preface v

I About the course 1

-2 The Course Architecture 3
-2.0.1 Weekly overview . . . 3
-2.1 About your learning . . . 6
-2.1.1 What do you learn? . . . 6
-2.1.2 Progression . . . 6
-2.1.3 How do you learn? . . . 6
-2.1.4 Course components — why? . . . 7
-2.1.5 A typical course week . . . 7

-1 Instructions 9
-1.1 Collaboration Groups . . . 9
-1.1.1 Collaboration Contract . . . 10
-1.1.2 Group Lab . . . 10
-1.2 Lectures . . . 11
-1.3 Exercises . . . 11
-1.4 Resource Times . . . 13
-1.5 Labs . . . 14
-1.6 Project Assignment . . . 15
-1.7 Oral Exam(CGI) . . . 16

0 How to contribute to the course material? 17
0.1 Contributions are warmly welcome! . . . 17
0.2 Instructions . . . 17
0.2.1 What do you need to be able to contribute? . . . 17
0.3 Example . . . 17

II Modules 21

1 Introduction 23
1.1 Theory . . . 24
1.1.1 How does a computer work? . . . 24
1.1.2 What is programming? . . . 24
1.1.3 What is a compiler? . . . 24

1.1.4 Virtual machine (VM) == abstract hardware 25
1.1.5 What does a program consist of? 25
1.1.6 Examples of programming languages 25
1.1.7 Different programming paradigms 26
1.1.8 Hello world 26
1.1.9 Development cycle 27
1.1.10 Development tools 27
1.1.11 Install tools on your own computer 27
1.1.12 Scala Command Line Interface (CLI) 28
1.1.13 Tips and tricks with `scala` in the terminal 28
1.1.14 Literals 28
1.1.15 Examples of built-in data types in Scala 29
1.1.16 Basic types in Scala 29
1.1.17 Ranges of basic types 30
1.1.18 Expressions 30
1.1.19 Variables 30
1.1.20 Rules for identifiers 31
1.1.21 Building strings: concatenation and interpolation 31
1.1.22 Integer arithmetic 32
1.1.23 Floating-point arithmetic 32
1.1.24 Define name for expressions 33
1.1.25 Function, argument, parameter 33
1.1.26 Built-in math functions in the package `scala.math` 33
1.1.27 Logical expressions 34
1.1.28 De Morgan's laws 34
1.1.29 Alternatives with if-expressions 35
1.1.30 Expression or statement? 35
1.1.31 Variable declaration and assignment statement 35
1.1.32 Assignment statements are *not* mathematical equality 36
1.1.33 Shortened assignment statements 36
1.1.34 Examples of shortened assignment statements 36
1.1.35 Variables that change values can be tricky 37
1.1.36 Control structures: alternatives and repetition 37
1.1.37 Scala 2 syntax for control structures works in Scala 3 37
1.1.38 Repeat many statements 38
1.1.39 Procedures 38
1.1.40 Problem solving: decomposition into abstractions that are then combined 39
1.1.41 Exercise `expressions` and lab `kojo` 39
1.2 Exercise `Excercise01` 40
1.2.1 Basic Tasks; preparation for the lab 40
1.2.2 Extra Tasks; additional practice 48
1.2.3 Advanced Tasks; challenges 51
1.3 Lab: `kojo` 54
1.3.1 Mandatory tasks 54
1.3.2 Checkpoint questions 57
1.3.3 Optional extra exercises 58

2 Programs and Control Structures **65**
2.1 Theory . . . 66
2.1.1 What is a data structure? . . . 66
2.1.2 Some collections in `scala.collection` . . . 66
2.1.3 Different structures for handling data . . . 67
2.1.4 What is a vector? . . . 67
2.1.5 A conceptual picture of a vector . . . 68
2.1.6 A collection of strings . . . 68
2.1.7 What is a control structure? . . . 68
2.1.8 Loop through the elements in a vector . . . 69
2.1.9 Build a new collection from existing with for-yield expression . 69
2.1.10 The collection `Range` keeps track of intervals . . . 69
2.1.11 Loop with Range . . . 70
2.1.12 Loop with Range created with `to` . . . 70
2.1.13 Loop through a collection with a `while`-statement . . . 71
2.1.14 What is an `Array`? . . . 71
2.1.15 Some similarities & differences between `Vector` and `Array` . . 71
2.1.16 A minimal standalone program in Scala . . . 72
2.1.17 Type-safe arguments to a program with @main . . . 72
2.1.18 What is an algorithm? . . . 73
2.1.19 Algorithm example: N-FACTORIAL . . . 73
2.1.20 Algorithm example: MIN . . . 73
2.1.21 Template for function definitions . . . 74
2.1.22 Better to have many small abstractions each doing one thing . 74
2.1.23 What is a block? . . . 74
2.1.24 Names in blocks become **local** . . . 75
2.1.25 Parameter and argument . . . 75
2.1.26 Procedures . . . 76
2.1.27 ”Nothing” *is* actually something in Scala . . . 76
2.1.28 Problem solving: decomposition into abstractions that are then combined . . . 77
2.1.29 Example of **functional decomposition** . . . 77
2.1.30 Why abstraction? . . . 77
2.1.31 From source code to machine code with JVM . . . 78
2.1.32 Packages . . . 78
2.1.33 Import . . . 79
2.1.34 Jar files . . . 79
2.2 Exercise `Excercise02` . . . 80
2.2.1 Basic Tasks . . . 80
2.2.2 Extra Tasks; additional practice . . . 86
2.2.3 Advanced Tasks; challenges . . . 88

3 Functions and Abstraction **91**
3.1 Theory . . . 92
3.1.1 What is abstraction? . . . 92
3.1.2 Examples of abstraction mechanisms in computer science . . . 92
3.1.3 Function: declaration and call . . . 92
3.1.4 Declare functions, overloading . . . 93
3.1.5 Functions with default arguments . . . 93
3.1.6 Functions with named arguments . . . 93
3.1.7 Uniform access . . . 94

3.1.8 Call stack and object heap . . . 94
3.1.9 Call stack and activation records . . . 95
3.1.10 What is a stack trace? . . . 95
3.1.11 How to read a stack trace? . . . 96
3.1.12 Local functions . . . 96
3.1.13 Functions are genuine values in Scala . . . 97
3.1.14 Function values can be arguments . . . 97
3.1.15 Apply functions to elements in collections with `map` . . . 97
3.1.16 Apply functions to elements in collections with `map` . . . 98
3.1.17 Pure functions . . . 98
3.1.18 Examples of impure functions: random numbers . . . 99
3.1.19 Random seed: get the same random numbers every time . . . 99
3.1.20 Anonymous functions . . . 99
3.1.21 Apply anonymous functions to elements in collections . . . 100
3.1.22 Placeholder syntax for anonymous functions . . . 100
3.1.23 Example of placeholder syntax with `reduceLeft` . . . 101
3.1.24 Predicate, with and without name . . . 101
3.1.25 Function value with empty parameter list: use a ”thunk” . . . 102
3.1.26 How does `repeat` in Kojo really work? . . . 102
3.1.27 Multiple parameter lists . . . 102
3.1.28 Call-by-value and call-by-name . . . 103
3.1.29 Braces with single parameter . . . 103
3.1.30 Create your own control structure . . . 104
3.1.31 Colon with single parameter . . . 104
3.1.32 Curried functions . . . 104
3.1.33 Function with captured variable scope: *closure* . . . 105
3.1.34 Recursive functions . . . 105
3.1.35 Looping with recursion . . . 105
3.1.36 Recursive data structures . . . 106
3.1.37 Recompile changed files at every save . . . 106
3.2 Exercise `Excercise03` . . . 107
3.2.1 Basic Tasks; preparation for the lab . . . 107
3.2.2 Extra Tasks; additional practice . . . 112
3.2.3 Advanced Tasks; challenges . . . 114
3.3 Lab: `irritext` . . . 117
3.3.1 Requirements . . . 117
3.3.2 Tips to get started . . . 118
3.3.3 Inspiration . . . 118

4 Objects and Encapsulation 121
4.1 Theory . . . 122
4.1.1 What does the turtle in Kojo hold in its state? . . . 122
4.1.2 What is an object? . . . 122
4.1.3 Declare, allocate, reference . . . 122
4.1.4 Different ways to allocate objects . . . 123
4.1.5 What is a singleton object? . . . 123
4.1.6 Allocation: memory reserved with space for data . . . 124
4.1.7 Dot notation, state change via assignment . . . 124
4.1.8 Dot notation and operator notation . . . 124
4.1.9 Namespace and shadowing . . . 125
4.1.10 Encapsulation: hiding internal parts . . . 125

4.1.11 Idiom: Private variables with underscore for ”collision” 126
4.1.12 The principle of uniform access 126
4.1.13 Example: singleton object with mutable state 126
4.1.14 Example: state, attributes 127
4.1.15 State change 127
4.1.16 Module 127
4.1.17 Declare package 128
4.1.18 Compile packages 128
4.1.19 Packages in REPL 129
4.1.20 What is a tuple? 129
4.1.21 Tuples as parameters and return values 129
4.1.22 A handy way to create 2-tuples with the method `->` 130
4.1.23 Type alias to abstract type names 130
4.1.24 Lazy variables with delayed initialization 131
4.1.25 Singleton objects are lazy 131
4.1.26 What is the difference between `val`, `var`, `def`, `lazy val`? . . . 131
4.1.27 Pitfall: initialization order and default values 132
4.1.28 Programming paradigms 132
4.1.29 Functions are true objects in Scala 133
4.1.30 Deepening: True function objects have function type 133
4.1.31 What is a class? 133
4.1.32 What is a class? 133
4.1.33 Using the class `Color` 134
4.1.34 Add methods afterward with `extension` 134
4.1.35 Collective extension methods 135
4.1.36 Import all names in a module 135
4.1.37 Rename on import 136
4.1.38 Exclude (hide) names on import 136
4.1.39 Local import declaration 136
4.1.40 Export 136
4.1.41 Using documentation for ready-made classes. 137
4.1.42 What is a jar file? 137
4.1.43 Open source on Maven Central 138
4.1.44 What is the *classpath*? 138
4.1.45 Ready-made graphic methods in class `PixelWindow` 138
4.1.46 Automatic dependencies with Scala CLI in REPL: 138
4.1.47 Run programs + code libraries with Scala CLI 139
4.1.48 Recompile on every change 139
4.2 Exercise `Excercise04` 140
4.2.1 Basic Tasks; preparation for the lab 140
4.2.2 Extra Tasks; additional practice 147
4.2.3 Advanced Tasks; challenges 149
4.3 Lab: `blockmole` 151
4.3.1 Background 151
4.3.2 Mandatory tasks 151
4.3.3 Control questions 156
4.3.4 Optional extra tasks 156

5 Classes and Data Modeling **161**
5.1 Theory . . . 162
5.1.1 A metaphor for class: Stamp . . . 162
5.1.2 What is a class? . . . 162
5.1.3 Data modeling . . . 162
5.1.4 Singleton object compared to class . . . 163
5.1.5 Reference equality or content equality? . . . 163
5.1.6 Example: reference equality and content equality . . . 164
5.1.7 Reference equality and custom classes . . . 164
5.1.8 Case classes provide content equality . . . 165
5.1.9 Equality and case classes . . . 165
5.1.10 Summary of case-class goodies . . . 165
5.1.11 Implementation missing: ??? . . . 165
5.1.12 Example: incomplete code . . . 166
5.2 Exercise `Excercise05` . . . 167
5.2.1 Basic Tasks; preparation for the lab . . . 167
5.2.2 Extra Tasks; additional practice . . . 173
5.2.3 Advanced Tasks; challenges . . . 177
5.3 Lab: `blockbattle0` . . . 182

6 Patterns and Exception Handling **183**
6.1 Theory . . . 184
6.1.1 The base type for all types: `Any` . . . 184
6.1.2 All types are subtypes of `Any` . . . 184
6.1.3 Your own reference types are subtypes of `AnyRef` . . . 184
6.1.4 What is matching? . . . 185
6.1.5 Decompose an object into its parts with patterns . . . 185
6.1.6 Check if one fits with nested if expressions . . . 186
6.1.7 Check if one fits with `match` expression . . . 186
6.1.8 Syntax for match expressions . . . 186
6.1.9 Matching with guard . . . 187
6.1.10 Matching with variable pattern . . . 187
6.1.11 Matching with or-pattern . . . 187
6.1.12 Matching with typed patterns . . . 188
6.1.13 Deepening: Union types and the type Matchable . . . 188
6.1.14 Constructor patterns with case classes . . . 189
6.1.15 Decompose collections with deep patterns . . . 189
6.1.16 Matching on tuples . . . 190
6.1.17 Pattern matching and enumeration with case objects . . . 190
6.1.18 Pattern matching and sealed types . . . 190
6.1.19 Pattern matching an enumeration . . . 191
6.1.20 Uppercase/lowercase initials in matching . . . 191
6.1.21 Uppercase/lowercase initials in matching . . . 192
6.1.22 Patterns in other places than in `match` . . . 192
6.1.23 Pattern parts and variable number of arguments . . . 192
6.1.24 Partial functions and the method `collect` . . . 193
6.1.25 Deepening: the method `unapply` . . . 193
6.1.26 How to handle missing values? . . . 194
6.1.27 A common base type for a value that may be missing . . . 194
6.1.28 Option for handling possibly missing values . . . 194
6.1.29 Some handy methods on `Option` . . . 195

6.1.30 Some collection methods that return an `Option`, exercise . . . 195
6.1.31 Some collection methods that return an `Option`, answers . . . 196
6.1.32 What is an exception? . . . 196
6.1.33 Cause exceptions indirectly with `require` and `assert` . . . 197
6.1.34 Throw exceptions directly with primitive `throw` . . . 197
6.1.35 A common base type for something that can fail . . . 197
6.1.36 Handle exceptions as a value with `Try` . . . 198
6.1.37 Primitive `try-catch` expressions . . . 198
6.1.38 Avoid exceptions if possible . . . 199
6.1.39 In-depth: Checked exceptions . . . 199
6.1.40 In-depth: Implementing `equals` with `match` . . . 200
6.1.41 In-depth: `equals` that works for final classes . . . 200
6.1.42 In-depth: 8-step recipe for inheritance-safe `equals` . . . 201
6.1.43 In-depth: Safer equality tests in Scala 3 . . . 201
6.2 Exercise `Excercise06` . . . 203
6.2.1 Basic Tasks; preparation for the lab . . . 203
6.2.2 Advanced Tasks; challenges . . . 208
6.3 Lab: `blockbattle1` . . . 214
6.3.1 Background . . . 214
6.3.2 Mandatory requirements . . . 215
6.3.3 Optional requirements – choose at least one . . . 215
6.3.4 Preparations for presentation . . . 215
6.3.5 Tips and suggestions . . . 216

7 Sequences and Enumerations 219
7.1 Theory . . . 220
7.1.1 What is a sequence? . . . 220
7.1.2 Example: A string is a sequence of characters . . . 220
7.1.3 Iterating over elements in a sequence . . . 220
7.1.4 Add elements at the beginning and at the end of a sequence . 221
7.1.5 Properties of some sequence collections in Scala . . . 221
7.1.6 Which sequence collection should I choose? . . . 222
7.1.7 Some quirks with Array . . . 222
7.1.8 Immutable or mutable? . . . 223
7.1.9 What is a sequence algorithm? . . . 223
7.1.10 Using ready-made sequence collection methods . . . 224
7.1.11 Some useful collection methods when implementing sequence algorithms . . . 224
7.1.12 Updated sequence with the powerful method `patch` . . . 224
7.1.13 Using for-expressions for filtering with guard . . . 225
7.1.14 Using the collection method `filter` for filtering . . . 225
7.1.15 Common sequence problems as function headers . . . 225
7.1.16 Implementation of sequence problems with `for`-expressions or ready collection methods . . . 226
7.1.17 Implementation of sequence problems with `map`, `filter` . . . 226
7.1.18 Hierarchy of collection types in `scala.collection` v2.13 . . . 227
7.1.19 Keep it open: use `Seq` . . . 227
7.1.20 Implementation with generic functions . . . 227
7.1.21 Deepening: Using Java collections in Scala with `CollectionConverters` 228
7.1.22 Deepening: Creating generic Array . . . 228
7.1.23 Repeated parameters become a sequence . . . 229

7.1.24 Sequence collection as argument to repeated parameters . . . 229
7.1.25 Enumerations have an order . . . 230
7.1.26 Enumerations can have parameters and members . . . 230
7.1.27 Enum can correspond to full-fledged case classes . . . 231
7.1.28 Enum and pattern matching . . . 231
7.1.29 Advantages of **enum** compared to enumerations with integers . 231
7.1.30 Registration . . . 232
7.1.31 Registration of Dice Rolls in `Array` . . . 232
7.1.32 Registration of Dice Rolls in `Array` . . . 233
7.1.33 Create solutions to sequence problems from scratch . . . 233
7.1.34 Create new sequence collection or change in place? . . . 233
7.1.35 Algorithm: SEQ-COPY . . . 234
7.1.36 Implementation of SEQ-COPY with `while` . . . 234
7.1.37 Type alias to abstract type names . . . 234
7.1.38 Example: SEQ-INSERT/REMOVE-COPY . . . 235
7.1.39 Pseudo-code for SEQ-INSERT-COPY . . . 235
7.1.40 Insertion/removal in a copy of primitive Array . . . 236
7.1.41 Example: PolygonWindow . . . 236
7.1.42 Implementing Polygon . . . 237
7.1.43 Example: PolygonArray, in-place modification . . . 237
7.1.44 Example: PolygonVector, variable reference to immutable data structure . . . 238
7.1.45 Example: Polygon as immutable case class . . . 238
7.1.46 Sorting and comparing strings lexicographically . . . 238
7.1.47 Comparing strings: equality . . . 239
7.1.48 Algorithm example: string equality, pseudocode . . . 239
7.1.49 Algorithm example: string equality, implementation . . . 240
7.1.50 Comparing strings: ”less than” . . . 240
7.1.51 Comparing strings: ”less than” . . . 240
7.1.52 Comparing strings: ”less than” . . . 241
7.1.53 Searching . . . 241
7.1.54 Linear searching: find index for element x . . . 242
7.1.55 Sorting . . . 242
7.1.56 Algorithmic complexity . . . 242
7.1.57 There are many different sorting algorithms . . . 243
7.1.58 Bogo sort . . . 243
7.1.59 Sort to new vector with insertion sort: pseudo-code . . . 243
7.1.60 Sort to new vector with insertion sort: implementation . . . 244
7.1.61 Sort to new collection with arbitrary ordering predicate . . . 244
7.1.62 Insertion sort in place – pseudo-code . . . 245
7.1.63 Insertion sort in place – implementation . . . 245
7.2 Exercise `Excercise07` . . . 246
7.2.1 Basic Tasks; preparation for the lab . . . 246
7.2.2 Extra Tasks; additional practice . . . 254
7.2.3 Advanced Tasks; challenges . . . 257
7.3 Lab: `shuffle` . . . 262
7.3.1 Background . . . 262
7.3.2 Given code . . . 263
7.3.3 Mandatory tasks . . . 267
7.3.4 Optional extra tasks . . . 268

7.3.5 Pictures with examples of different poker hands 268

III Solutions **271**

L Solutions to the exercises **273**

L.1 Solution `Excercise01` 274
L.1.1 Basic Tasks; preparation for the lab 274
L.1.2 Extra Tasks; additional practice 281
L.1.3 Advanced Tasks; challenges 285
L.2 Solution `Excercise02` 289
L.2.1 Basic Tasks 289
L.2.2 Extra Tasks; additional practice 294
L.2.3 Advanced Tasks; challenges 296
L.3 Solution `Excercise03` 298
L.3.1 Extra Tasks; additional practice 301
L.3.2 Advanced Tasks; challenges 303
L.4 Solution `Excercise04` 306
L.4.1 Basic Tasks; preparation for the lab 306
L.4.2 Extra Tasks; additional practice 313
L.4.3 Advanced Tasks; challenges 314
L.5 Solution `Excercise05` 317
L.5.1 Basic Tasks; preparation for the lab 317
L.5.2 Extra Tasks; additional practice 320
L.5.3 Advanced Tasks; challenges 323
L.6 Solution `Excercise06` 329
L.6.1 Basic Tasks; preparation for the lab 329
L.6.2 Advanced Tasks; challenges 335
L.7 Solution `Excercise07` 338
L.7.1 Basic Tasks; preparation for the lab 338
L.7.2 Extra Tasks; additional practice 346
L.7.3 Advanced Tasks; challenges 349

Part I

About the course

Chapter -2

The Course Architecture

-2.0.1 Weekly overview

W	*Module*	*Exercise*	*Lab*
W01	Introduction	Excercise01	kojo
W02	Programs and Control Structures	Excercise02	–
W03	Functions and Abstraction	Excercise03	irritext
W04	Objects and Encapsulation	Excercise04	blockmole
W05	Classes and Data Modeling	Excercise05	blockbattle0
W06	Patterns and Exception Handling	Excercise06	blockbattle1
W07	Sequences and Enumerations	Excercise07	shuffle
TP	–	–	–
W08	Nested and Generic Structures	Excercise08	life
W09	Sets and Maps	Excercise09	words
W10	Inheritance and Composition	Excercise10	snake0
W11	Variance and Context Parameters	Excercise11	snake1
W12	Advanced, Project	Excercise12	Project0
W13	Revision	Excercise13	Project1
W14	ORAL EXAM	Excercise14	Munta

The course consists of one **module** per study week with two **lectures**, one **exercise** and one **laboratory session** (except for some weeks that lack lab and/or exercise or have other activities, see weekly overview). The lectures provide an overview of the theory included in each module. By doing the exercises you process the theory and prepare yourself for the laboratory sessions. When you have completed the exercise and the laboratory session in a module you are ready to move on to the next. The table on the next page shows the concepts included in each module.
The course is divided into two study periods. The second study period ends with a larger **project**, an oral exam and an optional written **exam**.

W01	**Introduction**	sequence, alternative, repetition, abstraction, edit, compile, execute, parts of computer, virtual machine, literal, value, expression, identifier, variable, type, assignment, name, val, var, def, define and call function, function header, function body, procedure, built-in primitive types, println, Unit type, unit value (), string interpolator s, arithmetic, random numbers, logical expressions, De Morgan's laws, if, true, false, while, for, dod: operating system
W02	**Programs and Control Structures**	main program, program argument, input, scala.io.StdIn.readLine, control structure, iterate over elements in collection, for-expression, yield, map, foreach, collection, sequence, indexing, Array, Vector, range, Range, algorithm, implementation, pseudocode, algorithm examples: SWAP, SUM, MIN-MAX, MIN-INDEX, dod: version control
W03	**Functions and Abstraction**	abstraction, function, parameter, argument, return type, default argument, named arguments, parameter list, function header, function body, apply function to all elements in a collection, curried parameter list, define own control structure, function value, function type, pure function, step function, apply, anonymous functions, lambda, predicate, activation record, call stack, object heap, stack trace, value call, name call, curly braces and colon for single parameter, recursion, scala.util.Random, random seed, dod: typing
W04	**Objects and Encapsulation**	module, singleton object, dot notation, state, member, attribute, method, package, file structure, jar, classpath, documentation, JDK, import, selective import, renaming on import, export, tuple, multiple return values, block, local variable, shadowing, local function, functions are objects with apply method, namespace, visibility, private member, encapsulation, getter and setter, uniform access principle, method overloading, introprog.PixelWindow, initialization, lazy val, type alias, dod: machine code
W05	**Classes and Data Modeling**	application domain, data model, object-orientation, class, instance, Any, isInstanceOf, toString, new, null, this, access rules, private, private[this], class parameter, primary constructor, factory method, alternative constructor, mutable, immutable, case class, companion object, reference equality, structural equality, eq, ==
W06	**Patterns and Exception Handling**	pattern matching, match, Option, throw, try, catch, Try, unapply, sealed, flatten, flatMap, partial functions, collect, wildcard patterns, variable binding in patterns, sequence wildcard, literal patterns, implement equals, hashCode

W07	**Sequences and Enumerations**	overview of Scala's collection library and collection methods, class hierarchy in scala.collection, Iterable, Seq, List, ListBuffer, ArrayBuffer, WrappedArray, sequence algorithm, algorithm: SEQ-COPY, in-place vs copy, algorithm: SEQ-REVERSE, registration, algorithm: SEQ-REGISTER, linear search, algorithm: LINEAR-SEARCH, time complexity, memory complexity, overview of string methods, StringBuilder, order, built-in search methods, find, indexOf, indexWhere, built-in sorting methods, sorted, sortWith, sortBy, repeated parameters
TP	–
W08	**Nested and Generic Structures**	matrix, nested collection, nested for-statement, type parameter, generic function, generic class, free and bound type parameter, generic data structures, generic collections in Scala
W09	**Sets and Maps**	membership test, set, Set, mutable.Set, key-value table, Map, mutable.Map, hash code, java.util.HashMap, java.util.HashSet, persistence, serialization, text files, Source.fromFile, java.nio.file
W10	**Inheritance and Composition**	inheritance, composition, polymorphism, trait, extends, asInstanceOf, with, mix-in supertype, subtype, base type, override, Scala type hierarchy, Any, AnyRef, Object, AnyVal, Null, Nothing, top type, bottom type, reference types, value types, access rules in inheritance, protected, final, trait, abstract class
W11	**Variance and Context Parameters**	upper and lower type bound, variance, contravariance, covariance, type wildcard, context bound, type constructor, self type, type wildcard, given value (given), context parameter (using), ad hoc polymorphism, type class, api, code readability, reviews
W12	**Advanced, Project**	choose any advanced topic, start project
W13	**Revision**	practice exercises, report project, prepare for oral exam
W14	ORAL EXAM	

-2.1 About your learning

-2.1.1 What do you learn?

- Fundamental principles of programming:
  Sequence, Alternative, Repetition, Abstraction (SARA)
  $\Longrightarrow$ No prior programming knowledge required!
- Implementation of algorithms
- Thinking in abstractions, breaking down problems into subproblems
- Understanding several different approaches:
  - **imperative programming**
  - **object orientation**
  - **functional programming**
- The modern programming language **Scala**
- Development tools (editor, compiler, development environment)
- Implementing, reviewing, testing, debugging

-2.1.2 Progression

The course concepts advance **step by step**:

- Control structures
- Functions
- Objects
- Data structures
- Algorithms
- Nested structures
- More advanced abstraction mechanisms
  - Composition
  - Polymorphism
  - Contextual abstractions

We **iterate** over the concepts & **deepen** understanding **as we go**.

-2.1.3 How do you learn?

- Through practical **own work**: **Learn by doing!**
  - Exercises: apply concepts in different ways
  - Laboratory sessions: combine several concepts into a whole
- Through study of the course theory: **Create understanding!**
- Through cooperation with your course mates: **Go deeper!**

The compendium is the main course literature and defines the course content. Lectures, exercises, and laboratory sessions in the compendium are the primary knowledge

sources of the course, together with the open online resources referenced by the compendium. The compendium is open source and you are warmly welcome to contribute! If you prefer one or more more traditional textbooks as supplementary reading, the following are recommended:

- For those who have never coded, and want to read about coding from the ground up:
  - ”Introduction to Programming and Problem-Solving Using Scala” Second Edition (2016), Mark C. Lewis, Lisa Lacher.
  - Lewis & Lacher cover large parts of the course, but also contain some material included in later LTH courses. The order is quite different, but it is fine to read the book in a different order than it is written.
- For those who have already coded quite a lot in an object-oriented language:
  - ”Programming in Scala”, Fifth Edition (2021), Martin Odersky, Lex Spoon, and Bill Venners.
  - Martin Odersky is the creator of Scala and this well-written book contains a complete walkthrough of the Scala language with many examples and tips. ”Fifth Edition” covers new Scala 3. The book targets those who already have knowledge of some object-oriented language, e.g. Java or C#. There is a good index that makes it easy to adapt your reading to the course setup. The book’s approx. 800 pages contain much material that is of a more advanced level than this course, but you will benefit from its content in subsequent courses.

These textbooks do not directly follow the course’s scope and progression and you will have to do the useful homework yourself to relate their content to what we go through in the course’s different modules.

-2.1.4 Course components — why?

- **Lectures**: create overview, provide structure, explain theory, answer questions, motivate why.
- **Exercises**: process theory step by step, **basic exercises** for all, **extra exercises** if you want/need more practice, **in-depth exercises** if you want to go deeper; **preparation for the labs**.
- **Laboratories**: **mandatory**, assemble parts of theory into a larger program; solutions presented to coaches; pass all to skip for oral exam(CGI).
- **Resource times**: get help with exercises and lab preparations from tutors, ask whatever you want.
- **Collaboration groups**: group learning through cooperation, help each other.
- **Individual project assignment**: **mandatory**, demonstrate that you can create a larger program independently; presented to coach.
- **Oral exam**: **optional**, Show that you have sufficient understanding of course concepts to pass the next course.

-2.1.5 A typical course week

1. Attend **lectures** on **Monday–Wednesday**
2. **Work individually** with theory, exercises, lab preparations on **Monday–Wednesday**
3. **Meet** regularly in your **collaboration group** and help each other understand more and deepen learning, preferably at recurring times each week when everyone in the group can

4. Come to **resource times** and get help and tips from tutors and fellow students on **Friday**
5. Carry out the mandatory **laboratory session** on **Monday–Wednesday**

See details and exceptions in the schedule in MyTimeTables

Chapter -1

Instructions

This chapter contains instructions and guidelines for the various parts of the course. Read carefully so that you do not miss important information about the purpose behind the course components and what is expected of you.

-1.1 Collaboration Groups

Your learning in general, and your programming learning in particular, is deepened if it takes place in dialogue with others. In addition, your collaboration skills and your pedagogical abilities are crucial for your success as a professional system developer. Therefore, course participants are divided into *collaboration groups* of 4-6 people where members cooperate so that everyone in the group can reach as far as possible in their studies.
To manage and benefit from differences in prior knowledge, the collaboration groups are divided so that participants have *varying prior knowledge* based on a prior knowledge survey. Those who have already tried programming get the chance to practice their pedagogical skills, which are very important for system developers, while those who have not yet progressed as far can benefit from the group's collective competence in their learning. The competence variation in the group will change during the course, as different individuals learn at different speeds in different stages of their learning; those who initially have an advantage may later struggle to overcome a certain learning threshold.
The collaboration groups organize their work themselves and each group finds the forms of collaboration that suit its members best. Here are some experience-based tips:

1. Meet as soon as possible with the whole group and get to know each other. The faster you come together as a group and get the social interaction to work, the better. You will benefit from this investment throughout the semester and perhaps for the rest of your study time.

2. Agree on fixed meeting times and meeting places. Continuity in the work is important so that collaboration in the group develops and deepens. Meet at least once a week. Have a standing agenda, e.g., a round table where each person tells how far they have come and lists the concepts they currently need to focus on.

3. Help each other to identify and discuss your different individual study needs and ambitions. When learning to program, you encounter different learning thresholds that someone who has already passed the threshold can help overcome.

But it is important for the helper first to understand exactly what is difficult, or which specific puzzle pieces are missing, to best facilitate another student in overcoming the threshold. The key is to help *just enough* so that everyone independently has the chance to write their own code.

4. Be a fair comrade and act professionally, especially in situations where group members disagree. Communicate respectfully and seek constructive compromises. Social development is important for your future professional practice as a system developer and in the collaboration group you can practice and develop your cooperation skills.

-1.1.1 Collaboration Contract

You should establish a collaboration contract during the first week and present it to a supervisor. All group members must sign the contract. The supervisor must also sign to confirm that you have shown the contract.
The purpose of the contract is for you to discuss within the group how you want to work and which rules you consider reasonable. You decide the content of the contract yourselves. Below are suggestions of points that may be included. A contract template is available here: https://github.com/lunduniversity/introprog/blob/master/study-groups/collaboration-contract.tex

Collaboration Contract
We, who have signed this contract, promise to do our best to follow the collaboration rules below so that everyone can learn as much as possible.

1. Arrive on time to group meetings.
2. Be well prepared through self-study before group meetings.
3. Help each other understand, but do not solve tasks for anyone else.
4. Treat each other respectfully even if we have different opinions.
5. Include everyone in the community.
6. ...

-1.1.2 Group Lab

Lab `snake0` in study week W10 is a group lab. The following instructions apply especially to the group lab. (General instructions that apply to both individual labs and group labs are found in section -1.5.)

1. Discuss in your collaboration group how you will divide the code between you into several different parts, which you can work on separately. Such a part can be a class, a trait, an object, a package, or a function.
2. Each part should have a **main responsible** individual.
3. The division of work should be roughly evenly distributed among the group members.
4. The person who is the main responsible for a certain part presents that part.

5. You shall develop a common group checklist for code review. Each group member shall review at least one other group member's code according to the checklist.

6. The group collaboration takes place over **two weeks** divided into two partial presentations. At the first presentation, the work plan and ongoing development shall be presented. At the second occasion, the completed solutions shall be presented by the respective main responsible individual.

7. At the first presentation, you shall explain to the supervisor how you have divided the code and who is primarily responsible for what and what your responsibility includes, as well as how you practically work to synchronize your development.

8. The group laboratory work is an extra large assignment and group work requires lead time so that you have enough time to coordinate among yourselves. You therefore need to plan to work on some part of the group lab virtually every day during the available weeks, and be ready to contribute to discussions.

-1.2 Lectures

A normal study period week begins with two lecture sessions of 2 hours each. The lectures provide an overview of the course's theoretical content and go through the meaning of the concepts you should learn. The lectures contain many programming examples and the lecturer "live codes" now and then to illustrate the creative problem-solving process inherent in all programming. The lectures also cover the course organization and various practical details.
The lectures provide good opportunities to ask general questions about the theory and to discuss specific difficulties in plenary (individual teacher help is given during resource times, see section -1.4, and during labs, see section -1.5). Even if there are many people in the lecture hall, *do not hesitate to ask questions* — surely others wonder about the same thing as you!
Attendance at lectures is not mandatory, but it is very important that you go there, even if you at times feel that you have a good grasp of all theory. At the lectures you get a comprehensive subject structure and a concrete programming experience, which you share with your course mates and can discuss in collaboration groups. The lectures also prioritize the material and prepare you for the examination with practical advice and tips on how to focus your studies.

-1.3 Exercises

A normal study period week includes an exercise with several tasks and sub-tasks. The exercises form the foundation for your programming studies and offer a systematic review of all parts of the course theory through practical code examples that you carry out step by step at the computer with the help of an interactive tool called Read-Evaluate-Print-Loop (REPL). If you do the exercises in REPL, you ensure that you gain sufficient understanding of all the concepts included in the course and that you do not miss any important puzzle pieces.
The exercises also serve as preparation for the labs. If you do not complete the week's exercise, it is unlikely that you will pass the week's lab within a reasonable time.
Two things are especially important when learning to program:

- **Program!** It is not enough to just passively read about programming; you must *actively* write lots of code yourself and carry out your own programming experiments. It greatly helps if you embrace your curiosity and experimental spirit. All programming errors you make and all your mistakes, which in hindsight seem simple, are in fact indispensable steps on the way and provide crucial *"Aha!"* moments. The course's exercises are the foundation for this form of learning.

- **Be patient!** It is only when you have the ability to actively combine *many* different programming concepts that you can independently solve somewhat larger programming tasks. It can be frustrating at first before you reach the point where your toolbox of concepts is sufficiently large to create the code you want. Sometimes it takes extra patience before everything suddenly falls into place. Many programming teachers and students testify that "the penny suddenly drops" and everything clicks. The exercises aim to, step by step, build your toolbox so that it eventually becomes powerful enough for more advanced problem-solving.

Different students have different ambition levels, varying prior knowledge, different working capacities, more or less developed study techniques, and different ease of learning programming. To handle this variation, exercise tasks are offered in three different types:

- **Basic tasks**. Each week's basic tasks cover the core theory and help ensure you can proceed without knowledge gaps. The basic tasks also form the basis for the labs. All students should do all basic tasks. A good understanding of the content in the basic tasks gives good conditions to pass the final exam.

- **Extra tasks**. If you find the basic tasks difficult and want more practice, or if you want to ensure that you fully consolidate your basic knowledge, then you should do the extra tasks. These are on the same level as the basic tasks and provide additional training.

- **Advanced tasks**. If you want to go deeper and have the capacity to learn even more, do the advanced tasks. These complement the basic tasks with more advanced examples and go beyond what is required for a passing grade in the course. If you aim for one of the higher grades, you should do the advanced tasks. Some advanced tasks have a star in the margin. This symbol indicates that the ★ task is broadly educational but advanced and will not appear in the exam.

Solutions are available for each exercise at the back of this compendium. Do *not* look at the solution before you have first tried to solve the task yourself. The solutions often include comments and tips, so don't forget to review the week's solutions before you start preparing for that week's lab.
Keep in mind that there are often *many different solutions* to the same programming problem, which may be equivalent or have different advantages and disadvantages depending on the context. Feel free to discuss different solution variants with your course mates and supervisors — trying many different ways to solve a task deepens your learning considerably!
Many tasks state "try this in REPL and explain what happens," and the difficulty is often not in creating the code itself but in understanding how it works and *why*. In this way, you train your programming thinking with the help of a growing conceptual

apparatus. The purpose is often to illustrate a universal concept, and it is therefore especially good if you create your own exercises on the same theme and experiment with new variants that give you additional understanding.
The exercise tasks often include ready-made code snippets that you should type into REPL while it runs in a terminal window. REPL code is shown in the exercises with light text on dark background, like this:

```
scala> val msg = "Hello world!"
scala> println(msg)
```

The prompt `scala>` indicates that REPL is running and waiting for input. You should type the code *after* the prompt. More information on how to use REPL can be found in appendix C.4.2.
Even though the compendium is available for download, *do not* be tempted to copy and paste all code snippets into REPL. Instead, take the small time it takes to type the code line by line yourself. While you type, you have time to think, and the active writing promotes your learning and makes it easier to remember and understand.

-1.4 Resource Times

During each study period week, several resource times are scheduled. There is at least one time that fits your schedule group, but you are welcome to attend others and/or multiple sessions subject to availability. Resource times are scheduled in computer labs with Linux computers, and in each lab a supervisor is ready to answer your questions.
The following guidelines apply to resource times:

1. **Purpose**. The resource times are primarily for helping you forward if you get stuck with the exercises or lab preparations, but you can ask about anything related to the course as far as the supervisor can answer and has time.

2. **Collaboration**. Please help each other during resource times! If a fellow student is stuck, it is developmental and educational to help out. If your schedule and location permit, you are welcome to attend the same resource time slot as a member of your collaboration group, but you can also just as well help each other across group boundaries.

3. **Consideration**. When helping others, try to speak very quietly so you do not disturb others' concentration. Also keep in mind that everyone needs to practice a lot on their own without being overly controlled by a "backseat driver." Do not take over somebody else's keyboard; rather, give well-thought-out tips along the way and let your course mate retain control of the task solution.

4. **Focus**. You should *not* do and present the lab during resource times; these must be done and presented during lab time. However, if you have been sick or have not passed a particular lab, you may, if the supervisor has the time, ask to present your outstanding lab during a resource time.

5. **Progress Record**. On page iii there is a progress record for the exercises. Keep this updated as you complete the exercises and show the record when you ask for help from a supervisor. It will then be easier for the supervisor to see which knowledge you have acquired so far and to tailor the dialogue accordingly.

-1.5 Labs

A normal study period week ends with a teacher-supervised lab. While exercises train different parts of the theory in many smaller tasks, the labs aim to train combining several concepts and applying these together in a larger program with many interacting parts.
A lab lasts 2 hours and is scheduled in rooms with computers running Linux. The following instructions apply to the labs:

1. **Mandatory**. The labs are mandatory and an important part of the course examination. Approved labs show that you can apply the theory included in the course and that you have acquired a basic ability to independently, and in groups, develop larger programs with many components. *Note that all labs must be approved to skip the oral exam(CGI)!*

2. **Individual work and cheating.** You shall solve the individual labs *independently* through your own, individual work. You may help others to understand but must not give or receive finished solutions. Read carefully below what is allowed and what is not. Cheating can lead to suspension from the university and withdrawal of study grants. Document forgery can lead to prosecution in court.

   (a) It is allowed during preparation to discuss overall principles for lab solutions with others, but each person must independently create their own solution.

   (b) During the presentation, you must, upon request, explain in detail your individual solution and the concepts included in the learning goals to the supervisor.

   (c) Special instructions for group labs are found in section -1.1.2.

   (d) It is *not* allowed to post solutions online; this is complicity in cheating.

   (e) It is *not* allowed to use artificial intelligence to generate solutions. It is important that in this course you learn to independently develop basic solutions so that in the future you can review and evaluate the quality of AI-generated code. You should therefore disable the Copilot extension in VS Code (eng. *disable extension copilot*)[1] — please ask a supervisor for help on how to do this.

   (f) Read carefully on this webpage where the line between collaboration and cheating is: `https://cs.lth.se/utbildning/samarbete-eller-fusk/`

   (g) Cheating is not only risky and unethical, it also undermines your continued studies. The concepts you learn in this course are a fundamental prerequisite for you to benefit from subsequent courses, and deep learning in this course is fundamental for your entire education.

3. **Preparations**. For each lab there are preparations you should do *before* the lab. This is crucial for you to finish the lab within 2 hours. Get help from a friend or supervisor during resource times if you have questions during your preparation work. Before each lab, you should have:

   (a) studied relevant parts of the compendium;

[1] `https://stackoverflow.com/questions/75377406`

(b) done the basic tasks included in that week's exercise, and preferably also (some) extra and/or advanced tasks;

(c) read through the *entire* lab carefully;

(d) solved the preparation tasks. In the lab preparations, you should, where applicable, write parts of the code included in the lab. It is not required that everything you write is completely correct, but you should have made a reasonable attempt. Get help if you have problems with the tasks, but do not let anyone else solve the task for you.

If you do not complete all mandatory lab tasks, you may do the remaining tasks on your own and present them at the following lab session or resource time, and prepare *even* better for the next lab...

4. **Sick notification**. If you are sick at a lab occasion, you must notify the *course responsible* by email *before* the lab. If you have been ill, you should try to do the task on your own and then present it at the next lab or resource time. If you need help catching up after illness, come to one or more resource times and talk to a supervisor. If you are absent without notifying illness, the course responsible may decide that you have to wait until next academic year to present, and then you will not receive a final grade in the course this academic year.

5. **Written answers**. For some lab tasks, there is a pen symbol in the margin. This symbol indicates that you should write down and save a result that you will need later, and/or that you should show to the lab supervisor at a subsequent checkpoint or at the final presentation.

6. **Checkpoints**. For some lab tasks, there is an eye symbol with a checkmark in the margin. This means you have reached a checkpoint where you should discuss your results with a supervisor. Raise your hand and show what you have done before continuing. If there is a long wait before the supervisor can come, it is okay to proceed anyway, but don't forget to discuss with the supervisor later so that you together ensure that you have understood all partial results. The dialogue with your supervisor is an important chance to get feedback on your code — make use of it!

-1.6 Project Assignment

After completing the lab series, there is an *obligatory project assignment* where you on your own will create a large program with many different interacting parts. It is only when the amount of code becomes really large that you truly benefit from the various abstraction mechanisms you have learned during the course and your debugging skills are put to the test. The following instructions apply to the project assignment:

1. **Choice of project assignment**. You choose the project assignment yourself. In chapter 12 there are several proposals to choose from. Read through all the assignment alternatives before choosing which one you want to do. You can also, in consultation with a supervisor, define your own project assignment, but before you start a self-defined project, a written description of the assignment must be approved by a supervisor well before the presentation date. Choose an assignment according to what you believe you can handle and avoid both a too simple task and taking on more than you can manage.

2. Instructions 1 and 2 for the labs (see section -1.5) also apply to the project assignment: it is **mandatory** and the work should be done **individually**. You may discuss your project assignment at an overall level with others, and you can ask for help from supervisors during resource times on individual details if you get stuck, but the solution should be *yours* and you must have written the entire program yourself.

3. **Scope**. The difference between the project assignment and the labs is that it should be *significantly* larger than the biggest labs and that you complete the full solution *before* the presentation date. You should therefore start well in advance, preferably two weeks before the presentation date, to be sure to finish on time. It is important that you carefully consider the scope in relation to your choice of project assignment, preferably based on your self-awareness of what you need to practice. Feel free to discuss with a supervisor how to best use the project assignment for your learning.

4. **Documentation**. Before the presentation, you should create automatically generated documentation based on relevant documentation comments for at least half of your public methods, according to the instructions in Appendix E.

5. **Code storage and version control.** The project assignment can be a suitable opportunity to practice version control with git. It is, as with the labs, *not* allowed to store your solutions publicly on the internet. If you want to practice using a code repository, e.g., GitHub or GitLab, be sure to verify that the repository is closed (eng. *closed repository*), so you do not risk complicity in cheating. Using git and a code repository is optional.

6. **Presentation**. During the presentation, you use the time with the supervisor to go through your solution, explain how your code works, and discuss the pros and cons of your approach. You should also describe how your program evolved and how you stepwise debugged and improved the implementation. You should also go through the documentation of your code during the presentation.

-1.7 Oral Exam(CGI)

At the scheduled time you are invited to take a oral examination (CGI) for supervisors. You must have passed all laboratory exercises to be eligible to skip the oral exam. The purpose of the exam is to verify that you have an approved understanding of the concepts included in the course. You are recommended to prepare thoroughly for the exam, for example by reviewing fundamental concepts for each course module and revising basic exercises and laboratory work.
The exam will be a random sampling from the course content. You will be given some randomly selected questions where you will be asked to explain some of the concepts covered in the course. You will also receive assignments to write code similar to the course exercises and explain how the code works. You can practice typical questions here: `https://fileadmin.cs.lth.se/pgk/muntabot/`

Chapter 0

How to contribute to the course material?

0.1 Contributions are warmly welcome!

One of the main purposes of making this course material free and open is to enable contributions from anyone interested. Contributions from students who want to participate in developing the teaching are especially welcome.

0.2 Instructions

0.2.1 What do you need to be able to contribute?

If you find a problem, e.g. a simple typo, or have something more extensive that should be improved, but do not yet know or have access to the tools described below that are needed to make contributions, then contact someone who has already contributed to the material, so that someone else can implement your suggestion.
Before you can implement changes directly in the material yourself, you need to know, and have access to, one or more of the following tools (depending on what the change concerns):

- Latex: en.wikibooks.org/wiki/LaTeX
- Scala: en.wikipedia.org/wiki/Scala_%28programming_language%29
- git: https://en.wikipedia.org/wiki/Git_%28software%29
- GitHub: en.wikipedia.org/wiki/GitHub
- sbt: en.wikipedia.org/wiki/SBT_%28software%29

Read more about how to contribute here:
github.com/lunduniversity/introprog#how-to-contribute

0.3 Example

As an example of how it works in a typical open source project, what happened in a real case is described below: a documentation update of the Scala documentation after a bug was discovered. This example case is a typical scenario that illustrates how it can go, and what one might need to think about. The example also provides links to and insight into a really large open source project.

Scenario: *making a contribution upon discovery of a problem*

"To my great joy, this *Pull Request* (PR) was accepted in the documentation site for Scala. You can see my contribution here:
github.com/scala/scala.github.com/pull/517/commits/2624c305a8a6f24ea3398fe0fcbd0c72492bdd12
Starting by contributing to documentation is often a good way to get into an open source project, as there is a good chance to help without requiring deep knowledge about the code in the repository[1]. I describe below what happened step by step when I got a real PR accepted, as a typical example of how it often works.

1. I thought the documentation for the method `lengthCompare` on indexable collections on scala-lang.org/documentation was confusing. When I tested in the REPL it became obvious that something was wrong: either the documentation was incorrect or the method did not work as it should. Oh dear, maybe I discovered a new bug? A chance to contribute!

2. First I carefully searched among all issues under the 'issues' tab on GitHub to see if someone had already found this problem. If so, I could have commented on such an issue and written something to support that it needed fixing, or preferably offered to try to fix it. But I found no issue on this...

3. Therefore I created a new issue by clicking the *New issue* button in the web interface on GitHub and here is the result:
   `https://github.com/scala/scala.github.com/issues/515#`
   I carefully considered how to phrase it:
   - The issue title is extra important: it should summarize in a single line what it is all about so that the reader of the heading understands what the problem concerns.
   - Then I worked on writing a clear and detailed description of the problem and specified exactly which version it applied to. It is good to paste examples from Scala REPL and other test runs with input and output if relevant. It is important that the problem can be found and reproduced by others, therefore information about which version it applies to and a minimal test case that clarifies the problem is needed.
   - It is good to ask questions and come with suggestions to open a discussion about the issue. I specifically asked whether this was a documentation problem or a bug in the code.
   - NOTE! You should not open an issue before first checking carefully that it really is something that should be addressed and that it is not a duplicate or overlap with other issues: every time you open an issue it will generate work for others even if the issue in the end does not result in any action...
   - If it is a more open, general proposal, an improvement or a completely new feature, you can also create an issue (it doesn't have to be a pure bug). If you are unsure whether the issue is relevant, it is good to discuss it in the community mailing list first.

4. I quickly received comments on my issue, which is typical for a well-functioning community with alert repository maintainers. And when I got encouragement to contribute, I offered to implement the improvement.

[1] The word repo is an abbreviation of repository, here meaning a storage place for code.

5. Remember to always write all comments and answers in a factual, concise and friendly tone!

6. The next step is to 'fork' the repo on GitHub by simply clicking *Fork* in the web interface. I then got my own copy of the repo under my own GitHub user, where I have rights to change.

7. Then I cloned the repo to my local machine with the terminal command `git clone` *`https://...`* (or you can use the GitHub Desktop app).

8. Then I fixed the problem directly in the relevant file in an editor on my computer, in this case the file was in Markdown format (an easily readable text format from which one can generate HTML):
   raw.githubusercontent.com/scala/scala.github.com/master/overviews/collections/seqs.md

9. When I fixed the problem, I did `git add` on the file and then
   `git commit -m "well thought-out commit msg"`
   I thought carefully before writing the first line in the commit message so that it would be both short and concise. But I still forgot to include the issue number `:(`, see my comment on the commit, which I added afterwards when I finally discovered my blunder:
   scala.github.com/commit/2624c305a8a6f24ea3398fe0fcbd0c72492bdd12#comments

10. After I did `git commit` the change was still only local on my computer. Then I had to 'push' to my fork on GitHub with `git push` (or use the *Sync* button in the GitHub Desktop app).

11. Then I created a PR by simply clicking the *New pull request* button on the GitHub page for my fork. I thought carefully before writing the title that describes this PR. If this change had been more extensive, I would also have needed to write a detailed description of how the change was implemented to facilitate the review of my proposal. You can see this (now closed) PR here:
    `https://github.com/scala/scala.github.com/pull/517`

12. When I created a PR, those who maintain the repo received an automatic message about this new PR and the subsequent review phase began. It usually ends with one or more other people commenting on the PR in the web interface with 'LGTM'. LGTM = *"Looks Good To Me"* and means approximately "I have looked at this now and it seems (as far as I can judge) excellent and therefore ready for *merge*". If it doesn't look good, the reviewer is expected to suggest what needs to be improved in a factual and friendly tone.

13. When the PR is reviewed, a person who has permission to change can 'merge' the PR into the main branch, often called *master*, in the central repo, often called *upstream*.

14. Finally, the issue can be closed by those responsible for the repo. This issue is now marked "Closed" and no longer appears in the list of active issues.

Phew! Then it was done `:)` "

Part II

Modules

Chapter 1

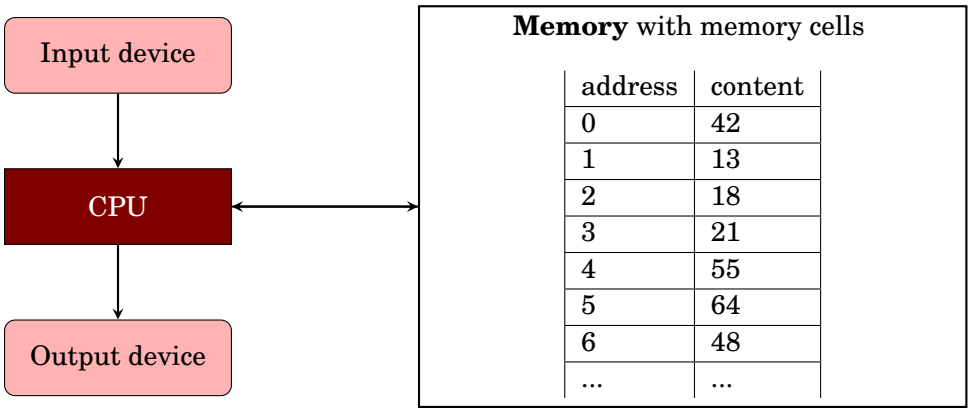
Introduction

Concepts included in this week's studies:

- ☐ sequence
- ☐ alternative
- ☐ repetition
- ☐ abstraction
- ☐ edit
- ☐ compile
- ☐ execute
- ☐ parts of computer
- ☐ virtual machine
- ☐ literal
- ☐ value
- ☐ expression
- ☐ identifier
- ☐ variable
- ☐ type
- ☐ assignment
- ☐ name
- ☐ val
- ☐ var
- ☐ def
- ☐ define and call function
- ☐ function header
- ☐ function body
- ☐ procedure
- ☐ built-in primitive types
- ☐ println
- ☐ Unit type
- ☐ unit value ()
- ☐ string interpolator s
- ☐ arithmetic
- ☐ random numbers
- ☐ logical expressions
- ☐ De Morgan's laws
- ☐ if
- ☐ true
- ☐ false
- ☐ while
- ☐ for
- ☐ dod: operating system

1.1 Theory

1.1.1 How does a computer work?



address	content
0	42
1	13
2	18
3	21
4	55
5	64
6	48
...	...

Memory contains only **integers** which represent **data and instructions**.

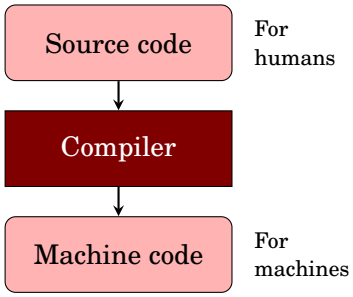
1.1.2 What is programming?

- Programming means giving instructions to a machine.
- A **programming language** is used by humans to write **source code** that can be translated by a **compiler** into **machine code** which in turn is **executed** by a computer.

- Ada Lovelace published the first program already in the 19th century intended for a mechanical gear computer.

- sv.wikipedia.org/wiki/Programming
- en.wikipedia.org/wiki/Computer_programming
- Have a picnic in Ada Lovelace Park at Brunnshög!

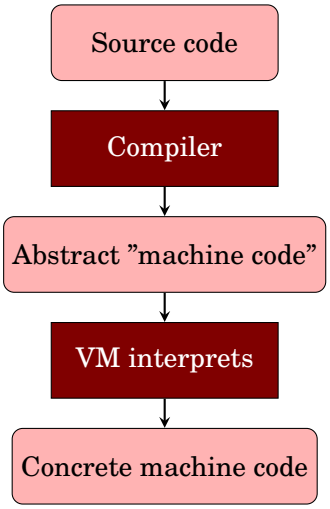
1.1.3 What is a compiler?



Grace Hopper invented the compiler in 1952.
en.wikipedia.org/wiki/Grace_Hopper

1.1.4 Virtual machine (VM) == abstract hardware

- A VM is a "computer" implemented in software that can interpret an abstract "machine code" which is **translated at run time** to the **real** machine's concrete machine code.
- With a VM the source code becomes **platform independent** and works on many different machines.
- Example JVM: **Java Virtual Machine**



1.1.5 What does a program consist of?

- Text that follows unambiguous language rules (grammar):
  - **Syntax**: the concrete appearance of the text
  - **Semantics**: the meaning of the text (what the machine does/calculates)
- **Keywords**: words with special meaning, e.g. `if`, `while`
- **Declarations**: definitions of new words: `def cucumber = 42`
- **Statements** are instructions that **do** something: `print("hello")`
- **Expressions** are instructions that calculate a **result**: `1 + 1`
- **Data** is information that is processed: e.g. the integer `42`
- Instructions are organized in code structures: **SARA**
  - **Sequence**: the order matters for what happens
  - **Alternative**: different outcomes depending on expression's value
  - **Repetition**: instructions are repeated many times
  - **Abstraction**: new building blocks created for reuse

1.1.6 Examples of programming languages

There are lots of different languages and new ones are constantly appearing.

- Java
- C
- C++
- C#
- Python
- JavaScript
- Scala
- Rust
- Go
- Kotlin
- ...

Some rankings:

- Redmonk
- PYPL
- TIOBE

1.1.7 Different programming paradigms

- There are many different programming paradigms (ways of programming), such as:
  - **imperative programming:** the program consists of statements that affect system state
  - **object-oriented programming:** a kind of imperative programming where the program consists of objects that encapsulate data and offer operations that process these data
  - **functional programming:** the program is built from cooperating functions that avoid changing data
  - **declarative programming, logic programming:** the program is built from logical expressions describing facts or conditions and execution is a proof procedure searching for values that satisfy those facts and conditions

This course covers the first three.

1.1.8 Hello world

Run line by line in Scala REPL (Read-Evaluate-Print-Loop):

```
> scala
Welcome to Scala 3.7.2 (21.0.5, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> println("Hello World!")
Hello World!
```

`@main` before any function indicates where a standalone program starts:

```
@main def hi = println("Hello world!")
```

Save the above text in file `hello.scala` and compile your program:

```
> scala compile hello.scala
```

Run your program with `scala run` which compiles automatically if needed.

```
> scala run hello.scala
Hello World!
```

1.1.9 Development cycle

edit; compile; find errors and improvements; edit; compile; find errors and improvements; edit; compile; find errors and improvements; edit; compile; find errors and improvements; edit; compile; find errors and improvements; edit; compile; find errors and improvements; ...

```
repeat(1000){
  edit
  compile
  test
}
```

1.1.10 Development tools

- Your tool knowledge is very important for your productivity.
- Learn shortcuts for common operations.
- All needed tools are preinstalled on **LTH's Linux computers**. If your own laptop is problematic: use the school's computers so you don't get delayed!
- Tools we use in the course:
  - Scala **REPL**: from exercise 1
  - Child-friendly Scala programming with Kojo: Lab 1
  - **Text editor** for code, e.g. **VS Code**: from exercise 2
  - Compile and run standalone programs with `scala`: from exercise 2
- Other tools worth learning:
  - Git for version control
  - GitHub for code storage – but **not** for lab solutions!
  - Linux/Ubuntu and useful terminal commands

1.1.11 Install tools on your own computer

When you want to write code in an editor, compile in the terminal and run your program as a **standalone application**, you need:

- An editor: **VS Code** with the extension **Scala (Metals)**
- The runtime environment **OpenJDK**
- Command line tools for the terminal: `scala`
- See instructions here: `https://lunduniversity.github.io/pgk/#verktyg`
- Read more in Appendix C.
- Tips if you use Windows: install the new Windows Terminal
- Installation help:
  1. Drop-in sessions organized by the department at lunchtime.
  2. Study evenings organized by SRD.
  3. `#questions-and-answers` on our Discord server
  4. Ask supervisors during resource time (if time permits).

1.1.12 Scala Command Line Interface (CLI)

- Development of a new command line interface (CLI) for Scala started in 2022 in an open source project led by Virtuslab.
- In August 2024, **`scala-cli`** became the new **`scala`** command. (Prior to Scala 3.5, the command was `scala-cli`.)
- Read more in Appendix C and F, and here: https://scala-cli.virtuslab.org/
- You can see what Scala CLI can do via the help option:

```
> scala help
```

  `scala-cli help` is the "old" command that is also included in the Scala installation.

1.1.13 Tips and tricks with `scala` in the terminal

- Type `:help` in the REPL to see available **commands**.
- You can **exit** the REPL with `:q` or by pressing Ctrl+D.
- A **vertical bar appears** if you press ENTER in the middle of an incomplete line. This indicates you can continue typing on a new line before evaluation takes place.
- If you want the REPL to wait to interpret the line you wrote and instead give you **another line**, press ESC first and then ENTER.
- To prevent REPL from giving a new line after ENTER on an incomplete line, write a **semicolon** and press ENTER.
- Start `repl` with a dot after a space if you want access to code in all scala files in the **current directory** in your REPL session:
  `scala repl .`
- Run with a dot after a space to compile and execute all scala files in the **current directory and** any **subdirectories**:
  `scala run .`

1.1.14 Literals

- A literal represents a fixed **value** in the code and is used to create **data** that the program will process.
- Examples:

`42`	integer literal
`42.0`	decimal literal
`'!'`	character literal, surrounded by 'single quotes'
`"hello"`	string literal, surrounded by "double quotes"
`true`	literal for the truth value "true"

- Literals have a **type** which determines what you can do with them.

1.1.15 Examples of built-in data types in Scala

- All values, expressions and variables have a **data type**, e.g.:
  - `Int` for integers
  - `Long` for *extra* large integers (uses more memory)
  - `Double` for decimal numbers, so-called floating point numbers
  - `String` for strings
- The compiler keeps track of expressions combined in a **type-safe** manner. Otherwise there is a **compilation error**.
- Scala and Java are so-called **statically typed** languages, meaning type information is checked at compile time[1].
- Scala compiler does **type inference**: you **do not have to write types** if the compiler can figure them out based on the subexpressions' types.

1.1.16 Basic types in Scala

These **basic types** (eng. *basic types*) are built into Scala:

English name	**Basic type**
integer types	`Byte`, `Short`, `Int`, `Long`, `Char`
floating point number types	`Float`, `Double`
numeric types	integer types and floating point types
string type	`String`
truth value type	`Boolean`

---

[1] Other languages, e.g. Python and Javascript, are **dynamically typed** and type checks are done at run time
What are the pros and cons of static vs dynamic typing?

1.1.17 Ranges of basic types

Basic type	Number of bits	Range: smallest & largest value
`Byte`	8	$-2^7 \ldots 2^7 - 1$
`Short`	16	$-2^{15} \ldots 2^{15} - 1$
`Char`	16	$0 \ldots 2^{16} - 1$
`Int`	32	$-2^{31} \ldots 2^{31} - 1$
`Long`	64	$-2^{63} \ldots 2^{63} - 1$
`Float`	32	$\pm\, 3.4028235 \cdot 10^{38}$
`Double`	64	$\pm\, 1.7976931348623157 \cdot 10^{308}$

Basic type `String` is stored as a *sequence* of 16-bit characters of type `Char` and can be arbitrary length (until memory runs out).

1.1.18 Expressions

- An **expression** consists of one or more parts which after **evaluation** yield a **result**.
- Parts in an expression can be e.g.:
  literals (42), operators (+), functions (math.sin), ...
- Examples:
  - A simple expression:
    `42.0`
  - Compound expressions:
    ```
    40 + 2
    (20 + 1) * 2
    math.sin(0.5 * math.Pi)
    "hello" + " to " + "you"
    ```
- When the program is interpreted the expression is **evaluated**, giving a result in the form of a **value** that has a **type**.

1.1.19 Variables

- A **variable** can be assigned the value of a simple or compound expression.
- A variable has a **variable name**, whose form follows the language's **identifier** rules.
- A new variable is introduced in a **variable declaration** and since it can be given a value, **initialized**. The name serves as a **reference** to the value.
- Examples of variable declarations in Scala, note the **keyword `val`**:

```
val a = 0.5 * math.Pi
val length = 42 * math.sin(a)
```

```
val exclamationMarks = "!!!"
val greetingSwedish = "Hello to you" + exclamationMarks
```

- During program execution the variable values are stored in memory and their respective value is fetched from memory when they are **referenced**.
- Variables declared with `val` can only be assigned a value **once**, at the initialization at declaration.

1.1.20 Rules for identifiers

- **Simple** identifier: e.g. `cucumber2Tomato`
  - Begin with a letter
  - ...followed by letters or digits
  - Can also contain underscores
- **Operator** identifiers, e.g. `+:`
  - Begin with an **operator character**, e.g. `+ - * / : ? ~ #`
  - Can be followed by more operator characters
- An identifier must **not** be a **reserved word**, see quick reference for all reserved words in Scala.
- **Literal** identifiers: `` `can contain anything` ``
  - Begin and end with **backticks** `` ` ``
  - Can contain anything (except backticks)
  - Can be used to avoid conflicts with reserved words: `` `val` ``

1.1.21 Building strings: concatenation and interpolation

- You can **concatenate** strings with the operator `+`
  `"hello" + " to " + "you"`
- After a string you can concatenate any expressions; expressions inside parentheses are evaluated first and the value is then converted to a string before concatenation:

```
val x = 42
val msg = "Double the value of " + x + " is " + (x * 2) + "."
```

- Scala can help you monitor string building with the **string interpolator s**:

```
val msg = s"Double the value of $x is ${x * 2}."
```

1.1.22 Integer arithmetic

- The four arithmetic operations are written as in mathematics (usual precedence):

```
scala> 3 + 5 * 2 - 1
res0: Int = 12
```

- **Parentheses** control **evaluation order**:

```
scala> (3 + 5) * (2 - 1)
val res1: Int = 8
```

- **Integer division** truncates decimals:

```
scala> 41 / 2
val res2: Int = 20
```

- **Modulo arithmetic** with remainder operator \%

```
scala> 41 % 2
val res3: Int = 1
```

1.1.23 Floating-point arithmetic

- Decimal numbers are represented with so-called **floating point numbers** of type `Double`:

```
scala> math.Pi
val res4: Double = 3.141592653589793
```

- Large numbers like $\pi * 10^{12}$ are written as:

```
scala> math.Pi * 1E12
val res5: Double = 3.141592653589793E12
```

- There are **not** infinitely many digits which causes problems with **rounding errors**:

```
scala> 0.1 + 0.2
val res6: Double = 0.30000000000000004

scala> 1E10 + 0.0000000000001
val res7: Double = 1.0E10

scala> BigDecimal("0.1") + BigDecimal("0.2")  // BigDecimal works
val res8: BigDecimal = 0.3
```

Read more here: https://0.30000000000000004.com

1.1.24 Define name for expressions

- With keyword **def** you can make a **name** mean the same as an **expression**.
- Example:

```
def cucumberLength = 42 + x
```

- The expression on the right is evaluated **each** time the **call** happens, i.e., each time the name is used elsewhere in code.

```
cucumberLength
```

1.1.25 Function, argument, parameter

- A **function** computes a **result** based on input called **arguments**.
- Arguments get names by declaring **parameters**.
- Example declaration of a function with one parameter:

```
def double(x: Int) = 2 * x
```

- Parameter types **must** be described after a **colon**.
- The compiler can infer the **return type**, but it can also advantageously be explicitly stated for clarity:

```
def double(x: Int): Int = 2 * x
```

- Note that the name `x` becomes a "fresh new" **local name** that **only exists and is visible** "inside" the function and is unrelated to any other `x` outside the function.
- The computation happens only at **call** of the function:

```
scala> double(42)
res1: Int = 84
```

1.1.26 Built-in math functions in the package `scala.math`

- The package **`scala.math`** contains many useful functions: e.g. `math.random()` gives random numbers between `0.0` and `0.99999999999999999`

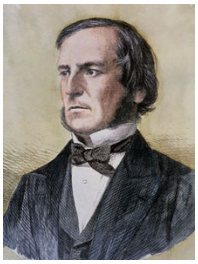
```
scala> val x = math.random()
x: Double = 0.27749191749889635

scala> val length = 42.0 * math.sin(math.Pi / 3.0)
length: Double = 36.373066958946424
```

- Study the documentation here: https://www.scala-lang.org/api/current/scala/math.html#

- The package `scala.math` often delegates to Java class **`java.lang.Math`** which is documented here:
  https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Math.html

---

1.1.27 Logical expressions



- The computer can "calculate" with truth and falsity: so-called Boolean algebra after George Boole
- Simple logical expressions (only two):

```
true
false
```

- Compound logical expressions with logical operators: && and, || or, ! not, == equality, != inequality, relations: > < >= <=
- Examples:

```
true && true
false || true
!false
42 == 43
42 != 43
(42 >= 43) || (1 + 1 == 2)
```

1.1.28 De Morgan's laws

De Morgan's laws describe what happens if you **negate** a logical expression. Can be used for **simplifications**.

- In all subexpressions connected with && or ||, change all && to || and vice versa.
- Negate all embedded subexpressions. A relation is negated by swapping == to !=, < to >=, etc.

Example of simplification using De Morgan's laws repeatedly:

```
! (a < b || (a == 1 && b == 1))              ⟺
! (a < b) && ! (a == 1 && b == 1)            ⟺
! (a < b) && (! (a == 1) || ! (b == 1))      ⟺
a >= b && (a != 1 || b != 1)
```

1.1.29 Alternatives with if-expressions

- An if-expression begins with keyword **if**, followed by a logical expression (condition) in parentheses and two branches.

```
def randomVegetable = if math.random() < 0.8 then "cucumber" else "tomato"
```

- The expression after **then** is the result if the condition is **true**
- The expression after **else** is the result if the condition is **false**

```
scala> randomVegetable
res13: String = cucumber

scala> randomVegetable
res14: String = cucumber

scala> randomVegetable
res15: String = tomato
```

1.1.30 Expression or statement?

Difference between expression and statement:

- An expression produces a **result**. Example: `1+1`
- A statement has an **effect**.
  Example: output, save to file, assign new value to variable.

Write an **expression** when you are interested in the **value** being calculated.
Write a **statement** when you want something to **be done**.

Both statements and expressions can in turn contain statements and expressions in arbitrarily complex **nested structures** (more on this later).

1.1.31 Variable declaration and assignment statement

- A **variable declaration** reserves **space in computer memory** so that values of the type the variable can refer to have space there.
- At declaration the variable must be **initialized** with a starting value.
- A **val** declaration gives a variable that cannot be changed after initialization.

These declarations...

```
var x = 42
val y = x + 1
```

... give this content somewhere in memory:

x	42
y	43

- With an **assignment statement** a previously **var** declared variable is given a new value:

```
x = 13
```

- The old value disappears forever and the new value is stored instead:

x	13
y	43

  Note that y is not affected by x changing value here.

1.1.32 Assignment statements are *not* mathematical equality

- The equals sign is used for **assigning** new values to variables and is **not** the same as mathematical equality. What happens here?

```
x = x + 1
```

- This syntax is inherited from older languages like C, Fortran etc.
- In other languages you might see e.g.
  `x := x + 1` or `x <- x + 1`

- This syntax perhaps better shows that assignment is a **stepwise process**:
  1. First the **expression to the right** of the assignment sign is evaluated.
  2. Then the *value* the variable name refers to is *replaced* by the evaluated expression. The old value **disappears forever**.

1.1.33 Shortened assignment statements

- It is common to want to assign a variable a new value depending on the old, like `x = x + 1`
- Therefore there are **shortened assignment statements** that save a few characters and make it clearer (?) what happens (once you've gotten used to this writing style):

```
x += 1
```

- The above expression gets expanded by the compiler to `x = x + 1`

1.1.34 Examples of shortened assignment statements

```
scala> var x = 42
val x: Int = 42

scala> x *= 2

scala> x
```

```
val res0: Int = 84


scala> x /= 3

scala> x
val res1: Int = 28
```

1.1.35 Variables that change values can be tricky

- Code containing variables that **change** over time is often harder to read and understand.
- Many bugs result from variables inadvertently being changed in incorrect and unexpected ways.
- Mutable values become especially difficult in concurrent (parallel) code.
- In production code with many users, `val` is therefore preferred, while `var` is used only if **really** needed.
- So: compute new values rather than changing existing ones.

1.1.36 Control structures: alternatives and repetition

Used to control (change) the sequence and create **alternative** paths through code. The path is decided at runtime.

- if-statement:

```
if math.random() < 0.8 then println("cucumber") else println("tomato")
```

Different kinds of **loops** to repeat statements. The number of repetitions is given at run time.

- `while` loop: good when you **do not know how many times** it will repeat.

```
while math.random() < 0.8 do println("cucumber")
```

- `for` loop: good when you **want to specify the number of repetitions**:

```
for i <- 1 to 10 do println(s"cucumber no. $i")
```

1.1.37 Scala 2 syntax for control structures works in Scala 3

In Scala 2 an older syntax was used for control structures that resembles C/C++/Java more. It is allowed in Scala 3, but the new more readable syntax is preferred.

- Scala 2 syntax for alternatives: parentheses but no `then`

```
if (math.random() < 0.8) println("cucumber") else println("tomato")
```

Scala 2 syntax for repetition:

- **while** loops: parentheses but no **do**

```
while (math.random() < 0.8) println("cucumber")
```

- **for** loops: parentheses but no **do**

```
for (i <- 1 to 10) println(s"cucumber no. $i")
```

- Kojo Desktop only works with Scala 2 and old syntax, but Kojo can also run with Scala 3 (see Appendix A).

1.1.38 Repeat many statements

If you want to do several things in sequence inside a repetition you can write many statements within **curly braces**:

```
while math.random() < 0.8 do {
  println("cucumber")
  println("tomato")
}
```

You can after some keywords (e.g. **do**, **then**, **else**) choose to omit curly braces (eng. *optional braces*).

```
while math.random() < 0.8 do
  println("cucumber")
  println("tomato")
```

Then the **indentation** determines which statements are included.
This works in Scala 3 (but not in Scala 2).

1.1.39 Procedures

- A **procedure** is a function that **does** something interesting, but **does not** return any interesting value.
- Example procedure in standard library: `println("hi")`
- You **declare your own procedures** by specifying **Unit** as return type. Then the value **()** is returned which means "nothing".

```
scala> def hi(x: String): Unit = println(s"Hello to you $x!")
hi: (x: String)Unit

scala> hi("Mr. Cucumber")
Hello to you Mr. Cucumber!
```

```
scala> val x = hi("Ms. Tomato")
Hello to you Ms. Tomato!
x: Unit = ()
```

- What **is done** is called (side) **effect**. Above the printing is the effect.
- Functions can also have side effects. They are then called **impure** functions.

1.1.40 Problem solving: decomposition into abstractions that are then combined

- One of the most important principles in programming is **functional decomposition** where **subprograms** in the form of functions and procedures are created to become building blocks combined into more advanced functions and procedures.
- Through the defined names **reusable abstractions** are created that encapsulate what the function does into a ”building block”.
- Good ”building blocks” make it easier to solve difficult programming problems.
- Abstractions that calculate or do **one single, well-defined thing** are easier to use compared with those that do many, very different things.
- Abstractions with **well thought-out names** are easier to use compared to cryptic or misleading names.

1.1.41 Exercise `expressions` and lab `kojo`

- In the exercises you run Scala REPL to practice SARA.
  Read in Appendix and on the course website under ”Tools” how to install and get Scala REPL running.
- In the lab you use child-friendly **Kojo** to practice SARA, focusing on abstraction.
- There are two ways to use Kojo:
  1. The graphics library in **`kojo`** in a standalone Scala program with vscode and running in terminal, see **Appendix A. Works well with new Scala 3**.
  2. The desktop app **Kojo Desktop** with built-in child-friendly editor (only Scala 2, old syntax etc).
  3. The web app **`http://kojo.lu.se/`** directly in the browser; not recommended – only Scala 2, more limited.

Option 1 is recommended, but if you are delayed by technical issues, get started with 2 for now until you get help.

1.2 Exercise `Excercise01`

Goals

- ☐ Understand what happens when statements are executed and expressions are evaluated.
- ☐ Be familiar with the meaning of the concepts sequence, alternative, repetition, and abstraction.
- ☐ Be familiar with literals for simple values, their types, and scope.
- ☐ Be able to declare and use variables and assignment, as well as be able to draw diagrams of the memory situation when variable values change.
- ☐ Understand the difference between different numeric types, be able to convert between them, and be aware of precision issues that may arise.
- ☐ Understand boolean expressions and the values **`true`** and **`false`**, and be able to simplify boolean expressions.
- ☐ Understand the difference between integer division and floating-point division, as well as the use of remainder in integer division.
- ☐ Understand precedence rules and the use of parentheses in expressions.
- ☐ Be able to use **`if`** statements and **`if`** expressions.
- ☐ Be able to use **`for`** loops and **`while`** loops.
- ☐ Be able to use `math.random()` to generate random numbers within different intervals.

Preparations

- ☐ Study the concepts in chapter 1
- ☐ You need a computer with Scala and Kojo, see appendix C and A.

1.2.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with description.*

Connect each concept with the (simplified) description that fits best:

literal	1	A	introduce new concepts that simplify coding
string	2	B	either true or false
statement	3	C	translate code into executable form
expression	4	D	represents a specific data value
function	5	E	decimal number with limited precision
procedure	6	F	a line of code that does something; can be separated with a semicolon
runtime error	7	G	a sequence of characters
compile error	8	H	combines values and functions into a new value
abstract	9	I	describes what the data can be used for
compile	10	J	performs a side effect when called; return value is empty
type	11	K	computes a return value when called
for-loop	12	L	to change the value of a variable
while-loop	13	M	may occur before execution starts
assignment	14	N	may occur while the program runs
floating-point	15	O	useful when the number of repetitions is known beforehand
boolean	16	P	useful when the number of repetitions is unknown beforehand

Task 2. *Output in Scala REPL.*

Start Scala REPL (eng. *Read-Evaluate-Print-Loop*).

```
> scala
Welcome to Scala 3.1.2 (17.0.2, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.
scala -version.
scala>
```

a) Write after the prompt `scala>` a statement that prints any (common/crazy) greeting phrase, by calling the procedure `println` with some string argument. Press *Enter* so that the statement compiles and executes.

b) Write the same statement again (or press the up arrow) but ”forget” to write the right parenthesis after the argument before pressing *Enter*. What happens?

> *Tip for continuation:* There are many useful shortcuts and other tricks to work fast in REPL. Ask someone who knows these tricks to show you how to work faster. Read appendix C.4.2 and then try copying and pasting text. Use arrow keys to scroll through history, Ctrl+A to go to the start of a line, Ctrl+K to delete from cursor to end of line, etc.

Task 3. *String concatenation.*

a) Write an expression that concatenates two strings, e.g. `"gurk"` and `"burk"`, using the operator + and observe the result. What is the value and type of the expression? What digit follows the word `res` in the variable that stores the result?

b) Use the result from the concatenation, e.g. `res0` (replace the `0` with the digit after `res` in the output from the previous evaluation if necessary), and write an expression

using the operator $*$ that repeats the result from the previous subtask 42 times.

Task 4. *When is the error detected?*

a) What is the type and value of the expression `"hej" * 3` ? Test in REPL.

b) Replace the 3 above with a sufficiently large integer so that memory runs out, but not so large that the number is outside the range of the base type `Int`. How does the error message start? Is this a runtime error or a compile-time error?

c) Choose a value for the argument after the operator $*$ such that a type error is generated. How does the error message start? Is this a runtime error or a compile-time error?

> *Tip for continuation:* Feel free to make mistakes when coding to learn more! Practice interpreting different error messages and ask someone for help if you don't understand. The compiler's messages can be very helpful, though sometimes cryptic. If you get stuck and cannot proceed yourself, ask for help, *but ask for hints rather than complete solutions* so you keep the initiative and control over the next step in your learning.

Task 5. *Literals and types.*

a) Use the REPL command `:type` (can be abbreviated `:t`) if needed to match the literals below with the correct type.

`1`	1	A	`String`
`1L`	2	B	`Boolean`
`1.0`	3	C	`Boolean`
`1D`	4	D	`Unit`
`1F`	5	E	`Int`
`'1'`	6	F	`Double`
`"1"`	7	G	`Long`
`true`	8	H	`Float`
`false`	9	I	`Char`
`()`	10	J	`Double`

b) What happens if you add 1 to the largest possible value of the `Int` type?
Tip: see the quick reference [2] under the heading "The Scala type system" section "Methods on numbers".

c) What is the difference between the types `Long` and `Int`?

d) What is the difference between the types `Double` and `Float`?

Task 6. *Mathematical functions. Using documentation.*

a) Suppose you have a chessboard with 64 squares. Imagine you start by placing a single grain of rice on the first square and then pile twice as many grains in a new pile for each subsequent square: 1, 2, 4, 8, ... etc. When you have done this for all squares, how many grains of rice have you placed on the chessboard in total?[3]

[2] https://fileadmin.cs.lth.se/pgk/quickref.pdf
[3] https://en.wikipedia.org/wiki/Wheat_and_chessboard_problem

Tip: You should calculate $2^{64} - 1$. If you type `math.` in the REPL and press TAB you will see built-in mathematical functions in Scala's standard library:

```
scala> math.    // Press TAB right after the dot and observe the list
```

Use the function `math.pow` and appropriate arguments. If you enter `math.pow` or `math.pow()` without arguments, you will see the function header with the parameter list.
If you browse to http://www.scala-lang.org/api/current/ and write `math` into the search box and then, after clicking **`scala.math`**, write **`pow`** in the box further down, the page is filtered and you find the documentation of `def pow` which you can click on and read more about.

b) Define the function `circumference` below in REPL. Can you omit the return type annotation? Why? Is there reason to keep it?

```
def circumference(radius: Double): Double = 2 * math.Pi * radius
```

c) Earth's (average) diameter (at the equator) is about 12 750 km. Write an expression that calls the function `circumference` above to calculate approximately how many kilometers per day you must travel if you want to go around the Earth in 80 days.

Task 7. *Variables and assignment. Mutable and immutable variables.*

a) Draw a *new* picture of the computer's memory after *each* executed line 1–6 below. Each picture should show all variables existing in memory along with their variable name, type, and value.

```
1 scala> var a = 13
2 scala> val b = a + 1
3 scala> var c = (a + b) * 2.0
4 scala> b = 0
5 scala> a = 0
6 scala> c = c + 1
```

After the first line the memory situation looks like this:

a:	Int	13

b) Why does line 4 cause an error? Is it a compile-time error or a runtime error? What does the error message say?

Task 8. *Random numbers with `math.random()`.*

a) What does the function `math.random()` return? What type? What are the maximum and minimum possible values?
Tip: Search here: http://www.scala-lang.org/api/current/and try in REPL.

b) Declare the parameterless function `def roll: Int = ???` that should represent a dice roll and return a random integer between 1 and 6. Test the function by calling it many times.
Tip: Use `math.random()` and multiply and add with appropriate integers. Surround the calculation with parentheses and end with `.toInt` to truncate decimals and convert the type from `Double` to `Int`.

Task 9. *Repetition with* **`for`**, `foreach` *and* **`while`**.

a) Here is an example of a **`for`** statement:

```
for i <- 1 to 10 do print(s"$i, ")
```

Use a **`for`** statement to print out the result of 100 dice rolls with the function `roll` from exercise 8.

b) Here is an example of a `foreach` statement:

```
(1 to 10).foreach(i => print(s"$i, "))
```

Use a `foreach` statement to print out the result of 100 dice rolls with the function `roll` from exercise 8.

c) Here is an example of a **`while`** statement:

```
var i = 1
while i <= 10 do { print(s"$i, "); i = i + 1 }
```

Use a **`while`** statement to print the result of 100 dice rolls with the function `roll` from exercise 8. What happens if you forget `i = i + 1`?

Task 10. *Alternatives with* **`if`** *statement and* **`if`** *expression.*

a) Here is an example of an **`if`** statement (note the double equal signs):

```
if roll == 3 then println("THREE") else println("NOT THREE")
```

Try the above in REPL. Then write a **`for`** statement that rolls 100 dice and prints the string `"CONGRATULATIONS! "` if it is a six, else a sad smiley: `":("`

b) Here is an example of an **`if`** expression:

```
if roll < 6 then 0 else 1
```

Try this in REPL. Then write a **`while`** statement that rolls 100 dice and counts the number of sixes. Print the count after the **`while`** statement.

c) Simplify the implementation of this function:

```
def isAdult(age: Int) = if age >= 18 then true else false
```

Task 11. *Sequence, statement and block.*

a) What do these statements do?

```
scala> def p = { print("true"); print("!"); println("hej")}
scala> p;p;p;p
```

b) Use the up arrow to get back the line where you defined the procedure `p`. Swap the strings in the print calls in the procedure `p` so that the output becomes:

```
hejtrue!
hejtrue!
hejtrue!
hejtrue!
```

c) How does the compiler interpret curly braces and semicolons? What is a block?

Task 12. *Integer division.* Which value and which type correspond to which expression? If you are unsure, test in REPL.

`4 / 42`	1	A	`4: Int`
`42.0 / 2`	2	B	`10: Int`
`42 / 4`	3	C	`21.0: Double`
`42 % 4`	4	D	`true : Boolean`
`4 % 42`	5	E	`false: Boolean`
`40 % 4 == 0`	6	F	`0: Int`
`42 % 4 == 0`	7	G	`2: Int`

Task 13. *Boolean values.* What is the value of these expressions?

a) `true && true`
b) `false && true`
c) `true || true`
d) `false || true`
e) `false || false`
f) `true == true`
g) `true != false`
h) `true > false`
i) `true && (1 / 0 > 1)`
j) `false && (1 / 0 > 1)`

Task 14. *Boolean variables.* What is printed on line 2 and 4 below?

```
1 scala> var monster = false
2 scala> if monster then println("watch out!!!")
3 scala> monster = true
4 scala> if monster then println("watch out!!!")
```

Task 15. *Turtle graphics with Kojo.* In this week's lab you will use Kojo to verify that you can use sequence, alternative, repetition, and abstraction. With Kojo you create Scala programs that draw colorful shapes using an easy-to-use Scala library for *turtle graphics*[4].

If you use Kojo as a graphics library (recommended) and run with `scala run` (see Appendix A) you can use Scala 3. But if you run Kojo Desktop, Scala 2 applies and even though most of this week's lab works similarly in Scala 2 and Scala 3, Scala 2 requires the old syntax for control structures with necessary parentheses around condition expressions, without either **do** or **then**, and neither optional curly braces nor indentation syntax.

Enter and run the program below with any method following Appendix A. Note the connection between the order of statements and what happens in the drawing window.

```
front; right
front; left
```

[4] https://en.wikipedia.org/wiki/Turtle_graphics

```
color(green)
front
```

If you use Kojo Desktop it is good to start the program with `erase` (why?[5]).

a) Write code that draws a square as in the image below.

Feel free to write your code in different ways *without* changing the result: write statements in sequence on multiple lines or statements in sequence on the same line separated by semicolons; use whitespace and blank lines in the code. How do you want to group your statements so they are easy for a person to read?

b) Draw stairs as shown in the image below.

c) Draw any picture on any background using some of the procedures in the table below. For example, you can draw a pink triangle with purple outlines on a black background. Try to ease code readability using appropriate line breaks and grouping.

`front(100)`	'The turtle moves forward 100 steps (25 if no argument).'
`color(pink)`	'Sets the pen's color to pink.'
`fill(purple)`	'Sets the filling color to purple.'
`fill(transparant)`	'Makes the turtle *not* fill anything when drawing.'
`width(20)`	'Makes the pen width 20.'
`background(black)`	'The background color becomes black.'
`penDown`	'Puts the turtle's pen down so it draws when moving.'
`penUp`	'Lifts the turtle's pen so it *does not* draw when moving.'
`right(45)`	'The turtle turns 45 degrees to the right.'
`left(45)`	'The turtle turns 45 degrees to the left.'
`jump`	'The turtle jumps 25 steps without drawing.'
`jump(100)`	'The turtle jumps 100 steps without drawing.'
`jumpTo(100, 200)`	'The turtle jumps to position (100, 200) without drawing.'
`goTo(100, 200)`	'The turtle turns and moves to position (100, 200).'

[5]When you press the play button in Kojo Desktop the canvas in the drawing window nor the turtle's state is reset. By starting your Kojo Desktop programs with `erase` you start execution in exactly the same initial condition: a blank canvas where the turtle points upward, the pen is down, and the pen color is red.

`east`	'The turtle turns so its nose points right.'
`west`	'The turtle turns so its nose points left.'
`north`	'The turtle turns so its nose points upward.'
`south`	'The turtle turns so its nose points downward.'
`setAngle(90)`	'The turtle turns its nose to angle 90 degrees.'

Tip for continuation: Keep both REPL and an editor open simultaneously. Then you can investigate how different code fragments work in REPL, while you *stepwise* create increasingly larger programs in the editor. This way of working will be very useful throughout the rest of the course. Regardless of which other tools you run, it is useful to keep REPL running in a separate window to help in your creative process, while chasing bugs, and while learning new concepts. Whenever you wonder how something works in Scala: **bring up REPL and try it!**

1.2.2 Extra Tasks; additional practice

Task 16. *Value and type.* Which value and type correspond to which expression? If you are unsure, test in REPL.

`1.0 + 18`	1	A	`1.042E42: Double`
`(41 + 1).toDouble`	2	B	`65: Int`
`1.042e42 + 1`	3	C	`113: Int`
`12E6.toLong`	4	D	`48: Int`
`32.toChar.toString`	5	E	`" ": String`
`'A'.toInt`	6	F	`0: Int`
`0.toInt`	7	G	`'*': Char`
`'0'.toInt`	8	H	`19.0: Double`
`'9'.toInt`	9	I	`12000000: Long`
`'A' + '0'`	10	J	`'q': Char`
`('A' + '0').toChar`	11	K	`42.0: Double`
`"*!%#".charAt(0)`	12	L	`57: Int`

Task 17. *Statements and expressions.*

a) What is the difference between a statement and an expression?

b) Give examples of statements that are not expressions?

c) Explain what happens for each evaluated line:

```
scala> def valueMissing = ()
scala> valueMissing
scala> valueMissing.toString
scala> println(valueMissing)
scala> println(println("hej"))
```

d) What is the type of the literal `()`?

e) What is the return type of `println`?

Task 18. *Procedure with parameter.* A procedure is a function that causes a side effect, for example printing or a variable assignment, but that does not return any interesting result value.[6]

a) Declare a mutable variable `highscore` initialized to 0.

b) Declare a procedure `updateHighscore` that takes a parameter `points` and assigns `highscore` a new value if `points` is greater than `highscore`, and prints the string `"REKORD!"`. If `points` is not greater than `highscore`, print the string `"DON'T GIVE UP!"`. Test the procedure in REPL.

c) Make a new variant of `updateHighscore` that is *not* a procedure but instead a function returning a string for later printing. Test the function in REPL.

[6] In Scala, procedures are functions returning the *empty value*, written `()` and typed `Unit`. In Java and several other languages there is no empty value and a special syntax for procedures using the keyword `void`.

Task 19. *Floating point arithmetic.*

a) What is the smallest positive value of type `Double`?

b) What is the value of this expression? Why does it become so?

```
scala> Double.MaxValue + Double.MinPositiveValue == Double.MaxValue
```

Task 20. ***if*** *statements.* For each line below, describe what is printed.

```
scala> if !true then println("true") else println("false")
scala> if !false then println("true") else println("false")
scala> def flipCoin = if math.random() < 0.5 then "kop" else "munt"
scala> for i <- 1 to 5 do print(s"$i:$flipCoin")
```

Task 21. Declare the following variables with these initial values:

```
scala> var vegetable = "cucumber"
scala> var fruit = "banana"
```

For each line below, give the value and type of the expression:

```
scala> if vegetable == "tomato" then "good" else "not good"
scala> if fruit == "banana" then "good" else "not good"
scala> if true then vegetable else 42
scala> if false then vegetable else 42
```

Task 22. *Modulo operator* `%` *and Boolean values.*

a) Declare a function `def isEven(n: Int): Boolean = ???` that returns `true` if the number `n` is even, else `false`.

b) Declare a function `def isOdd(n: Int): Boolean = ???` that returns `false` if the number `n` is even, else `true`.

Task 23. *Differences between* ***var***, ***val***, ***def***.

a) Evaluate each line one at a time in order in Scala REPL. For each line below: explain what happens and note value and possible errors.

```
scala> var x = 30
scala> x + 1
scala> x = x + 1
scala> x == x + 1
scala> val y = 20
scala> y = y + 1
scala> var z = { println("hej z!"); math.random() }
scala> def w = { println("hej w!"); math.random() }
scala> z
scala> z
scala> z = z + 1
scala> w
scala> w
scala> w = w + 1
```

b) What is the difference between `var`, `val` and `def`?

Task 24. *Difference between* ***`if`*** *and* ***`while`***. What is the result of line 3 and 4?

```
1 scala> def lotto1 = if math.random() > 0.5 then print("win :) ")
2 scala> def lotto2 = while math.random() > 0.5 do print("win :) ")
3 scala> lotto1
4 scala> lotto2
```

1.2.3 Advanced Tasks; challenges

Task 25. *Logic and De Morgan's Laws.* Simplify the following expressions. Assume `point` and `highscore` are integer variables and `clear` is of Boolean type.

a) `point > 100 && point > 1000`

b) `point > 100 || point > 1000`

c) `!(point > highscore)`

d) `!(point > 0 && point < highscore)`

e) `!(point < 0 || point > highscore)`

f) `clear == true`

g) `clear == false`

Task 26. *The string interpolator s.* Putting an `s` in front of a string literal allows the compiler to safely insert variable values into the string. Variable names must be preceded by a dollar sign, e.g. `s"Hej $namn"`. If you want to evaluate an expression, place it in braces immediately after the dollar sign, e.g. `s"Doubled length: ${namn.size * 2}"`

a) What is printed below?

```
scala> val f = "Kim"
scala> val e = "Finkodare"
scala> println(s"The name '$f $e' has ${f.size + e.size} letters.")
```

b) Create the following output using the `s` string interpolator and the variables `f` and `e` from the previous subtask.

```
Kim has 3 letters.
Finkodare has 9 letters.
```

Task 27. *Assignment operators.* You can shorten an assignment that changes a variable, e.g. `x = x + 1`, by using so-called assignment operators and write `x += 1` which means the same. Draw a new picture of computer memory after each line below. The pictures should show variables' names, type, and value.

```
scala> var a = 40
scala> var b = a + 40
scala> a += 10
scala> b -= 10
scala> a *= 2
scala> b /= 2
```

Task 28. *Big numbers.* If we want to compute $2^{64} - 1$ as an exact integer[7] it becomes larger than `Int.MaxValue`, so unfortunately we cannot use fast `Int`. Luckily, `BigInt` to the rescue.

a) Read about `BigInt` and `BigDecimal` in http://www.scala-lang.org/api/current/ Note what they can be used for.

b) You create a `BigInt` integer using `BigInt(2)` and can call the method `pow` on a `BigInt` with dot notation. Compute $2^{64} - 1$ as an exact integer.

[7] https://en.wikipedia.org/wiki/Wheat_and_chessboard_problem

c) What disadvantages do `BigInt` and `BigDecimal` have?

Task 29. *Precedence rules* Evaluation order can be controlled with parentheses. What value and type do the following expressions have?

a) `23 + 2 * 2 + (23 + 2) * 2`

b) `(-(2 - 42)) / (1 + 1 + 1)`

c) `(-(2 - 42)) / (-1)/(1 + 1 + 1)`

Task 30. *Documentation of packages in Java and Scala.*

a) By pressing the tab key you can see what is in various packages. What is the constant π called in `java.lang.Math` (note capital M) and `scala.math.`?

```
scala> java.lang.Math.    //press TAB after the dot
scala> scala.math.        //press TAB after the dot
```

b) Compare the documentation for class `java.lang.Math` here:
https://docs.oracle.com/javase/8/docs/api/
with the documentation for the package `scala.math` here:
http://www.scala-lang.org/api
Give examples of what you can do on the Scala documentation website that you *cannot* do on the corresponding Java documentation website.

c) What does the method `hypot` do? Why is it better to use `hypot` instead of implementing the calculation yourself with square root, multiplication, and addition?

Task 31. *Accuracy and exceptions in arithmetic expressions.* What is the result of the expressions below? Note exceptions and accuracy issues.

a) `Int.MaxValue + 1`

b) `1 / 0`

c) `1E8 + 1E-8`

d) `1E9 + 1E-9`

e) `math.pow(math.hypot(3,6), 2)`

f) `1.0 / 0` ★

g) `(1.0 / 0).toInt` ★

h) `math.sqrt(-1)` ★

i) `math.sqrt(Double.NaN)` ★

j) `throw new Exception("PANG!!!")`

Task 32. *Modulo arithmetic with negative numbers.* Read about modulo arithmetic here: ★
en.wikipedia.org/wiki/Modulo_operation
and investigate how it works with different signs (positive vs negative) for modulo calculation with $dividend\%divisor$ in Scala.

Task 33. *Literal identifiers.* Read about identifiers in Scala and especially *literal identifiers* here: `http://www.artima.com/pins1ed/functional-objects.html#6.10`. ★

a) Explain what happens below:

```
scala> val `literal val` = 42
scala> println(`literal val`)
```

b) Scala and Java have different sets of reserved words. In what way can 'backticks' be useful considering this?

★ **Task 34.** *`java.lang.Integer`, hexadecimal literals, BigDecimal.*

a) Find the documentation for `java.lang.Integer`.
Use the methods `toBinaryString` and `toHexString` to fill in the table below.

decimal integer	binary value	hexadecimal value
33		
42		
64		

b) How do you write the hexadecimal integer 10c (which equals 268 decimal) as a literal in Scala?

c) What is `0x10` raised to the power c = speed of light in m/s? *Tip:* Use `BigDecimal`.

★ **Task 35.** *String formatting.* Read about the `f`-interpolator here:
http://docs.scala-lang.org/overviews/core/string-interpolation.html
How can you use `f`-interpolator to produce the following output in REPL? Change line 2 at `???` so that the floating point number `g` is rounded to three decimals before printing.

```
1 scala> val g = 2 / 3.0
2 scala> val str = f"The giant cucumber is $g??? meter lang"
3 scala> println(str)
4 The giant cucumber is 0.667 meter lang
```

Task 36. *Multiplication warning.* Look up the documentation for `java.lang.Math.multiplyExact` and read what that method does.

a) What happens here?

```
scala> Math.multiplyExact(1, 2)
scala> Int.MaxValue * 2
scala> Math.multiplyExact(Int.MaxValue, 2)
```

b) Why might one want to use `java.lang.Math.multiplyExact` instead of 'regular' multiplication?

★ **Task 37.** *Extra operators for exact multiplication.* Kim CodeWizard thinks `Math.multiplyExact` is too cumbersome to write and extends the type `Int` with an extra operator:

```
extension (i: Int) def *!(j: Int) = Math.multiplyExact(i,j)
```

a) Paste the above code into REPL and try the extra operator.

b) Help Kim CodeWizard add more operators for values of type `Int` that allow using `Math.subtractExact` and `Math.addExact` conveniently.

c) Test a compound expression that uses all extra methods on `Int`. Do you find it more readable or more cryptic with the new operators?

1.3 Lab: `kojo`

Goals

- ☐ Be able to apply and combine the principles of sequence, alternative, repetition, and abstraction in creating own programs of at least 20 lines of code.
- ☐ Be able to explain what a program does in terms of sequence, alternative, repetition, and abstraction.
- ☐ Be able to format own programs so that they become easy to read and understand.
- ☐ Be able to explain what a variable is and be able to declare immutable and mutable variables, as well as make assignments.
- ☐ Be able to carry out repeated rounds in the cycle edit-execute-debug/improve to gradually build up increasingly developed programs.

Preparations

- ☐ Review this week's lecture materials.
- ☐ Do exercise `Excercise01` in section 1.2
- ☐ Read about Kojo in appendix A.
- ☐ Read through the entire lab below. Consider possible solutions for the tasks that are marked with a pencil in the margin.
- ☐ Fetch the given code via the course GitHub repository.

1.3.1 Mandatory tasks

If there is a pencil in the margin, you should make notes in preparation for the presentation.

Task 1. *Install and start Kojo.* Open the terminal and enter the following commands to download the file `kojo.scala` from the internet, save it in the directory `~/w01-kojo`, and start Kojo.

```
> mkdir ~/w01-kojo
> cd ~/w01-kojo
> curl -sLO https://fileadmin.cs.lth.se/kojo.scala
> scala repl ~/w01-kojo/kojo.scala
```

You should now be able to write `activateCanvas()` in the terminal, press ENTER, and see a window appear on the screen.

```
Welcome to Scala 3.7.3 (21.0.8, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> activateCanvas()

scala>
```

Task 2. *Sequence and repetition.* Draw a square using `repeat(n){ ??? }` where you replace `n` with the number of repetitions and `???` with the statements to repeat.

Task 3. *Variable and repetition.*

a) The function `System.currentTimeMillis` is part of Java's standard library and returns an integer of type `Long` with the current number of milliseconds since midnight on January 1, 1970. With the Kojo procedure `setAnimationDelay(0)` there is no delay when the turtle draws and the drawing happens as fast as possible. Enter the following program in the REPL and explain what happens. *Note:* hold down `ESC` while pressing `ENTER` to create a line break without running the program.

```
clear()
animationDelay(0)
val n = 800 * 4
val t1 = System.currentTimeMillis
repeat(n):
  repeat(4):
    forward
    right
val t2 = System.currentTimeMillis
println(s"$n square rounds took ${t2 - t1} milliseconds")
```

✎ b) Note approximately how many square rounds per second the turtle can draw at its fastest. Run several times because the virtual machine needs to "warm up" for the machine code to be optimized. Some runs may be slower if the garbage collector needs to spend time freeing memory.

✎ c) What are the names of the variables in the code above? What values do the variables have?

d) Draw a square again, but now using a **`while`** statement and a loop variable.

```
slow(100)
var i = 0
while ??? do
  forward
  right
  i = ???
```

✎ e) What is the difference between variables declared with **`val`** and variables declared with **`var`**?

f) Draw a square again, but now using a **`for`** statement. Print the value of the local variable `i` in each loop iteration.

```
for i <- 1 to ??? do
  ???
```

✎ g) Is it possible to assign a new value to the variable `i` inside the loop?

✎ h) Is it possible to refer to the name `i` outside the loop?

i) Draw a square again, but now using `foreach`. Print the loop variable's value in each iteration.

```
(1 to ???).foreach: i =>
  ???
```

```
def square =
  for ??? do
    ???

for ??? do
  ???
```

Figure 1.1: A column of squares.

Task 4. *Abstraction*.

a) Use a repetition to abstract the sequence below so that the program becomes shorter:

```
forward; right; jump; forward; left; jump; forward; right;
jump; forward; left; jump; forward; right; jump; forward;
left; jump; forward; right; jump; forward; left; jump;
forward; right; jump; forward; left; jump
```

b) Define your own procedure named `square` using the keyword **`def`** which, when called, draws a square using a **`for`**-loop.

```
def square =
  for ??? do
    ???
```

c) Call your abstraction after it has been declared and after you have executed: `setAnimationDelay(100)`

d) Call your abstraction inside a **`for`**-loop so that the turtle draws a column consisting of 10 squares as shown in the image below.

e) Examine how calling the procedure `square` affects the execution sequence of your statements by making appropriate printouts so you can see when different parts of the code are executed. At which points in the program does a "jump" in the sequence occur instead of the following statement being executed? Use an appropriate argument to `setAnimationDelay` so that you have time to study the execution.

f) Draw the same picture with 10 stacked squares (see figure 1.1 on page 56), but now *without* using the abstraction `square` – instead use a nested repetition (i.e., a repetition inside a repetition). Which of the two ways (with and without the abstraction `square`) is easiest to read?

g) Generalize your abstraction `square` by giving it a parameter `side: Double` that specifies the square's size. Draw several squares as in figure 1.2 on page 57.

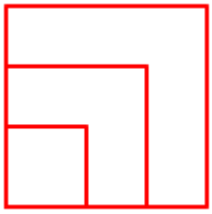


Figure 1.2: Squares of different sizes.

Task 5. *Alternative.*

a) Run the program below. Explain what happens.

```
setAnimationDelay(5000)

def move(key: Int): Unit =
  println("key: " + key)
  if key == 87 then forward(10)
  else if key == 83 then forward(-10)

move(87); move('W'); move('W')
move(83); move('S'); move('S'); move('S')
```

b) Run the program below. Note the `activateCanvas()` call so you don't need to click inside the drawing window before you can control the turtle. The call `onKeyPress(move)` makes `move` be called when a key is pressed. Add code in `move` so that the key A causes a 5-degree left rotation while the key D causes a 5-degree right rotation. With `onKeyPress` you define which procedure should run upon keypress.

```
setAnimationDelay(0)
activateCanvas()

def move(key: Int): Unit =
  println("key: " + key)
  if key == 'W' then forward(10)
  else if key == 'S' then forward(-10)

onKeyPress(move)
```

1.3.2 Checkpoint questions

✓ Review the theory for this week and be ready to answer these questions when it is your turn to present what you have done during the lab:

1. What does sequential execution of statements mean?
2. What is the difference between a statement and an expression?
3. What is the difference between a procedure and a function?

4. Does the order of arguments matter when calling a function with multiple parameters?
5. What is a variable? Give examples of declaration, initialization, and assignment of variables, as well as usage of variables in expressions.
6. What is a logical expression? Give examples of usage of logical expressions.
7. What is abstraction? Give examples of use of abstraction.
8. What is the benefit of abstraction?
9. How is a variable declared and initialized whose value is mutable?
10. How is a variable declared and initialized whose value is immutable?
11. Is it a runtime error or a compile-time error to assign a new value to an immutable variable?
12. Indicate which of `for` and `while` is most appropriate in these cases:
    A. Sum the first hundred integers.
    B. Count the number of characters in a string until the first space character.
    C. Draw 100 random numbers between 1 and 6 and sum those numbers less than 3.
    D. Sum the integers starting from 1 upwards until the sum is at least 100.

1.3.3 Optional extra exercises

Do the following tasks in any order if you wish and depending on your exercise needs.

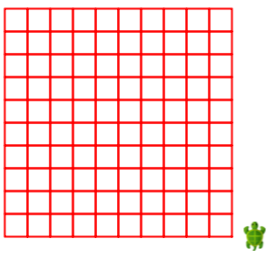
Task 6. *Abstraction and generalization.*

a) Create an abstraction `def stapel = ???` that uses your abstraction `square`.

b) Now you will *generalize* your procedure so that it does not just draw exactly 10 squares in a column. Give the procedure `stapel` a parameter `n` which controls how many squares are drawn.

```
def square = ???
def stapel(n: Int) = ???

animationDelay(100)
stapel(42)
```

c) Draw the image below using the abstraction `stapel`. There are a total of 100 squares and each square has side length 25. *Tip:* With a negative argument to the procedure `hop` you can make the turtle jump backwards without drawing, e.g. `hop(-10*25)`



d) Generalize your abstractions `square` and `stapel` so that you can control the size of the squares that are drawn.

e) Create an abstraction `grid` with suitable parameters so that one can draw grids with varying square sizes and numbers of squares in both horizontal and vertical directions.

f) Generalize your abstractions `square` and `stapel` so that one can influence the fill color and pen color for the squares that are drawn.
The colors in Kojo are of type `java.awt.Color`. The type is accessible under the name `Color` because the name is made directly available with **`export`** `java.awt.Color` in the file `kojo.scala` (more about the keywords **`export`** and **`import`** in reading week 4).

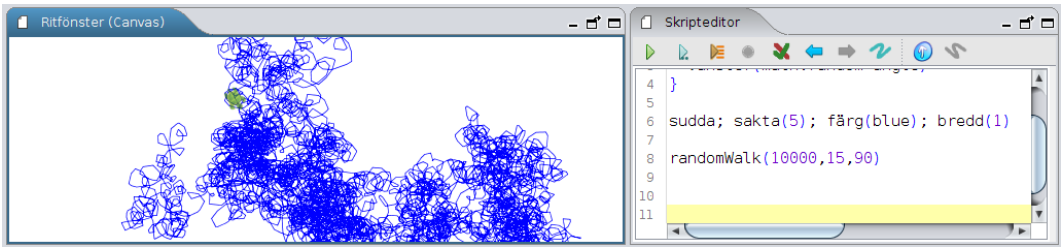
Task 7. *Toggling with Boolean values.*

a) Build upon the program in task 5 and add the code below at the beginning of the program. Also add code so that when pressing the G key, the grid is toggled on and off alternately. Note that exactly *one* procedure is called at `onKeyPress`.

```
var isGridOn = false

def toggleGrid =
  if isGridOn then
    gridOff
    isGridOn = false
  else
    gridOn
    isGridOn = true
```

b) Make it so that when pressing the X key, the coordinate axes are toggled on and off alternately. Use a variable `isAxesOn` and define an abstraction `toggleAxes` which calls `axesOn` and `axesOff` similarly to the previous task.

Task 8. *Repetition.* Write a procedure `randomWalk` with this signature:
`def randomWalk(n: Int, maxStep: Int, maxAngle: Int): Unit`
which makes the turtle take `n` steps of random length between `0` and `maxStep`, and after each step turn left by a random angle between `0` and `maxAngle`. Call your procedure with different arguments and examine how the values affect the image's appearance.
Tip: The expression `math.random() * 100` gives a number from 0 to (almost) 100. You can control how slowly the turtle draws by calling `setAnimationDelay(???)` (try to find an appropriate integer argument instead of `???`).



Task 9. *Variables, naming and formatting.*

a) Paste the strangely formatted program below *exactly* as it is with whitespaces, indentations, and line breaks. Run the program and explain what happens.

```
// A strangely formatted program with some strange names.

def cucumber(x: Double, y: Double, name: String, type: String, value:String) = {
val tomato = 15
val h = 30
setPosition(x,y)
north
write(name+": "+type)
setPosition(x+tomato*(name.size+type.size),y)
write(value); south; front(h); left
front(tomato * value.size); left
front(h); left
front(tomato* value.size); left }
erase; color(black);
val s = 130
val h = 40
var x = 42; cucumber(10, s-h*0, "x","Int", x.toString)
var y = x; cucumber(10, s-h*1, "y","Int", y.toString)
x = x + 1; cucumber(10, s-h*2, "x","Int", x.toString)
cucumber(10, s-h*3, "y","Int", y.toString); invisible
```

b) Write down the names of all variables that appear in the program. ✎

c) Which of these variables are local? ✎

d) Which of these variables can change after initialization? ✎

e) Suggest three changes to the program above (for example, renaming) that make it easier to read and understand. ✎

f) Do search-and-replace of `cucumber` to a better name.

g) Do automatic formatting of the code using an appropriate shortcut key. Note the differences. Which autoformatting changes make the program easier to read? Which manual formatting changes do you think should be done to increase readability? Give the function `cucumber` a better name. Discuss the readability with a supervisor.

Task 10. *Timing.* How fast is your computer?

a) Enter the code below in the REPL and run repeatedly by typing `main`. Does it take the same time every time? Why?

```
object timer:
  def now: Long = System.currentTimeMillis
  var saved: Long = now
  def elapsedMillis: Long = now - saved
  def elapsedSeconds: Double = elapsedMillis / 1000.0
  def reset: Unit =
    saved = now

// MAIN PROGRAM:
def main =
  timer.reset
  var i = 0L
```

```
  while i < 1e8.toLong do
    i += 1
  val t = timer.elapsedSeconds
  println("Counted to " + i + " in " + t + " seconds.")
```

b) Change the loop in task a so that it counts to 4.4 billion. How long does it take your computer to count that far?[8]

c) If you run on a Linux machine: Run the Linux command below repeatedly in a terminal window. At what MHz is your CPU clock running currently? How does the clock frequency compare with the number of iterations in the while loop in the previous task? (It may happen that your computer can vary the CPU clock frequency. Try both while you run the timing in Kojo and while your computer is "idle". What is the benefit of a processor being able to vary its clock frequency?)

```
> lscpu | grep MHz
```

d) Change the code in task a so that the **while** loop runs only 5 times.

e) Add the code below to your program and try to find out approximately how far your computer can count in one second for `Long` and `Int` variables respectively. Write `calculateFast` in the terminal to run the main program.

```
def timeLong(n: Long): Double =
  timer.reset
  var i = 0L
  while i < n do
    i += 1
  end while
  timer.elapsedSeconds

def timeInt(n: Int): Double =
  timer.reset
  var i = 0
  while i < n do
    i += 1
  end while
  timer.elapsedSeconds

def show(msg: String, sec: Double): Unit =
  print(msg + ": ")
  println(sec + " seconds")

def report(n: Long): Unit =
  show("Long " + n, timeLong(n))
  if n <= Int.MaxValue then
    show("Int  " + n, timeInt(n.toInt))

// MAIN PROGRAM, measurements:
def calculateFast =
  report(Int.MaxValue)
  for i <- 1 to 10 do
    report(4.26e9.toLong)
```

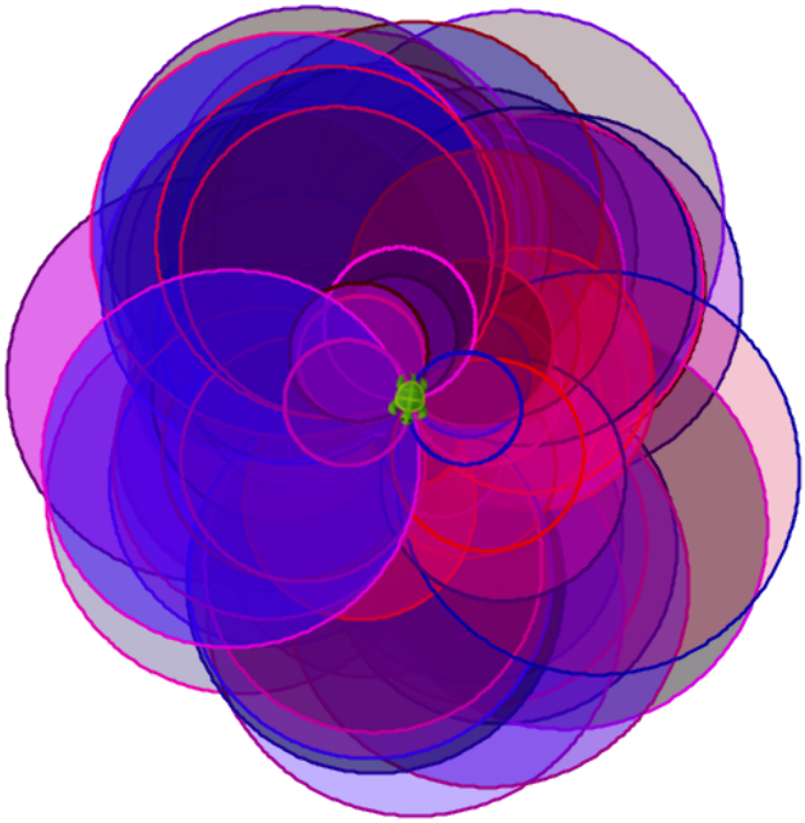
[8] It is possible to do roughly one integer addition per clock cycle per core. The first electronic computer ENIAC had a clock frequency corresponding to 5 kHz. The computer on which this exercise was created had an i7-4790K turbo clocked up to 4.4 GHz.

f) How much faster is it to count with `Int` variables compared to `Long` variables? Discuss your answer with a supervisor if you wish.

Task 11. Play with color in Kojo. Search the internet for documentation of the class `java.awt.Color` and study which integer parameters the last constructor in the list of constructors takes to create sRGB colors. If you write `selectColor()` in the REPL, a new window opens where you can pick a color. If you then press OK or the ENTER key, the chosen color will become a value in the REPL. Try this in the REPL:

```
scala> val chosenColor = selectColor() // choose a color
val chosenColor: java.awt.Color = java.awt.Color[r=0,g=255,b=226]

scala> val c = new java.awt.Color(124,10,78,100)
val c: java.awt.Color = java.awt.Color[r=124,g=10,b=78]

scala> c.  // press TAB
asInstanceOf    getColorComponents      getRGBComponents
brighter        getColorSpace           getRed
createContext   getComponents           getTransparency
darker          getGreen                isInstanceOf
getAlpha        getRGB                  toString
getBlue         getRGBColorComponents

scala> c.getAlpha
val res3: Int = 100
```

Write a program that draws many figures with different colors, for example circles as below. If you use the alpha channel, the colors become transparent.



Task 12. Download ”Missions with Kojo” from lth.se/programmera/uppdrag and do some exercises that you find interesting.

Chapter 2

Programs and Control Structures

Concepts included in this week's studies:

- ☐ main program
- ☐ program argument
- ☐ input
- ☐ scala.io.StdIn.readLine
- ☐ control structure
- ☐ iterate over elements in collection
- ☐ for-expression
- ☐ yield
- ☐ map
- ☐ foreach
- ☐ collection
- ☐ sequence
- ☐ indexing
- ☐ Array
- ☐ Vector
- ☐ range
- ☐ Range
- ☐ algorithm
- ☐ implementation
- ☐ pseudocode
- ☐ algorithm examples: SWAP
- ☐ SUM
- ☐ MIN-MAX
- ☐ MIN-INDEX
- ☐ dod: version control

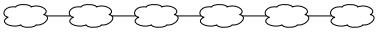
2.1 Theory

A program contains statements and expressions. A **control structure**, e.g. `while`, controls in which **order** statements and expressions are executed. Data can be placed in a **data structure**, e.g. a `Vector`, so that one can access the data again later.

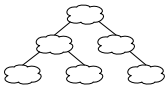
2.1.1 What is a data structure?

- A data structure is a structure for organizing data that...
  - can contain **many** elements,
  - can be **referred to** as a **whole**, and
  - provides the possibility to **access individual elements**.
- A **collection** is a data structure that can contain many elements of **the same type**.
- Examples of different collections where the elements are organized differently:

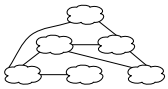
Sequence



Tree



Graph



More about sequences & trees in pfk. More about trees, graphs in Discrete structures.

2.1.2 Some collections in `scala.collection`

- A **collection** is a data structure that can contain many elements of **the same type**.
- A **sequence** is a collection where all elements are ordered.
- Examples of **built-in collections** in Scala's standard library where elements are internally organized in **different** ways so that the collection gets different properties suited for **different use cases**:
  - `scala.collection.immutable.Vector`, sequence with fast access **everywhere**.
  - `scala.collection.immutable.List`, sequence with fast access **at the beginning**.
  - `scala.collection.immutable.Set`, `scala.collection.mutable.Set`, set with unique elements; not ordered but fast membership test.
  - `scala.collection.immutable.Map`, `scala.collection.mutable.Map`, set with key & associated value pairs, fast access via key.
  - `scala.collection.mutable.ArrayBuffer`, mutable sequence that can change size.
  - `scala.Array`, mutable sequence that **cannot** change size. All elements are stored consecutively in memory: fastest access of all collections, but has special limitations.

2.1.3 Different structures for handling data

- **Tuple**
  - gather several data values e.g. `(1, "hej", true)` in elements `_1`, `_2`, `_3`
  - elements can be of **different** types
- **Enumeration** (also called *enumeration*)
  - Name enumerated values e.g. `enum Color { case Red, Black }`
  - The values have ordinal numbers and are all of **the same** type (here `Color`)
- **Class**
  - collects data in **attributes** with (well chosen!) names
  - attributes can be of **different** types
  - also defines **methods** that use the attributes
    (also called **operations** on data)
- **Built-in collection**
  - special classes that collect data in elements of **the same** type
  - example: `scala.collection.immutable.Vector`
  - often has *many* built-in **convenience methods**,
    see quick reference https://fileadmin.cs.lth.se/pgk/quickref.pdf
- **User-implemented collections** → advanced course pfk

2.1.4 What is a vector?

A **vector**[1] (eng. *vector*) is a **sequence** that is **fast** to **index**. Access to elements in a sequence named e.g. `xs` occurs in Scala with `xs.apply(index)`:

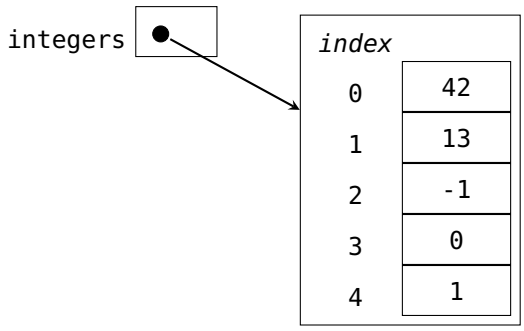
```
scala> val integers = Vector(42, 13, -1, 0, 1)
val integers: scala.collection.immutable.Vector[Int] = Vector(42, 13, -1, 0, 1)

scala> integers.apply(0)   // index counted from zero
val res0: Int = 42

scala> integers(1)         // in Scala one can omit .apply before (
val res1: Int = 13

scala> integers(5)         // runtime error since sixth position does not exist
java.lang.IndexOutOfBoundsException: 5
  at scala.collection.immutable.Vector.checkRangeConvert(Vector.scala:132)
```

If you omit `.apply` the compiler automatically inserts a call to `apply`.

[1] Vector is sometimes called a field in Swedish, but this causes much confusion since the English word *field* often is used for *attributes* (explained later).

2.1.5 A conceptual picture of a vector

```
scala> val integers = Vector(42, 13, -1, 0, 1)

scala> integers(0)
val res0: Int = 42
```



2.1.6 A collection of strings

- A vector can store **many** values of the same type.
- The elements can be for example integers or strings.
- Or actually anything. (A so-called *generic* collection.)

```
scala> val vegetables = Vector("cucumber","tomato","pepper","celery")
vegetables: scala.collection.immutable.Vector[String] =
  Vector(cucumber, tomato, pepper, celery)

scala> val v = vegetables(1)
val v: String = tomato

scala> val xs = Vector(42, "cucumber", true, 42.0)
val xs: Vector[Matchable] = Vector(42, cucumber, true, 42.0)
```

Note the type `Matchable` which means "**almost any type**"
(More on `Matchable` later.)

2.1.7 What is a control structure?

- A **control structure** affects in which order (sequence) statements are executed and expressions evaluated.

  Examples of **built-in** control structures:
  `if`–`then`–`else`–expression

  `for`–`yield`–expression
  `for`–`do`–statement
  `while`–`do`–statement

- In Scala one can define **custom** control structures.

Example: `repeat` which you used in Kojo
`repeat(4){forward; right}`

2.1.8 Loop through the elements in a vector

A **for-do-statement** that prints all elements in a vector:

```
scala> val vegetables = Vector("cucumber","tomato","pepper","celery")

scala> for v <- vegetables do println(v)
cucumber
tomato
pepper
celery
```

for ... **do** ... does the following:

- Extract **each element** from the collection.
- **The name** before the arrow (here v) **refers** to a **new** value for each loop iteration.
- This name corresponds to a **local val**-variable.

2.1.9 Build a new collection from existing with for-yield expression

A **for-yield-expression** that **creates a new collection**.

```
for v <- vegetables yield s"good $v"
```

```
scala> val vegetables = Vector("cucumber","tomato","pepper","celery")

scala> val opinions = for v <- vegetables yield s"good $v"
val opinions: Vector[String] =
  Vector(good cucumber, good tomato, good pepper, good celery)
```

2.1.10 The collection `Range` keeps track of intervals

- With a `Range(start, end)` you can create an **interval**:
  from and including `start` up to (but not including) **end**

```
scala> Range(0, 42)
val res0: Range =
  Range(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
    15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28,
    29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41)
```

- But all values in between are not created until they are needed:

```
scala> val hugeRange = Range(0, Int.MaxValue)
val hugeRange: Range.Exclusive = Range 0 until 2147483647

scala> hugeRange.end
val res1: Int = 2147483647

scala> hugeRange.toVector
java.lang.OutOfMemoryError: Java heap space
```

2.1.11 Loop with Range

Range is used in for-loops to keep track of the number of iterations.

```
scala> for i <- Range(0, 6) do print(s" cucumber $i")
 cucumber 0 cucumber 1 cucumber 2 cucumber 3 cucumber 4 cucumber 5
```

You can create a Range with `until` after an integer:

```
scala> 1 until 7
val res1: Range =
  Range(1, 2, 3, 4, 5, 6)

scala> for i <- 1 until 7 do print(s" tomato $i")
 tomato 1 tomato 2 tomato 3 tomato 4 tomato 5 tomato 6
```

With the method `indices` you can get a Range with all indices:

```
scala> val xs = Vector("cucumber1","cucumber2","tomato1")
val xs: Vector[String] = Vector(cucumber1, cucumber2, tomato1)

scala> xs.indices
val res0: Range = Range 0 until 3
```

2.1.12 Loop with Range created with `to`

With `to` after an integer you get a Range up to and **including** the last:

```
scala> 1 to 6
res2: Range.Inclusive =
  Range(1, 2, 3, 4, 5, 6)

scala> for i <- 1 to 6 do print(" cucumber " + i)
 cucumber 1 cucumber 2 cucumber 3 cucumber 4 cucumber 5 cucumber 6
```

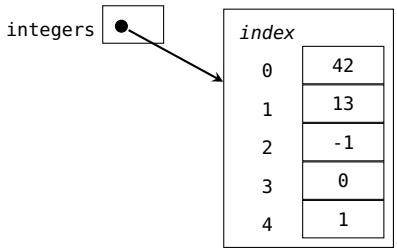
2.1.13 Loop through a collection with a `while`-statement

```
scala> val xs = Vector("Hi","to","you","!!!")
val xs: Vector[String] =
  Vector(Hi, to, you, !!!)

scala> xs.size
val res0: Int = 4

scala> var i = 0
val i: Int = 0

scala> while i < xs.size do { println(xs(i)); i = i + 1 }
Hi
to
you
!!!
```

2.1.14 What is an `Array`?

- An `Array` resembles a `Vector` but has a special status in the JVM:
  - Stored as a sequence in memory at consecutive addresses.
  - **Advantage**: fastest collection for element access in JVM.
  - But there are several **disadvantages** which we will see later.

```
scala> val integers = Array(42, 13, -1, 0 , 1)
```



2.1.15 Some similarities & differences between `Vector` and `Array`

```
scala> val xs = Vector(1,2,3)
```

```
scala> val xs = Array(1,2,3)
```

Some similarities between `Vector` and `Array`

- Both are collections that can contain many elements.
- With both you can quickly access any element: `xs(2)`

Some important differences:

Vector

- Is **immutable**: you can trust that element references never ever change.
- Is **fast at creating a partially changed copy**, e.g. addition/removal/update in the middle of the sequence.

Array

- Is **mutable**: `xs(2) = 42`
- Is **fast** if you only want to read or write existing positions.
- Can't **change size**: addition or removal in the middle requires **slow** copying of the rest.

2.1.16 A minimal standalone program in Scala

Save the below Scala code in the file `hej.scala`:

```
@main def run = println("Hej Scala!")
```

Compile and run in the terminal:

```
> scala run hej.scala
Compiling project (Scala 3.7.2, JVM (21))
Compiled project (Scala 3.7.2, JVM (21))
Hej Scala!
```

Before running your code files are automatically compiled if needed. You can see the machine code in a subdirectory under the directory `.scala-build`:

```
> ls .scala-build/*/classes/main
'hej$package.class'  'hej$package$.class'  'hej$package.tasty'   run.class   ru
```

2.1.17 Type-safe arguments to a program with @main

Write and save the code below in the file `helloargs2.scala`

```
@main def hej(integer: Int, rest: String*): Unit =  // note the * after String
  for i <- 0 until integer do println(rest(i))
```

Compile and run with program arguments after `--`

```
> scala run helloargs2.scala -- 2 carrot cucumber tomato
carrot
cucumber
> scala run helloargs2.scala -- aj carrot cucumber tomato
Illegal command line: java.lang.NumberFormatException: For input string: "aj"
```

With `@main` a primitive main is automatically generated that checks that arguments have the correct type.

2.1.18 What is an algorithm?

An algorithm is a sequence of instructions that describe how to solve a problem.

Examples:

- bake a cake
- calculate your pension prognosis
- drive a car
- check if a highscore in a game

2.1.19 Algorithm example: N-FACTORIAL

```
Input  : integer n
Output : the product of the first n positive integers

prod ← 1
i ← 2
while i ≤ n do
    prod ← prod * i
    i ← i + 1
end
prod
```

- What happens if n is zero?
- What happens if n is one?
- What happens if n is two?
- What happens if n is three?

2.1.20 Algorithm example: MIN

```
Input  : Array args with strings all containing integers
Output : smallest integer

min ← the largest integer that can appear
n ← number of integers
i ← 0
while i < n do
    x ← args(i).toInt
    if ( x < min) then
        min ← x
    end
    i ← i + 1
end
min
```

Test with input: `args = Array("2", "42", "1", "2")`

A program is often divided into many different **functions**. A function can have parameters and return a result. If you divide your program into many simple functions with good names, your program becomes easier to read and understand. If a well-tested and bug-free function is used in many places, the chance of bugs can also be reduced.

2.1.21 Template for function definitions

def functionName(parameterDeclarations): returnType = expression

Example:

```
def increase(i: Int): Int = i + 1
```

Return type can be inferred by the compiler:

```
def increase(i: Int) = i + 1
```

But to get help from the compiler it is good to specify return type!
If multiple parameters use commas. If multiple statements use indentation (and optional curly braces).

```
def isHighscore(points: Int, high: Int): Boolean = {
  val highscore: Boolean = points > high
  if highscore then println(":)") else println(":(")
  highscore
}
```

The above function has a **side effect** of printing a smiley.

2.1.22 Better to have many small abstractions each doing one thing

```
def isHighscore(points: Int, high: Int): Boolean = points > high

def printSmiley(isHappy: Boolean): Unit =
  if isHappy then println(":)") else print(":(")
```

```
printSmiley(isHighscore(113,99))
```

- This better `isHighscore` is now a **pure function** that always returns the same answer for the same input parameters and **has no side effects**; these functions are often easier to understand.
- Functions that return a boolean value are called **predicates**.

2.1.23 What is a block?

- A block **encapsulates** several statements/expressions and looks ”from outside” like a single statement/expression.
- A block is created with curly braces (”curly braces”)

```
{ expression1; expression2; ... expressionN }
```

- In Scala (unlike many other languages) a block has a **value** and is thus an **expression**.
- The value is given by the **last expression** in the block.

```
scala> val x = { println(1 + 1); println(2 + 2); 3 + 3 }
2
4
x: Int = 6
```

2.1.24 Names in blocks become local

Visibility rules:

1. Identifiers declared inside a block become **local**.
2. Local names **shadow** names in outer blocks if the same.
3. Names are visible in nested sub-blocks.

```
scala> def a = { val localName = 42; println(localName) }
scala> a
42

scala> println(localName)
1 |println(localName)
  |         ^^^^^^^^^^
  |         Not found: localName

scala> def b = { val x = 42; { val x = 76; println(x) }; println(x) }
scala> def c = { val x = 42; { val b = x + 1; println(b) } }
scala> b  // what happens?
scala> c  // what happens?
```

2.1.25 Parameter and argument

Distinguish parameter and argument!

- A **parameter** is the declared name used **locally** in a function to refer to...
- **argument** which is the value passed **at call time** and bound to the local parameter name.

```
scala> val oneArgument = 42

scala> def increase(myParameter: Int) = myParameter + 1

scala> increase(oneArgument)
```

Special syntax: call with so-called **named argument**

```
scala> increase(myParameter = oneArgument)
```

Named arguments can be given in any order; then you avoid wrong order errors.

2.1.26 Procedures

- A **procedure** is a function that **does** something interesting, but **does not** return an interesting value.
- Example of an existing procedure: `println("hej")`
- You **declare your own procedures** by specifying **`Unit`** as the return type. Then the value **`()`** is given which means "nothing".

```
scala> def hej(x: String): Unit = println(s"Hello to you $x!")

scala> hej("Mr Cucumber")
Hello to you Mr Cucumber!

scala> val x = hej("Mrs Tomato")
Hello to you Mrs Tomato!

scala> :type x
Unit

scala> println(x)    // what happens?
```

- What is **done** is called (side) **effect**. Above, printing is the effect itself.
- Functions can also have side effects. They are then called **impure** functions.

2.1.27 "Nothing" *is* actually something in Scala

- In many languages (Java, C, C++, ...) functions that lack values are special. Java etc. have special syntax for procedures with the keyword **`void`**, but Scala **does not**.
- In Scala procedures are not special cases; they are ordinary functions that return a value representing nothing, namely () which is of type Unit.
- In this way procedures are no exceptions but follow the usual syntax and semantics just like all other functions.
- This is typical for Scala: generalize concepts and we avoid bothersome exceptions! (But we must understand the generalization...) https://en.wikipedia.org/wiki/Void_type https://en.wikipedia.org/wiki/Unit_type

---

2.1.28 Problem solving: decomposition into abstractions that are then combined

- One of the most important principles in programming is **functional decomposition** where **subprograms** in the form of functions and procedures are created to become building blocks that combine into more advanced functions and procedures.
- Through the names that are defined **reusable abstractions** are created that encapsulate what the function does.
- The problem becomes easier to solve with good building blocks.
- Abstractions that compute or do **one single, well-defined thing** are easier to use compared to those that do many, very different things.
- Abstractions with **well-thought-out names** are easier to use compared to cryptic or misleading names.

2.1.29 Example of functional decomposition

Kojo lab gave examples of **functional decomposition** where a number of abstractions are created and reused.

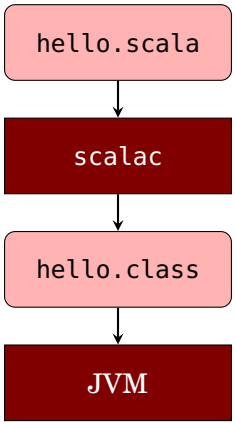
```
// create abstractions that build on each other

def square = repeat(4) { forward(25); right(90) }

def column =
  repeat(10) { square; jump(25) }
  jump(-10 * 25)

def grid = repeat(10) { column; right(90); forward(25); left(90) }

@main def mainProgram =
  clear(); speed(200)
  grid
```

2.1.30 Why abstraction?

- Large programs need to be divided up otherwise it becomes very difficult to understand and develop the program.
- We need to be able to choose names for things in code *locally*, without clashing with the same name in other parts of the code.
- Abstractions help to manage and encapsulate complex parts so that they become easier to use repeatedly.
- Examples of **abstraction mechanisms** in Scala:
  - Classes are "building blocks" with code used to create objects, containing parts that belong together.
    Keywords: `class` and `object`

- Methods are functions that exist in classes/objects and are used to solve specific tasks. Keywords: **def**
- Packages are used to create namespaces and organize machine code in a hierarchical directory structure.
  Keyword: **package**

2.1.31 From source code to machine code with JVM



Source code file

The compiler creates *abstract* machine code (so-called bytecode)

`.class` file with bytecode

Java Virtual Machine
Translates bytecode into *concrete* machine code that suits your specific CPU **during runtime** (so-called interpretation)

2.1.32 Packages

```
package greeting

@main def run = println("Hello world!")
```

- Package (eng. *package*) creates namespaces in a hierarchical structure.
- Packages can be **nested**: often there are packages in packages in packages.
- Packages are especially good if you have a lot of code in many code files.
- The compiler places the machine code in directories according to the package structure.[2]
  If you are curious, check subdirectories in `.scala-build`:

```
ls -R .scala-build
```

---

[2]The directory structure for the source code *must* in many other languages, e.g. Java, *exactly correspond to the package structure*, but this is not necessary in Scala – all Scala code files can be placed in the same top-level directory or in a subdirectory with any name, regardless of how your code uses **package**.

2.1.33 Import

Using dot notation you access content in a package.

```
val age = scala.io.StdIn.readLine("Enter your age:")
```

An **import** statement...

```
import scala.io.StdIn.readLine
```

...makes the name visible **directly**, so you don't have to write the whole path to the name:

```
val age = readLine("Enter your age:")
```

The imported name becomes directly visible in the current **namespace** (eng. *in scope*).

2.1.34 Jar files

- `jar` files resemble `zip` files and are used to bundle many compiled code files in **one compressed file** for easy distribution and execution.
- You use jar files with the option `--jar`

  ```
  scala run . --jar introprog.jar
  ```

- You can create your own jar files with `scala package` where the option `--library` means that only the compiled code is included. Without the option `--library`, the jar file is executable. With the option `--assembly`, everything needed to run the jar file standalone with a double-click or `java -jar myapp.jar` is included.

  ```
  scala package . --library --output myapp.jar
  scala run --jar myapp.jar
  scala package . --assembly --output my-fat-jar-app.jar
  java -jar my-fat-jar-app.jar
  ```

  The `--assembly` option requires power mode according to instructions in the warning.
  Read more about jar files in Appendix F.

2.2 Exercise `Excercise02`

Goals

- ☐ Be able to create, compile, and run a simple application in the terminal.
- ☐ Be able to create the collections Range, Array, and Vector with integers and strings.
- ☐ Be able to index an indexable collection, e.g., Array and Vector.
- ☐ Be able to call the operations size, mkString, sum, min, max on collections containing integers.
- ☐ Be familiar with the differences and similarities between the collections Range, Array, and Vector.
- ☐ Understand the difference between a while statement and a for expression.
- ☐ Be able to create collections with integer values as a result of simple for expressions.
- ☐ Understand the difference between an algorithm in pseudo-code and its implementation.
- ☐ Be able to implement the algorithms SUM, MIN, MAX with an indexable collection and a while loop.

Preparations

- ☐ Study the concepts in chapter 2
- ☐ Familiarize yourself with basic terminal commands, see appendix B.
- ☐ Familiarize yourself with VS Code, see appendix C.
- ☐ Retrieve given code via the course's GitHub repository.

2.2.1 Basic Tasks

Task 1. *Match concepts with descriptions.*

Match each concept with the (simplified) description that fits best:

compile	1	A	used in for-expressions to create a new collection
script	2	B	an immutable, indexable sequence collection
object	3	C	machine code is generated from one or more source files
@main	4	D	a mutable, indexable sequence collection
program argument	5	E	where execution of the compiled program starts
data structure	6	F	data structure with elements in a certain order
collection	7	G	single code file, no main program needed
sequence collection	8	H	step-by-step description of a solution to a problem
Array	9	I	a specific realization of an algorithm
Vector	10	J	many elements in a whole; element-wise access
Range	11	K	data structure with elements of the same type
yield	12	L	can be passed via the args parameter to main
algorithm	13	M	groups variables and functions
implementation	14	N	a collection representing a range of integers

Task 2. *Using the terminal.* Read about the terminal in appendix B.

a) What three commands should you run to 1) create a directory named `hello` and 2) navigate into the directory and 3) display the current directory name? Open a terminal window and run these three commands.

b) What two commands should you run to 1) navigate back "up" one step in the directory tree and 2) list all files and directories at that location? Run these two commands in the terminal.

Task 3. *Create and run a Scala script.*

a) Create a file named `sum.sc` in the `hello` directory you created in the previous task using an editor, e.g. VS `code`.

```
> cd hello
> code sum.sc
```

The file should contain the following lines:

```
val n = 1000
val summa = (1 to n).sum
println(s"The sum of the first $n numbers is: $summa")
```

Save the file and run the command `scala run sum.sc` in the terminal:

```
> scala run sum.sc
```

What is the sum of the first 1000 numbers?

b) Modify the file `sum.sc` so that the right parenthesis on the last line is missing. Save the file (Ctrl+S) and run the script file again in the terminal (up arrow key). What is the error message? Is it a runtime error or a compilation error?

c) Modify `sum.sc` so that instead of `1000` it says `args(0).toInt` after `val n =` and save and rerun your program with the argument 5001 as follows:

```
> scala run sum.sc -- 5001
```

What is the sum of the first 5001 numbers?

d) What error message do you get if you forget to give the script an argument? Is it a runtime error or a compilation error?

Task 4. *Scala application with `@main`.* Create, using an editor, a file named `hello.scala`.

```
> code hello.scala
```

Write the following code in the file:

```
@main def run(): Unit = {
  val message = "Hello world!"
  println(message)
}
```

a) Compile with `scala compile hello.scala`. What are the names of the files the compiler creates? Look for files ending with .class in the directories under the folder starting with project...

```
> scala compile hello.scala
> ls .scala-build/project*/classes/main/
```

b) How should you modify your code so that the compiler gives the following error message:
`Syntax Error: '}' expected, but eof found?`

c) In Scala, curly braces are optional ((eng. *optional braces*)) and code is instead structured in contiguous blocks using indentation[3]. It is fine to switch styles in the same file if you think this makes the code more readable.
The above code can be written as:

```
@main def run(): Unit =
  val message = "Hello world!"
  println(message)
```

What happens if you remove the indentation of the last line?

d) What does the `@main` annotation mean?

Task 5. *Create and use collections.* In Scala's standard library there are many different collections that can be used uniformly (with some exceptions for `Array`). Match the expressions that create or use collections with the explanations, so that all connections are correct (at least one explanation fits more than one expression, but there is only one solution where all connections are pairwise correct):

`val xs = Vector(2)`	1	A	new collection with a zero appended at the end
`val ys = Array.fill(9)(0)`	2	B	new collection, elements converted to integers
`Vector.fill(9)(' ')`	3	C	new reference to a mutable sequence
`xs(0)`	4	D	new collection, elements converted to strings
`xs.apply(0)`	5	E	shortcut for `apply(0)`
`xs :+ 0`	6	F	indexing, returns the first element
`0 +: xs`	7	G	new string with commas between elements
`ys.mkString`	8	H	new collection with a zero prepended at the beginning
`ys.mkString(",")`	9	I	new reference to a sequence of length 1
`xs.map(_.toString)`	10	J	new immutable sequence with blank characters
`xs.map(_.toInt)`	11	K	new string with all elements concatenated

Practice with your own variants in the REPL until you have learned to use expressions like the above by heart. Then you will find it easier to get started with coming labs.

Task 6. *Compare `Array` and `Vector`.* Match each collection type with the description that fits best:

a) What applies regarding mutability?

[3]Optional curly braces and significant indentation came with the new Scala 3. In old Scala 2, curly braces were necessary if multiple statements should be combined and indentation did not affect meaning.

Vector	1	A	mutable
Array	2	B	immutable

b) What applies when adding elements at the beginning ((eng. *prepend*)) and end ((eng. *append*)), or changing a subsequence at an arbitrary position (also called *to patch*)?

Vector	1	A	slow when changing size (requires copying everything)
Array	2	B	creates variants with more/other elements quickly from existing ones

c) What applies regarding equality testing?

Vector	1	A	`xs == ys` is **`true`** if all elements are equal
Array	2	B	unlike other Scala collections, == does not check content equality

Task 7. *Calculate sum, min and max in `args`.* Write a program that prints the sum, min, and max for a sequence of integers in `args`. You can assume the program is only run with integer program parameters. *Tip:* Using expressions `args.sum`, `args.min`, and `args.max` gives the sum, smallest and largest values respectively.
Example of running in terminal:

```
> code sum-min-max.scala
> scala run sum-min-max.scala -- 1 2 42 3 4
52 1 42
```

What error message appears if you give the argument `hej` when an integer is expected?

Task 8. *Algorithm: SWAP.* It is common when working with mutable data structures to need to swap places between elements, and then the SWAP algorithm is needed, as illustrated here by swapping values:
Problem: Swap values of two variables.
Solution idea: Use a temporary variable for storage.

a) Write the SWAP algorithm in *pseudo-code* (step by step in common language) below.
Input: Two integer variables x and y
???
Output: Variables x and y with their values swapped.

b) Implement the SWAP algorithm. Replace `???` below with code that swaps the values in the variables `x` and `y`:

```
scala> var x = 42; var y = 43
scala> ???
scala> println(s"x is $x, y is $y")
x is 43, y is 42
```

Task 9. *Indexing and assignment in Array with SWAP.* Write a program that swaps the first and last elements in the parameter `args`. The swap should only happen if there are at least two elements in `args`. Regardless of whether the change happened or not, `args` should then be printed with spaces between the arguments. *Tip:* You can access the last element with `args(args.length - 1)`
Example run in terminal:

```
> code swap-args.scala
> scala run swap-args.scala -- hej alla barn
barn alla hej
```

Task 10. ***for*** *expressions and* `map` *expressions.* The variable `xs` below refers to the collection
`Vector(1, 2, 3)`. Match the expressions on the left with the correct value on the right.

`for x <- xs yield x * 2`	1	A	`Vector(2, 4, 6)`
`for i <- xs.indices yield i`	2	B	`Vector(1, 2)`
`xs.map(x => x + 1)`	3	C	`Vector(1, 2, 3)`
`for i <- 0 to 1 yield xs(i)`	4	D	`Vector(2, 3, 4)`
`(1 to 3).map(i => i)`	5	E	`Vector(0, 1, 2)`
`(1 until 3).map(i => xs(i))`	6	F	`Vector(2, 3)`

Practice with your own variants in REPL until you have learned to use expressions like above by heart. Then it will be easier to get started with upcoming labs.

Task 11. *Algorithm: SUMBUG* Below is the pseudo-code for SUMBUG.

Input : integer n
Output : sum of the positive integers 1 through n

$sum \leftarrow 0$
$i \leftarrow 1$
while $i \leq n$ **do**
  $sum \leftarrow sum + 1$
end
sum

a) Step through the algorithm with pen and paper, writing down how the values for each variable change. There are two bugs in the algorithm. What are they? Correct the bugs and test again by "running" the algorithm on paper and check it works for $n = 0$, $n = 1$, and $n = 5$. What happens if $n = -1$?

b) Using an editor, create the file `sumn.scala`. Implement the SUM algorithm according to the corrected pseudocode and place the implementation in a `@main` annotated method named `sumn`. You can create input `n` to the algorithm with this declaration at the beginning of your method:
`val n = args(0).toInt`
or directly have `n` as a parameter to the method.
What does the application print if you run it with the argument 8888?

```
scala sumn.scala -- 8888
```

Check that your implementation calculates correctly by comparing the answer with the value of this expression evaluated in Scala REPL:

```
scala> (1 to 8888).sum
```

2.2.2 Extra Tasks; additional practice

Task 12. *Algorithm: MAXBUG* Below is the pseudo-code for MAXBUG.

```
Input  : Array args of strings all containing integers
Output : the largest integer
max ← the smallest integer value possible
n ← number of integers
i ← 0
while i < n do
    x ← args(i).toInt
    if ( x > max) then
        max ← x
    end
end
max
```

a) Run with pen and paper. There is a bug in the above algorithm. What is it? Fix the bug.

b) Implement the MAX algorithm (without bug) as a Scala application. Tips:

- The smallest `Int` value that can ever occur: `Int.MinValue`
- Number of elements in *args* given by: `args.length` or `args.size`

```
> code maxn.scala
> scala maxn.scala -- 7 42 1 -5 9
42
```

c) Rewrite the algorithm so that variable *max* is initialized to the first number in the sequence.

d) Implement the new algorithm variant from task c and test the program. Make sure the program works even if *args* is empty.

Task 13. *Algorithm MIN-INDEX.* Implement the MIN-INDEX algorithm that searches for the index of the smallest integer in a sequence. Pseudocode for the MIN-INDEX algorithm:

```
Input  : Sequence xs with n integers.
Output : Index of the smallest number or −1 if xs is empty.
minPos ← 0
i ← 1
while i < n do
    if xs(i) < xs(minPos) then
        minPos ← i
    end
    i ← i + 1
end
if n > 0 then
    minPos
else
    −1
end
```

a) Try the algorithm with pen and paper on the sequence $(1, 2, -1, 4)$ and draw the memory state after each round in the loop. What difference occurs in the execution flow if the loop variable i is initialized to 0 instead of 1?

b) Implement the MIN-INDEX algorithm in a Scala program with the following function:

```
def indexOfMin(xs: Array[Int]): Int = ???
```

- Let the program have a `main` function that creates a new array of integers from `args` which is sent to `indexOfMin`, and then print the result.
- Test for several cases:
  - empty sequences
  - sequence with only one number
  - long sequence with the minimal number first, somewhere in the middle, and last.

Task 14. *The data structure `Range`.* Evaluate the following expressions in Scala REPL. What are the value and type of each expression?

a) `Range(1, 10)`

b) `Range(1, 10).inclusive`

c) `Range(0, 50, 5)`

d) `Range(0, 50, 5).size`

e) `Range(0, 50, 5).inclusive`

f) `Range(0, 50, 5).inclusive.size`

g) `0.until(10)`

h) `0 until (10)`

i) `0 until 10`

j) `0.to(10)`

k) `0 to 10`

l) `0.until(50).by(5)`

m) `0 to 50 by 5`

n) `(0 to 50 by 5).size`

o) `(1 to 1000).sum`

2.2.3 Advanced Tasks; challenges

Task 15. *Rock-Paper-Scissors game.* Build on the code below to make a Rock-Paper-Scissors game[4]. The code works as it should, except the `winner` function cheats in favor of the computer. Add also a main function so the program can be compiled and run in the terminal. The game becomes more fun if you count the number of wins and losses. You can also make the computer choose not with uniform distribution.

```
object Game:
  val choices = Vector("Rock", "Paper", "Scissors")

  def printChoices(): Unit =
    for i <- 1 to choices.size do println(s"$i: ${choices(i - 1)}")

  def userChoice(): Int =
    printChoices()
    scala.io.StdIn.readLine("What do you choose? [1|2|3]<ENTER>:").toInt - 1

  def computerChoice(): Int = (math.random() * 3).toInt

  /** Should return "You", "Computer", or "None" */
  def winner(user: Int, computer: Int): String = "Computer"

  def play(): Unit =
    val u = userChoice()
    val c = computerChoice()
    println(s"You chose ${choices(u)}")
    println(s"Computer chose ${choices(c)}")
    val w = winner(u, c)
    println(s"$w is the winner!")
    if w == "None" then play()
```

Task 16. *Compare execution time for resizing between `Array` and `Vector`.* ★
Paste the code below into REPL:

```
def time(block: => Unit): Double =
  val t = System.nanoTime
  block
  (System.nanoTime-t)/1e6  // gives milliseconds
```

a) Write code that performs these steps in order:
   1. declare a `val` `as` which is an `Array` filled with one million integer zeros,
   2. declare a `val` `vs` which is a `Vector` filled with one million integer zeros,
   3. run `time(as :+ 0)` 10 times and calculate the average of the timings,
   4. run `time(vs :+ 0)` 10 times and calculate the average of the timings.

b) Which of `Array` and `Vector` is fastest for adding elements? Why is that?

[4] https://en.wikipedia.org/wiki/Rock_paper_scissors

★ **Task 17.** *Memory usage for Range.* The data structure `Range` keeps track of start and end values and step size for an enumeration, but all the numbers in the enumeration are not generated until requested. An `Int` takes 4 bytes in memory. Approximately how much space in memory do the objects the variables (a) `interval` and (b) `sequence` refer to below take?

```
scala> val interval = (1 to Int.MaxValue by 2)
scala> val sequence = interval.toArray
```

Tip: Use expression `BigInt(Int.MaxValue) * 2` in your calculations.

★ **Task 18.** *Examine the generated byte-code.* The compiler generates bytecode, pronounced "byte code", which the virtual machine interprets and translates to machine code while the program runs.
Create a file plusxy.scala with:

```
@main def plusxy(x: Int, y: Int) = x + y
```

Compile the program with

```
scala compile plusxy.scala
```

Navigate with `cd .scala-build/` and further down with `ls` and `cd` as deep as you can until you reach directory `main`. Note what files the compiler has created with `ls`. With the command `javap -v 'plusxy$package$.class'` you can inspect the bytecode directly in the terminal.

```
javap -v 'plusxy$package$.class'
```

a) Find the line `public int plusxy(int, int);` and study the code after `Code:` and try to guess which instruction performs the actual addition.

b) What happens if we add a parameter?
Create a new file `plusxyz.scala`:

```
@main def plusxyz(x: Int, y: Int, z: Int) = x + y + z
```

Compile and then study the bytecode with `javap -v 'plusxyz$package$.class'`. What differs in bytecode between `plusxy` and `plusxyz`?

c) Read about bytecode here: en.wikipedia.org/wiki/Java_bytecode. What does the initial letter in the addition instruction mean?

Chapter 3

Functions and Abstraction

Concepts included in this week's studies:

- ☐ abstraction
- ☐ function
- ☐ parameter
- ☐ argument
- ☐ return type
- ☐ default argument
- ☐ named arguments
- ☐ parameter list
- ☐ function header
- ☐ function body
- ☐ apply function to all elements in a collection
- ☐ curried parameter list
- ☐ define own control structure
- ☐ function value
- ☐ function type
- ☐ pure function
- ☐ step function
- ☐ apply
- ☐ anonymous functions
- ☐ lambda
- ☐ predicate
- ☐ activation record
- ☐ call stack
- ☐ object heap
- ☐ stack trace
- ☐ value call
- ☐ name call
- ☐ curly braces and colon for single parameter
- ☐ recursion
- ☐ scala.util.Random
- ☐ random seed
- ☐ dod: typing

3.1 Theory

3.1.1 What is abstraction?

- **Abstraction** means creating a simplified **model** from concrete details
- We "invent" new **concepts** that give us reusable "building blocks" for our thoughts and communication
- We get an abstract **name** that can be used instead of lots of **concrete details**
- Distinguish between the abstraction's **name** (concept, notion), its **use** (call), and its detailed **description** (definition, implementation)
- **Functions** (which you already know from mathematics) are one of our **most** important abstraction mechanisms

https://sv.wikipedia.org/wiki/Abstraktion https://en.wikipedia.org/wiki/Abstraction

---

3.1.2 Examples of abstraction mechanisms in computer science

We will cover several different, increasingly **powerful** abstraction mechanisms in this course:

- Functions
- Objects
- Classes
- Inheritance
- Generic structures
- Contextual abstractions

These abstraction mechanisms become **extra powerful** if they are **combined**!

3.1.3 Function: declaration and call

`def` functionName(parameterDeclarations): returnType = expression

- A function has a **header** and after = comes its **body**.
- A **named** function is **declared** with the keyword `def`
- A function can have **parameters** declared in its header.
- The **body** should be an **expression** (possibly a block with several expressions).
- **Parameters** are bound to **arguments** at **call**.
- The expression in the function body is **evaluated** at **each call**.
- The value of the expression becomes the function's **return value**.

Example:

```
def increase(a: Int, b: Int): Int = a + b
```

```
scala> increase(42, 1)
val res0: Int = 43
```

3.1.4 Declare functions, overloading

- Overloaded functions in the same namespace:

```
scala> object mathOps:
         def increase(a: Int): Int = a + 1
         def increase(a: Int, b: Int): Int = a + b

scala> mathOps.increase(1)
val res0: Int = 2

scala> mathOps.increase(1, 2)
val res1: Int = 3
```

- Both functions above can coexist! Even though they have **the same name**, they are **different functions**; the compiler can distinguish them by their **different parameter lists**.
- This is called **overloading** of functions.
- Overloading provides **flexibility in usage**; we do not have to invent a new name like `increase2` for 2 parameters.

3.1.5 Functions with default arguments

- We can often achieve the same flexibility as with overloading, but with **a single** function, if we instead use **default arguments**:

```
scala> def inc(a: Int, b: Int = 1) = a + b

scala> inc(42, 2)
val res0: Int = 44

scala> inc(42, 1)
val res1: Int = 43

scala> inc(42)
val res2: Int = 43
```

- If an argument is omitted and the parameter has a default argument declared, that default is applied. The compiler fills in the argument for us, if it is **unambiguous** which parameter is meant.

3.1.6 Functions with named arguments

- By using **named arguments**, you do not have to keep track of the order of parameters, as long as you know the parameter names.
- Named arguments can be **combined** with default arguments.

```
scala> def name(
         firstName: String,
```

```
        lastName: String,
        firstNameFirst: Boolean = true,
        label: String = "Name:"
      ): String =
        if firstNameFirst
        then s"$label $firstName $lastName"
        else s"$label $lastName, $firstName"

scala> name(label = "Name:", lastName = "Coder", firstName = "Kim")
val res0: String = Name: Kim Coder
```

3.1.7 Uniform access

- If a function is **declared with** empty parameter list `()`, it *must* be **called with** empty parameter list. (Exception: Java methods)

```
scala> def emptyParamList() = 42

scala> emptyParamList()
val res1: Int = 42

scala> emptyParamList
1 |emptyParamList
  |^^^^^^^^^^^^^^^^
  |method emptyParamList must be called with () argument
```

- A parameterless function declared **without** `()` must be called **without** `()`.

```
scala> def noParamList = 42
scala> noParamList()
1 |noParamList()
  |^^^^^^^^^^^^^
  |method noParamList does not take parameters
```

- Declaration without `()` enables **uniform access**: the implementation can be changed from `val` to `def` or vice versa, **without** affecting the **usage**.

3.1.8 Call stack and object heap

Memory that holds a program's data is divided into two parts:

- **Call stack**:
  - On the call stack, an **activation record** (eng. *stack frame*[1], *activation record*) for each function call is placed with space for **parameters** and **local variables**.
  - The activation record is **removed** when the *return value* has been delivered.

[1] en.wikipedia.org/wiki/Call_stack

  - The stack **grows** during *nested function calls*, when a function calls another function.

- **Object heap**: In the object heap[2,3] all objects (data) allocated during runtime are stored. The heap is periodically cleaned by the *garbage collector*, and memory no longer in use is freed.

3.1.9 Call stack and activation records

Nested calls cause a growing call stack. At every call, an **activation record** (eng. *activation record*) is allocated in memory for parameters, local variables, and possibly the return value. When the function finishes, the activation record is removed and the stack shrinks.

```
scala> def h(x: Int, y: Int): Unit = { val z = x + y; println(z) }
       def g(a: Int, b: Int): Unit = { val x = 1; h(x + 1, a + b) }
       def f(): Unit = { val n = 5; g(n, 2 * n) }

scala> f()
```

The stack with 3 activation records when f calls g which calls h:

variable	value	Call of...
n	5	f
a	5	g
b	10	
x	1	
x	2	h
y	15	
z	17	

3.1.10 What is a stack trace?

When you look for bugs at runtime errors, it is useful to **carefully study** the **printout of the call stack** (eng. *stack trace*):

```scala
// Program in file bmi.scala

@main
def bmi(heightCm: Int, weightKg: Int) =
  safeDiv(weightKg, heightCm * heightCm)

def safeDiv(numerator: Int, denominator: Int): (Int, String) =
  if denominator == 0 then (numerator / denominator, "")  // do you see the bug?
  else (0, "division by zero")
```

[2] en.wikipedia.org/wiki/Memory_management
[3] Not to be confused with the data structure heap sv.wikipedia.org/wiki/Heap

```
> scala run bmi.scala -- 0 42
Exception in thread "main" java.lang.ArithmeticException: / by zero
        // HERE COMES STACK TRACE due to runtime error - see next slide
```

3.1.11 How to read a stack trace?

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at bmi$package$.safeDiv(bmi.scala:8)
        at bmi$package$.bmi(bmi.scala:5)
        at bmi.main(bmi.scala:3)
```

- A **stack trace** is printed after a crash due to a runtime error.
- Runtime errors are recognized by the word **Exception**.
- First comes a description of the error that caused the crash, here: `java.lang.ArithmeticException: / by zero`
- Then the call stack is shown.
- For each function call: **`class.method(sourcefile:lineNumber)`**
- Main functions are placed in a singleton object in a special package
- Singleton objects in Scala are compiled as Java classes with a dollar sign after the name, because JVM does not have singleton objects.

3.1.12 Local functions

With local functions, subproblems can be solved with nested abstractions.

```
def guessNumber(max: Int, min: Int = 1): Unit =
  def guessed = io.StdIn.readLine(s"Guess the number between $min and $max: ").toInt

  val secret = (math.random() * (max - min) + min).toInt

  def printHintIfNotCorrect(guess: Int): Unit =
    if guess > secret then println(s"$guess is too big :(")
    else if guess < secret then println(s"$guess is too small :(")

  def notCorrect(guess: Int): Boolean =
    printHintIfNotCorrect(guess)
    guess != secret


  def loop: Int = { var i = 1; while notCorrect(guessed) do i += 1; i }

  println(s"You found the number $secret after $loop guesses :)")
```

Local, nested function declarations are unfortunately not allowed in many other languages, e.g., Java.[4]

[4] stackoverflow.com/questions/5388584/does-java-support-inner-local-sub-methods

3.1.13 Functions are genuine values in Scala

- A function is a **genuine value**.
- We can, for example, assign a **function value** to a variable.
- Using **only** the **function name** gives us the function as a **value** (no arguments applied yet):

```
scala> def add(a: Int, b: Int) = a + b

scala> val f = add
val f: (Int, Int) => Int = Lambda7210/0x0000000841e4e040@1ce2db23

scala> f(21, 21)
val res0: Int = 42
```

- A function value has a **type** like any value:
  `f: (Int, Int) => Int`
- A function value does not have a name unlike a function declaration (the variable `f` has a name but not the function itself). It is therefore called an **anonymous** function or **lambda** (more about this soon).

3.1.14 Function values can be arguments

Functions can have functions as parameters:

```
scala> def twice(x: Int, f: Int => Int) = f(f(x))

scala> def increase(x: Int) = x + 1

scala> def decrease(x: Int) = x - 1

scala> twice(42, increase)
val res1: Int = 44

scala> twice(42, decrease)
val res1: Int = 40
```

A function that has function values as input (or output) is called a **higher-order function**.

3.1.15 Apply functions to elements in collections with `map`

```
def increase(x: Int) = x + 1

def decrease(x: Int) = x - 1
```

```
val xs = Vector(1, 2, 3)
```

The method `map` works on all Scala collections and takes **a function as argument** and applies this function to all elements and **creates a new collection** with the results:

```
scala> xs.map(increase)
val res0: ???   // what is the result?

scala> xs.map(decrease)
val res1: ???   // what is the result?
```

3.1.16 Apply functions to elements in collections with `map`

```
def increase(x: Int) = x + 1

def decrease(x: Int) = x - 1

val xs = Vector(1, 2, 3)
```

The method `map` works on all Scala collections and takes **a function as argument** and applies this function to all elements and **creates a new collection** with the results:

```
scala> xs.map(increase)
val res0: scala.collection.immutable.Vector[Int] = Vector(2, 3, 4)

scala> xs.map(decrease)
val res1: scala.collection.immutable.Vector[Int] = Vector(0, 1, 2)
```

The method `map` is a handy and frequently used **higher-order function**.

3.1.17 Pure functions

- A **pure** function is a function that produces a result that **only** depends on its arguments. Exactly as in mathematics.
- A pure (mathematical) function is *referentially transparent*. That means that **each call** *can be replaced* by **the value of** *the function body* where parameters have been substituted with corresponding arguments before evaluation.
- A pure function has **no side effects**, e.g., printing, reading/writing files, or updating variables **visible outside** the function.
- Example:

```
def add(x: Int, y: Int): Int = x + y              // pure function
def rnd(n: Int): Int = (math.random() * n).toInt  // impure function
```

  - The expression `add(41, 1)` can be replaced with 41 + 1 which in turn can be replaced with 42 without affecting the result. The result of `add(41, 1)` is **the same every time** the function is applied with these arguments

- The expression `rnd(42)` **cannot** be replaced by a specific value.
  In other words: *not referentially transparent*.

3.1.18 Examples of impure functions: random numbers

- Functions whose values depend somehow on randomness are **not** pure functions.
- Even if the same arguments are given at repeated application, the result can be different.
- Check the documentation for `scala.util.Random` here:
  https://www.scala-lang.org/api/current/scala/util/Random.html
- You will use the function `Random.nextInt` and a random seed in this week's exercises.

3.1.19 Random seed: get the same random numbers every time

- If you use random numbers, it can be difficult to debug because it becomes **different every time** you run the program and the bug might only occur sometimes.
- Using class `scala.util.Random` you can create **pseudo**-random sequences.
- If you provide a so-called **seed** (eng. *seed*), an integer, as an argument to the constructor when creating an instance of class `scala.util.Random`, you get the same "random" sequence **every time** you run the program.

```
val seed = 42
val rnd = util.Random(seed) // create new random generator with seed 42
val r = rnd.nextInt(6)      // something among integers 0, 1, 2, 3, 4, 5
```

- If you **do not** provide a **seed**, the seed is set to "*a value very likely to be distinct from any other invocation of this constructor*". Then the seed is unknown and it is different each time you run the program.

```
val rnd = util.Random() // DIFFERENT seed at each program run
val r = rnd.nextInt(6)
```

3.1.20 Anonymous functions

- You don't have to give functions names. They can instead be created with **function literals**.[5]
- A function literal has ...
  1. a parameter list (without function name, without return type),

[5] Also called "lambda value" or just "lambda" after the so-called lambda calculus. en.wikipedia.org/wiki/Anonymous_function

2. then the reserved symbol combination **=>**
3. and then an expression (or a block).

- Example:

```
(x: Int, y: Int) => x + y
```

What type does this function literal have? `(Int, Int) => Int`

- If the compiler can infer the types from the context, the parameter types do not need to be specified in the function literal itself:

```
val f: (Int, Int) => Int = (x, y) => x + y
```

3.1.21 Apply anonymous functions to elements in collections

Anonymous function created with function literal directly in the call:

```
scala> val xs = Vector(1, 2, 3)

scala> xs.map((x: Int) => x + 1)
res0: scala.collection.immutable.Vector[Int] = Vector(2, 3, 4)
```

Since the compiler here can infer the type `Int` it is not needed:

```
scala> xs.map(x => x + 1)
res1: scala.collection.immutable.Vector[Int] = Vector(2, 3, 4)
```

If you only use the parameter once in the function, you can replace the parameter name with an underscore.

```
scala> xs.map(_ + 1)
res2: scala.collection.immutable.Vector[Int] = Vector(2, 3, 4)
```

3.1.22 Placeholder syntax for anonymous functions

Underscores in function literals are called **placeholders** (eng. *placeholder*) and allow a shortened syntax **if** the parameter represented by the underscore is used **only once**.

```
_ + 1
```

The above is expanded by the compiler to the following function literal (where the parameter name is arbitrary):

```
x => x + 1
```

There may be several underscores; the first corresponds to the first parameter, the second to the second parameter, etc.

```
_ + _
```

... expands to:

```
(x, y) => x + y
```

3.1.23 Example of placeholder syntax with `reduceLeft`

The method `reduceLeft` applies a function to the first two elements of a sequence and then takes the result as the first argument and the next element as the second argument, repeating this through the entire collection.

```
scala> def sum(x: Int, y: Int) = x + y

scala> val xs = Vector(1, 2, 3, 4, 5)

scala> xs.reduceLeft(sum)
res20: Int = 15

scala> xs.reduceLeft((x, y) => x + y)
res21: Int = 15

scala> xs.reduceLeft(_ + _)
res22: Int = 15

scala> xs.reduceLeft(_ * _)
res23: Int = 120
```

3.1.24 Predicate, with and without name

- A function that has `Boolean` as a return type is called a **predicate**.
- Example:

```
def isTooLong(name: String): Boolean = name.length > 10

def isTall(heightInMeters: Double, limit: Double = 1.78): Boolean =
  heightInMeters > limit
```

- Predicates are often given names starting with `is` or `has` so that it is easy to see that it is a predicate when reading code calling the function.
- Many of the collection methods in Scala's standard library take predicates as function arguments. Example with predicate as anonymous function:

```
scala> val parts = Vector(3, 1, 0, 5).partition(_ > 1)
val parts: (Vector[Int], Vector[Int]) =
  (Vector(3, 5),Vector(1, 0))
```

- Study the quick reference and try to find collection methods that take predicates as function arguments. https://fileadmin.cs.lth.se/pgk/quickref.pdf
  In example calls with predicate arguments letter `p` is used.

3.1.25 Function value with empty parameter list: use a "thunk"

- We have seen earlier:
  If you want the function as a value – **write only the name** without the parameter list:

```
scala> val f = add     // add not called
val f: (Int, Int) => Int = Lambda7210/0x0000000841e4e040@1ce2db23
```

- BUT: For **empty parameter lists** you need an anonymous function that **delays the call** to get the corresponding function value:

```
scala> def a() = 42

scala> val b = a
1 |val b = a
  |        ^
  |        method a must be called with () argument

scala> val b = () => a()   // anonymous function, delayed evaluation
val b: () => Int = Lambda7214/0x0000000841e50440@565d794
```

- Note the type: `() => Int` Such a function value is called a **thunk**
  https://en.wikipedia.org/wiki/Thunk
- Suggested Swedish name: **tånk** (since thunk is a pun on *think*)

3.1.26 How does `repeat` in Kojo really work?

```
repeat(10) {
  println("hello")
}
```

We will now see how, by combining several concepts, we can create our own control structures like `repeat` above:

- braces with single parameter
- multiple parameter lists
- named call (delayed evaluation)

3.1.27 Multiple parameter lists

We have earlier seen that you can have more than one parameter:

```
scala> def add(a: Int, b: Int) = a + b

scala> add(21, 21)
res0: Int = 42
```

You can also have **more than one parameter list**:

```
scala> def add(a: Int)(b: Int) = a + b

scala> add(21)(21)
res1: Int = 42
```

(eng. *multiple parameter lists*)
docs.scala-lang.org/style/declarations.html#multiple-parameter-lists

---

3.1.28 Call-by-value and call-by-name

What we have seen so far is **call-by-value**: the argument is evaluated **first** before its **value** is *then* applied:

```
scala> def byValue(n: Int): Unit = for i <- 1 to n do print(" " + n)

scala> byValue(21 + 21)
 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42

scala> byValue({print(" hello"); 21 + 21})
 hello 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42
```

But with **=>** before the parameter type, you can get **call-by-name**: the argument is **"spliced in"** instead of the **name** and evaluated **each time** (also called **delayed evaluation**):

```
scala> def byName(n: => Int): Unit = for i <- 1 to n do print(" " + n)

scala> byName({print(" hello"); 21 + 21})
 hello hello 42 hello 42 hello 42 hello 42 hello 42 hello 42 hello 42 hello 42
```

Quiz: Why is "hello" printed one extra time in the beginning? hint: `1 to n`

3.1.29 Braces with single parameter

We have just seen that you can do this:

```
scala> def twice(action: => Unit): Unit = { action; action }

scala> twice( { print("hello"); print("friend ") } )
hellofriend hellofriend
```

It looks a bit clumsy with `({` and `})` or what do you think? But... For all functions `f` it holds that:
it is perfectly OK to replace regular parentheses: `f(expression)`
with braces: `f{expression}`
if the parameter list has **exactly one** parameter.

You can thus omit the outer parentheses for **better readability**:

```
scala> twice { print("hello"); print("friend ") }
```

3.1.30 Create your own control structure

- By **combining multiple parameter lists** with **call-by-name** with **braces with single parameter** we can create our own control structure: `repeat`

```
repeat(42){
  if math.random() < 0.5 then print(" cucumber")
  else print(" tomato")
}
```

How so? For example like this:

```
def repeat(n: Int)(block: => Unit) = for i <- 0 until n do block
```

```
cucumber cucumber cucumber tomato tomato cucumber cucumber cucumber cucum
```

3.1.31 Colon with single parameter

In Scala 3, instead of using braces for a single parameter, you can use a colon to get fewer "braces" (eng. *fewer braces*).

```
repeat(42):
  if math.random() < 0.5
  then print(" cucumber")
  else print(" tomato")
```

This simplified syntax was preceded by lengthy discussions before it was finally accepted.[6]

3.1.32 Curried functions

If a function has multiple parameter lists, you can create **curried functions**, also called **partially applied functions**.

```
scala> def add(x: Int)(y: Int) = x + y

scala> val increase = add(1)
val increase: Int => Int = Lambda7339/0x0000000841eb7040@19c8add7

scala> Vector(1,2,3).map(increase)
val res0: Vector[Int] = Vector(2, 3, 4)

scala> Vector(1,2,3).map(add(2))
val res1: Vector[Int] = Vector(3, 4, 5)
```

---

[6] The curious can read the proposal before the vote here:
https://docs.scala-lang.org/sips/fewer-braces.html

3.1.33 Function with captured variable scope: *closure*

```
def f(x: Int): Int => Int =
  val a = 42 + x
  def g(y: Int): Int = y + a
  g
```

Function g **captures** the local variable a in a function object.

```
scala> val funkis = f(1)
val funkis: Int => Int = Lambda7356/0x0000000841ed2840@1bda26bc

scala> funkis(2)
val res0: Int = 45
```

A function object with "captured" variables is called a **closure**.
(More about functions as objects later.)

3.1.34 Recursive functions

- Functions that **call themselves** are called **recursive**.

```
scala> def factorial(n: Int): Int =
         if n < 2 then 1 else n * factorial(n - 1)

scala> factorial(5)
val res0: Int = 120
```

- Each new call adds a new activation record on the stack.
- The activation record stores each return value so that `5 * (4 * (3 * (2 * 1)))` can be computed.
- The recursion stops when the **base case** is reached, here `n < 2`
- A recursive function **must** have a return type.

3.1.35 Looping with recursion

```
def guessNumber(max: Int, min: Int = 1): Unit =
  def guessed =
    io.StdIn.readLine(s"Guess the number between [$min, $max]: ").toInt

  val secret = (math.random() * (max - min) + min).toInt

  def printHintIfNotCorrect(guess: Int): Unit =
    if guess > secret then println(s"$guess is too big :(")
    else if (guess < secret) println(s"$guess is too small :(")

  def isCorrect(guess: Int): Boolean =
    printHintIfNotCorrect(guess)
    guess == secret
```

```
def loop(n: Int = 1): Int = if isCorrect(guessed) then n else loop(n + 1)

println(s"You found the number $secret after ${loop()} guesses :)")
```

3.1.36 Recursive data structures

- The data structures List and Tree are examples of data structures well suited for recursion.
- Both data structures can be described recursively:
  - A list consists of a head and a list, which in turn consists of a head and a list, which in turn...
  - A tree consists of branches to trees which in turn consist of branches to trees which in turn...
- These data structures are preferably processed with recursive algorithms.
- In this course recursion is only ”for knowledge”:
  you should know what it is and be able to create a simple recursive function, e.g., factorial calculation. You will work more with recursion and recursive data structures in an advanced course.

3.1.37 Recompile changed files at every save

- The creative programming process includes many short cycles of coding, changing, testing.
- With **each small change**, you want to **recompile** what has changed and see if it still compiles without errors.
- This is done automatically in vscode and you get red underlines for errors.
- But you can also use the terminal:
  `scala compile . --watch`
  Changes are monitored, and you can clearly see any error messages in a compiler output that happens only when you save your changes with Ctrl+S.
- You **avoid** error messages caused by incomplete code...

3.2 Exercise Excercise03

Goals

- ☐ Be able to create and use functions with one or more parameters, default arguments, and named arguments.
- ☐ Be able to explain nested function calls with activation records on the stack.
- ☐ Be able to explain the difference between genuine and "pseudo" functions.
- ☐ Be able to apply a function to all elements in a collection.
- ☐ Be able to use functions as first-class values.
- ☐ Be able to create and use anonymous functions (also known as lambda functions).
- ☐ Be aware that functions can have a split parameter list.
- ☐ Be aware that it is possible to partially apply arguments to functions with split parameter lists to create so-called curried functions.
- ☐ Be aware of recursion and be able to describe the characteristics of a recursive function.
- ☐ Be aware that it is possible to create custom control structures using name calls.
- ☐ Be aware of the difference between call-by-value and call-by-name.
- ☐ Be able to interpret a stack trace.

Preparations

- ☐ Study the concepts in chapter 3

3.2.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with description.* Match each concept with the (simplified) description that fits best:

function header	1	A	describes names and types of parameters
function body	2	B	argument evaluated before call
parameter list	3	C	has parameter list and optionally a return type
block	4	D	delayed evaluation of argument
named arguments	5	E	function without a name, also called lambda
default argument	6	F	applies a function to each element of a collection
value call	7	G	a function returning a boolean value
name call	8	H	arguments can be passed in any order
map	9	I	always returns same result for same arguments
pure function	10	J	list of calls at runtime error
predicate	11	K	a function that calls itself
random seed	12	L	arguments can be omitted
anonymous function	13	M	produces a repeatable pseudo-random sequence
recursive function	14	N	code executed on function call
stack trace	15	O	may have local names; last line gives value

Task 2. *Define and call functions.* A function with one parameter is defined with the following syntax in Scala:

```
def name(parameter: Type = defaultArgument): ReturnType = returnValue
```

a) Define the function `increase` that has an integer parameter `x` and whose return value is the argument plus 1. The default argument should be 1. Specify the return type explicitly.

b) What is the value of the expression `increase(increase(increase(increase())))`?

c) Define the function `reduce` that has an integer parameter `x` and whose return value is the argument minus 1. The default argument should be 1. Specify the return type explicitly.

d) What is the value of the expression `increase(reduce(increase(increase(reduce(reduce()))))))`

e) What is the difference between parameter and argument?

Task 3. *Implement function in different ways.* Create a function that can sum the first `n` positive integers.

a) First write the function header with `???` as the function body. Give the function a good name. Specify the return type. Check that your function compiles without errors before proceeding.

b) Implement the function using a range and the method `sum`. Test that the function works. What happens if you give a negative argument?

c) Implement the function using **`while-do`**. What happens if you give a negative argument?

Task 4. *The text game AliensOnEarth.* Download the game below [7] and study the code.

```
object AliensOnEarth:
  def readChoice(msg: String, options: Vector[String]): String =
    options.indices.foreach(i => println(s"$i: ${options(i)}"))
    val selected = scala.io.StdIn.readLine(msg).toInt
    options(selected)

  def isAnswerYes(msg: String): Boolean =
    scala.io.StdIn.readLine(s"$msg (Y/n)").toLowerCase.startsWith("y")

  def randomChoice(options: Vector[String]): String =
    val selected = scala.util.Random.nextInt(options.size)
    options(selected)

  def playGame(alien: String, maxPoints: Int = 1000): Int =
    val xs = Vector("penguin", "window", "apple")
    val correct = if math.random() < 0.5 then xs(0) else randomChoice(xs)
    val cheatCode = (xs.indexOf(correct) + 1) * math.Pi
    println(s"""|Hello $alien!
                |You are an alien on Earth.
                |Your encrypted password is $cheatCode.
                |You see three strange Earth objects.""".stripMargin)
    val choice = readChoice(s"$alien wants? ", xs)
    if choice == correct then maxPoints else 0

```

[7] https://raw.githubusercontent.com/lunduniversity/introprog/master/compendium/examples/AliensOnEarth.scala

```
  def main(args: Array[String]): Unit =
    try
      val name = if args.size > 0 then args(0) else "Captain Zoom"
      val points = playGame(alien = name)
      if points > 0 then println(s"Congratulations $name! :)")
      println(s"You got $points points.")
    catch case e: Exception =>
      println(s"Game over. The Earth was hit by an asteroid. :(")
      if isAnswerYes("Do you want to trace the asteroid?") then
        e.printStackTrace()
```

a) While reading the code, try to figure out what is the best strategy to get as many points as possible. Compile and run the game in the terminal with your favorite name as argument. Which of the three objects on planet Earth has the highest probability to be the best option?

b) Match the code snippets below with the best description.[8]

`options.indices`	1	A	convert string to lowercase
`"1X2".toLowercase`	2	B	integer sequence with all indices in a sequence
`Random.nextInt(n)`	3	C	random number in range `0 until n`
`try { } catch { }`	4	D	removes margin up to vertical bar
`""" ... """`	5	E	catch exceptions to prevent crash
`s.stripMargin`	6	F	string spanning multiple lines
`e.printStackTrace`	7	G	prints exception information

Tip: Using the REPL you can find out how different parts work, for example:

```
scala> val xs = Vector("p", "w", "a")
scala> xs.indices
scala> xs.indices.foreach(i => println(i))
scala> xs.indexOf("w")
scala> xs.indexOf("gurka")
scala> Vector("hej", "hejsan", "hej").indexOf("hej")
scala> try 1 / 0 catch case e: Exception => println(e)
```

Tips for continuing:

- When I created `AliensOnEarth` I started with a very small program with a simple `main` function that just printed something fun. Then I built the program step-by-step and compiled and tested after every small change.
- When I code, I have REPL running in one terminal window and my code editor in another. In a third window I have a terminal running compilation in *watch mode*, see appendix F.2.1. Ask a supervisor how you can work efficiently with stepwise experimentation in REPL to build larger programs little by little.
- This way of working takes time to get used to, but is a good way to invent larger and better programs. Large programs are easiest built stepwise and debugging is much easier if you just add small parts at a time.
- It also becomes much easier for you to understand your program if you divide the code into many short functions with good names. You can then more easily

[8] Do your best even if not everything is clear. Some things will be covered in detail only in later course modules.

invent more advanced functions by reusing existing ones.

- During this week's lab you will develop your own text game. You will benefit from reusing the input and random choice functions from the example program `AliensOnEarth`.

Task 5. *Pure functions.* A pure function[9] (eng. *pure function*) always gives the same result with the same arguments (as we are used to in mathematics) and has no externally observable side effects (for example printing).
Which of the functions below are pure functions?

```
var x = 0
val y = x

def inc(i: Int) = i + 1

def nöff(i: Int) =
  x = x + i
  "nöff " * x
end nöff

def addX(i: Int) = x + i

def addY(i: Int) = y + i

def isPalindrome(s: String) = s == s.reverse

def rnd(min: Int, max: Int) = math.random() * max + min
```

Tip: Copy and test the functions in REPL one by one so that you understand exactly what happens.

Task 6. *Apply function to each element in a collection. Function as argument.*
Declare the function `increase` and the variable `xs` below in REPL:

```
scala> def increase(x: Int) = x + 1
scala> val xs = Vector(3, 4, 5)
```

Match the expressions below on the left with the expressions on the right that have the same value. If you wonder about something, test the expressions and various variants in REPL.

`for i <- 1 to 3 yield increment(i)`	1	A	`xs`
`Vector(2, 3, 4).map(i => increment(i))`	2	B	`Vector(4, 5, 6)`
`xs.map(increment)`	3	C	`()`
`xs.map(increment).map(increment)`	4	D	`Vector(5, 6, 7)`
`xs.foreach(increment)`	5	E	`Vector(2, 3, 4)`

[9] Pure functions by definition satisfy *referential transparency* (eng. *referential transparency*) which you can read more about here: simple.wikipedia.org/wiki/Referential_transparency

Task 7. *Anonymous functions.* We have several times seen the syntax `i => i + 1`, for example in a loop `(1 to 10).map(i => i + 1)` where the function `i => i + 1` is applied to all integers from 1 through 10 and the result is a new sequence collection. The syntax `(i: Int) => i + 1` is a literal to create a *function value* (also called an *anonymous function* or *lambda expression*). The syntax resembles that for function declarations, but the keyword `def` is missing in the function header and instead of equal sign `=>` separates the parameter list from the function body. If the compiler can infer the type from the context, the short form `i => i + 1` can be used.
There is an *even* shorter way to write an anonymous function *if* the type can be inferred *and* it only uses its parameter *once*; then function literals can be written with so-called *placeholder syntax* using underscores, such as `_ + 1` which the compiler automatically expands to `ngtnamn => ngtnamn + 1` (the name of the parameter does not matter; the compiler chooses some internal one).
Match expressions on the left with expressions on the right that have the same value:

`(0 to 2).map(i => i + 1)`	1	A	`Vector(9.0, 16.0, 25.0)`
`(1 to 3).map(_ + 1)`	2	B	`(2 to 4).map(i => i - 1)`
`(2 to 4).map(math.pow(2, _))`	3	C	`Vector(2.0, 2.5, 3.0)`
`(3 to 5).map(math.pow(_, 2))`	4	D	`Vector(2, 3, 4)`
`(4 to 6).map(_.toDouble).map(_ / 2)`	5	E	`Vector(4.0, 8.0, 16.0)`

Function literals are called *anonymous functions* since they have no name, unlike e.g. `def increase(i: Int): Int = i + 1`, which is named `increase`. Another common name is *lambda expression* after the computer science mathematical tool lambda calculus.

Task 8. *Create your own control structure using call-by-name parameter.* Call-by-name is written with a colon-rocket after the colon before the parameter type and means that the argument is evaluated on the spot each time.

a) Use call-by-name combined with a split parameter list to create your own control structure as below.[10]

```
def repeat(n: Int)(block: => Unit): Unit =
  var i = 0
  while i < n do
    ???
  end while
```

b) Test your control structure in REPL. Let it repeat 100 times rolling a random number between 1 and 6 and then print it out. Also try using fewer braces with help of colon.

c) Why is call-by-name needed here?

Task 9. *Learn to read a stack trace.* Write a program in the file `fel.scala` that causes a *runtime error* and then run it in terminal with `scala run fel.scala`. Study the stack trace printed. What does a `stack trace` contain? Discuss with the supervisor how you can benefit from a stack trace when debugging.

[10] This is how the loop `repeat` in Kojo is defined.

3.2.2 Extra Tasks; additional practice

Task 10. *Function with multiple parameters.*

a) Define in REPL two functions `sum` and `diff` with two integer parameters that return the sum respectively the difference of the arguments:

```
def sum(x: Int, y: Int): Int = ???

def diff(x: Int, y: Int): Int = ???
```

What are the values of the expressions below? Explain what happens.

b) `diff(0, 100)`

c) `diff(100, sum(42, 43))`

d) `sum(sum(42, 43), diff(100, sum(0, 0)))`

e) `sum(diff(Byte.MaxValue, Byte.MinValue), 1)`

Task 11. *Average value.* Write and test a function `avg` that computes the average between two integers and returns a `Double`.

Task 12. *Function calls with named arguments.*

```
1 scala> def skrivNamn(efternamn: String, förnamn: String) =
2          println(s"Namn: $efternamn, $förnamn")
3 scala> skrivNamn(förnamn = "Stina", efternamn = "Triangelsson")
4 scala> skrivNamn(efternamn = "Oval", "Viktor")
```

a) What is printed after line 3 and line 4 above?

b) Name three advantages with named arguments.

Task 13. *Function as a genuine value.* Functions are *genuine values* in Scala. This means variables can have functions as values and function values can be arguments to functions that have function parameters. Functions that take functions as arguments are called *higher-order functions*.

A function that has an integer parameter and returns an integer has the function type `Int => Int` (pronounced *int-to-int*) and the value of the function forms an object which has a method called `apply` with the corresponding function type.

a) Declare the functions and variables below in REPL. Then match the expressions on the left with the expressions on the right that create the same output. If you wonder about something, test the expressions and various variants in REPL.

```
scala> def health(): Unit = println("Hej!")
scala> def multipleCalls(aantal: Int, f: () => Unit): Unit =
         for _ <- 1 to aantal do f()
scala> val f1 = () => health()
scala> var f2 = (s: String) => println(s)
scala> val f3 = () => f2("Thunk")
```

`multipleCalls(1, greet)`	1	A	`f2("Hello!\nHello!")`
`multipleCalls(3, greet)`	2	B	`multipleCalls(3, f1)`
`multipleCalls(2, f1)`	3	C	`f3()`
`multipleCalls(1, f3)`	4	D	`f2("Hello!")`

b) What types do the variables `f1`, `f2` and `f3` have?

c) Does this work? Why? `f2 = f1`

d) Does this work? Why? `val f4 = multipleCalls`

e) Does this work? Why? `val f4 = health`

f) Does this work? Why? `val f4: () => Unit = health`

Task 14. *Discarded result values and return type `Unit`.* Investigate the code below in REPL and explain what happens.

a)

```
scala> def tom = println("")
scala> println(tom)
```

b)

```
scala> def wasted: Unit = 1 + 1
scala> println(wasted)
```

c)

```
scala> def wasted2 = { val x = 1 + 1 }
scala> println(wasted2)
```

d) Why is it good to explicitly specify `Unit` as the return type for procedures?

Task 15. *Call-by-name.*
Declare this procedure in REPL:

```
def doThisTwice(b: => Unit): Unit = { b; b }
```

Call `doThisTwice` with a block as parameter. The block should contain a print statement. Explain what happens.

3.2.3 Advanced Tasks; challenges

Task 16. *Mutability of parameters.* What do you think about this: Is a parameter mutable in the function body ...

a) ... in Scala? (Yes/No)

b) ... in Java? (Yes/No)

c) ... in Python? (Yes/No)

Task 17. *Call-by-value and call-by-name.* Normally in Scala (and in Java) *call-by-value* is used when calling functions, which means the argument expression is evaluated *before* the binding to the parameter name happens.
One can also in Scala (but not in Java) with syntax `=>` before the parameter type declare that *call-by-name* shall happen, which means the evaluation of the argument expression is *delayed* and happens *each time* the name is used in the method body.
Declare below functions in REPL.

```
def snark: Int = { print("snark "); Thread.sleep(1000); 42 }
def callByValue(x: Int):   Int = x + x
def callByName(x: => Int): Int = x + x
lazy val zzz = snark
```

Explain what happens when the following expressions are evaluated.

a) `snark + snark`

b) `callByValue(snark)`

c) `callByName(snark)`

d) `callByName(zzz)`

Task 18. *Create your own control structure for iteration with loop variable.*

a) The advantage of `repeat` in task 7 is that it is concise and easy to use. But it is not as easy to use if you need access to a loop variable. Implement the control structure below.

```
def repeat(n: Int)(p: Int => Unit): Unit =
  var i = 0
  while i < n do
    ???
```

b) Use `repeat` to write the loop variable and a random decimal number between 0 and 1 one hundred times.

Task 19. *Split parameter list and curried functions.* One can split the parameters to a function into several parameter lists. The function `add1` below has a parameter list with two parameters while `add2` has two parameter lists with one parameter each:

```
  def add1(a: Int, b: Int) = a + b
  def add2(a: Int)(b: Int) = a + b
```

a) When calling the function `add2` the arguments must be written in two separate parentheses. How can you use `add2` to compute `1 + 1`?

b) One advantage with split parameter lists is that one can create so-called *curried functions*[11] where arguments are partially applied. Try the curried function value `singLa` below. What is printed after lines 3 and 5?

```
1 scala> def repeat(s: String)(n: Int): String = s * n
2 scala> val song = repeat("doremi ")(3)
3 scala> println(song)
4 scala> val singLa = repeat("la")
5 scala> println(singLa(7))
```

★ **Task 20.** *Recursion.* A recursive function calls itself.

a) Explain what happens below.

```
scala> def countdown(x: Int): Unit =
         if x > 0 then {println(x); countdown(x - 1)}
scala> countdown(10)
scala> countdown(-1)
scala> def finalCountdown(x: Byte): Unit =
         {println(x); Thread.sleep(100); finalCountdown((x-1).toByte); 1 / x}
scala> finalCountdown(Byte.MaxValue)
```

b) What happens if you put the statement that risks division by zero *before* the recursive call in the function `finalCountdown` above?

c) Explain what happens below. Why does the last line take longer than the penultimate line?

```
scala> def signum(a: Int): Int = if a >= 0 then 1 else -1
scala> def add(x: Int, y: Int): Int =
         if y == 0 then x else add(x + 1, y - signum(y))
scala> add(100, 100)
scala> add(Int.MaxValue, 0)
scala> add(0, Int.MaxValue)
```

★ **Task 21.** *Investigate tail recursion by throwing exceptions.* Explain what happens. Can you find evidence that the compiler can optimize recursion into an ordinary loop?

```
scala> def explode = throw Exception("BANG!!!")
scala> explode
scala> def countdown(n: Int): Unit =
         if n == 0 then explode else countdown(n-1)
scala> countdown(10)
scala> countdown(10000)
scala> def countdown2(n: Int): Unit =
         if n == 0 then explode else {countdown2(n-1); print("no tailrec")}
scala> countdown2(10)
scala> countdown2(10000)
```

★ **Task 22.** *`@tailrec` annotation.* You can ask the compiler to give an error message if it cannot optimize the code to an equivalent while-loop. This can be used in cases where you want to be absolutely sure that the compiler can optimize the code and there is no risk of a *stack overflow* due to too deep call nesting.
Try the lines below in REPL and explain what happens.

[11] Also called Curry functions after the mathematician and logician Haskell Brooks Curry.

```
scala> def countNoTailrec(n: Long): Unit =
         if n <= 0L then println("Klar! " + n) else {countNoTailrec(n-1L); ()}
scala> countNoTailrec(1000L)
scala> countNoTailrec(100000L)
scala> import scala.annotation.tailrec
scala> @tailrec def countNoTailrec(n: Long): Unit =
         if n <= 0L then println("Klar! " + n) else {countNoTailrec(n-1L); ()}
scala> @tailrec def countTailrec(n: Long): Unit =
         if n <= 0L then println("Klar! " + n) else countTailrec(n-1L)
scala> countTailrec(1000L)
scala> countTailrec(100000L)
scala> countTailrec(Int.MaxValue.toLong * 2L)
```

3.3 Lab: irritext

Goals

- ☐ Be able to create a larger program with your own code based on your own ideas.
- ☐ Be able to use an editor and the terminal to iteratively edit, compile, and test your code.
- ☐ Be able to use variables in combination with conditionals and repetition at multiple levels.
- ☐ Be able to gradually improve your code to facilitate changes and increase readability.
- ☐ Be able to create and use abstractions to generalize and enable code reuse.

Preparations

- ☐ Do exercise `Excercise03` and review exercise `Excercise02` before you start the lab.
- ☐ Read appendix B and C.
- ☐ Download given code via the course's GitHub page.
- ☐ Develop a first, playable version of your text game, which you can further develop during the lab.
- ☐ Find someone who plays an early version of your game and reads your code, then gives feedback on the code's readability. Write down the feedback you receive.
- ☐ Play someone else's text game and give feedback on the code's readability.

3.3.1 Requirements

- You shall create a moderately annoying text game using an editor, for example VS `code` (see appendix C.1.1). The game shall run in the terminal.

- During the presentation of the lab, you shall explain which programming concepts you have practiced during the development of your text game. You shall also describe for the instructor how you have improved your code based on the feedback you received from someone who played your game and read the code.

- Your text game shall be *moderately* annoying if the player has read the code, while the game may well be unreasonably annoying for the one who *has not* read the code. It should be possible to complete the game (you decide what that means) and thereby end the program within a reasonable time having knowledge of the code.

- Try to make your code *easy to read and understand*, even if the game itself at times may be more or less incomprehensible, quirky, or troublesome for the player who does not have access to the code... Note that your code does not need to be "perfect" from the start. Start freely and improve gradually.

- As your program becomes longer, you should reshape and split your code into many, short abstractions with well-chosen names to increase readability.

- Your code should use the important concepts that the course has covered so far, with special focus on what you specifically need to practice the most.

3.3.2 Tips to get started

- Create a directory that contains a Scala code file with any name.
- Write a simple `@main` method in the newly created code file that only prints the string `"Hello World!"`.
- Compile and run, fix any errors until the program works correctly.
- When the program works, begin extending the `@main` method in your code file and implement more functionality, take a look under inspiration below.
- Start simple and try to formulate what your program should do with *pseudocode* as comments before you write the code.
- Compile and run at every addition and keep each addition as small as possible, so you avoid sorting out a lot of difficult subsequent errors at compilation and any runtime errors become more understandable.
- Keep extending until the lab requirements have been met.

3.3.3 Inspiration

Here follows a list of different suggestions for features that you can choose from, combine, and vary in different ways. You can also include completely other features in your game. The most important thing is that you combine joy of coding with educational challenges :)

- Ask the user to log in. Give silly error messages if the user does not know the password.
- Let the user end up in an annoying endless loop of pointless questions if they make a ”wrong” choice.
- Describe a scary fantasy place where the user is located, for example a cave | a basement | a spaceship | the Chemistry Center.
- Let the user choose between silly weapons, for example a floor mop | cotton swab | foil hat | poisoned chocolate biscuit.
- Let the user choose between different paths | doors | tunnels | section cafés. Let the choice dictate which monsters are encountered. Let the user fight the monster with different weapons.
- Introduce some kind of score that is shown during the game and at the end.
- Introduce different kinds of points for health, combat power, achieved skill level, etc.
- Ask the user about more or less relevant details: name | shoe size | favorite pet. Give silly comments where these details are included as substrings.
- Play rock | paper | scissors with the user.
- Play ”guess the number” and give hints if the number is too low or too high.

- Measure how long it takes for the user to complete your game and assign points accordingly.
- Check the user's reaction time by measuring how long it takes to press Enter after waiting a random time for the string `"NOW!"` to be printed. If the user presses Enter before the start print, the measured time is 0 and thus your program can detect that the user pressed too early. Measure reaction time repeatedly and give points based on the average.
- Let the user write different words backwards as fast as possible within a time limit.
- Ask the user to write a palindrome. Give points based on length.
- Train the user in the multiplication table against the clock.
- Let the user answer multiple-choice questions about your favorite movie.
- Make it possible to provide an extra argument with a ”cheat code” that gives the user special abilities or otherwise eases the gameplay.

Chapter 4

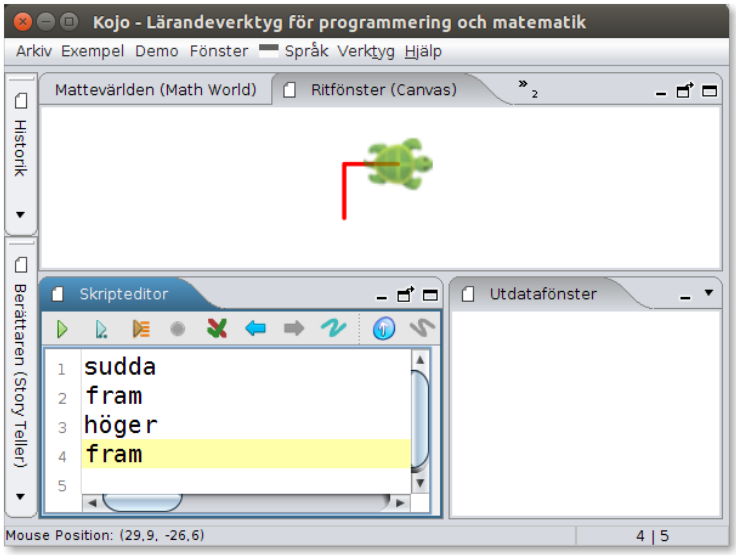
Objects and Encapsulation

Concepts included in this week's studies:

- ☐ module
- ☐ singleton object
- ☐ dot notation
- ☐ state
- ☐ member
- ☐ attribute
- ☐ method
- ☐ package
- ☐ file structure
- ☐ jar
- ☐ classpath
- ☐ documentation
- ☐ JDK
- ☐ import
- ☐ selective import
- ☐ renaming on import
- ☐ export
- ☐ tuple
- ☐ multiple return values
- ☐ block
- ☐ local variable
- ☐ shadowing
- ☐ local function
- ☐ functions are objects with apply method
- ☐ namespace
- ☐ visibility
- ☐ private member
- ☐ encapsulation
- ☐ getter and setter
- ☐ uniform access principle
- ☐ method overloading
- ☐ introprog.PixelWindow
- ☐ initialization
- ☐ lazy val
- ☐ type alias
- ☐ dod: machine code

4.1 Theory

4.1.1 What does the turtle in Kojo hold in its state?



position, direction, color, width, pen up/down, fill color

4.1.2 What is an object?

- An object is an abstraction that...
  - can contain **data** that the object "keeps track of" and
  - can offer **operations** that *do* something or return a *value*

- Example: The turtle in Kojo
  - What **data** is stored by the turtle?
    position, direction, pen color, ...
  - Which **operations** can you ask the turtle to perform?
    forward, right, left, ...
- Terminology:
  - the object's **data** is stored in variables called **attributes**
  - all variables' **values** together form the object's **state**
  - the **operations** are functions in the object and are called **methods**
  - the object's **parts** (attributes, methods, etc.) are called **members**

4.1.3 Declare, allocate, reference

Different things you can do with objects:

- **declare**: write code that describes the object;
  there are several ways: singleton object, class, tuple, ...

- **allocate**: create space in memory for the object at runtime
- **reference**: use the object via a name;
  you access the contents of an object with **dot notation**:
  `ref.member`
- (**deallocate**): release memory for objects no longer used; this **happens automatically** in Scala, Java, C#, etc. thanks to the **garbage collector**, but in many other languages, e.g. C++, you have to manage deallocation yourself, which is tricky and easily leads to difficult bugs.

4.1.4 Different ways to allocate objects

1. Use a **ready-made function** that creates an object for us, e.g. `apply`:

   ```
   Vector(1,2,3)  // create a Vector object with the apply method
   Vector.apply(1,2,3)  // explicit apply
   ```

   A function that creates objects is called a **factory method**.

2. Do `new` on a class (more on classes later):

   ```
   new introprog.PixelWindow() // create a window object
   ```

   With `new` you can create **many instances** of the same type of object.
   In Scala 3 `new` can often be omitted: `introprog.PixelWindow()`
3. Declare a **singleton object** with the keyword `object`
   - A singleton object exists in exactly **one** instance.
   - Allocated **automatically** the first time the object is referenced;
     you do not need, and cannot, write `new`.
4. Use a **tuple**, example: `val p = (200, 300)`

4.1.5 What is a singleton object?

- A singleton object (eng. *singleton*) is declared with the keyword `object` and is used to gather **members** (eng. *members*) that **belong together**.
- A singleton object is also called a **module**.
- Members can e.g. be **variables** (`val`, `var`) and **methods** (`def`).
- A **method** is a **function** that exists in an object. Methods are also called **operations**.
- Example: singleton object/module that manages highscore:

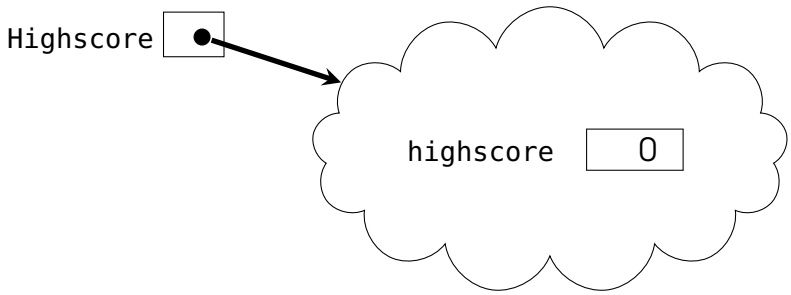
  ```
  object Highscore {
    var highscore = 0
    def isHighscore(points: Int): Boolean = points > highscore
  }
  ```

- Curly braces are optional in Scala 3:
  you can use colon and indentation instead.

- The idea is often that abstractions should be usable in other code, to facilitate when building applications, and are then called an **API** (Application Programming Interface). Example: a highscore API.

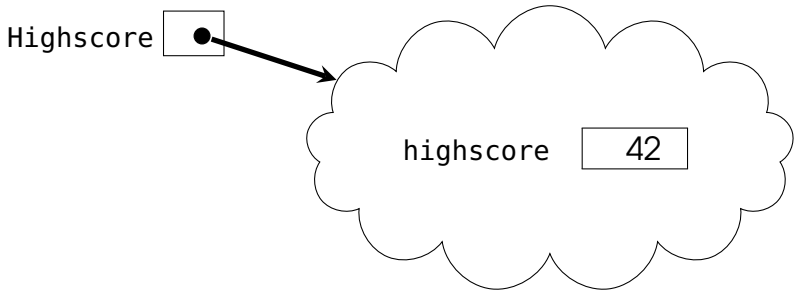
4.1.6 Allocation: memory reserved with space for data

```
object Highscore:
  var highscore = 0
  def isHighscore(points: Int): Boolean = points > highscore
```



4.1.7 Dot notation, state change via assignment

```
scala> Highscore.isHighscore(5)
res0: Boolean = true

scala> Highscore.highscore = 42
```



4.1.8 Dot notation and operator notation

Dot notation where the method call has **a single** argument:

```
object.method(argument)
```

can also be written in infix **operator notation**:

```
object method argument
```

Example:

```
1 + 2                                                    1.+(2)
1 to 42                                                 1.to(42)
 Highscore isHighscore 1000
```

Methods whose name **starts with letters trigger warnings** when called with operator notation unless declared with **infix** before **def**, to encourage consistent use of dot notation.

4.1.9 Namespace and shadowing

- A **namespace**[1] (eng. *namespace*) is an environment (context) in which all names are unique. By creating several different namespaces you can avoid "**collisions**" between identical names with different meanings (homonyms).
  Example: email addresses `kim@company1.se` ≠ `kim@company2.se`
- Members in a singleton object exist in their own namespace, where all names must be unique at the same level. They don't "collide" with names "outside" the object. However, **shadowing** (eng. *shadowing*) can occur:

```
object Game:

  val highscore = 42   // a different value than Game.Highscore.highscore

  object Highscore:
    var highscore = 0  // a different value than Game.highscore
    def isHighscore(points: Int): Boolean = points > highscore
```

4.1.10 Encapsulation: hiding internal parts

With the keyword **private** internal parts are hidden from the outside world. Private members can only be referenced *inside* the object. This principle is called **encapsulation**.

```
object Highscore:
  private var myHighscore = 0        // the name myHighscore is not visible externally
  def highscore: Int = myHighscore   // a so-called getter gives an attribute value
  def isHighscore(points: Int): Boolean = points > myHighscore
  def update(points: Int): Unit = if isHighscore(points) then myHighscore = points
```

Why is this useful?

- Prevent accidentally changing object's state in the wrong way.
- Prevent usage of code that may change in the future.
- Offer a simpler "outside" by hiding complexity "inside".
- Don't "pollute" the namespace with "unnecessary" names.

Disadvantages?

[1] https://en.wikipedia.org/wiki/Namespace

- Limits usage, you don't have access to all parts.
- Harder to experiment with an API while trying to understand it.

4.1.11 Idiom: Private variables with underscore for "collision"

Idiom: (i.e., a typical, generally accepted way to write code)

- If a private variable's name collides with the name of a getter, you usually start the private name with an underscore:

```
object Highscore:
  private var _highscore = 0
  def highscore: Int = _highscore
  def isHighscore(points: Int): Boolean = points > _highscore
  def update(points: Int): Unit = if isHighscore(points) then _highscore = points
```

Name collisions between methods and variables do not occur in Java and others, where these are in *different* namespaces. But in Scala the principle of **uniform access** applies and all members (both methods and variables) are therefore in a common namespace.

4.1.12 The principle of uniform access

- In Scala, access of attributes and calls to methods declared without parameter lists look the same.

```
object A1 { val a = 42 }
object A2 { def a = (41 + math.random()).round.toInt }
```

```
scala> A1.a
scala> A2.a
```

- Many other languages have different syntax for attribute access and method calls (e.g. Java etc., where all method calls must have parentheses).
- Advantage: It is easy to change the implementation back and forth between using attributes and using methods without code that uses your implementation needing to change.
- Disadvantage: Name collisions between methods and attributes can happen because they are in the same namespace.

4.1.13 Example: singleton object with mutable state

```
object myBankAccount:
  val accountNumber: Long        = 123456789L
  var balance: Int               = 1000
  def isOverdrawn: Boolean       = balance < 0
```

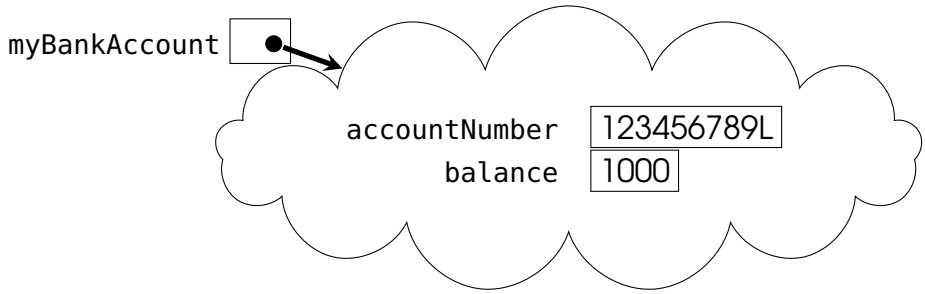
```
scala> myBankAccount.balance -= 25000

scala> myBankAccount.isOverdrawn
res0: Boolean = true
```

(Next week we will see how to create many instances of the same type of object with classes, so we can have multiple different bank accounts.)

4.1.14 Example: state, attributes

An object's **state** is the collection of all values of the attributes in the object.

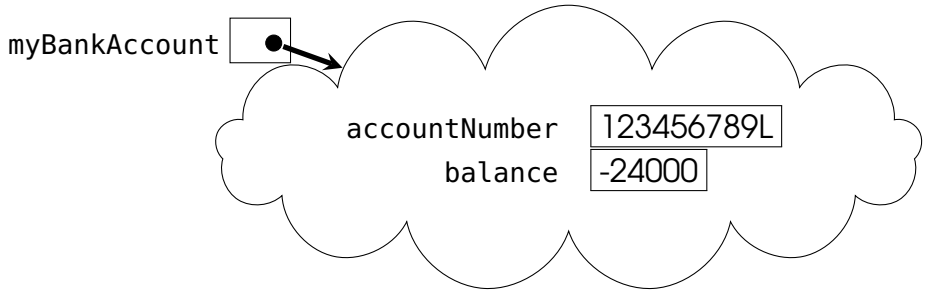
```
object myBankAccount:
  val accountNumber: Long        = 123456789L
  var balance: Int               = 1000
  def isOverdrawn: Boolean       = balance < 0
```



4.1.15 State change

When a variable is assigned a new value a **state change** occurs. A **mutable object** has **mutable state**.

```
scala> myBankAccount.balance -= 25000

scala> myBankAccount.balance
res1: Int = -24000
```



4.1.16 Module

- A module gathers code that forms a cohesive, delimited **set of abstractions** that can be used by other code to solve a specific (sub)problem.

- In Scala there are two ways to create modules:[2]
  - **singleton object** with the keyword `object` and
  - **package** with the keyword `package`
  - They are similar; e.g. you can use dot notation and do `import` on members in both singleton objects and packages.
  - Differences:
    - for each package the compiler creates **subdirectories** for machine code
    - packages can be split across **multiple** code files – an object must be in **one** code file
    - objects can **inherit members** from classes & traits (more on this later)

4.1.17 Declare package

With the keyword `package` at the start of a code file all declarations get a common namespace.
This code is in the file `f1.scala`:

```
package mypackage

object A:
  def greet: Unit = println(B.greeting)
```

This code is in the file `f2.scala`:

```
package mypackage

object B:
  def greeting: String = "hello"
```

The singleton objects `A` and `B` both exist in the namespace `mypackage`.

4.1.18 Compile packages

Package declarations cause the compiler to put bytecode files in a directory with the same name as the package:

```
> scala compile . // compile all files in current directory
> ls .scala-build/*/classes/main/
mypackage
> ls .scala-build/*/classes/main/mypackage
'A$.class'   A.class   A.tasty  'B$.class'   B.class   B.tasty
```

Idioms, syntax and semantics:

- Package names usually consist of only lowercase letters.

[2] en.wikipedia.org/wiki/Modular_programming

- If package names contain dots, nested subpackages are created, for example: `p1.p2.p3` compiles code to the directory `p1/p2/p3`
- You can have multiple packages and even nested packages in the **same** code file, by using curly braces (or colon+indentation):
  `package p1 { object A; package p2 { object B }}`
  (but this is not common)

4.1.19 Packages in REPL

Packages don't work in the REPL:

```
scala> package mypackage { def hi = println("Hi") }
-- [E103] Syntax Error: -------------------------------
1 |package mypackage { def hi = println("Hi") }
  |^^^^^^^
  |this kind of statement is not allowed here
```

4.1.20 What is a tuple?

- An n-tuple is an object that gathers n objects in a simple data structure with concise syntax; you just need parentheses and commas to create tuple objects: `(1,'a',"hello")`
- The elements can be of **different** types.
- `(1,'a',"hello")` is a **3-tuple** of type: `(Int, Char, String)`
- You can access the individual elements with **_1**, **_2**, ... **_*n***
- You can also use **`apply(0)`**, **`apply(1)`**, ... **`apply(n-1)`**

```
scala> val t = ("hello", 42, math.Pi)
t: (String, Int, Double) = (hello,42,3.141592653589793)

scala> t._1        // 1-based access
res0: String = hello

scala> t(1)        // 0-based indexing, implicit apply
res1: Int = 42
```

- Tuples are convenient when you don't want the somewhat larger effort of creating your own class. (But with classes you can do much more than with tuples.)

4.1.21 Tuples as parameters and return values

- Tuples are handy as **parameters** when you want to combine values that belong together, for example x and y values of a point: `(3, 4)`
- Tuples are handy when you want a function to **return more than one value** in a simple and type-safe way.

```
scala> def length(p: (Double, Double)): Double = math.hypot(p._1, p._2)

scala> def angle(p: (Double, Double)): Double = math.atan2(p._1, p._2)

scala> def polar(p: (Double, Double)): (Double, Double) = (length(p), angle(p))

scala> polar((3,4))
res2: (Double, Double) = (5.0,0.6435011087932844)
```

- If the types fit, you can omit extra parentheses for **single tuple arguments**:

```
scala> polar(3,4)
res3: (Double, Double) = (5.0,0.6435011087932844)
```

https://en.wikipedia.org/wiki/Polar_coordinate_system

4.1.22 A handy way to create 2-tuples with the method ->

There is a method named -> that can be used on objects of **any** type to **create pairs**:

```
scala> ("Age", 42)
res0: (String, Int) = (Age,42)

scala> "Age".->(42)
res1: (String, Int) = (Age,42)

scala> "Age" -> 42
res2: (String, Int) = (Age,42)

scala> Vector("Age" -> 42, "Height" -> 178, "Weight" -> 65)
res3: scala.collection.immutable.Vector[(String, Int)] =
        Vector((Age,42), (Height,178), (Weight,65))
```

4.1.23 Type alias to abstract type names

With the keyword **type** you can declare a **type alias** to provide an **alternative** name for a certain type. Example:

```
scala> type Pt = (Int, Int)            // type alias
scala> type Pts = Vector[Pt]           // nested type alias

scala> def distToOrigin(pt: Pt): Double = math.hypot(pt._1, pt._2)

scala> val xs: Pts = Vector((1,1), (2,2), (3,4))
val xs: Pts = Vector((1,1), (2,2), (3,4))

scala> xs.head
val res0: Pt = (1,1)

```

```
scala> xs.map(distToOrigin)
val res1: Vector[Double] = Vector(1.4142135623730951, 2.8284271247461903, 5.0)
```

Type aliases are useful when:

- you have a long and complicated type and want to use a shorter name,
- you want to easily change the implementation later
  (e.g. if you want to use your own class instead of a tuple).

4.1.24 Lazy variables with delayed initialization

With the keyword `lazy` before `val` delayed ("**lazy**") evaluation occurs of the initialization expression.
The opposite (normal in Scala) is called **strict** evaluation.

```
scala> val strict = Vector.fill(1000000)(math.random())
strict: scala.collection.immutable.Vector[Double] =
 Vector(0.7583305221813246, 0.9016192590993339, 0.770022134260162, 0.1566771818

scala> lazy val lazyVal = Vector.fill(1000000)(math.random())
lazyVal: scala.collection.immutable.Vector[Double] = <lazy>

scala> lazyVal
res0: scala.collection.immutable.Vector[Double] =
  Vector(0.5391685014341797, 0.14759775960530275, 0.722606095900537, 0.90255727
```

A `lazy val` is **not** initialized at declaration but **later** when it is referenced for the **first time**.

4.1.25 Singleton objects are lazy

- Singleton objects are **not** allocated immediately at declaration; allocation happens first when the object is referenced for the first time.
- Example:

```
object myLazyObject:
  println("I am lazy")
  val bigArray = { println("creating big Array"); Array.fill(10000)(42) }
  lazy val evenBiggerArray = Array.fill(Int.MaxValue)(42)
```

When are the prints executed?
When are the variables allocated?

4.1.26 What is the difference between `val`, `var`, `def`, `lazy val`?

```
object example:
  println("hello example")
  val alwaysSameRef  = {println("hello val"); math.random()}
```

```
var canChangeWithAssignment = {println("hello var"); math.random()}
def evaluatedAtEachCall  = {println("hello def"); math.random()}
lazy val delayedInit = {println("hello lazy val"); math.random()}
```

In what order are the prints executed?

Lazy evaluation is an important principle in functional programming that enables efficient, immutable data structures where elements are allocated only when needed. en.wikipedia.org/wiki/Lazy_evaluation

---

4.1.27 Pitfall: initialization order and default values

```
scala> object X:
         val a = b
         val b = 42

scala> val test = X.a
val test: Int = 0        // AAAARGH!
```

If you use a variable in an object before it is declared you get a default value (zero or null). More about this in w05.

Solution: change the order if possible, otherwise make a a `lazy val`

```
scala> object Y:
         lazy val a = b
         val b = 42

scala> val works = Y.a
val works: Int = 42        // :)
```

4.1.28 Programming paradigms

en.wikipedia.org/wiki/Programming_paradigm:

- **Imperative programming**: the program is built from sequences of statements that read and **change** state
- **Object-oriented programming**: a kind of imperative programming where the program consists of objects that encapsulate state and offer operations that read and **change** state.
- **Functional programming**: the program is built from cooperating (pure) functions that **avoid** mutable data and state changes. Immutable data structures create efficient programs in combination with lazy evaluation and recursion.

4.1.29 Functions are true objects in Scala

Scala shows how one can **unify** (eng. *unify*)
object-orientation and **functional programming**:

A function is an object with an `apply` method.

```
scala> object increment:
         def apply(x: Int) = x + 1

scala> increment.apply(1)
res0: Int = 2

scala> increment(1)   // the apply method does not need to be explicitly writte
res1: Int = 2
```

4.1.30 Deepening: True function objects have function type

More precisely:
A function is an object of function type that has an `apply` method.

```
scala> object increment extends (Int => Int):
         def apply(x: Int) = x + 1

scala> increment(1)
res2: Int = 2

scala> Vector(1,2,3).map(increment)
res3: scala.collection.immutable.Vector[Int] = Vector(2, 3, 4)

scala> increment.   // press TAB
... andThen   apply   compose ... toString ...
```

More about `extends` later in the course...

4.1.31 What is a class?

Singleton objects exist in exactly ONE instance:

```
object myBankAccount:
  val accountNumber: Long        = 123456789L
  var balance: Int               = 1000
  def isOverdrawn: Boolean       = balance < 0
```

If we want multiple bank accounts we need a **class**.

4.1.32 What is a class?

A class can be used to create many objects of the same type. Each instance has its own state and is called an **instance** of the class (more on this next week).

```
class BankAccount(val accountNumber: Long, var balance: Int): // class description
  def isOverdrawn: Boolean = balance < 0
```

```
scala> val bk1 = new BankAccount(123456789L, 1000 )   // instantiate a class
bk1: BankAccount = BankAccount@5d7399f9

scala> val bk2 = new BankAccount(6789012L, -200 )
bk2: BankAccount = BankAccount@286855ea

scala> bk1.balance
res0: Int = 1000

scala> bk2.isOverdrawn
res1: Boolean = true
```

4.1.33 Using the class `Color`

- In the JDK (Java Development Kit) there are hundreds of packages (modules) and thousands of ready-made classes. [3]
- One of these classes is called `Color` and is located in package `java.awt` and is used to represent RGB colors with a number that describes the amount of Red, Green, and Blue.

```
scala> val red = java.awt.Color(255, 0, 0)    // fully red color

scala> import java.awt.Color  // name Color available in current namespace

scala> Color.    // press TAB and see all public members
```

- Use the class `java.awt.Color` in this week's exercise.
- How do I know how to use a ready-made class?
  1. Read the **documentation**, which shows the "outside" which is easier (?) than the "inside"
  2. **Experiment** with the help of REPL and/or an IDE
  3. Read the code, shows the "inside" with all its complexity; can be tricky...

4.1.34 Add methods afterward with `extension`

- Often you want to add methods to arbitrary types afterwards, especially types that exist in code written by someone else.
- This can be done in Scala with the keyword **`extension`**:
  `extension (s: String) def shoutReverse = s.reverse.toUpperCase`

[3] https://stackoverflow.com/questions/3112882/

- An **extension method** can be called with **dot notation** as if it were a member of the type.
- It can also be called as a standalone function without dot notation.

```
scala> extension (s: String) def shoutReverse = s.reverse.toUpperCase
def shoutReverse(s: String): String

scala> "hello".shoutReverse
val res1: String = OLLEH

scala> shoutReverse("goodbye")
val res2: String = EYBDOOG
```

4.1.35 Collective extension methods

- It is possible to gather several functions under the same `extension` like this:

```
extension (s: String)
  def reverse = s.reverse
  def shout = s.toUpperCase
```

- This is called **collective extension methods**.
- Note that there *must not* be a colon after the first line of the `extension` declaration.

4.1.36 Import all names in a module

- You can import **all** names in a module (singleton object or package). This is called a *wildcard import*.
  - Syntax: `import p1.p2.*`
- Example:

```
scala> import java.awt.*  // import ALL names in the awt package
```

- **Advantages**:
  1. No need to write import for each individual name.
  2. The abstractions that are meant to be used together become all visible in the current namespace (eng. *in scope*).
- **Disadvantages**:
  1. Can cause name collisions and difficult bugs due to shadowing.
  2. Pollutes your namespace with names that may not be intended for use but can have effect if misused, e.g., misspelling.
  3. You cannot tell exactly which names are used by looking at imports, which can make code harder to understand.

4.1.37 Rename on import

- You can avoid name collisions by **renaming on import**.
- Syntax: `import p1.p2.existingName as newName`
- Example:

```
scala> import java.awt.Color as JColor // import and rename

scala> val green = JColor(0, 255, 0)  // create instance with new name
green: java.awt.Color = java.awt.Color[r=0,g=255,b=0]
```

4.1.38 Exclude (hide) names on import

- You can avoid name collisions by excluding some names (eng. *import hiding*).
- Syntax: `import p1.p2.excludeMe as _`
- Example:

```
scala> import java.awt.{Event as _, *}  // import everything EXCEPT Event
```

- Can be combined with renaming and wildcard import:

```
scala> import java.awt.{Event as _, Color as JColor, *}
```

4.1.39 Local import declaration

- You can limit "polluting" the namespace by making import declarations as local as possible, for example inside an object or inside a function body.
- Example:

```
object A:
  def x =
    import java.awt.Color.RED
    /* ... the name RED is only visible locally in this function */
```

4.1.40 Export

- `import` gives direct visibility **locally** inside a namespace
- With `export` you can do the *opposite* of import:
  make members directly visible **outside** a namespace.

```
object A:
  import java.awt.Color.* // make colors directly visible inside this object
  def test = RED          // color RED is directly visible in local namespace
```

```scala
object B:
  export java.awt.Color.* // RED becomes a member visible outside via B.RED
  export math.{sin, cos}  // sin and cos become methods in B
```

```
scala> A.RED
-- [E008] Not Found Error: -------------------------------------------
1 |A.RED
  |^^^^^
  |value RED is not a member of object A

scala> B.RED
val res0: java.awt.Color = java.awt.Color[r=255,g=0,b=0]

scala> (B.cos(0), B.sin(0))
val res1: (Double, Double) = (1.0,0.0)
```

4.1.41 Using documentation for ready-made classes.

- Documentation for the Scala standard library is here: https://www.scala-lang.org/api/
- Exercise: Find the documentation for the method `reduceLeft` in the class `Vector`.

- Documentation for the Java standard library is here: https://docs.oracle.com/en/java/javase/21/docs/api/index.html
- Exercise: Find the documentation for `java.awt.Color`
- Read more in Appendix E about documentation.

4.1.42 What is a jar file?

- Jar files are used to distribute compiled code so others can use it easily
- The abbreviation **jar** comes from "Java Archive"
- A **jar** file follows a standardized file format and is used to **package multiple files** into one file, for example:
  - `.class` files with bytecode
  - resource files for an application, e.g. images `.png`, `.jpg`, etc.
  - information about which class contains the `main` function
  - etc.
- A `.jar` file is compressed like a `.zip` file.
- For the interested: https://en.wikipedia.org/wiki/JAR_(file_format)

4.1.43 Open source on Maven Central

- On **Maven Central** managed by Sonatype there are thousands of publicly available code libraries published as jar files.
- You can search among all Scala libraries here:
  https://index.scala-lang.org/
- You can search among all libraries here:
  https://search.maven.org/

4.1.44 What is the *classpath*?

- How does the compiler find ready-made modules?
- The compilers `scalac` and `javac` and the programs `scala` and `java` that start the JVM use **a list of file paths** called **classpath** when searching for compiled code.
- Scala's standard library is automatically put on the classpath.
- Using the option `--jar` you can add a jar file to the classpath.
- Example: (dot is used to indicate current directory)

```
scala run . --jar introprog.jar
```

4.1.45 Ready-made graphic methods in class `PixelWindow`

- In the lab you will use a `.jar` file with the code library `introprog`.
- There is the class `PixelWindow` which can create drawing windows.
- You can start REPL like this if you have downloaded the jar file manually from
  https://fileadmin.cs.lth.se/introprog.jar

```
> scala repl --jar introprog.jar
```

- Test `PixelWindow` in REPL with:

```
scala> val w = introprog.PixelWindow(300, 200, "hello")
```

- Study the documentation for `introprog.PixelWindow` here:
  https://fileadmin.cs.lth.se/pgk/api

4.1.46 Automatic dependencies with Scala CLI in REPL:

- You can instead let `scala-cli` **automatically** download a ready-made code library published on Maven Central and add it to classpath with the option `--dep` which is short for *dependency*.
- Note the number of colons in the address to the code library:

```
> scala repl . --dep se.lth.cs::introprog:1.4.0
Welcome to Scala 3.7.3 (17.0.3, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> introprog.Dialog.show("hello introprog")
```

4.1.47 Run programs + code libraries with Scala CLI

- `scala-cli` can include code libraries from Maven Central if you write a "magic" comment at the start of your `.scala` file:

```
//> using scala 3.7.3
//> using dep se.lth.cs::introprog:1.4.0

@main def run = introprog.Dialog.show("hello introprog")
```

  Note the > after `//`
- When you run your program like this Scala CLI will download the code library if it has not already been done:

```
> scala run .
```

- Read more here: https://index.scala-lang.org/lunduniversity/introprog-scalalib and in Appendix C, the part about Scala CLI. More about `//> using` here: https://scala-cli.virtuslab.org/docs/reference/directives

---

4.1.48 Recompile on every change

Specify the option `--watch` so the command runs every time you save a scala file with Ctrl+S.

```
> scala compile . --watch
```

Can also be written shorter:

```
> scala compile . -w
```

Also works for the run command, but is less useful if the app is interactive and waits for user input before it ends.

```
> scala run . -w
```

Make as small changes as possible and compile and test at **every** change! Many changes can cause hard-to-find cascading bugs...

4.2 Exercise `Excercise04`

Goals

- ☐ Be able to create and use objects as modules.
- ☐ Be able to explain how nested blocks affect name visibility and name shadowing.
- ☐ Be able to explain the concepts of visibility, private member, namespace, and name shadowing.
- ☐ Be able to create and use tuples.
- ☐ Be able to create functions that have multiple return values.
- ☐ Be able to explain the semantic relationship between functions and objects in Scala.
- ☐ Be able to explain the connection between package structure and code file structure.
- ☐ Be able to use ready-made code libraries in jar files.
- ☐ Be able to use import of members in objects and packages.
- ☐ Be able to rename on import.
- ☐ Be able to explain the difference between import and export.
- ☐ Be able to create and use variables with delayed initialization.

Preparations

- ☐ Study the concepts in chapter 4
- ☐ Read about how to fix bugs in appendix D.

4.2.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with descriptions.*

Connect each concept with the (simplified) description that fits best:

module	1	A	function that is a member of an object
singleton object	2	B	module that can have state; exists in a single instance
package	3	C	code unit with abstractions that can be reused
import	4	D	module creating namespace; machine code gets own folder
export	5	E	belongs to an object; accessed via dot notation if visible
lazy initialization	6	F	makes name locally available without full path
member	7	G	allocation occurs only when name is referenced
attribute	8	H	variable representing (part of) an object's state
method	9	I	environment where all names are unique
private	10	J	methods with same name but different parameter types
overloading	11	K	modifies visibility of an object member
name shadowing	12	L	local name hides same name in enclosing block
namespace	13	M	changing between def and val does not affect usage
uniform access	14	N	alternative name for type, often improves readability
dot notation	15	O	used to access non-private parts
type alias	16	P	makes name visible outside as a member of this object

Task 2. *Nested singleton objects, import, visibility and dot notation.* In the two-dimensional Underworld live the Mole and the Worm. The Worm has hidden from the Mole and is located far away. The Worm has also made parts of its position invisible to the outside world:

```
object Underworld:
  var x = 0
  var y = 1

  object Mole:
    var x = Underworld.x + 10
    var y = Underworld.y + 9

  object Worm:
    private var x = Mole.x
    var y = Mole.y + 190
    def isMoleFood: Boolean = ???
```

a) Create the above code in the file `Underworld.scala` with an editor and implement the predicate `isMoleFood` so that it returns true if the mole's coordinates are the same as the worm's.

b) Test the life in Underworld by pasting your module in REPL. Import Underworld's members with an asterisk so that you can see Mole and Worm. Using assignments, move Worm's y-coordinate so that it ends up at the same location as cwthe Mole. Check that the predicate `isMoleFood` works as intended.

c) Then import everything from Mole and then everything from Worm and assign `x` a new value according to lines 1–3 below. What values do the expressions in lines 4–6 below give? Explain what happens in terms of name shadowing and visibility.

```
1 scala> import Mole.* // Mullvaden.*
2 scala> import Worm.* // Masken.*
3 scala> x = -1
4 scala> Mole.x
5 scala> Worm.x
6 scala> Underworld.x
```

Task 3. *Export.*

a) Compare **`import`** and **`export`** by describing one similarity and one difference.

b) Create an example in REPL demonstrating the usefulness of **`export`**.

Task 4. *Tuples.* Tuples combine multiple different values into an immutable object. Below, tuples are used to represent a 3D point in the Underworld with coordinates `(x, y, z)` of type `(Int, Int, Double)`, where the z-coordinate indicates how deep down in the underworld the point is. At a secret location is the entrance to the uberworld.

```
object Underworld3D:
  private val secret = ("the ascent to the uberworld", (0, 0, 0.0))

  object Mole:
    var pos = (5, 3, math.random() * 10 + 1)
```

```
    def depth  = ???

  object Worm:
    private var pos = (0, 0, 10.0)
    def isMolefood : Boolean = ???
    def isRightUnderTheRise: Boolean = ???
```

a) The function `depth` should give the z-coordinate for Mole. What is the type of `depth`?

b) What is the type of `secret`?

c) Write the code for `Underworld3D` in an editor and implement the missing parts. The predicate `isMoleFood` should be true if Worm is at the same place as Mole. The predicate `isRightUnderTheRise` should be true if the x and y coordinates match the secret entrance to the uberworld. Test your implementation in REPL.

d) A tuple with n values is called an n-tuple. If you consider the empty value `()` as a tuple, what can this value be called?

Task 5. *Lazy initialization.* With `lazy val` you can delay initialization.

a) What do lines 2 and 3 below produce?

```
1 scala> lazy val z = { println("nu!"); Array.fill(1e1.toInt)(0)}
2 scala> z
3 scala> z
```

b) Try again with a so large array that the memory gets full. When does the allocation happen?

c) Singleton objects are lazy. Initialization order can be wrong.

```
object test:
  object zzz    { val a = { println("nu!"); 42} }
  object buggig { val a = b ; val b = 42        }
  object funkar { lazy val a = b; val b = 42    }
```

Paste the module `test` into REPL. When is `"nu!"` printed?

d) What happens in REPL if you refer to the three different `a` variables?

e) What is the difference between `lazy val a = expression` and `def b = expression` ?

Task 6. *Extension methods.* Extension methods allow dot notation on values of existing types.

a) Create an extension method on integers that allows incrementing.

```
scala> 42.inc
val res0: Int = 43
```

b) Create an extension method on integers that allows decrementing.

```
scala> 42.dec
val res1: Int = 41
```

c) Combine the extension methods so that they become *collective*, i.e. under the same `extension`. Also use `math.incrementExact` and `math.decrementExact` after

consulting the documentation here: https://docs.oracle.com/en/java/javase/17/docs/api/

d) What is the advantage of `math.incrementExact` and `math.decrementExact`?

Task 7. *Jar file. Classpath. Package.* A jar file is used to collect compiled programs, code, documentation, resource files, etc, into a single file. A jar file is compressed similarly to a zip file. In the course we use a package named `introprog` which is in a jar file named something like `introprog_3-1.4.0.jar` (or later version) where the first number indicates the Scala version the library is compiled for, and the second number indicates the library version which changes with each new release.

a) In this week's lab we will use the class `PixelWindow` that exists in the package `introprog`. What parameters does the class `PixelWindow` have and which default arguments are present? How do you write to create an instance of `PixelWindow`?
Tip: See the code for `PixelWindow` here (search for the class's parameters):
https://github.com/lunduniversity/introprog-scalalib/blob/master/src/main/scala/introprog/PixelWindow.scala

b) Download the latest release of the jar file with the `introprog` package here:
https://github.com/lunduniversity/introprog-scalalib/releases
Save the file named `introprog_3-1.4.0.jar` (or later version) in a suitable location.

c) Test `PixelWindow` in REPL as below. Use the option `-jar` with the jar file name as argument. Write code that draws a square with side 100 and upper left corner at point (100, 100), continuing the partially started code below (adjust the jar file name according to the version you downloaded):

```
> scala repl --jar introprog_3-1.4.0.jar
scala> val w = introprog.PixelWindow(400,300,"HEJ")
scala> w.line(100, 100, 200, 100)
scala> w.line(200, 100, 200, 200)
scala> // continue so that a full square is drawn
```

d) Write the program below with an editor in the file `hello-window.scala` and fill in the missing parts so that a red square is drawn, guided by the documentation:
https://fileadmin.cs.lth.se/pgk/api

```scala
package hello

object Main:
  val w = new introprog.PixelWindow(400, 300, "HEJ")

  var color = java.awt.Color.red

  /** Square with upper corner at point p and size side pixels. */
  def square(p: (Int, Int))(side: Int): Unit =
    if side > 0 then
      // side == 1 gives a square that is a single pixel
      val d = side - 1

      w.line(p._1,     p._2,     p._1 + d, p._2,     color)
      w.line(p._1 + d, p._2,     p._1 + d, p._2 + d, color)
      w.line(p._1 + d, p._2 + d, p._1,     p._2 + d, color)
      ???
```

```
  def main(args: Array[String]): Unit =
    println("Draw square:")
    square(300,100)(50)
```

Run the program with

```
> scala run hello-window.scala --jar introprog_3-1.4.0.jar
Found several main classes. Which would you like to run?
[0] hello.Main
[1] introprog.examples.TestBlockGame
[2] introprog.examples.TestIO
[3] introprog.examples.TestPixelWindow
```

Besides your own main program named `hello.Main`, there are several example main programs in package `introprog.examples`. When several main programs are detected, you are asked which one you want to run. Choose your own main program.

e) You can avoid the question by explicitly specifying the main program with the option `--main-class`. Try that!

f) You can avoid downloading `introprog` by yourself using the option `--dep` when running from terminal, as described in the library's `README.md` on github here:
https://github.com/lunduniversity/introprog-scalalib
Try it!

g) You can also add the dependency inside your code file using a magic comment, also described in the above `README.md`. Try that!

Task 8. *Color.* There are many ways to describe colors. In natural language we have different names for colors, e.g. *white*, *pink* and *magenta*. In computer image memories it is common to describe colors as a mixture of *red*, *green* and *blue* in the so-called RGB system.
In this week's lab we will use `PixelWindow`, which describes RGB colors with the class `java.awt.Color`. There are some predefined colors in `java.awt.Color`, e.g. `java.awt.Color.black` for black and `java.awt.Color.green` for green, see further documentation of `java.awt.Color` in JDK[4]. Other colors can be created by specifying the particular amount of red, green and blue needed to blend a certain color. The three parameters to `new java.awt.Color(r, g, b)` specify how much *red*, *green* and *blue* the color should contain, and the amounts should be in the range 0–255. The color $(153, 102, 51)$ means quite a lot of red, somewhat less green and even less blue and it is perceived as brown.

a) In the lab you need these three brownish colors and it is convenient to collect them in their own namespace via a singleton object called `Color` as below.

```
object Color:
  val mole   = new java.awt.Color( 51,  51,   0)
  val soil   = new java.awt.Color(153, 102,  51)
  val tunnel = new java.awt.Color(204, 153, 102)
```

But we would prefer importing `java.awt.Color` so that we can use the class name without repeating the entire path, even though the name collides with the name of our singleton object. Rewrite the above code using import aliasing so that colors can

[4] https://docs.oracle.com/en/java/javase/21/docs/api/

be created with `new JColor(...)`. Make the import local inside the singleton object `Color`.

b) Inspired by the REPL experiments below, modify your program in `hello-window.scala` so that *three* overlapping color-filled squares are drawn as in the top picture to the right. Instead of drawing with the existing method `fill` from `PixelWindow`, you practice iteration by implementing the drawing procedures `straight` and `fill` yourself as below. The procedure `straight` should draw a horizontal line with left point `p` and length `d` pixels. The procedure `fill` should, with many horizontal lines, draw a filled square with upper left corner at point `p` and side `s` pixels. The drawing should look like the top picture to the right. If you for example include one pixel too many in your coordinate calculations, you may get the incorrect picture below.

```
> scala repl --dep se.lth.cs::introprog:1.4.0
scala> val w = new introprog.PixelWindow(400,300,"Three shades of brown")
scala> type Pt = (Int, Int)
scala> var color = java.awt.Color.red
scala> def straight(p: Pt)(d: Int) = w.line(p._1, p._2, ???, ???, color)
scala> def fill(p: Pt)(s: Int) = for i <- ??? do straight((p._1, ???))(s)

scala> object Color:
     |   ???

scala> color = Color.soil
scala> fill(100,100)(75)

scala> color = Color.tunnel
scala> fill(100,100)(50)

scala> color = Color.mole
scala> fill(150,150)(25)
```

c) On which calls above is the omission of tuple parentheses utilized?

Task 9. *Events.* In this week's lab, you will implement a simple game where the user can control a block mole with the keyboard. With `introprog.PixelWindow` you can handle events generated when the user presses or releases a keyboard or mouse button.

a) Study the documentation for the singleton object `introprog.PixelWindow.Event`. What is the name of the immutable integer variable that represents a key press event? What is its value?

b) From the documentation for singleton object `introprog.examples.TestPixelWindow` you can access the code that implements the object by clicking the `Source` link above the search box. Which line in the main program in the `main` method takes care of the case that a key press event has occurred?

c) Run with `scala run .` (where the dot stands for current directory) the main program in `TestPixelWindow` with options
`--main-class introprog.examples.TestPixelWindow` and
`--dep se.lth.cs::introprog:1.3.1`
A test window opens when `main` runs. Click in the window at different places and press different keys and observe what is printed. What is printed when the up arrow key is pressed and released?

d) Inspired by the implementation of `TestPixelWindow`, write a program that draws green lines between the positions for each mouse button press and mouse button release the user makes.
Tip: When the mouse button is pressed, save the position in a variable named `start`. When the mouse is released, draw the line from the saved position to `w.lastMousePos`.

4.2.2 Extra Tasks; additional practice

Task 10. *Functions are objects with an `apply` method.*
The method `apply` is special.

```
scala> object plus { def apply(x: Int, y: Int) = x + y }
scala> plus.apply(42, 43)
```

Is it possible to omit `.apply` and call `plus` as a function?

Task 11. *Create modules using singleton objects.*

a) Investigate in REPL what the expression `"pearIceScream".split('i')` evaluates to.

b) What is printed if you call `Test()` which calls the `apply` method below?

```
object stringUtils:
  object split:
    def sentences(s: String): Array[String] = s.split('.')
    def words(s: String): Array[String] = s.split(' ').filter(_.nonEmpty)

  object count:
    def letters(s: String):   Int = s.count(_.isLetter)
    def words(s: String):     Int = split.words(s).size
    def sentences(s: String): Int = split.sentences(s).size

  object statistics:
    var history = ""
    def printFreq(s: String = history): Unit =
      println(s"\n--- FREQUENCY ANALYSIS OF:\n\$s")
      println(s"# letters : \${count.letters(s)}")
      println(s"# words   : \${count.words(s)}")
      println(s"# sentences: \${count.sentences(s)}")
      history = (s"\$history \$s").trim

object Test:
  import stringUtils.*
  def apply(): Unit =
    val s1 = "Five ants are more than four elephants. Eat cucumber."
    val s2 = "Galaxies in my breams. Tomato is good. Pear split."
    statistics.printFreq(s1)
    statistics.printFreq(s2)
    statistics.printFreq()
```

c) Which of the objects in the module `stringUtils` has state? Is it mutable?

d) Change the methods in the singleton object `count` to become extension methods so they can be called like this:

```
scala> import stringUtils.count

scala>  val s = "Hejsan hoppsan. Gurka är gott."
val s: String = Hejsan hoppsan. Gurka är gott.

scala>  (s.nbrOfLetters, s.nbrOfWords, s.nbrOfSentences)
val res0: (Int, Int, Int) = (24,5,2)
```

Task 12. *Tuples as parameters.* Implement below different variants of calculating the distance between two points. *Tip:* Use `math.hypot`.

```
def distxy(x1: Int, y1: Int, x2: Int, y2: Int): Double = ???
def distpt(p1: (Int, Int), p2: (Int, Int)):     Double = ???
def distp(p1: (Int, Int))(p2: (Int, Int)):      Double = ???
```

Task 13. *Tuples as function results.* Tuples enable a function to return several different values simultaneously. Implement the function statistics below. It should return a 3-tuple containing the number of elements in `xs`, the average of the elements, and a 2-tuple with the range (min,max). Specify the return type explicitly in your implementation. Test that it works in REPL. *Tip:* You can use methods `size`, `sum`, `min` and `max` which work on numeric sequences.

```
/** Returns the size, the mean, and the range of xs */
def statistics(xs: Vector[Double]) = ???
```

Task 14. *Create modules using packages.*

a) The code below is in the file `paket.scala`. Draw a picture of the directory structure that is created in the current library under subdirectory `main` in `.scala-build` when the code below is compiled with: `scala compile paket.scala`

```
package gurka.tomat.banan

package p1:
  package p11:
    object hello:
      def hello = println("Hej paket p1.p11!")
  package p12:
    object hello:
      def hello = println("Hej paket p1.p12!")

package p2:
  package p21:
    object hello:
      def hello = println("Hej paket p2.p21!")

object Main:
  def main(args: Array[String]): Unit =
    import p1.*
    p11.hello.hello
    p12.hello.hello
    import p2.{p21 as orange}
    orange.hello.hello
```

b) What is printed when the program is run?

c) Are packages allowed to have state variables without placing them inside a singleton object or a class?

4.2.3 Advanced Tasks; challenges

Task 15. *How to do without* **do while** *in Scala 3?* In many languages there is a construct with the following syntax: **do** <statements> **while** <condition> where the <statements> run at least once before the condition is tested. This "backwards while" is not used often but can be convenient if you want to run a repetition at least once.
This construct exists in Scala 2 but not Scala 3 because the keyword **do** in Scala 3 is used for optional braces and indentation syntax in "regular while". One reason it is considered okay to remove **do** <statements> **while** <condition> is that in Scala 3 the "backwards while" can be rewritten as a regular while by enclosing the statements to be done at least once in a block at the condition place and having the loop body be the empty value, i.e.:

```
while
  <statements>
  <condition>
do ()
```

a) The below works in Scala 2, but what happens if you try to do this in Scala 3:

```
> scala repl --scala 2
Welcome to Scala 2.13.8 (OpenJDK 64-Bit Server VM, Java 17.0.3).
Type in expressions for evaluation. Or try :help.

scala> var i = 0
var i: Int = 0

scala> do i += 1 while (i < 10)

scala> i
val res20: Int = 10
```

b) Rewrite the "backwards" **do while** to an equivalent "regular" **while do** that works in Scala 3.

Task 16. *Postfix operators for increment and decrement.* In many languages, e.g. Java, C++, C, you can write `i++` and `i--` to increment or decrement an integer variable `i`. Use Scala's extension methods to make it possible to use the operators ++ and -- on integers as below:

```
scala> 42.++
val res0: Int = 43

scala> 42.--
val res1: Int = 41

scala> import language.postfixOps    // allows postfix operator notation

scala> 43 ++
val res2: Int = 44

scala> 43 --
val res3: Int = 42
```

```
scala> val i = 42
val i: Int = 42

scala> i++
val res4: Int = 43

scala> i--
val res5: Int = 41
```

Task 17. *Use ready package: Color chooser.* In the lab it is useful to be able to mix your own colors so you can draw clear blue sky and lush grass. You can create a color chooser with the `introprog` package as below.

```
> scala repl --dep se.lth.cs::introprog:1.3.1
scala> introprog.Dialog.selectColor()
```

a) What happens if you press **Ok** after selecting a green color?

b) What happens if you press **Cancel** ?

c) What happens if you press **Reset** ?

d) Read the documentation for the method `selectColor` in the singleton object `Dialog` in the package `introprog`. Call `selectColor` with the default color `java.awt.Color.green`.

Task 18. *Use ready package: user dialogs.*

a) Read the documentation for the singleton object `Dialog` in the package `introprog`.

b) Use the procedure `introprog.Dialog.show` and give a message to the user that it is `"Game over!"`.

c) Use the function `introprog.Dialog.input` to ask the question `"What is your name?"` and find out the user's name. What happens if the user clicks *Cancel*?

d) Use the function `introprog.Dialog.select` to ask the user to choose among rock, scissors and paper. What is the return type?

Task 19. *Create your own `jar` file.* ★

a) Write the command `jar` in terminal and check with `jar --help` what options exist. Which options should you use to create a jar in a named file with verbose output about what happens?

b) Create with an editor in the file `hello.scala` a simple program that prints `"Hello package!"` or similar. The code should be in package `hello` and contain an object `Main` with a `main` method. Compile your file with the option `--destination .` so that your code ends up in the current directory instead of in `.scala-build`.

c) Write a jar command in terminal that packages the code into a jar file named `my.jar` and start REPL with the jar on the classpath. Call your `main` function in REPL by specifying the path *`packagename.objectname.methodname`* with an empty array as argument.

d) With which command can you run the compiled and jar-packaged program directly in terminal (i.e. without starting REPL)?

Task 20. *How large is JDK8?* Find out using `http://stackoverflow.com/` how many classes and packages the Java platform JDK8 includes. ★

4.3 Lab: blockmole

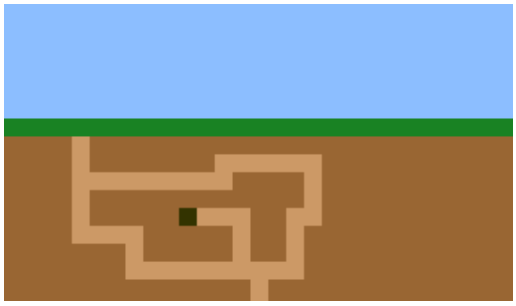
Goals

- ☐ Be able to explain how singleton objects can be used as modules.
- ☐ Be able to explain how access to members in singleton objects occurs.
- ☐ Be able to create code that reacts to and changes the state of objects.
- ☐ Be able to explain the benefit of collecting named constants in a separate module.
- ☐ Be able to explain how import affects the visibility of names.
- ☐ Be able to give an example of a situation where renaming upon import is useful.
- ☐ Be able to describe the difference between packages and singleton objects.
- ☐ Be able to create and use tuples.

Preparations

- ☐ Do exercise `Excercise04` and review exercise `Excercise03`.
- ☐ Review appendices B, C, and D.
- ☐ Obtain given code via the course's GitHub repository.

4.3.1 Background



Blockmole (*Talpa laterculus*) is a fantasy creature in the mole family. It is known for its distinctive square appearance. It mostly lives alone in its underground tunnels which, unlike the real mole's (*Talpa europaea*) tunnels, have completely straight walls.

4.3.2 Mandatory tasks

Task 1. *Create directory and code file.* Step by step, you will create a program that lets the user interact with a living blockmole. Use an editor, e.g., VS `code`, compile your program in the terminal with `scala compile . --watch` and run it in another terminal window with `scala run .`

a) Create a new file named `blockmole.scala` in a new directory in your home directory, for example `~/pgk/w04/lab/blockmole.scala`, where ~ is your home directory.

```
> mkdir -p ~/pgk/w04/lab
> code ~/pgk/w04/lab/blockmole.scala
```

b) Navigate to your new directory and check that your new file is there.

```
> cd ~/pgk/w04/lab/
> ls
blockmole.scala
```

c) Make a package declaration at the beginning of the file `blockmole.scala` so that the code you will write below is part of the package `blockmole`.

d) Then declare a single object named `Main` with a `@main def run` procedure that prints the text: `"Keep on digging!"`

e) Compile your program with `scala compile .` and look for files ending with `.class` in subdirectories of `.scala-build`. What is the name of the subdirectory with your program's machine code files? Why did this subdirectory get this name? ✎

f) Run the command `scala run . --main-class blockmole.run` to execute your program and check the output in the terminal window.

Now you have written a program that encourages a blockmole to keep digging. That program is not very useful since moles cannot read. The next step is therefore to write a graphical program.

Task 2. *Create a basic structure for the program.* In smaller programs, it works well to gather all functions in a single object, but in large programs it becomes easier to find your way in the code and understand what it does if you have several modules with different responsibilities. Your program should have the following overall structure:

```
package blockmole

object Color:
  // Creates different colors needed in other modules
  ???

object BlockWindow:
  // Has an introprog.PixelWindow and draws block graphics
  ???

object Mole: // Represents a blockmole that can dig
  def dig(): Unit = println("This is where the digging happens!")

object Main:
  def drawWorld(): Unit = println("Will draw the underground!")

  @main def run =
    drawWorld()
    Mole.dig()
```

Create the above program skeleton in the file `blockmole.scala` and make sure the code compiles without errors and can be run with outputs as expected. The function `???` in the skeleton is used as a placeholder so that the code can compile even though the single object's bodies are currently empty (more about this in chapter 5). Replace `???` with the actual code for Color and BlockWindow in the upcoming subtasks.
In this lab, we place all modules in the same file, but in other situations when the modules grow large and/or need to be reused by several different programs, it is good to have them in separate files so they can be compiled and tested separately.

Task 3. *Add colors in the color module.* In the single object `Color`, we will create colors using the Java class `java.awt.Color`. Since our single object name conflicts with the name of the Java color class, we rename the Java class to `JColor` in the import declaration.

a) Insert an import declaration with the renaming immediately after the package declaration. We place the import so that it is visible throughout the package because several objects need access to `JColor`. Ensure the code still compiles without errors.

b) Then create the following colors in the object `Color`:

```
object Color:
  val black  = new JColor(  0,   0,   0)
  val mole   = new JColor( 51,  51,   0)
  val soil   = new JColor(153, 102,  51)
  val tunnel = new JColor(204, 153, 102)
  val grass  = new JColor( 25, 130,  35)
```

Task 4. *Create a drawing window in the block graphics module.* Add the following three variables to the single object `BlockWindow`:

```
  val windowSize = (30, 50)  // (width, height) in number of blocks
  val blockSize  = 10        // number of pixels per block

  val window = new PixelWindow(???, ???, ???)
```

- Import `introprog.PixelWindow` locally inside `BlockWindow`. (A local import declaration is good here since only this object needs access to `PixelWindow`.)
- Make the size of `window` correspond to the block size times width respectively height in `windowSize`.
- Give the window an appropriate title, e.g., `"Digging Blockmole"`.
- When compiling, you need to make sure that `introprog` is available on the `classpath` (see exercise `Excercise04`).
- If you forgot the parameter order to the class `PixelWindow` check the documentation for `PixelWindow` [5]. Use named arguments when creating the window. Do you think the code becomes more readable with named arguments? [6]

To test the window, add a simple test drawing by using `BlockWindow.window` in procedure `drawWorld`, e.g.:

```
  def drawWorld(): Unit =
    BlockWindow.window.line(100, 10, 200, 20)
```

Compile and run and ensure everything works as expected.

Task 5. *Create procedure for block graphics.* Now you have made a graphical program, but no mole is yet visible. It is time to create the coordinate system in the blockmole's block world.

[5] https://fileadmin.cs.lth.se/pgk/api

[6] Unfortunately you cannot use named arguments when instantiating Java classes in Scala, but PixelWindow is implemented in Scala so here it works fine.

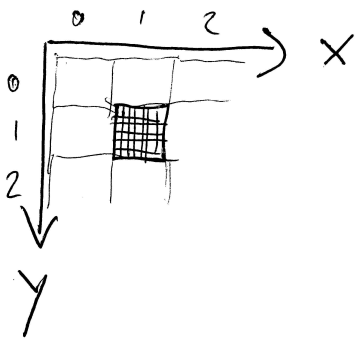


Figure 4.1: Each block consists of many pixels. The marked block has coordinate (1,1) in block coordinates, while the block's top-left pixel has coordinate (7,7) in `PixelWindow` coordinates, if e.g. there are seven-by-seven pixels per block. What is the block coordinate of the block to the right of the marked block in the picture? What are its `PixelWindow` coordinates for the top-left and bottom-right pixels?

a) Make sure you can explain how coordinates in a `PixelWindow` are interpreted, by drawing a simple sketch on paper showing roughly where the positions $(0,0)$, $(300,0)$, $(0,300)$ and $(300,300)$ are in a window that is 300 pixels wide and 500 pixels high. Use figure 4.1 to explain the relationship between underlying window coordinates and block coordinates. Note that the y-axis points downward.

b) Coordinates in `BlockWindow` shall have square, *large* pixels that consist of many window pixels. We call these large pixels *blocks* to distinguish them from the one-pixel image points in `PixelWindow`.
In the block coordinate system for `BlockWindow`, the following applies:

> *The block size* indicates the side length in the square for a block counted in the number of pixels. If the block size is b, then coordinate (x,y) in `BlockWindow` lies at coordinate (bx,by) in `PixelWindow`.

Implement the function `block` in the module `BlockWindow` as below, so that a square is drawn when the procedure is called. The parameter `pos` indicates the block coordinate and the parameter `color` indicates the color. The type-alias declaration of `Pos` gives a descriptive type name for a 2-tuple of integers, which we can use in parameter lists to denote positions in a `BlockWindow`. See the documentation of the `fill` method in `PixelWindow`. Note that you need to convert the block coordinates in `pos` to window coordinates in `window.fill`. Fill in the missing parts below.

```
type Pos = (Int, Int)

def block(pos: Pos)(color: JColor = JColor.gray): Unit =
  val x = ??? // calculate block's x-coordinate in pixel window
  val y = ??? // calculate block's y-coordinate in pixel window
  window.fill(???)
```

Make sure the code compiles without errors.

c) To test your procedure, call the function `BlockWindow.block` several times in `Main.drawWorld`, both with the default argument omitted and with different colors

from the color module. Compile and run your program and verify that everything works as it should.

Task 6. *Create rectangle procedure and underground.* Now you shall write a procedure named `rectangle` that draws a rectangle using the `block` procedure. Then you shall use `rectangle` in `Main.drawWorld` to draw the mole's underground world.

a) Add the procedure `rectangle` in the graphics module. The procedure header should have the following parameters split into three different parameter lists, and return type `Unit`:

```
(leftTop: Pos)(size: (Int, Int))(color: JColor = JColor.gray)
```

The parameter `leftTop` indicates the block coordinate for the rectangle's upper left corner, and `size` indicates `(width, height)` expressed in number of blocks.
Use this nested repetition to draw the rectangle:

```
for y <- ??? do
  for x <- ??? do
    block(x, y)(color)
```

✎ b) In which order are the blocks in the rectangle drawn (vertically or horizontally)? If you are unsure, you can insert a printout of `(x, y)` in the innermost loop to see the order.

c) Add the following code in `Main.drawWorld` so the program draws the underground (that is, lots of soil where the blockmole can dig its tunnels) and some grass.

```
def drawWorld(): Unit =
  BlockWindow.rectangle(0, 0)(size = (30, 4))(Color.grass)
  BlockWindow.rectangle(0, 4)(size = (30, 46))(Color.soil)
```

d) Call `Main.drawWorld` in `Main.run` and test that it works. If any part of the window remains black instead of grass or soil color, check that `block` and `rectangle` are correctly implemented.

Task 7. In `PixelWindow` there are functions to detect keystrokes and mouse clicks. You shall use those functions to control a blockmole. Study the documentation for `awaitEvent` and `Event` in `PixelWindow`, as well as the code in the example program `TestPixelWindow` in the package `introprog.examples`.

a) Add this function in `BlockWindow`:

```
  val maxWaitMillis = 10

  def waitForKey(): String =
    window.awaitEvent(maxWaitMillis)
    while window.lastEventType != PixelWindow.Event.KeyPressed do
      window.awaitEvent(maxWaitMillis) // skip other events
    println(s"KeyPressed: ${window.lastKey}")
    window.lastKey
```

There are different kinds of events a `PixelWindow` can respond to, like keystrokes and mouse clicks. The function you just added waits for an event in your `PixelWindow` using (`window.awaitEvent`) until a keypress event (`KEY_EVENT`) comes. When a keypress

arrives, `window.lastKey` is called to find out which letter or key it was, and this result is also the output of `waitForKey`, since it is the last expression in the block.

b) Extend the procedure `Mole.dig` as below:

```
def dig(): Unit =
  var x = BlockWindow.windowSize._1 / 2
  var y = BlockWindow.windowSize._2 / 2
  var quit = false
  while !quit do
    BlockWindow.block(x, y)(Color.mole)
    val key = BlockWindow.waitForKey()
    if      key == "w" then ???
    else if key == "a" then ???
    else if key == "s" then ???
    else if key == "d" then ???
    else if key == "q" then quit = true
  end while
```

c) Fill in all `???` so that `'w'` moves the mole one step upward, `'a'` one step to the left, `'s'` one step downward and `'d'` one step to the right.

d) Check that `main` only contains two calls: one to `drawWorld` and one to `dig`. Compile and run your program to see if the program responds to keys w, a, s and d.

e) If the program works, there will be many moles forming a long worm, which is a bit odd. Add a call in `Mole.dig` that draws a piece of tunnel at position (x, y) after the call to `BlockWindow.waitForKey` but before the **`if`**-statements. Compile and run your program to dig tunnels with your blockmole.

4.3.3 Control questions

Review the theory of this week and be prepared to answer these questions when it's your turn to report what you have done during the lab:

1. How do the mole's coordinates change when it moves upward on the screen?
2. How are colors represented with RGB?
3. What is a tuple and how are tuples used in this lab?
4. What does dot notation mean?
5. Give an example of using **`import`** and explain what happens.
6. What is the advantage of shadowing and local names?
7. We used several single objects as so-called `modules` in this lab. What is the advantage of dividing the code into modules?
8. Go through the goals of the lab and check that you have met them.

4.3.4 Optional extra tasks

Task 8. The mole can currently dig outside the window. Add some **`if`** statements at the start of the **`while`** loop which detects if `x` or `y` is outside the window's edge and move the mole back just inside the edge if so.

Task 9. Moles are not so interested in life above ground, but it might be nice to see how far down the mole has dug. Add a sky color in the object `Color` and draw sky above the grass in `Mole.drawWorld`. Also adjust what you did in the previous task so that the mole stays on the ground. *Tip:* You can benefit from an interactive color picker that you get by calling `introprog.Dialog.selectColor()` in Scala REPL.

Task 10. Change so that the mole does not leave a tunnel behind it when it runs on grass.

Task 11. Change your `rectangle` method so that it draws the same rectangle but *without* using nested loops. This can be accomplished by a call to `PixelWindow.fill`.

Task 12. Let the mole keep digging even if no key is pressed. Keystroke shall change the direction.

a) Create a new method `BlockWindow.waitForKeyNonBlocking` that allows non-blocking keyboard reading as below:

```
def waitForKeyNonBlocking(): String  =
  import PixelWindow.Event.{KeyPressed, Undefined}

  window.awaitEvent(maxWaitMillis)
  while
    window.lastEventType != KeyPressed  &&
    window.lastEventType != Undefined)
  do window.awaitEvent(maxWaitMillis)
  if window.lastEventType == KeyPressed then window.lastKey else ""
```

b) Add a new method `BlockWindow.delay` that should make it possible to slow down the blockmole so it does not run too fast:

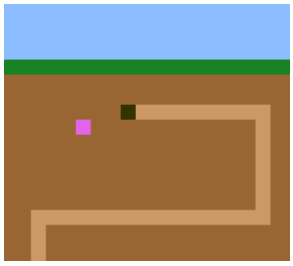
```
def delay(millis: Int): Unit = Thread.sleep(millis)
```

c) Create a new method `Mole.keepOnDigging` that initially is a copy of the method `dig`. Make the following additions/changes:

1. Add two variables `var dx` and `var dy` at the beginning, which keep track of the direction the blockmole digs. Initialize them to `0` and `1` respectively.
2. Add a delay of 200 milliseconds in the infinite loop. Declare a constant `delayMillis` at an appropriate place in `Mole` and use this constant as argument to `delay`.
3. Call `waitForKeyNonBlocking` instead of `waitForKey` and check for key presses like in the code skeleton below. Fill in the missing parts so that the blockmole moves one step in the correct direction each loop iteration.

```
      if      key == "w" then { dy = -1; dx = 0 }
      else if key == "a" then { ??? }
      else if key == "s" then { ??? }
      else if key == "d" then { ??? }
      else if key == "q" then { quit = true }
      y += ???
      x += ???
```

Task 13. *Catch the blockworm.*



Blockworm (*Lumbricus quadratus*) is a fantasy creature in the earthworm family. Known for its ability to teleport instantly from one place to another, making it hard to catch. It has the same RGB color (225, 100, 235) as the real earthworm (*Lumbricus terrestris*) but is square and exactly one block in size. The blockworm is a coveted prey among blockmoles.

a) Add the module `Worm` below in your code and use the procedures in `keepOnDigging` so the blockmole gets a blockworm to chase.

```
object Worm:
  import BlockWindow.Pos

  def nextRandomPos(): Pos =
    import scala.util.Random.nextInt
    val x = nextInt(BlockWindow.windowSize._1)
    val y = nextInt(BlockWindow.windowSize._2 - 7) + 7
    (x, y)

  var pos = nextRandomPos()

  def isHere(p: Pos): Boolean = pos == p

  def draw(): Unit  = BlockWindow.block(pos)(Color.worm)

  def erase(): Unit = BlockWindow.block(pos)(Color.soil)

  val teleportProbability = 0.02

  def randomTeleport(notHere: Pos): Unit =
    if math.random() < Worm.teleportProbability then
      erase()
      while
        pos = nextRandomPos()
        pos == notHere
      do ()
      draw()

end Worm
```

b) The code in `Worm` assumes that sky exists in the top 7 blocks of the window. How many blocks are sky can ideally be a constant with a good name in a good place. This constant should be used also in `drawWorld`. Fix that!

c) Make the text `"WORM CAUGHT!"` print to the terminal if the blockmole is at the same place as the blockworm.

d) Use the parameter `notHere` to prevent the blockworm from teleporting to the same spot as the blockmole.

e) Make the blockmole get 1000 points each time it catches the blockworm.

f) Make the game last a certain reasonable amount of time before `Game Over`. Use `System.currentTimeMillis` which gives the current number of milliseconds since January 1, 1970. When the game ends, print out the total points the blockmole has accumulated to the terminal.

g) Make the game speed up (i.e. the delay in the game loop decreases) after some time. Concurrently, the probability that the blockworm teleports also increases.

Chapter 5

Classes and Data Modeling

Concepts included in this week's studies:

- ☐ application domain
- ☐ data model
- ☐ object-orientation
- ☐ class
- ☐ instance
- ☐ Any
- ☐ isInstanceOf
- ☐ toString
- ☐ new
- ☐ null
- ☐ this
- ☐ access rules
- ☐ private
- ☐ private[this]
- ☐ class parameter
- ☐ primary constructor
- ☐ factory method
- ☐ alternative constructor
- ☐ mutable
- ☐ immutable
- ☐ case class
- ☐ companion object
- ☐ reference equality
- ☐ structural equality
- ☐ eq
- ☐ ==

5.1 Theory

The concept of **class** is an important abstraction mechanism in **object-oriented programming** (OOP) to model data in an application domain, e.g. data about *users* and their *favorite music* in the application domain *music player*. Classes are used to collect functions and data. A class has a name and can have parameters. A class is declared with the keyword `class` and is a description of how a certain type of object should be designed when they are eventually created. It is possible to create **many** objects from the same class.

5.1.1 A metaphor for class: Stamp

A class is like a **stamp**.

- A stamp can be **manufactured** – corresponds to **declaration** of the class.
- Nothing happens until you **stamp** – corresponds to **instantiation**.
- Then **impressions** of the stamp are created – corresponds to **allocation of an object** which is an **instance** of the class.
- Allocation is also called **construction** and the function/code that does the allocation is called the **constructor**.

5.1.2 What is a class?

- A class is a template (eng. *template*) for creating objects.
- Objects can be created with `new ClassName(parameters)`, which is called **instantiation**.
- In Scala 3, `new` is optional; `ClassName(parameters)` is sufficient.
- An object created with the class `ClassName` as template is called an **instance** of the class `ClassName`.
- A class contains **members**, which can include:
  - **attributes**, also called fields: `val`, `lazy val`, `var`
  - **methods**, also called operations: `def`
- Each instance has its **own** set of attribute values, which together constitute the instance's **state**.

5.1.3 Data modeling

Why are classes needed?

- In a certain **application domain**, e.g., the tax authority's declaration system, a **model of domain-specific data** is needed, e.g., persons, personal identity numbers, addresses, incomes, deductions, properties, etc.
- With classes you can create **new** types (beyond `Int`, `String` ...) that better represent the domain's data.
- With classes you implement models representing essential **attributes** from the application domain.
- With **methods** (functions in classes) you can create and process domain data.
- Data modeling in Scala is often done with so-called **`case`** classes and **immutable** instances.

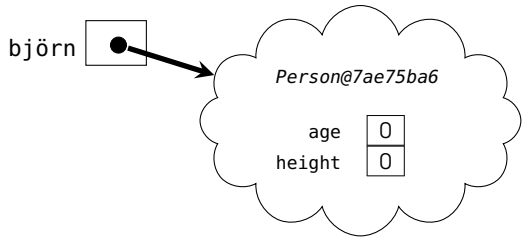
5.1.4 Singleton object compared to class

We have previously declared **singleton objects** that only exist in a **single** copy:

```
scala> object Björn { var age = 54; val height = 178 }
```

With a **class** you can create **arbitrary many instances of the class** using the keyword **`new`** followed by the class name:

```
scala> class Person { var age = 0; var height = 0 }

scala> val björn = new Person   // allocate space in memory
björn: Person = Person@7ae75ba6  // unique id for the instance
```



5.1.5 Reference equality or content equality?

There are two **fundamentally different** kinds of **equality**:

- **Reference equality**: two references are considered equal if they refer to the **same instance** in memory.
- **Content equality**, also called structural equality: two references are considered equal if they refer to objects with the **same content**.
- In Scala there are several methods that test equality:
  - the method `eq` tests **reference equality** and `r1.eq(r2)` yields **`true`** if `r1` and `r2` refer to the **same** instance.
  - the method `ne` tests reference **inequality** and `r1.ne(r2)` yields **`true`** if `r1` and `r2` refer to **different** instances.
  - the method `==`, which calls the method `equals`, by default tests reference equality with `eq` but which **can be overridden** if you **want to determine** whether it should be reference equality or structural equality.

- Scala's **standard library** and **primitive types** such as `Int`, `String`, etc., test **content equality** using the method ==.

5.1.6 Example: reference equality and content equality

In Scala's standard library, `equals` has been overridden so that the method == tests for **content equality** between instances:

```
scala> val v1 = Vector(1,2,3)
v1: scala.collection.immutable.Vector[Int] = Vector(1, 2, 3)

scala> val v2 = Vector(1,2,3)
v2: scala.collection.immutable.Vector[Int] = Vector(1, 2, 3)

scala> v1 eq v2                // reference equality test: different instances
res0: Boolean = false

scala> v1 ne v2
res1: Boolean = true

scala> v1 == v2                // content equality test: same content
res2: Boolean = true

scala> v1 != v2
res3: Boolean = false
```

5.1.7 Reference equality and custom classes

If you do nothing special with your own classes, the method == tests **reference equality** between instances:

```
scala> class Cucumber(val weight: Int)

scala> val g1 = new Cucumber(42)
g1: Cucumber = Cucumber@2cc61b3b

scala> val g2 = new Cucumber(42)
g2: Cucumber = Cucumber@163df259

scala> g1 == g2       // same content but different instances
res0: Boolean = false

scala> g1.weight == g2.weight
res1: Boolean = true
```

5.1.8 Case classes provide content equality

Besides other "free goodies", you also get this with **case class**:

- The method == provides **content equality** (and not reference equality).

5.1.9 Equality and case classes

The method `equals` is automatically overridden in case classes so that the method == tests structural equality.

```
scala> case class Cucumber(weight: Int)

scala> val c1 = Cucumber(42)
c1: Cucumber = Cucumber(42)

scala> val c2 = Cucumber(42)
c2: Cucumber = Cucumber(42)

scala> c1 eq c2          // different instances
res0: Boolean = false

scala> c1 == c2          // same content!
res1: Boolean = true
```

5.1.10 Summary of case-class goodies

Checklist of "goodies" in **case** classes so far:

1. class parameters become **val** attributes
2. nice toString
3. automatic factory method `apply` in companion object
4. == provides content equality (eng. *structural equality*)

   ...

But we haven't seen all the goodies yet:
Pattern matching (more on that later).

5.1.11 Implementation missing: ???

- Often one wants to build code iteratively and step by step add various functionality.
- The standard function `???` raises the exception **NotImplementedError** when called and can be used in places in the code where one is not yet finished.
- `???` allows **compilation of incomplete code**.
- Exceptions have the bottom type `Nothing` which is a subtype of *all* types and can therefore be assigned to references of any type.

```
scala> lazy val willExplodeSoon: Int = ???

scala> willExplodeSoon + 42
scala.NotImplementedError: an implementation is missing
```

5.1.12 Example: incomplete code

```scala
case class Person(name: String, age: Int):
  def isTeenager = age >= 13 && age <= 19
  def isYoung = !isOld
  def isOld: Boolean = ???   // implementation not yet finished
```

```
scala> Person("Björn", 51).isTeenager
res23: Boolean = false

scala> Person("Sandra", 39).isYoung
scala.NotImplementedError: an implementation is missing
```

5.2 Exercise Excercise05

Goals

- ☐ Be able to declare classes with class parameters.
- ☐ Be able to create instances with and without **new**.
- ☐ Be able to provide arguments during instantiation.
- ☐ Understand the meaning of reference variables and the value **null**.
- ☐ Be able to use the keyword **private** to control the visibility of attributes and constructor parameters.
- ☐ Understand the purpose of getters and setters.
- ☐ Be able to explain access rules for companion objects.
- ☐ Be able to create a factory method in a companion object.
- ☐ Be aware of the usefulness of a private constructor.
- ☐ Understand the difference between reference equality and structural equality.
- ☐ Be aware of the difference between == and eq, as well as != and ne.
- ☐ Be able to explain how case classes handle instantiation.
- ☐ Be aware of how case classes handle equality.

Preparations

- ☐ Study the concepts in chapter 5

5.2.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with description.*

Connect each concept with the (simplified) description that fits best:

class	1	A	indirect assignment to attribute value
instance	2	B	instances are considered different even if states are equal
constructor	3	C	keyword for direct instance creation of a class
class parameter	4	D	sees private members of class with same name
reference equality	5	E	helper function for indirect construction
structural equality	6	F	no need to write new; automatic structural equality
case class	7	G	a value that does not refer to any instance
getter	8	H	template for creating multiple instances of the same type
setter	9	I	copy of an object with its own state memory
companion object	10	J	instances considered equal if they have the same state
factory method	11	K	binds to arguments given at construction
null	12	L	indirect access to attribute value
new	13	M	creates instance, allocates space for state memory

Task 2. *Class and instance.* You have in exercise Excercise04 seen how singleton objects in their own namespace can collect functions (methods) and have state (attributes). But singleton objects exist only in one copy. If you want to create many objects of the same type you need a *class*. An object instance created from a class is

called an *instance* of the class. Each instance has its own state. Declare the singleton object and the class below and paste into REPL.

```
object Singelpunkt { var x = 1; var y = 2 }
class  Punkt       { var x = 3; var y = 2 }
```

a) Suppose the expressions on the left are evaluated from top to bottom. Which result on the right corresponds to each expression? Try in REPL if you are unsure.[1]

`Singelpunkt.x`	1	A	`java.lang.NullPointerException`
`Punkt.x`	2	B	`1`
`val p  = new Singelpunkt`	3	C	`Not found: type`
`val p1 = new Punkt`	4	D	`p1: Punkt = Punkt@27a1a53c`
`val p2 = Punkt()`	5	E	`3`
`{ p1.x = 1; p2.x }`	6	F	`p2: Punkt = Punkt@51ab04bd`
`(new Punkt).y`	7	G	`value is not a member of object`
`{ val p: Punkt = null; p.x }`	8	H	`2`

b) In three cases there is an error. Why? Is it a compile-time error or runtime error?

Task 3. *Class parameters.* The class Punkt in the previous task is not so handy to use because you *after* instantiation can only give the attributes `x` and `y` the coordinate values you want, and this must be done with explicit assignment statements.
You can solve this problem with *class parameters* that allow you to initialize attributes with constructor arguments and thereby specify an initial state directly at instantiation.
Declare the class below in the REPL.

```
class Point(var x: Int, var y: Int)
```

a) Suppose the expressions on the left are evaluated from top to bottom. Which result on the right corresponds to each expression? Try in REPL if you are unsure.

`val p1 = Point(1, 2)`	1	A	`missing argument for parameter`
`val p2 = Point()`	2	B	`2`
`val p2 = Point(3, 4)`	3	C	`p1: Point = Point@30ef773e`
`p2.x - p1.x`	4	D	`too many arguments for constructor`
`Point(0, 1).y`	5	E	`p2: Point = Point@218cf600`
`Point(0, 1, 2)`	6	F	`1`

b) In two cases there is an error. Why? Is it a compile-time error or runtime error?

Task 4. *Immutable class with default arguments.* What applies to parameters and arguments to functions also applies to class parameters, e.g. default arguments and

[1] The string after the `@`-sign is a hexadecimal representation of the integer assigned to each object so that the system can distinguish different instances. https://stackoverflow.com/questions/4712139

named arguments. One can *additionally* put the keywords **var** and **val** before class parameters and then the parameter becomes a visible attribute. If one wants private attributes one can specify e.g. **private val** before the class parameter name. If nothing is specified before a class parameter, the most restrictive visibility **private**[this] **val** applies, meaning that the name is only visible within the current instance[2].
Declare below the class in REPL.

```
class Point3D(val x: Int = 0, val y: Int = 0, z: Int = 0)
```

a) Suppose the expressions on the left are evaluated from top to bottom. Which result on the right corresponds to each expression? Try in REPL if you are unsure.

`val p1 = Point3D()`	1	A	`false`
`val p2 = Point3D(y = 1)`	2	B	`Reassignment to val`
`Point3D(z = 2).z`	3	C	`p1: Point3D = Point3D@2eb37eee`
`p2.y = 0`	4	D	`true`
`p2.y == 0`	5	E	`value cannot be accessed`
`p1.x == Point3D().x`	6	F	`p2: Point3D = Point3D@65a9e8d7`

b) What is the problem with the above class if you want to use it to represent points in 3 dimensions?

Task 5. *Case class, `this`, equality, `toString` and companion object.*
Paste the classes below into REPL.

```
case class Pt(x: Int = 0, y: Int = 0):
  def moved(dx: Int = 0, dy: Int = 0): Pt = Pt(x + dx, y + dy)

class MutablePt(private var p: (Int, Int) = (0, 0)):
  def x: Int = p._1
  def y: Int = p._2
  def move(dx: Int = 0, dy: Int = 0) = { p = (x + dx, y + dy); this }
  override def toString = s"MPt($x,$y)"
```

a) Suppose the expressions on the left are evaluated from top to bottom. Which REPL responses on the right correspond to each expression? Try in REPL if you are unsure.

`val p1 = Pt(1, 2)`	1	A	`MPt(5,6)`
`val p2 = Pt(y = 3)`	2	B	`false`
`val p3 = MutablePt(5, 6)`	3	C	`Pt(0,3)`
`val p4 = Mutable()`	4	D	`Not found`
`p2.moved(dx = 1) == Pt(1, 3)`	5	E	`Pt(1,2)`
`p3.move(dy = 1) == MutablePt(5, 7)`	6	F	`true`

[2]For case classes, as we shall soon see, **val** applies meaning visibility and immutability (and not **private**[this] **val**).

b) What return type will the compiler infer here for the function `MutablePt.move`?

c) What is the difference between instantiation with universal apply methods and instantiation with **`new`**? Is there any case where **`new`** must be used?

d) What are methods like **`def`** `x` and **`def`** `y` above called?

Task 6. *Implement parts of the classes `Pos`, `KeyControl`, `Mole` and `BlockWindow` needed during the lab `blockbattle1`.* In the next lab you will build upon the `blockmole` lab and make a game for two players where each player controls their *own* instance of a `blockmole`. We must then change `Mole` so that it becomes a class instead of a singleton object. Complete the classes below and test carefully so that they work.
All classes should belong to **`package`** `blockbattle` and be in separate files with the same name as the class, e.g. `Pos.scala`.
Tip: Keep a separate terminal window open and run Scala CLI with change watching according to the command below. Then your changed code is automatically compiled each time you save a scala file in the current directory.

```
scala compile . --watch
```

The option `--watch` can be shortened to `-w` instead.

a) During the lab it is convenient to be able to represent movable positions in a pixel window. Implement the case class `Pos` in a new terminal window as below so that it works according to the following REPL tests.

```
package blockbattle

case class Pos(x: Int, y: Int):
  def moved(delta: (Int, Int)): Pos = ???
```

Test that `Pos` works with REPL as below:

```
> scala repl .
Welcome to Scala 3.3.0 (17.0.6, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> blockbattle.Pos(1,2)
val res0: blockbattle.Pos = Pos(1,2)

scala> import blockbattle.*

scala> val p = Pos(1,2)
val p: blockbattle.Pos = Pos(1,2)

scala> p.moved(0,1)
val res1: blockbattle.Pos = Pos(1,3)
```

Also test calling `moved` on the class name, e.g. `Pos.moved(0,1)`. Does this work? Why/why not? How do method calls differ on methods in singleton objects vs. class instances?

b) During the lab it is convenient to be able to represent which keys correspond to the different directions a user can control their mole in. Complete the case class `KeyControl` as below so that it works according to the following REPL tests. The method `direction` should give a delta step in the correct `(x, y)` direction for a given key name. The method `has` should return **`true`** if the key name exists in any of the four directional keys in this `KeyControl` instance, otherwise **`false`**.

```scala
package blockbattle

case class KeyControl(left: String, right: String, up: String, down: String):
  def direction(key: String): (Int, Int) = ???

  def has(key: String): Boolean = ???
```

```
scala> import blockbattle.*

scala> val kc1 = KeyControl(right="d",left="a",up="w",down="s")
val kc1: blockbattle.KeyControl = KeyControl(a,d,w,s)

scala> val kc2 = KeyControl("Left","Right","Up","Down")
val kc2: blockbattle.KeyControl = KeyControl(Left,Right,Up,Down)

scala> kc2.left
val res0: String = Left

scala> kc2.has("a")
val res1: Boolean = false

scala> kc2.has("Up")
val res2: Boolean = true

scala> kc1.direction("a")
val res3: (Int, Int) = (-1,0)

scala> kc1.direction("s")
val res4: (Int, Int) = (0,1)

scala> kc1.direction("d")
val res5: (Int, Int) = (1,0)

scala> kc1.direction("w")
val res6: (Int, Int) = (0,-1)

scala> Pos(1,2).moved(kc1.direction("a"))
val res7: blockbattle.Pos = Pos(0,2)
```

c) Complete the class `Mole` as below. Mole is a class representing a block mole with mutable attributes for position, direction and score. Each instance also has immutable attributes that keep track of its name, color and which keys can be used to control the mole. Implement the class members one by one and test carefully with suitable test cases after each addition/bugfix. Create a main program e.g. in the file `Main.scala` with your tests creating instances and printing out attributes etc.

```scala
package blockbattle

class Mole(
  val name: String,
  var pos: Pos,
  var dir: (Int, Int),
  val color: java.awt.Color,
  val keyControl: KeyControl
):
```

```
  var points = 0

  override def toString =
    s"Mole[name=$name, pos=$pos, dir=$dir, points=$points]"

  /** If keyControl.has(key) then the direction dir is updated according to keyControl */
  def setDir(key: String): Unit = ???

  /** Updates dir to the opposite direction. */
  def reverseDir(): Unit = ???

  /** Updates pos to become nextPos */
  def move(): Unit = ???

  /** Returns the next position according to the direction dir without updating pos */
  def nextPos: Pos = ???
```

d) During the lab you need a class `blockbattle.BlockWindow` which with the help of `introprog.PixelWindow` offers block graphics. Each instance of `BlockWindow` should have an attribute that refers to a `PixelWindow` instance. This is called **aggregation** (eng. *aggregation*).[3]
To be able to compile and test your `BlockWindow` class you need to have the `introprog` package on the classpath. Download the file https://fileadmin.cs.lth.se/introprog.jar via your browser or with the command `curl` below (note the capital letter `O` and not a zero in option `-sLO`):

```
curl -o introprog.jar -sLO  https://fileadmin.cs.lth.se/introprog.jar
scala run . --jar introprog.jar
```

Then `introprog.jar` is automatically on the classpath.
Complete the class `BlockWindow` below. The method `setBlock` should using the method `pixelWindow.fill` fill a square area with side `blockSize` pixels at a certain position `pos` in block coordinates and with a certain color `color`. The method `getBlock` should using the method `pixelWindow.getPixel` return the color for the upper-left corner in the block at position `pos` in block coordinates.

```
package blockbattle

class BlockWindow(
  val nbrOfBlocks: (Int, Int),
  val title: String = "BLOCK WINDOW",
  val blockSize: Int = 14
):
  import introprog.PixelWindow

  val pixelWindow = new PixelWindow(
    nbrOfBlocks._1 * blockSize, nbrOfBlocks._2 * blockSize, title)

  def setBlock(pos: Pos, color: java.awt.Color): Unit = ???

  def getBlock(pos: Pos): java.awt.Color = ???

  def write(
    text: String,
```

[3] https://en.wikipedia.org/wiki/Object_composition#Aggregation

```
    pos: Pos,
    color: java.awt.Color,
    textSize: Int = blockSize
  ): Unit = pixelWindow.drawText(
      text, pos.x * blockSize, pos.y * blockSize, color, textSize)

  def nextEvent(maxWaitMillis: Int = 10): BlockWindow.Event.EventType  =
    import BlockWindow.Event._
    pixelWindow.awaitEvent(maxWaitMillis)
    pixelWindow.lastEventType match
      case PixelWindow.Event.KeyPressed   => KeyPressed(pixelWindow.lastKey)
      case PixelWindow.Event.WindowClosed => WindowClosed
      case _                              => Undefined

object BlockWindow:
  def delay(millis: Int): Unit = Thread.sleep(millis)

  object Event:
    trait EventType
    case class  KeyPressed(key: String) extends EventType
    case object WindowClosed            extends EventType
    case object Undefined               extends EventType
```

In the instructions to lab `blockbattle1` there are tips on how you can handle events in a `BlockWindow` using the method `nextEvent` above.

e) Make the main program in `Main.scala` draw some arbitrary blocks in an instance of the class `BlockWindow`. Also create a **`while`** `(!quit)` loop which using `nextEvent()` prints events to the terminal that are not of type `Undefined`.
The method `nextEvent()` is in the class `BlockWindow`. Each loop round should also contain a `200` milliseconds delay by calling the `delay` method defined in the companion object `BlockWindow` above. If the event `WindowClosed` occurs the loop should terminate. Run the main program and check that the result is as expected.

5.2.2 Extra Tasks; additional practice

Task 7. *Instantiation with* ***`new`*** *and the value* ***`null`***. Instances of classes are created with **`new`**. Then the constructor is called and space is reserved in the computer's memory for the object. Variables of reference type that do not refer to any object have the value **`null`**.

a) What happens below? Which lines give error messages and if so, what does the error say?

```
1 scala> class Cucumber(val weight: Int)
2 scala> var g: Cucumber = null
3 scala> g.weight
4 scala> g = new Cucumber(42)
5 scala> g.weight
6 scala> g = null
7 scala> g.weight
```

b) Draw the memory situation after lines 2, 4, 6.

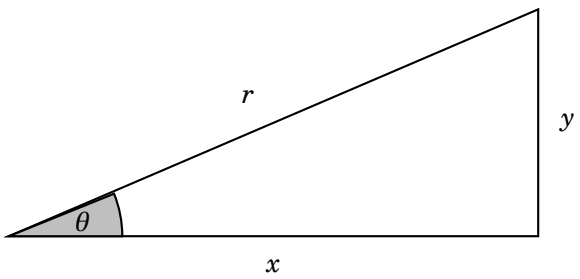
Task 8. *Create a point class that can handle polar coordinates and a class that represents a polygon using these points.* You shall create an immutable case class `Point` which can represent a coordinate both with "normal" Cartesian coordinates[4] and with polar coordinates [5]. Then you shall use this class to create regular polygons with an immutable case class `Polygon`.

a) Create code with the help of an editor, e.g. VS Code, in the file `Point.scala` following these guidelines:

1. `Point` shall belong to the package `graphics`.
2. `Point` shall have the following two public, immutable class parameters:
   - `x: Double` for the x-coordinate.
   - `y: Double` for the y-coordinate.
3. `Point` shall have the following public members (two immutable attributes and one method):
   - `val r: Double` shall give the corresponding polar coordinate's distance to origin.
   - `val theta: Double` shall give the polar coordinate's angle in radians.
   - `def +(p: Point): Point` shall give a new point whose coordinate is the sum of the x and y coordinates of this instance and point `p`.
4. `Point` shall have a companion object with a method that constructs a point from polar coordinates. The method shall have this header:
   `def polar(r: Double, theta: Double): Point`

Tips:

- You will find useful the methods $r =$ `math.hypot(x, y)` and $\theta =$ `math.atan2(y, x)` when converting to polar coordinates:



- You will find useful the methods `math.cos(theta)` and `math.sin(theta)` when converting from polar coordinates.

- Note that the class's attributes are of type `Double` and not `Int`, even though later you will use the point to describe a discrete pixel position in a `PixelWindow`. The reason for this is that rounding errors may occur during numerical computations. This is particularly noticeable during repeated calculations with small values, e.g. when drawing an approximated circle with many line segments.

b) The class `PolygonWindow` below contains a `PixelWindow` and provides the ability to draw polygons. Copy the code for `PolygonWindow` to a new code file `PolygonWindow.scala` in the same directory as your `Point` above.

[4] https://sv.wikipedia.org/wiki/Kartesiskt_koordinatsystem
[5] https://sv.wikipedia.org/wiki/Pol%C3%A4ra_koordinater

```
//> using dep "se.lth.cs::introprog:1.4.0"
package graphics

import introprog.PixelWindow
import java.awt.Color

extension (p: Point) def toPixels: Seq[Int] =
  Seq(p.x.round.toInt, p.y.round.toInt)

class PolygonWindow:
  val black     = Color(0, 0, 0)
  val coolGreen = Color(0, 255, 111)
  val width     = 500
  val height    = 500

  val window =
    PixelWindow(width, height, title = "Polygons",
                background = black, foreground = coolGreen)

  def draw(polygon: Polygon): Unit =
    for i <- 0 until polygon.nbrOfCorners do
      val from = polygon.points(i).toPixels.map(_ + width / 2)
      val next = polygon.points((i+1) % polygon.nbrOfCorners)
      val to = next.toPixels.map(_ + width / 2)
      window.line(from(0), from(1), to(0), to(1), lineWidth = 2)
```

Create a case class named `Polygon` with a parameter `points: Vector[Point]` and an attribute `val nbrOfCorners: Int`. The case class `Polygon` shall also be in the package `graphics`.
Like the class `Point` above, `Polygon` shall have a companion object. The companion object shall have a method `regular` which creates a regular polygon. The method shall have the following parameters: `nbrOfCorners: Int, radius: Double, midPoint: Point` Consider how the case class `Polygon` and its companion object should look for the code above in `PolygonWindow` to work as intended. Test that everything works in REPL.

c) Can you use the method `regular` to draw circles? Can you use `Polygon` to represent irregular polygons? Test in REPL.

Task 9. *Classes, instances and garbage.* Long ago in a galaxy far far away...

```
case class Arm(isToLeft: Boolean)

case class Leg(isToLeft: Boolean)

case class Head(hasHair: Boolean = true)

case class SpaceCreature(
      arm1:   Arm   = Arm(true),
      arm2:   Arm   = Arm(false),
      leg1:   Leg   = Leg(true),
      leg2:   Leg   = Leg(false),
```

```
      head1: Head = Head(hasHair = false),
  var head2: Head = Head()
):
  def isBald = !head1.hasHair && !head2.hasHair
```

a) Paste the above space code in REPL and evaluate the lines below. Draw the memory situation after line 5 and describe what happens.

```
1 scala> val alien = SpaceCreature()
2 scala> alien.isBald
3 scala> val predator = SpaceCreature()
4 scala> predator.isBald
5 scala> predator.head1 = alien.head1
6 scala> predator.head2 eq alien.head1  // test of reference equality
7 scala> println(predator)
8 scala> predator.isBald
```

b) What happens eventually to the original head2 object in predator after the assignment on line 5? Is it possible to refer to this object in some way?

Task 10. *Case class. Immutable square.*

a) Implement the square below with an editor and paste it in REPL.

```
case class Square(val x: Int = 0, val y: Int = 0, val side: Int = 1):
  val area: Int = ???

  /** Creates a new Square moved to position (x + dx, y + dy) */
  def moved(dx: Int, dy: Int): Square = ???

  def isEqualSizeAs(that: Square): Boolean = ???

  /** Multiplies the side with factor and rounded to nearest integer */
  def scale(factor: Double): Square = ???

object Square:
  /** A Square at (0, 0) with side 1 */
  val unit: Square = ???
```

b) Test your square as below. Explain what happens.

```
1 scala> val (s1, s2) = (Square(), Square(1, 10, 1))
2 scala> val s3 = s1 moved (1,-5)
3 scala> s1 isEqualSizeAs s3
4 scala> s2 isEqualSizeAs s1
5 scala> s1 isEqualSizeAs Square.unit
6 scala> s2.scale(math.Pi) isEqualSizeAs s2
7 scala> s2.scale(math.Pi) isEqualSizeAs s2.scale(math.Pi)
8 scala> s2.scale(math.Pi) eq s2.scale(math.Pi)
9 scala> Square.unit eq Square.unit
```

5.2.3 Advanced Tasks; challenges

Task 11. *Content equality between different types.* Paste below classes into REPL and investigate what happens.

```
class Cucumber(val weight: Int)

class Car(val typ: String)
```

```
scala> class Cucumber(val weight: Int)
     |
     | class Car(val typ: String)
// defined class Cucumber
// defined class Car

scala> 42 == "Fourtytwo"

scala> Cucumber(50) == Car("Sedan")
```

Is there any result that is problematic, and if so, why?

Task 12. *Attribute representation. Private constructor. Factory method.* Kim Kodkunnig long ago created this class which is used in many places in existing code:

```
class Point private (val x: Int, val y: Int)
object Point:
  def apply(x: Int = 0, y: Int = 0): Point = new Point(x, y)
  val origo = apply()
```

a) What happens if you try to instantiate Kim Kodkunnig's class directly with the keyword `new`?

b) Why does Kim Kodkunnig use a companion object with a factory method? What access rules apply between a companion object and a class with the same name?

c) Help Kim Kodkunnig change the attribute representation so that the immutable state consists of a 2-tuple `val p: (Int, Int)` instead. Existing code should not have to change and the class `Point` should behave from the "outside" just like before.

Task 13. *Visibility of class parameters and constructor,* ***`private`****`[this]`*.

a) One of the cucumber classes below is broken. Why and what error do you get?

```
class Cucumber1(weight: Int)

class Cucumber2(val weight: Int)

class Cucumber3(private val weight: Int)

class Cucumber4(private val weight: Int, buddy: Cucumber4):
  def buddyWeight = buddy.weight

class Cucumber5(private[this] val weight: Int, buddy: Cucumber5):
  def buddyWeight = buddy.weight
```

```
class Cucumber6 private (weight: Int)

class Cucumber7 private (var weight: Int)
object Cucumber7:
  def apply(weight: Int) =
    require(weight >= 0, "negative weight: " + weight)
    new Cucumber7(weight)
```

b) Investigate below what effect the keywords **val** and **private** have. Explain what happens. Which lines give which error messages?

```
scala> new Cucumber1(42).weight
scala> new Cucumber2(42).weight
scala> new Cucumber3(42).weight
scala> val noneCucumber: Cucumber4 = null
scala> new Cucumber4(42, noneCucumber).buddyWeight
scala> new Cucumber4(42, new Cucumber4(84, null)).buddyWeight
scala> new Cucumber6(42)
scala> new Cucumber7(-42)
scala> Cucumber7(-42)
scala> val g = Cucumber7(42)
scala> g.weight
scala> g.weight = -1
scala> g.weight
```

Task 14. *User-defined setter combined with private constructor.* Paste this code in REPL:

```
class Cucumber8 private (private var _weight: Int):
  def weight = _weight
  def weight_=(v: Int): Unit =
    require(v >= 0, "negative weight: " +v)
    _weight = v

object Cucumber8:
  def apply(weight: Int) =
    require(weight >= 0, "negative weight: " + weight)
    new Cucumber8(weight)
```

a) Explain what happens below. Which lines produce which error messages?

```
scala> val g = Cucumber8(-42)
scala> val g = Cucumber8(42)
scala> g.weight
scala> g.weight = 0
scala> g.weight = -1
scala> g.weight += 42
scala> g.weight -= 1000
```

b) What is the advantage of being able to create user-defined setters?

Task 15. *Object with mutable state.* You shall implement a model of a jumping frog fulfilling the following requirements:

1. Each frog object shall keep track of where it is.
2. Each frog object shall keep track of how far the frog has jumped in total.
3. Each frog object shall be able to calculate how far it is between the frog's current position and the starting position.
4. All frogs start their jumping at the origin.
5. A frog can jump according to two methods:
   - relative move according to the parameters `dx` and `dy`,
   - random relative move $[1, 10]$ in x-direction change and $[1, 10]$ in y-direction change.

a) Implement the class `Frog` according to the code skeleton below and the above requirements.

```
class Frog private (initX: Int = 0, initY: Int = 0):
  def x: Int = ???
  def y: Int = ???

  def jump(dx: Int, dy: Int): Unit = ???
  def randomJump: Unit = ???

  def distanceToStart: Double = ???
  def distanceJumped: Double = ???
  def distanceTo(that: Frog): Double = ???

object Frog:
  def spawn(): Frog = ???
```

Tips:

- If the name you want to give a private mutable attribute "collides" with a method name, it is common to prefix the attribute name with underscore, e.g. `private var _x` to avoid name conflict.
- Introduce one method at a time and paste the new frog in REPL after each extension and test.

b) Create a method `def test(): Unit` in a singleton object `FrogTest` that contains code that does at least one check for each requirement. If no check fails, `"Test Ok!"` should be printed, otherwise execution should abort. *Tip:* Use `assert(b, msg)` which aborts execution and prints `msg` if `b` is false.

c) What is a method called that only returns the value of a private attribute?

d) Introduce setters for the attributes tracking the x and y position. Changes of position in x- or y-direction should count as a jump and thus be recorded in the attribute that holds the accumulated jump distance.

e) Simulate massive frog jumping with collision detection by creating 100 frogs initially placed on the x-axis at distance 8 units apart. For each round in a `while` loop, let a randomly selected frog perform a `randomJump` until some frog pair is closer than 0.5 length units, which is the definition of collision. Count how many loop rounds are needed before any frog pair collides and print the number. Also print the total number of

Tips: Start with pseudocode on paper. Use a frog vector.

Task 16. *Object with mutable state.* The webshop **UberSquare** sells movable squares. In the business model it includes charging per movement. You shall help UberSquare develop a simple prototype to impress venture capitalists.

a) Implement `Square` according to the documentation comments in the following code sketch and according to these requirements:

1. Each instance of `Square` shall count the number of moves made since the instance was constructed.
2. To monitor their customers UberSquare also want to count the total number of moves made by all squares ever created (so-called *big data*).
3. Each time a move happens a certain amount shall be added to the accumulated cost for the respective square, according to the cost calculation in requirement 4.
4. UberSquare worries that the squares get moved too far away and therefore decides that for each move the accumulated square cost is increased by the new position's distance to the origin at square construction multiplied by the current size of the square.
5. To appear generous UberSquare advertises that scaling squares is free. [6]

```
/** A mutable and expensive Square. */
class Square private (val initX: Int, val initY: Int, val initSide: Int):
  private var nMoves = 0
  private var sumCost = 0.0

  private var _x = initX
  private var _y = initY

  private var _side = initSide

  private def addCost(): Unit =
    sumCost += ???

  def x: Int = ???
  def y: Int = ???

  def side = ???

  /** Scales the side of this square and rounds it to nearest integer */
  def scale(factor: Double): Unit = ???

  /** Moves this square to position (x + xd, y + dy) */
  def move(dx: Int, dy: Int): Unit = ???

  /** Moves this square to position (x, y) */
  def moveTo(x: Int, y: Int): Unit = ???

  /** The accumulated cost of this Square */
  def cost: Double = ???

  /** Returns the accumulated cost. Sets the accumulated cost to zero. */
```

[6] That is, a call to the method `scale` does not cause any immediate cost.

```
  def pay: Double = ???

  override def toString: String =
    s"Square[($x, $y), side: $side, #moves: $nMoves times, cost: $sumCost]"


object Square:
  private var created = Vector[Square]()

  /** Constructs a new Square object at (x, y) with size side */
  def apply(x: Int, y: Int, side: Int): Square =
    require(side >= 0, s"side must be positive: $side")
    ???

  /** Constructs a new Square object at (0, 0) with side 1 */
  def apply(): Square = ???

  /** The total number of moves that have been made for all squares. */
  def totalNumberOfMoves: Int = ???

  /** The total cost of all squares. */
  def totalCost: Double = ???
```

b) Test your square prototype in REPL. Use e.g. the code below:

```
scala> val xs = Vector.fill(10)(Square())
scala> xs.foreach(_.move(2, 3))
scala> xs.foreach(_.scale(2.9))
scala> val (m, c) = (Square.totalNumberOfMoves, Square.totalCost)
val m: Int = 10
val c: Double = 36.055512754639885
```

★ **Task 17.** *Auxiliary constructor.* In previous tasks we have enabled alternative ways to create instances through default arguments and factory methods in companion objects. Another way to do this, which in Scala is unusual[7], is to define several constructors inside the class body. In Scala, such an extra constructor is called an **auxiliary constructor**.

You create an auxiliary constructor in Scala by defining a method with the special name `this`, i.e. a declaration `def this(...) = ...`. Auxiliary constructors must begin by calling another constructor, either the primary constructor (i.e. the one defined in the class header) or a previously defined auxiliary constructor.

a) Read more about auxiliary constructors here:
www.artima.com/pins1ed/functional-objects.html#6.7

b) Invent your own task with auxiliary constructors, based on one of the classes from previous exercises.

[7] But in Java this is very common since default arguments etc. are not part of the language.

5.3 Lab: `blockbattle0`

Preparations

- ☐ Do exercise `Excercise05` in section 5.2.
- ☐ You have two weeks to complete `blockbattle`. Already now, read through all exercises in section 6.3, but first do the basic exercises before starting the lab, especially exercise 6 in section 5.2.

Chapter 6

Patterns and Exception Handling

Concepts included in this week's studies:

- ☐ pattern matching
- ☐ match
- ☐ Option
- ☐ throw
- ☐ try
- ☐ catch
- ☐ Try
- ☐ unapply
- ☐ sealed
- ☐ flatten
- ☐ flatMap
- ☐ partial functions
- ☐ collect
- ☐ wildcard patterns
- ☐ variable binding in patterns
- ☐ sequence wildcard
- ☐ literal patterns
- ☐ implement equals
- ☐ hashCode

6.1 Theory

6.1.1 The base type for all types: `Any`

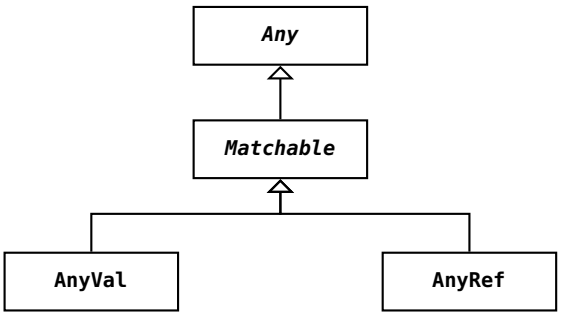
Scala's type system is **complete**:

- Exactly all existing values in Scala are **objects** that have a **type**.
- All types are subtypes of the base type `Any`, and is therefore called **top type**.
- The type `Nothing` is a subtype of all types, and is therefore called **bottom type**.
  No value has only the type `Nothing`; all object types are less specific than that.
- A simplified description of the top type `Any` that gives all objects `toString` etc:

```
trait Any:
  // methods that can be overridden
  def toString: String                        // a string representation
  def equals(other: Any): Boolean = eq(other) // defines equality
  def hashCode: Int                           // should give the same integer if equals is true

  // final methods cannot be overridden:
  final def eq(other: Any): Boolean       // always provides reference equality
  final def ne(other: Any): Boolean = !eq(other)
  final def ==(other: Any) = equals(other)
  final def !=(other: Any) = !equals(other)
  final def isInstanceOf[T]: Boolean      // type test at runtime
  final def asInstanceOf[T]: T            // unsafe type cast at runtime
  final def ## = hashCode                 // different for value types and null
```

https://www.scala-lang.org/api/current/scala/Any.html
(More about base types, traits, equals, hashCode, ... later.)

6.1.2 All types are subtypes of `Any`

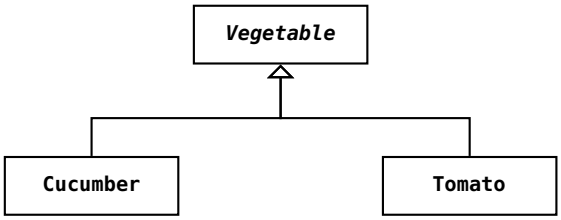


- All **value types**, e.g. `Int`, `Double`, `Boolean`, are subtypes of `AnyVal`
- All **reference types**, e.g. `String` are subtypes of `AnyRef`
- Values of type `Matchable` can be used in so-called pattern matching.
- (There are also so-called opaque types that cannot be pattern matched.)

6.1.3 Your own reference types are subtypes of `AnyRef`

All types you create are subtypes of `AnyRef` without you needing to write it.

```
trait Vegetable:                                    // your own base type
  def weight: Int

case class Cucumber(weight: Int) extends Vegetable    // your own subtype
case class Tomato(weight: Int) extends Vegetable      // another subtype
```



More about type hierarchies and **extends** will come in the week about inheritance.

In a match expression you can match on a certain value or on a certain type and match expressions are often used instead of nested if expressions, as they are often easier to read and understand. With match expressions you can also do **pattern matching** on case class instances, for example to easily check if attributes have special values. Match expressions in Scala are a more powerful version of `switch` statements found in many other languages.

6.1.4 What is matching?

- Matching is done when you want to compare a value against other values and find correspondence (eng. *match*) according to various **patterns**.
- With patterns you can also **decompose** objects into their parts.

6.1.5 Decompose an object into its parts with patterns

```
scala> case class Point(x: Int, y: Int)

scala> val p = Point(1, 2)      // construct a point
val p: Point = Point(1,2)
```

```
scala> val Point(a, b) = p      // decompose a point
val a: Int = 1
val b: Int = 2
```

`Point(a, b)` is called a **constructor pattern**.
The arbitrary names `a` and `b` become new variables.
There are *many* different kinds of patterns.
The most common use of patterns is in **match** expressions.

6.1.6 Check if one fits with nested if expressions

A common problem:
to check which among many values fits

Can be done with nested **if-then-else** expressions:

```
val g = scala.io.StdIn.readLine("Enter a vegetable:")
val taste =
  if      g == "cucumber" then "good!"
  else if g == "tomato"   then "very good!"
  else if g == "broccoli" then "quite good..."
  else "not good :("

println(g + " is " + taste)
```

6.1.7 Check if one fits with `match` expression

Instead of nested **if** you can use Scala's powerful **match expression**:

```
val g = scala.io.StdIn.readLine("Enter a vegetable: ")
val taste =
  g match
    case "cucumber"    => "good!"
    case "tomato"      => "very good!"
    case "broccoli"    => "quite good..."
    case _ => "less good..."
```

- Each **case** branch is tested separately in order **from top to bottom**.
- What is between **case** and **=>** is called a **pattern**.
- If a pattern matches, what is after **=>** is executed.
- **No subsequent case branches are tested after a successful match**.
- Above is an example of matching on **constant patterns**,
  in this case three string constant patterns.
- The last default branch above is called a **wildcard pattern**: **case _ =>**
- There are many other ways to write patterns.

6.1.8 Syntax for match expressions

A **match** expression consists of any number of **case ... => ...**

```
  valueToExamine match {
    case pattern1 => result1
    case pattern2 => result2
    case pattern3 => result3
```

```
    case patternN => resultN
  }
```

- Curly braces after **match** are optional if **case** starts on a new line.
- Each result expression can consist of many lines.
- Curly braces are not needed after **=>** when many lines.

If many lines follow **case**, the last expression is the result.
We will now see examples of many different patterns

6.1.9 Matching with guard

One can insert a so-called **guard** before the arrow **=>** to condition the matching: (note **if** without **then**)

```
val g = scala.io.StdIn.readLine("Enter a vegetable: ")
val taste = g match
  case "cucumber" if math.random() > 0.5 => "good sometimes!"
  case "tomato" => "very good!"
  case "broccoli" => "quite good..."
  case _ => "less good..."
```

The **case** branch with guard only results in a successful match
if the expression after **if** is true; otherwise the next branch is tried, etc.

6.1.10 Matching with variable pattern

If there is a name after **case** that starts with a lowercase letter, this name becomes a variable that is automatically bound to the expression before **match**:

```
val g = scala.io.StdIn.readLine("Enter a vegetable: ")
val taste = g match
  case "cucumber" if math.random() > 0.5 => "good sometimes!"
  case "tomato" => "very good!"
  case "broccoli" => "quite good..."
  case other => "tastes weird: " + other.reverse
```

A **simple variable pattern**, like
`case other => ...`
in the example above, matches **everything**!
`other` thus gets the value of `g` if `g` is **not** `"cucumber"`, `"tomato"`, or `"broccoli"`.

6.1.11 Matching with or-pattern

If you have the same outcome for different branches they can be combined and the pattern separated with vertical bar: `|`

```
val g = scala.io.StdIn.readLine("Enter a vegetable: ")
val taste = g match
  case "cucumber" => "good"
  case "tomato"   => "good"
  case "onion"    => "good"
  case _ => "not good"
```

More concise with or-pattern:

```
val g = scala.io.StdIn.readLine("vegetable:")
val taste = g match
  case "cucumber" | "tomato" | "onion" => "good"
  case _ => "not good"
```

6.1.12 Matching with typed patterns

Assume we have the below fake function `f` that we want to match on:

```
def f() =   // What return type is inferred by the compiler?
  if math.random() < 0.5 then 42 + math.random()
  else s"cucumber ${math.random()}"
```

With a type annotation after a variable you get a **typed pattern**. Upon successful match the value is **converted** to the specific type and bound to the variable.

```
val i: Int = f() match
  case x: Double => x.round.toInt     // typed pattern that checks if Double
  case s: String => s.length          // typed pattern that checks if String
```

Matching against specific types as above is used idiomatically in Scala rather than `isInstanceOf` and `asInstanceOf` but you can do the equivalent as below:

```
val i2: Int =
  val x = f()
  if x.isInstanceOf[Double] then x.asInstanceOf[Double].round.toInt
  else if x.isInstanceOf[String] then x.asInstanceOf[String].length
  else throw scala.MatchError(x)
```

6.1.13 Deepening: Union types and the type Matchable

- Example: For unrelated types `String` and `Int` the most specific type that can be inferred is `Int | String`, read "Int or String" and called a **union type**.

```
scala> def f() = math.random() match
          case a if a > 0.5 => 42
          case a if a < 0.2 => "hello"

def f(): Int | String
```

- All values that can be examined with **match** have type `Matchable`.
- The type `Matchable` is almost as general as the top type `Any`.

```
scala> (f().isInstanceOf[Matchable], f().isInstanceOf[Any])
val res0:  (Boolean, Boolean) = (true,true)
```

- `Matchable` was introduced in Scala 3 with **opaque type aliases** that are guaranteed never to be boxed but cannot be pattern matched. (Not included in this course.)
- More on `Matchable` and **opaque type** in Scala 3 is here: https://dotty.epfl.ch/docs/reference/other-new-features

---

6.1.14 Constructor patterns with case classes

A base class with common parts and two subtypes:

```
trait Vegetable:
  def weight: Int
  def isRotten: Boolean

case class Cucumber(weight: Int, isRotten: Boolean) extends Vegetable
case class Tomato(weight: Int, isRotten: Boolean) extends Vegetable
```

Thanks to the case classes you can use **constructor patterns** to see what is **inside** an instance:

```
def test(v: Vegetable): String = v match
  case Cucumber(w, false) => "good, weighs " + w
  case Cucumber(_, true)  => "not good"
  case Tomato(w, r)       => (if r then "not " else "") + s"good, weighs $w"
  case _ => s"unknown vegetable: $v"
```

Constructor patterns ”**decompose**” what is matched and bind variables to the attributes in the case class constructor.

6.1.15 Decompose collections with deep patterns

- You can decompose the contents of a collection like this:

```
def show(xs: Vector[Vegetable]): String = xs match
  case Vector()                 => "empty vegetable vector"
  case Vector(Cucumber(w, true)) => s"a rotten cucumber weighing $w"
  case Vector(v)                => s"exactly one vegetable: $v"
  case Vector(v1, v2)           => s"exactly two vegetables: $v1, $v2"
  case Vector(v, vs*)           => s"first a $v and then the tail: $vs"
```

- What happens if you swap the order of the 2nd and 3rd pattern?
- `Vector(v, vs*)` can also be written as `v +: vs`

6.1.16 Matching on tuples

It works fine to decompose tuples with pattern matching:[1]

```
var pair = ("hello", 42)

pair match
  case (a, b) if b == 42 => s"the meaning of life is found: $a"
  case (_, b)            => s"lacking meaning: $b"
```

Underscore means you are not interested in binding a variable name to the value.

6.1.17 Pattern matching and enumeration with case objects

A base type and specific singleton objects of common type:

```
trait Color
case object Spades  extends Color // works without case but we want nice toString
case object Hearts extends Color
case object Diamonds extends Color
case object Clubs  extends Color

def parallelColor(c: Color): Color = c match
  case Spades  => Clubs
  case Clubs   => Spades
  case Hearts  => Diamonds
```

Which case branch have we forgotten? Can the compiler help us?

```
scala> parallelColor(Diamonds)
scala.MatchError: Diamonds
```

Exception at runtime :(

6.1.18 Pattern matching and sealed types

With the keyword **sealed** you get a **sealed** type: no more subtypes are allowed than those that are in the **same source file**. You get a **warning** if you **forgot a case** in pattern matching.

```
sealed trait Color                  // ESC+Enter in REPL compiles multiple lines at once
case object Spades  extends Color
case object Hearts extends Color
case object Diamonds extends Color
case object Clubs  extends Color

def parallelColor(c: Color): Color = c match
  case Spades  => Clubs
  case Clubs   => Spades
  case Hearts  => Diamonds
```

[1] https://youtu.be/aboZctrHfK8

```
1 warning found
  |def parallelColor(c: Color): Color = c match
  |                                     ^
  |                         match may not be exhaustive.
  |
  |                         It would fail on pattern case: Diamonds
```

Warning at compile time :) Thanks compiler!

6.1.19 Pattern matching an enumeration

Instead of `sealed trait ... case object ...` you can use an **enumeration** (aka. enumeration type).

```
enum Color:
  case Spades, Hearts, Diamonds, Clubs

def parallelColor(c: Color): Color =
  import Color.*
  c match
    case Spades  => Clubs
    case Clubs   => Spades
    case Hearts  => Diamonds
```

```
1 warning found
  |  c match
  |  ^
  |  match may not be exhaustive.
  |
  |  It would fail on pattern case: Diamonds
```

Here too we get a helpful warning at compile time :) Thanks compiler!

6.1.20 Uppercase/lowercase initials in matching

Pitfall: matching a **value** that starts with a **lowercase** letter.

```
scala> val meaningOfLife = 42

scala> def isMeaningOfLifeBuggy(answer: Int) = answer match
         case meaningOfLife => true    // local name matching everything!
         case _ => false

scala> isMeaningOfLifeBuggy(43)
val res0: Boolean = true

scala> val MeaningOfLife = 42   // uppercase initial

scala> def isMeaningOfLife(answer: Int) = answer match
         case MeaningOfLife => true    // works fine!
         case _ => false

```

```
scala> isMeaningOfLife(43)
val res1: Boolean = false
```

6.1.21 Uppercase/lowercase initials in matching

One way around the problem with lowercase initial:
backticks to the rescue!

```
scala> val meaningOfLife = 42

scala> def isMeaningOfLifeBackTicks(answer: Int) = answer match
         case `meaningOfLife` => true    // now it works!
         case _ => false

scala> isMeaningOfLifeBackTicks(43)
val res2: Boolean = false
```

6.1.22 Patterns in other places than in `match`

Patterns in **declarations**:

```
scala> case class Point(x: Int, y: Int)

scala> val p = Point(0, 1)

scala> val Point(x, y) = p          // constructor pattern with case class
val x: Int = 0
val y: Int = 1

scala> val (x, y, z) = (0, 1, 2)    // constructor pattern with tuple
val x: Int = 0
val y: Int = 1
val z: Int = 2
```

Patterns in **for-expressions**:

```
scala> val xs = for (x, y) <- Vector((1,2), (3,4)) yield x
val xs: Vector[Int] = Vector(1, 3)
```

6.1.23 Pattern parts and variable number of arguments

With two different special characters it is possible to

- bind variables to **pattern parts** with `@`
  `case Vector(xs@Vector(a), Vector(42)) => ...`

- match **variable number of arguments** with ∗
  `case Vector(a, _, c) => ...` matches if 3 elements, _ ignored
  `case Vector(a, tail*) => ...` matches if at least one element
  `case Vector(a, _*) => ...` interested in first, tail ignored

6.1.24 Partial functions and the method `collect`

- A **partial function** is, unlike a **total function**, not defined for all parameter values.
- Partial functions can be created with `case` without `match`:

```
val pf: PartialFunction[Int, Double] = { case z if z != 0 => 1.0 / z }
```

- The function is not defined for argument `0`:

```
scala> pf(0)
scala.MatchError: 0
```

- This is useful together with the collection method `collect` which applies a partial function only to defined values:

```
scala> Vector(1, 2, 0, 4).collect(pf)
val res0: Vector[Double] = Vector(1.0, 0.5, 0.25)

scala> Vector(1 -> 2, 0 -> 3, 42 -> 0).collect{ case (a,b) if a > 0 => a }
val res1: Vector[Int] = Vector(1, 42)
```

- Note that **curly braces are needed** when using `case` without `match`.

6.1.25 Deepening: the method `unapply`

When you declare a case class, the compiler will **automatically generate a method** named `unapply`.

```
scala> case class Cucumber(weight: Int, isRotten: Boolean)

scala> Cucumber.unapply // press ENTER to see the type
val res0: Cucumber => Cucumber = Lambda1914/0x00000008408cf840@b0e7bde

scala> val g = Cucumber(100, false)

scala> Cucumber.unapply(g)
val res1: Cucumber = Cucumber(100,false)
```

What's this good for? The method `unapply` is generated by the compiler and used internally during matching; it is this method that makes case classes usable in constructor patterns. The principle is general: You can create **your own** so-called **extractors** that can decompose a value via pattern matching, even without a case class.
For the curious: https://docs.scala-lang.org/scala3/reference/changed-features/

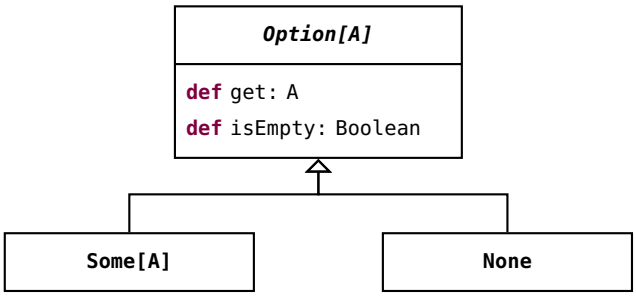
pattern-matching.html

6.1.26 How to handle missing values?

Different ways to handle missing values:

- Invent a special value: for example -1 for missing value
- `null` if value is missing (common in Java and other languages, very rare in Scala)
- Use a collection and let empty collection represent missing value:
  `val sums = Vector(Vector(42),Vector(32),Vector(),Vector(21))`
- `Option[A]` common base type for:
  `None` which represents **missing value**, and
  `Some[A]` which represents that **value exists**

6.1.27 A common base type for a value that may be missing




```
scala> var x: Option[Int] = Some(42)

scala> x.isEmpty
val res0: Boolean = false

scala> x = None

scala> x.isEmpty
val res1: Boolean = true
```

6.1.28 Option for handling possibly missing values

Not everyone wants to tell Facebook their gender.
Improve Facebook's code with a small Scala program:

```
enum Gender:
  case Male, Female

case class Person(name: String, gender: Option[Gender])
```

```
scala> val p1 = Person("Björn",  Some(Gender.Male))
scala> val p2 = Person("Sandra", Some(Gender.Female))
scala> val p3 = Person("Kim",  None)

scala> val g2 = p2.gender
val g2: Option[Gender] = Some(Female)

scala> def show(g: Option[Gender]): String = g match
         case Some(x) => x.toString
         case None    => "undefined"

scala> show(g2)                              // returns "Female"
scala> show(p3.gender)                       // returns "undefined"
scala> val ps = Vector(p1,p2,p3)
scala> val binary = ps.flatMap(_.gender)     // flatMap ignores None
val binary: Vector[Gender] = Vector(Male, Female)
```

6.1.29 Some handy methods on `Option`

The method `getOrElse` often allows avoiding pattern matching.

```
var opt: Option[Int] = None

val x = opt.getOrElse(42)   // give me the value if it exists otherwise default valu
```

Several common collection methods work, e.g. `foreach` and `map`.

```
opt.foreach(x => println(x)) // does nothing if value is missing

opt.map(x => x + 1)          // nothing is computed if value is missing

opt = Some(42)               // assign opt some value

opt.foreach(x => println(x)) // this runs when value exists

opt.map(x => x + 1)          // new option with updated value
```

6.1.30 Some collection methods that return an `Option`, exercise

```
scala> val (xs, ys) = (Vector(1,2,3), Vector())

scala> xs.headOption
???

scala> ys.headOption
???

scala> xs.find(_ > 1)
???
```

```
scala> xs.find(_ > 5)
???

scala> (xs.lift(0), ys.lift(0))
???

scala> val capital = Map("Sverige" -> "Sthlm", "Skåne" -> "Malmö")

scala> capital.get("Skåne")
???

scala> capital.get("Danmark")
???
```

6.1.31 Some collection methods that return an `Option`, answers

```
scala> val (xs, ys) = (Vector(1,2,3), Vector())

scala> xs.headOption
val res0: Option[Int] = Some(1)

scala> ys.headOption
val res1: Option[Nothing] = None

scala> xs.find(_ > 1)
val res2: Option[Int] = Some(2)

scala> xs.find(_ > 5)
val res3: Option[Int] = None

scala> (xs.lift(0), ys.lift(0))
val res4: (Option[Int], Option[Nothing]) = (Some(1),None)

scala> val capital = Map("Sverige" -> "Sthlm", "Skåne" -> "Malmö")

scala> capital.get("Skåne")
val res5: Option[String] = Some(Malmö)

scala> capital.get("Danmark")
val res6: Option[String] = None
```

6.1.32 What is an exception?

Exceptions represent an error or an abnormal condition detected during execution that needs to be handled specially alongside the normal execution flow.

sv.wikipedia.org/wiki/Exception_handling

Examples of exceptions:

- Indexing outside the vector's index bounds.

- Reading beyond the end of a file.
- Attempting to open a file that does not exist.
- Out of memory.
- Integer division by zero causes `java.lang.ArithmeticException`.
- `"hej".toInt` causes `java.lang.NumberFormatException`.

6.1.33 Cause exceptions indirectly with `require` and `assert`

- With the function `require(b)` an `IllegalArgumentException("requirement failed")` is created if b is **`false`**
- `require` is used to restrict which arguments are valid
- With the function `assert(b)` an `AssertionError("assertion failed")` is created if b is **`false`**
- `assert` is used to prevent invalid states

See the implementation of `require` here:
https://github.com/scala/scala/blob/v2.13.17/src/library/scala/Predef.scala#L322

6.1.34 Throw exceptions directly with primitive `throw`

You can generate an exception yourself with **`throw`**, which is called **throwing** an exception that (if not **caught**), causes execution to **abort**.

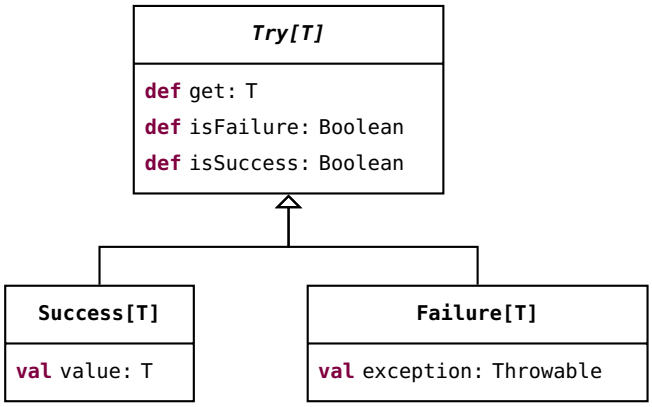
```
scala> def bang = throw Exception("BANG!")
bang: Nothing

scala> bang
java.lang.Exception: BANG!
```

Different ways to handle exceptions and prevent execution from aborting:

- **`try catch`** expressions convert exceptions into some appropriate value.
- `scala.util.Try` **wraps** code that can produce exceptions.

6.1.35 A common base type for something that can fail

```
import scala.util.{Try, Success, Failure}
```



6.1.36 Handle exceptions as a value with `Try`

```
scala> def bang = throw new Exception("BANG!")

scala> def maybeBang = if math.random() < 0.5 then 42 else bang

scala> import scala.util.{Try, Success, Failure}

scala> def attempt = Try { maybeBang }

scala> val xs = Vector.fill(15){attempt}

scala> val thirteenth = xs(12) match
         case Success(value) => value
         case Failure(e) => println(e); -1

scala> (xs(12).isSuccess, xs(12).isFailure)

scala> xs(12).getOrElse(0)

scala> xs(12).toOption

scala> attempt.foreach(println)

scala> attempt.map(_ + 1)

scala> for Success(x) <- xs yield x
```

6.1.37 Primitive `try-catch` expressions

You can catch exceptions directly with a **`try`** ... **`catch`** expression:

```
def carola =
  try
    if math.random() > 0.5 then throw Exception("storm wind")
    42
  catch
    case e: Exception =>
```

```
    println("Caught a " + e.getMessage)
    -1
```

```
scala> Vector.fill(5)(carola)
Caught a storm wind
Caught a storm wind
Caught a storm wind
val res0: Vector[Int] = Vector(-1, 42, 42, -1, -1)
```

https://www.youtube.com/watch?v=4Ml6pJqc_bw

6.1.38 Avoid exceptions if possible

Advantages of exceptions:

- For serious errors when there is not much to do but restart, e.g. `OutOfMemoryException`, it is good to know what went wrong.
- Rare abnormal cases can be handled separately (e.g. in the main program) without complicating the code for the normal case.

Disadvantages of exceptions:

- A kind of "goto" that makes the execution flow hard to follow.
- Creating stack-trace takes time; exceptions that happen often affect performance.

Example: exception-free `toIntOption` is both safe and fast!

```
scala> def time(op: => Unit): Long = {val t0 = System.nanoTime; op; System.nanoTime - t

scala> def min(op: => Unit, n: Int = 1000): Long = Seq.fill(n)(time(op)).drop(n / 20).m

scala> min(util.Try("hello".toInt))
val res0: Long = 3549

scala> min(try "hello".toInt catch (_: Throwable) => ())
val res1: Long = 3046

scala> min("hello".toIntOption)
val res2: Long = 157
```

6.1.39 In-depth: Checked exceptions

- There is a risk that exception handling is forgotten.
  What happens then? **Bang!** – it becomes a runtime error and abrupt stop :(
- There are capabilities in Scala to let the compiler check if exceptions are handled with so-called **checked exceptions**
- The curious can read more here:
  https://docs.scala-lang.org/scala3/reference/experimental/canthrow.html

---

When you compare values with ==, the method `equals` present for all types is called. In your own classes, you can override `equals` with your own definition of what equality should mean. It is then suitable to use pattern matching. However, it is quite an extensive task to implement a correct equality comparison that works under all circumstances. A recipe for a complete implementation of `equals` is given in the in-depth section below.

6.1.40 In-depth: Implementing `equals` with `match`

It turns out that **content equality** is **surprisingly complicated** to implement, especially in connection with inheritance.

- The simpler case: Do the in-depth exercise *"The `equals` method"* and implement `equals` for content equality without inheritance.
  A good practice for using **`match`**!
- Harder: Do the in-depth exercises *"Override `equals`"* and *"Override equals with inheritance"* if you want to see how a **complete** `equals` should look to work **in all situations**.

It is not required in this course that you yourself can implement a generally working `equals`. But you should understand the difference between reference equality and content equality. More about `equals` in the advanced course, but a small insight into the problem now...

If a class is marked **`final`**, it cannot have any subclasses. The compiler checks that this applies to all final classes and gives a compilation error if you try to do **`extends`** on a final class. If a class is guaranteed to have no subclasses, the implementation of `equals` can be simpler.

6.1.41 In-depth: `equals` that works for final classes

Recipe for implementing `equals` that works for types that **do not** have any subtypes:

```
final class Cucumber(val weight: Int, val isEatable: Boolean):
  override def equals(other: Any): Boolean = other match
    case that: Cucumber => weight == that.weight && isEatable == that.isEatable
    case _ => false

  override def hashCode: Int = (weight, isEatable).## // provides good hashcode
```

- You **must** always override `hashCode` too if you override `equals`, otherwise cucumber collections don't work (long story ...)
- Note the type `Any` – this follows the design choice made in Java (unfortunately?).
- A **type-safer** content equality test that **guarantees** only comparing a cucumber with a cucumber and nothing else:

```
def ===(other: Cucumber): Boolean =
  weight == other.weight && isEatable == other.isEatable
```

6.1.42 In-depth: 8-step recipe for inheritance-safe `equals`

1. Introduce this method: `def canEqual(other: Any): Boolean`
   Note that the parameter type should be `Any`. If subclassed, needs `override`.
2. The method `canEqual` should return `true` if `other` is of the same type as `this`, e.g.:
   `override def canEqual(other: Any): Boolean = other.isInstanceOf[Cucumber]`
3. Introduce the method `equals` and be careful that the parameter type is `Any`:
   `override def equals(other: Any): Boolean`
4. Implement the method `equals` with a match expression that begins like this:
   `other match`
5. The match expression should have two branches. The first branch should have a typed pattern for the class to be compared, e.g.:
   `case that: Cucumber =>`
6. If you implement `equals` in the class that introduces `canEqual`, start with:
   `(that canEqual this) &&`
   and then create a continuation based on the contents of the class, e.g.:
   `this.weight == that.weight && this.length == that.length`
   If you override equals, you probably want to start with `super.equals(that) &&`
7. The second branch in the match should be: `case _ => false`
8. Override `hashCode`, e.g., with a tuple of attribute values and the method ##:
   `override def hashCode: Int  = (weight, length).##`

http://www.artima.com/pins1ed/object-equality.html

6.1.43 In-depth: Safer equality tests in Scala 3

- **Problem**: `equals` takes values of any type.
- This is called **universal equality**.

```
scala> case class Hund(namn: String)
scala> case class Katt(namn: String)
scala> Hund("bob") == Katt("bob") // weird comparison; can never be true
val res0: Boolean = false         // but the compiler lets you perform the equali
```

- In Scala 3 you can get type-safe equality test with `derives CanEqual`
- This is called **multiversal equality**.

```
scala> case class Hund(namn: String) derives CanEqual
scala> Hund("bob") == Katt("bob")   // thanks compiler for the error:
-- Error:
1 |Hund("bob") == Katt("bob")
  |^^^^^^^^^^^^^^^^^^^^^^^^^^
  |Values of types Hund and Katt cannot be compared with == or !=
```

- You can **skip** writing `derives CanEqual` if you do:
  `import scala.language.strictEquality`
- Read more here: https://docs.scala-lang.org/scala3/reference/contextual/multiversal-equality.html

6.2 Exercise Excercise06

Goals

- ☐ Be able to create and use **match** expressions with constant values, guards, and pattern matching with case classes.
- ☐ Be able to create and use case objects for matching on enumerated values.
- ☐ Be able to handle missing values using the `Option` type and pattern matching on `Some` and `None`.
- ☐ Be able to catch exceptions with `scala.util.Try`.
- ☐ Know about **try**, **catch**, and **throw**.
- ☐ Know the keyword **sealed** and understand the usefulness of sealed types.

Preparations

- ☐ Study the concepts in chapter 6

6.2.1 Basic Tasks; preparation for the lab

Task 1. *Match on constant values.*

a) Write the program below with a code editor and save it in the file `Match.scala`. Compile and run and give as argument your favorite vegetable. What happens? Explain how a **match** expression works.

```
object Match:
  def main(args: Array[String]): Unit =
    val favorite = if args.length > 0 then args(0) else "selleri"
    println("Your favourite vegetable: " + favorite)
    val firstChar = favorite.toLowerCase.charAt(0)
    val meThink = firstChar match
      case 'c' => "cucumber is good!"
      case 't' => "tomato is good!"
      case 'b' => "broccoli is good !"
      case  _  =>  s"$favorite is less good..."
    println(s"I think that $meThink")
```

b) What error message do you get if you remove the case branch for default values and input is chosen such that no case branch matches? Is it a runtime error or a compile-time error?

Task 2. *Guards in case branches.* With the help of a guard in a case branch, you can limit with a condition whether the branch should be chosen.
Start from the code in task 1a and replace the case branch for `'c'` matching with the variant below with a guard using the keyword **if** (note that parentheses around the condition are not needed):

```
    case 'c' if math.random() > 0.5 => "cucumber is sometimes good..."
```

Recompile and run the program repeatedly with different inputs until all branches in the **match** expression have executed. Explain what happens.

Task 3. *Pattern match on attributes in case classes.* Scala's `match` expression is extra powerful if used with `case` classes: then attributes can be automatically extracted and bound to local variables directly in the case branch as the example below shows (note that `w` and `rotten` do not need to be explicitly declared). This is called **pattern matching**. What is printed below? Why? Try changing the names `w` and `rotten`.

```
scala> case class Cucumber(weight: Int, isRotten: Boolean)
scala> val g = Cucumber(100, true)
scala> g match { case Cucumber(w,rotten) => println("G" + w + rotten) }
```

Task 4. *Match on case objects and the usefulness of `sealed`.* Write the following lines of code in a REPL one by one. Note the keyword `sealed`, which is used to seal a type. A **sealed type** must have all its subtypes in the same source file.

```
scala> sealed trait Color
scala> case object Spades extends Color
```

a) What is the error message and why does it occur? Is it a compile-time or a runtime error?

b) Now create the code below in an editor and paste it in the REPL.

```
object Deck:
  sealed trait Color
  object Color:
     val values = Vector(Spades, Hearts, Diamonds, Clubs)
  case object Spades extends Color
  case object Hearts extends Color
  case object Diamonds extends Color
  case object Clubs extends Color
```

c) Create a function `def paracolor(f: Color): Color` in an editor, which by means of a match expression returns the parallel color for a color. The parallel color to `Hearts` is `Diamonds` and vice versa, while the parallel color to `Clubs` is `Spades` and vice versa. Paste the function in the REPL. Also take the opportunity to write an `import` statement for the outer object **Deck**, so that the members of the object can be reached easily.

```
scala> paracolor(Spades)
scala> val xs = Vector.fill(5)(Color.values((math.random() * 4).toInt))
scala> xs.map(paracolor)
```

d) Now we will investigate what happens if you forget one of the case branches in the match in `paracolor`. Intentionally "forget" one of the case branches and paste the new `paracolor` with the incomplete match. What is the warning? Does the warning occur at runtime or compilation?

e) Call `paracolor` with the "forgotten" color. What is the error message? Is it a compile-time or runtime error?

f) Explain what the keyword `sealed` means and what benefit there is of **sealing** a supertype.

Task 5. *Pattern match enumeration.* Now we will investigate and compare the difference between the keywords **enum** and **sealed trait**. Write the code below in a REPL.

```
enum Color:
  case Spades, Hearts, Diamonds, Clubs
```

a) Create again with the help of an editor a function **def** `paracolor(f: Color): Color`, nearly identical to the one we created in subtask 4c. The function should again use a match expression to return the parallel color of the given argument. Note that this time `Color` is not a **sealed trait**, but instead an enumeration (**enum**). Paste the function in the REPL.

```
scala> paracolor(Color.Diamonds)
scala> val xs = Vector.fill(5)(Color.values((math.random() * 4).toInt))
scala> xs.map(paracolor)
```

b) Reflect on the differences and similarities between using **sealed trait** with **case** objects versus using **enum** when pattern matching.

Task 6. *Meaning of lowercase and uppercase initial letters when matching.* To enable names to be bound to variables during matching without needing prior declaration (as we saw in task 3), identifiers starting with a lowercase letter have a special meaning: the compiler interprets it as you wanting a variable to be bound to a value during matching. An identifier starting with an uppercase letter is interpreted as a constant value (e.g. a case object or a case class pattern).

a) *A case branch that captures everything*. A case branch with an identifier starting with a lowercase letter without a guard will match everything. Try the following in REPL, but try to anticipate what will happen. What happens?

```
scala> val x = "ucumber"
scala> x match
         case str if str.startsWith("c") => println("maybe cucumber")
         case whatever => println("not cucumber: " + whatever)
scala> val c = "cucumber"
scala> c match
         case str if str.startsWith("c") => println("maybe cucumber")
         case whatever => println("maybe cucumber: " + whatever)
```

b) *Pitfall with lowercase initial letters.* Before trying below in REPL, try to guess what will happen. What happens? What are the warnings and what do they mean?

```
scala> val any: Any = "neither tomato nor cucumber"
scala> case object Cucumber
scala> case object tomato
scala> any match
         case Cucumber => println("cucumber")
         case tomato => println("tomato")
         case _ => println("everything else")
```

c) *Use backticks to force a match on a constant value.* There is a workaround if you don't want the compiler to create a new local variable: use the special character *backtick*, written ` and requiring special keyboard strokes.[2] Rewrite the previous task

[2] Ask someone if you don't know how to type backtick ` on your keyboard.

but enclose the identifier `tomato` in the tomato case branch in backticks, like this:
`case` `tomato` `=>` `...`

Task 7. *Match on the contents of a Vector.* Run the code below in REPL. What is printed? Explain what happens.

```
scala> val xss = Vector(Vector("hej"),Vector("on", "dough"),Vector("4","x","2")
scala> xss.map( _ match
  case Vector() => "tom"
  case Vector(a) => a.reverse
  case Vector(_, b) => b.reverse
  case Seq(a, "x", b) => a + b
  case _ => "OTHERWISE THIS"
  ).foreach(println)
```

Task 8. *Use `Option` and match on potentially missing values.* One often needs to write code to handle values that may be missing, e.g., missing phone numbers in a person database. This situation is common enough that many languages have special support for missing values.
In Java[3] the `null` value is used to indicate that a reference lacks a value. One must remember to test if the value is missing each time such a value is handled, e.g. using `if (ref != null) { ...} else { ... }`. Another common trick is to use `-1` to indicate missing positive integers, e.g., missing indices, which must be handled with `if (i != -1) { ...} else { ... }`.
In Scala, there is a special type `Option` that allows smooth and type-safe handling of missing values. If a possibly missing value is wrapped in an `Option`, it is in a special kind of collection that can only contain *none* or *some* value, thus has size either `0` or `1`.

a) Explain what happens below.

```
scala> var maybe: Option[Int] = None
scala> maybe.size
scala> maybe = Some(42)
scala> maybe.size
scala> maybe.isEmpty
scala> maybe.isDefined
scala> def increaseIfExists(opt: Option[Int]): Option[Int] = opt match
         case Some(i) => Some(i + 1)
         case None    => None
scala> val anotherMaybe = increaseIfExists(maybe)
scala> def increase(i: Int) = i + 1
scala> val moreMaybe = maybe.map(increase)
```

b) The pattern matching above is at least as clumsy as an `if` statement, but since `Option` is a kind of (small) collection there are smoother methods. Explain what happens below.

```
val sentence = Some(42)
val notSentence = Option.empty[Int]
sentence.map(_ + 1)
notSentence.map(_ + 1)
notSentence.map(_ + 1).orElse(Some("missing")).foreach(println)
sentence.map(_ + 1).orElse(Some("missing")).foreach(println)
```

[3] Scala also has `null` but it is only needed when interoperating with Java code.

c) *Collection methods that return an `Option`.* Explain what happens for each line below. One line gives an error message; which line and what error?

```
val xs = (42 to 84 by 5).toVector
val e = Vector.empty[Int]
xs.headOption
xs.headOption.get
xs.headOption.getOrElse(0)
xs.headOption.orElse(Some(0))
e.headOption
e.headOption.get
e.headOption.getOrElse(0)
e.headOption.orElse(Some(0))
Vector(xs, e, e, e)
Vector(xs, e, e, e).map(_.lastOption)
Vector(xs, e, e, e).map(_.lastOption).flatten
xs.lift(0)
xs.lift(1000)
e.lift(1000).getOrElse(0)
xs.find(_ > 50)
xs.find(_ < 42)
e.find(_ > 42).foreach(_ => println("FOUND!"))
```

d) What are the advantages of `Option` compared to **`null`** or `-1` if one forgets to handle missing values in code?

Task 9. *Throw exceptions.* If you want to signal that an error or abnormal situation has occurred, you can **throw** an **exception**. Then the program aborts immediately with an error message unless you **catch** the exception. a) What happens below?

```
scala> throw new Exception("PANG!")
scala> java.lang.   // Press TAB after the dot
scala> throw new IllegalArgumentException("wrong wrong wrong")
scala> val carola =
         try
           throw new Exception("stormWind!")
           42
         catch
           case e: Throwable =>
             println("Handled by a " + e)
             -1
```

b) Name a couple of exceptions in package `java.lang` that you can guess their meaning and in what situations they are thrown.

c) What type does variable `carola` have above? What would its type be if the catch branch returned a string instead?

Task 10. *Catch exceptions with `scala.util.Try`.* In package `scala.util` there is type `Try` (capital T), a kind of collection that can contain either a ”successful” or ”failed” value. If the computation succeeds and no exceptions are thrown, the value is wrapped in a `Success`, otherwise the thrown exception is wrapped in a `Failure`. One can extract the value or exception via pattern matching but it is usually more convenient to use collection methods `map` and `foreach`, like with `Option`. There is also a convenient method `recover` on `Try` where one can provide code that is run if an exception occurs.

a) Explain what happens below.

```
scala> def pang = throw new Exception("PANG!")
scala> import scala.util.{Try, Success, Failure}
scala> Try{pang}
scala> Try{pang}.recover{case e: Throwable =>   "disarmed bomb: " + e}
scala> Try{"silent"}.recover{case e: Throwable => "disarmed bomb: " + e}
scala> def maybePang = if math.random() > 0.5 then "silent" else pang
scala> def maybeOk = Try{maybePang}
scala> val xs = Vector.fill(100)(maybeOk)
scala> xs(13) match
         case Success(x) => ":)"
         case Failure(e) => ":( " + e
scala> xs(13).isSuccess
scala> xs(13).isFailure
scala> xs.count(_.isFailure)
scala> xs.find(_.isFailure)
scala> val badOpt = xs.find(_.isFailure)
scala> val goodOpt = xs.find(_.isSuccess)
scala> badOpt
scala> badOpt.get
scala> badOpt.get.get
scala> badOpt.map(_.getOrElse("bomb disarmed!")).get
scala> goodOpt.map(_.getOrElse("bomb disarmed!")).get
scala> xs.map(_.getOrElse("bomb disarmed!")).foreach(println)
scala> xs.map(_.toOption)
scala> xs.map(_.toOption).flatten
scala> xs.map(_.toOption).flatten.size
```

b) What is the return type of the function `pang`?

c) Why does the function `maybePang` have the inferred return type `String`?

6.2.2 Advanced Tasks; challenges

Task 11. *Use matching or dynamic binding?* One can implement the distinction of edible vegetables in task 3 via dynamic binding instead of **`match`**.

a) Make a new variant of your program as per the guidelines below, save the modified code in `vegopoly.scala` and compile and run.

- Remove the predicate `isWorthEating` from object `Main` and instead add an abstract method **`def`** `isWorthEating: Boolean` in the trait `Vegetable`.
- Add concrete **`val`** members in each vegetable class that define edibility.
- Modify the main program accordingly so that `isWorthEating` is invoked as a method on the harvested vegetable objects when filtering edible ones.

b) Add a new vegetable **`case class`** `Broccoli` and define its edibility. Modify the randomization functions so broccoli becomes less common than cucumber.

c) Compare the solution with **`match`** in task 3 and the solution above with polymorphism. What are pros and cons of each? Discuss two different scenarios at a hypothetical company developing software for the agricultural sector: 1) the set of vegetables does not change often but edibility definitions change very frequently, and 2) the vegetable set changes very often but edibility definitions rarely change.

Task 12. *Method equals.* If you override the existing `equals` method, then == will behave differently. You can achieve content equality instead of reference equality. We start by studying the existing equals with reference equality.

a) What happens below? Inspect parameter type and return type for `equals`.

```
1 scala> class Cucumber(val weight: Int, val isEatable: Boolean)
2 scala> val g1 = new Cucumber(42, true)
3 scala> val g2 = g1
4 scala> val g3 = new Cucumber(42, true)
5 scala> g1 == g2
6 scala> g1 == g3
7 scala> g1.equals  // press ENTER to see function type
```

b) Draw the memory situation after line 4.

c) *Override methods `equals` and `hashCode`.*

Background: It turns out surprisingly complicated to implement content equality with `equals` so that it works correctly in all edge cases. For example, one must also override a method called `hashCode` if one overrides `equals`, because these are used jointly for efficiency to create internal storage of objects in some collections. Missing this can cause objects to become "invisible" in hash-code-based collections – more about this in later courses. When objects are in an open inheritance hierarchy it is also more complicated; it is easier if dealing with final classes. Also, special considerations are needed if the class has type parameters.

Define the class below in REPL with overridden `equals` and `hashCode`; it does not inherit anything and is final.

```
// works well if the class is final and does not inherit anything
final class Cucumber(val weight: Int, val isEatable: Boolean):
  override def equals(other: Any): Boolean = other match
    case that: Cucumber => weight == that.weight && isEatable == that.isEatable
    case _ => false
  override def hashCode: Int = (weight, isEatable).## //explained later
```

d) What happens now below, where `Cucumber` now has overridden `equals` with content equality?

```
scala> val g1 = new Cucumber(42, true)
scala> val g2 = g1
scala> val g3 = new Cucumber(42, true)
scala> g1 == g2
scala> g1 == g3
```

e) How do you notice above that the overridden `equals` causes == now to give content equality? Compare with subtask a.
In task 18 you get to try the full 8-step recipe to override `equals` according to all the rules. In the following course there will be more training in handling content equality and hash codes. In Scala one gets an object's hash code via the method ##.[4]

Task 13. *Polynomial.* With the code below, one can do the following:

```
scala> import polynomial.*

scala> Const(1) * x
```

[4] If interested in hash codes, read more here: en.wikipedia.org/wiki/Hash_function

```
res0: polynomial.Term = x

scala> (x*5)^2
res1: polynomial.Prod = 25x^2

scala> Poly(x*(-5), y^4, (z^2)*3)
res2: polynomial.Poly = -5x + y^4 + 3z^2
```

a) Explain what happens above by studying the code below[5].

```scala
object polynomial:

  sealed trait Term:
    def *(that: Term): Term

  case class Const(value: BigDecimal) extends Term:

    def toSilentString: String = this match
      case Const.One      => ""
      case Const.MinusOne => "-"
      case _              => value.toString

    override def toString = value.toString

    override def *(that: Term): Term = that match
      case Const(d)    => Const(d * value)
      case v: Var      => Prod(this, Set(v))
      case Prod(c, vs) => Prod(Const(c.value * value), vs)

    def *(d: BigDecimal): Const = Const(d * value)

    def ^(e: Int): Const = Const(value.pow(e))


  object Const:
    final val Zero     = Const(BigDecimal(0))
    final val One      = Const(BigDecimal(1))
    final val MinusOne = Const(BigDecimal(-1))

  case class Var(name: Char, exp: Int = 1) extends Term:

    private def silentExpString: String =
      if exp == 1 then "" else "^"+exp.toString

    override def toString = s"$name$silentExpString"

    def ^(e: Int): Var = Var(name, e * exp)

    def *(c: BigDecimal) = Prod(Const(c), Set(this))

    override def *(that: Term): Term = that match
```

[5]The code is also available here:
github.com/lunduniversity/introprog/tree/master/compendium/examples/polynomial

```
      case c: Const => Prod(c, Set(this))

      case v: Var =>
        if v.name == name then Var(name, v.exp + exp)
        else Prod(Const.One, Set(this, v))

      case p: Prod => p * this


  object Var:

    def apply(d: BigDecimal, name: Char): Prod =
      Prod(Const(d), Set(Var(name)))

    def apply(d: BigDecimal, name: Char, exp: Int): Prod =
      Prod(Const(d), Set(Var(name, exp)))

    def addExp(v1: Var, v2: Var): Var = Var(v1.name, v1.exp + v2.exp)

    def multiply(v1: Var, vs: Set[Var]): Set[Var] =
      if !vs.contains(v1) then vs + v1
      else vs.map(v2 => if v1.name == v2.name then addExp(v1, v2) else v2)

    def multiply(vs1: Set[Var], vs2: Set[Var]): Set[Var] =
      var result = vs2
      vs1.foreach{ v1 => result = multiply(v1, result) }
      result


  case class Prod(const: Const, vars: Set[Var]) extends Term :

    override def toString = s"${const.toSilentString}${vars.mkString}"

    override def *(that: Term): Term = that match
      case Const(d) => Prod(Const(d * const.value), vars)

      case v: Var => Prod(const, Var.multiply(v, vars))

      case Prod(Const(d), vs)  =>
        Prod(Const(const.value * d), Var.multiply(vs, vars))

    def ^(e: Int) = Prod(const ^ e, vars.map(_ ^ e))

  case class Poly(xs: Set[Term]):
    override def toString = xs.mkString(" + ")

  object Poly:
    def apply(ts: Term*) : Poly = Poly(ts.toSet)

  val (x, y, z, s, t) = (Var('x'), Var('y'), Var('z'), Var('s'), Var('t'))
```

b) Build further on **object** `polynomial` and implement addition between different terms.

Task 14. *`Option` as a collection.* Study the documentation for `Option` here and see if you recognize any methods that also exist on the collection `Vector`:
www.scala-lang.org/api/current/scala/Option.html
Explain how the method `contains` on an `Option` works using the documentation examples.

Task 15. *Catch exceptions with `catch` in Java and Scala.* Make the analogous program in Scala as shown in task 12, utilizing that Scala's `try-catch` is an expression. Compile and run and test that from user's point of view it works exactly the same way. Note the main differences between the two programs.

Task 16. *Polynomial, continuation: reduction.* Build on `object polynomial` in task 13 on page 209 and implement the method `def reduce: Poly` in case class `Poly` which simplifies polynomials if multiple `Prod` terms can be added.

Task 17. *Type-safe content test with method ===.* Methods `equals` and == allow comparison with anything. Sometimes one wants a type-safe content comparison that only allows comparison of objects of a more specific type and gives a compile error otherwise. One usually defines a method === with a parameter `that` having a suitably specific desired type. Introduce the abstract method === in trait `polynomial.Term` in task 13 on page 209 and override it in all subclasses of Term. Test that you get a compilation error if you try to compare a `Term` with something completely different, e.g. a `String` or a `Vector`.

```
  def ===(that: Term): Boolean
```

Task 18. *Override `equals` with content equality also for non-final classes.* Below are parts of class `Complex` representing a complex number with real and imaginary parts. Instead of, as often done in Scala, using a case class and an `equals` method that automatically gives content equality, you should practice implementing your own `equals`.

```
class Complex(val re: Double, val im: Double):
  def abs: Double = math.hypot(re, im)
  override def toString = s"Complex($re, $im)"
  def canEqual(other: Any): Boolean = ???
  override def hashCode: Int  = ???
  override def equals(other: Any): Boolean = ???

case object Complex:
  def apply(re: Double, im: Double): Complex = new Complex(re, im)
```

Follow this **recipe**[6] in 8 steps to override `equals` with content equality that also works for non-final classes:

1. Introduce this method: `def canEqual(other: Any): Boolean`
   Note that the parameter type must be `Any`. If done in a subclass to a class that already has `canEqual`, you should also add `override`.

2. Method `canEqual` should return `true` if `other` is of the same type as `this`, e.g.:
   `def canEqual(other: Any): Boolean = other.isInstanceOf[Complex]`

[6] This recipe is based on `http://www.artima.com/pins1ed/object-equality.html`

3. Introduce method `equals` making sure parameter has type `Any`:
   `override def equals(other: Any): Boolean`

4. Implement `equals` with a match expression starting as:
   `other match`

5. The match should have two branches. The first branch should have a typed pattern for the class to compare:
   `case that: Complex =>`

6. If you implement `equals` in the class introducing `canEqual`, start the expression with:
   `(that canEqual this) &&`
   and continue with a comparison based on class content, e.g.: `this.re == that.re && this.im ==`
   If overriding a different equals than the standard one from `AnyRef`, you probably want to start the logical expression with calling super's equals method: `super.equals(that) &&` but you need to carefully consider what equality between subclasses should mean in your particular case.

7. The second branch in the match should be: `case _ => false`

8. Override `hashCode`, for example by making a tuple of the class contents and calling method ## on the tuple to get a good hash code:
   `override def hashCode: Int = (re, im).##`

Task 19. *Override equals with inheritance.* Build on the example below and override equals with inheritance by following the recipe in task 18.

```
trait Number:
  override def equals(other: Any): Boolean = ???

class Complex(re: Double, im: Double) extends Number:
  override def equals(other: Any): Boolean = ???

class Rational(numerator: Int, denominator: Int) extends Number:
  override def equals(other: Any): Boolean = ???
```

Task 20. *Special matchings.* Read about usage of special matchings here:
dotty.epfl.ch/docs/reference/changed-features/vararg-splices.html

a) Try variable binding with `@` in a match in REPL.

b) Try sequence patterns with `_` and `_*` in a match in REPL.

Task 21. *Extractors.* Read more about extractors here:
dotty.epfl.ch/docs/reference/changed-features/pattern-matching.html
Create your own extractor object for HTTP addresses like e.g.:
`http://my.host.domain/path/to/this`
that extracts `my.host.domain` and `path/to/this` with method `unapply` and test in a match.

Task 22. *Polynomial, continuation: polynomial division.* Implement polynomial division appropriately by building on `object polynomial` in task 13 on page 209.
Read more about polynomial division here: sv.wikipedia.org/wiki/Polynomdivision

6.3 Lab: `blockbattle1`

Goals

- ☐ Be able to explain differences and similarities between a single object and objects that are instances of classes.
- ☐ Be able to explain the difference between mutable and immutable objects.
- ☐ Be able to define and instantiate classes and case classes, as well as be able to describe when a case class is most suitable and give some examples of what it offers beyond a regular class.
- ☐ Be able to create and use classes whose instances contain references to other instances (aggregation).
- ☐ Understand the meaning of the instance reference `this`.
- ☐ Be able to create simple match expressions.

Preparations

- ☐ Do exercise `Excercise05` in section 5.2, especially exercise 6.
- ☐ Do exercise `Excercise06` in section 6.2.
- ☐ Read through the entire lab and plan your work.
- ☐ Retrieve given code via the course GitHub location.

6.3.1 Background

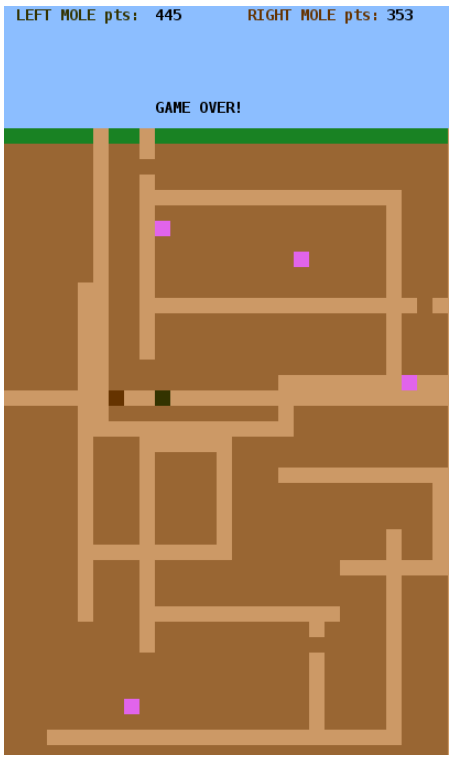


Figure 6.1: A duel over blockworms between two Lund-style block moles captured in an intense digging moment.

In this lab you will practice declaring classes and creating multiple instances of the same class. You will also practice building a larger program from scratch. You will develop a game for two players who share the same keyboard, where the left player controls a block mole with the keys A,S,D,W, and the right player controls another block mole with the arrow keys.

In the picture on the left you can see how the game can look. There is a light brown and a dark brown mole. The scoring is shown at the top in the sky. There are four pink blockworms (see exercise 13 in lab `blockmole`) that the moles compete to catch. When a blockworm teleports to a new random position, it leaves soil behind. When a mole digs up to the grass surface, holes appear in the grass. It gives points to dig tunnels and to catch blockworms.

You decide how scoring should work and what the criteria for when the game ends are etc.

6.3.2 Mandatory requirements

The following functional requirements must be fulfilled by your program:

- ☐ Each mole moves in its current direction until the user changes direction by pressing ”its” corresponding key, e.g. W or arrow-up.
- ☐ When a mole moves in dark brown soil, light brown tunnels are dug.
- ☐ When a block mole reaches the window edge or the sky, its direction should be reversed.
- ☐ Digging tunnels should yield points.
- ☐ Each player’s points should be displayed during the game.
- ☐ A game should end and *Game over* be displayed when any chosen criterion has been met.

Your code should be designed according to these design requirements:

- ☐ A `Game` is created in the main program with the method `start` which runs the game.
- ☐ Constants should be named and placed in an appropriate companion object.
- ☐ Each class with any corresponding companion object should be in its own source file and belong to the package `blockbattle`.
- ☐ You should start from the classes you implemented in task 6 in exercise `Excercise05`.
- ☐ The class `BlockWindow` converts to internal window coordinates. Other classes should use block coordinates.

6.3.3 Optional requirements – choose at least one

You should implement at least one (preferably several) of these requirements:

- ☐ There should be an appropriate number of blockworms (see lab `blockmole` ex. 13, p. 158).
- ☐ The block moles should also have an attribute representing health, e.g. a numeric value between 0 and 100. Health should be weakened slightly when digging tunnels. Health should be visible in the game window, e.g. as a sequence of red blocks in the sky indicating the proportion of max health for each player.
- ☐ Running on grass should affect points and/or health.
- ☐ Catching blockworms should affect points and/or health.
- ☐ There should be yellow block diamonds that give many points if caught first.
- ☐ At game start it should be possible to choose a name for each block mole and the name should appear in the game next to the scoring.
- ☐ It should be faster to move in tunnels compared to digging in soil.
- ☐ If a block mole catches a blockworm, its digging speed should increase.
- ☐ If a block mole collides with another block mole something should happen, e.g. its direction reverses.
- ☐ Show *highscore* at *Game Over*. Highscore is saved with `introprog.IO` in a file which is created if it does not exist, otherwise it is read at startup if available and updated when needed. Save the entire highscore list or just the highest score so far.

6.3.4 Preparations for presentation

✓ 👁 Before you present your implementation you should be able to orally explain the following:

- ☐ Study someone else’s game and give your peer at least one tip on how the readability of their code can be improved. Write down your tips and describe

them at the presentation.

- ☐ Describe what measures you have taken to make your code easy to read and understand.
- ☐ Describe how you have gradually developed your program from simpler to more advanced functionality, as well as bugs you have discovered and fixed.
- ☐ Describe which optional requirements your implementation fulfills.
- ☐ Describe how you would need to change the class `Mole` so that it is possible to write
  `new Mole().move().move().reverseDir().move()`

6.3.5 Tips and suggestions

1. **Many small steps.** Run compilation with change monitoring using `--watch` in a separate terminal window, so that at every change you can fix any compilation errors. Run and test your program in another terminal window.

2. **Choose good names**. Your code becomes easier to read and modify if you come up with good names for members and place them appropriately. For example, you can gather global game constants in the companion object of the class `Game`. You can build upon the code below and add members as you discover they are needed. Below is, for example, a function that gives the background color for a given y-coordinate, which is useful when you need to restore the background after a mole has moved.

```
package blockbattle

object Game:
  val windowSize = (30, 50)
  val windowTitle = "EPIC BLOCK BATTLE"
  val blockSize = 14
  val skyRange    = 0 to 7
  val grassRange  = 8 to 8
  object Color { ??? }
  /** Used with the different ranges and eraseBlocks */
  def backgroundColorAtDepth(y: Int): java.awt.Color = ???

class Game(
  val leftPlayerName: String  = "LEFT",
  val rightPlayerName: String = "RIGHT"
):
  import Game.* //direct access to members of companion object

  val window    = new BlockWindow(windowSize, windowTitle, blockSize)
  val leftMole: Mole  = ???
  val rightMole: Mole = ???

  def drawWorld(): Unit = ???

  /** Use to erase old points, e.g updated score */
  def eraseBlocks(x1: Int, y1: Int, x2: Int, y2: Int): Unit = ???

  def update(mole: Mole): Unit = ???  // update, draw new, erase old

  def gameLoop(): Unit = ???

  def start(): Unit =
```

```
  println("Start digging!")
  println(s"$leftPlayerName ${leftMole.keyControl}")
  println(s"$rightPlayerName ${rightMole.keyControl}")
  drawWorld()
  gameLoop()
```

3. **Divide your code into functions.** Your code becomes easier to read and modify if you split it into many small functions with good names. In the `Game` class above there are examples of some useful functions. As you extend your program you can add more functions with names like `showPoints`, `gameOver`, etc.

4. **Consider the overall structure.** The program you will write in this lab is larger than what you have done before. Therefore, it is important to consider the structure of your program, which classes are responsible for what and how they cooperate. Feel free to discuss with an instructor if you are uncertain about how the pieces of code you developed in the previous week's exercise 6, the classes `Pos`, `KeyControl`, `Mole`, and `BlockWindow`, are intended to interact. Be sure to test that the different classes and their methods work on their own.

5. **Design of `gameLoop()`.** In a game a so-called game loop is needed which repeats the code that should run on every new screen frame. Each round in the game loop updates data and draws in the game window, as well as includes a suitable delay. A sketch of a typical game loop is shown below:

```
var quit = false
val delayMillis = 80

def gameLoop(): Unit =
  while !quit do
    val t0 = System.currentTimeMillis
    handleEvents()    // changes direction upon key press etc.
    update(leftMole)  // moves, draws, erases, etc.
    update(rightMole)

    val elapsedMillis = (System.currentTimeMillis - t0).toInt
    Thread.sleep((delayMillis - elapsedMillis) max 0)
  end while
end gameLoop
```

6. **Event handling.** A `BlockWindow`, which you implemented in task 6 in exercise `Excercise05`, can, via the call `nextEvent`, return `KeyPressed(key)` upon a key press and `WindowClosed` when the window is closed. If no event is available it returns `Undefined`. Use a loop to process all events until `Undefined` is encountered, as below:

```
def handleEvents(): Unit =
  var e = window.nextEvent()
  while e != BlockWindow.Event.Undefined do
    e match
      case BlockWindow.Event.KeyPressed(key) =>
        ???  // change direction for respective mole

      case BlockWindow.Event.WindowClosed =>
        ???  // end the game loop
```

```
      e = window.nextEvent()
    end while
  end handleEvents
```

7. **Flicker-free graphics.** To reduce flicker it is best in each iteration of the game loop (1) only to redraw what has changed in order to minimize the time spent drawing, and (2) when making changes draw the new parts before erasing the old parts. To avoid mole flicker you can "*draw first – erase after*" as in the following.[7]

```
window.setBlock(mole.nextPos, mole.color) // draw new
window.setBlock(mole.pos, Color.tunnel)   // erase old
mole.move()                               // update
```

[7] In game development one typically uses so-called *double buffering* (or even *triple buffering*) to achieve completely flicker-free graphics. However, this is beyond the course scope and is not supported by `PixelWindow`.

Chapter 7

Sequences and Enumerations

Concepts included in this week's studies:

- ☐ overview of Scala's collection library and collection methods
- ☐ class hierarchy in scala.collection
- ☐ Iterable
- ☐ Seq
- ☐ List
- ☐ ListBuffer
- ☐ ArrayBuffer
- ☐ WrappedArray
- ☐ sequence algorithm
- ☐ algorithm: SEQ-COPY
- ☐ in-place vs copy
- ☐ algorithm: SEQ-REVERSE
- ☐ registration
- ☐ algorithm: SEQ-REGISTER
- ☐ linear search
- ☐ algorithm: LINEAR-SEARCH
- ☐ time complexity
- ☐ memory complexity
- ☐ overview of string methods
- ☐ StringBuilder
- ☐ order
- ☐ built-in search methods
- ☐ find
- ☐ indexOf
- ☐ indexWhere
- ☐ built-in sorting methods
- ☐ sorted
- ☐ sortWith
- ☐ sortBy
- ☐ repeated parameters

7.1 Theory

7.1.1 What is a sequence?

- A sequence is a **series of elements** which
  - has **ordinal numbers** (e.g. numbered from zero)
  - is of a certain **type** (e.g. integers).
- A sequence can contain multiple elements that are equal.
- A sequence can be **empty** and then has length zero.
- Example of a non-empty sequence with duplicates:

```
scala> val xs = Vector(42, 0, 42, -9, 0, 5)
xs: scala.collection.immutable.Vector[Int] =
  Vector(42, 0, 42, -9, 0, 5)
```

- **Indexing** gives an element via its ordinal number:

```
scala> xs(2)
res0: Int = 42

scala> xs.apply(2)
res1: Int = 42
```

7.1.2 Example: A string is a sequence of characters

```
scala> "haj po daj"
```

Length? What is at the first position? Element type? Duplicates?

```
scala> "haj po daj".length
res1: Int = 10

scala> "haj po daj".apply(0)
res2: Char = h

scala> "haj po daj"(0)
res3: Char = h

scala> "haj po daj".distinct
res4: String = haj pod
```

7.1.3 Iterating over elements in a sequence

- To **iterate** means to (eng. *iterate*), aka traverse (eng. *traverse*), which involves **going through** and processing elements in a collection.
- Examples of iteration with `foreach`, `map`, `for`:

```
scala> val xs = Vector(1,2,3)
val xs: Vector[Int] = Vector(1, 2, 3)

scala> xs.foreach(x => println(x + 1))
2
3
4

scala> xs.map(_ + 1)
val res0: Vector[Int] = Vector(2, 3, 4)

scala> for x <- xs yield x - 1
val res1: Vector[Int] = Vector(0, 1, 2)
```

7.1.4 Add elements at the beginning and at the end of a sequence

- Using the methods `+:` and `:+` you can create a new sequence with new elements added at the start or at the end, respectively.
- Mnemonic: "**Colon on the collection side**"

```
scala> val xs = Vector(1,2,3)
scala> xs :+ 42         // gives new Vector(1, 2, 3, 42)
scala> 42 +: xs         // gives new Vector(42, 1, 2, 3)
```

- Semantics: operator notation with operators that **end with a colon** are **right-associative**
- The call `42 +: xs` is rewritten by the compiler to `xs.+:(42)`

```
scala> xs.+:(42)
res4: scala.collection.immutable.Vector[Int] = Vector(42, 1, 2, 3)
```

- Concatenation (joining) of sequences: `xs ++ ys`

7.1.5 Properties of some sequence collections in Scala

- `Vector`
  - **Immutable**. Fast to create copies with small changes.
  - Versatile performance: **good for most things**.
- `List`
  - **Immutable**. Fast to create copies with updates **at the beginning**.
  - Fast to work **at the beginning**, but **slow** to work at the **end** of the list.
  - Convenient & fast for **recursive** algorithms.
  - Slow for repeated **indexing** at arbitrary positions.
- `ArrayBuffer`

  - **Mutable**: **fast indexing & updating**.
  - Can **change size** after allocation. Fast to index everywhere.
- `ListBuffer`
  - **Mutable**: fast indexing & updating **at the beginning**.
  - Fast if you build up a sequence by many additions at the beginning.
- `Array` or `scala.collection.mutable.ArraySeq`
  - **Mutable**: **fast indexing & updating**.
  - Cannot **change size**; size is set at allocation.
  - Has special status in the JVM: provides fast allocation and access.

7.1.6 Which sequence collection should I choose?

- Choose `Vector` if ...
  a) you want immutability: `val xs = Vector[Int](1,2,3)`
  b) you need mutability (note `var`):
     `var xs = Vector.empty[Int]`
  c) you do not yet know which sequence collection is best; you can always change after you have measured performance and identified bottlenecks in repeated runs.
- Choose `List` if ...

  you have a **recursive** sequence algorithm or mainly work at the beginning.
- Choose `ArrayBuffer` if ...

  it is needed for performance reasons and you **do not** know the size at allocation:
  `val xs = scala.collection.mutable.ArrayBuffer.empty[Int]`
- Choose `ListBuffer` if ...

  it is needed for performance reasons and you only need to add at the beginning:
  `val xs = scala.collection.mutable.ListBuffer.empty[Int]`
- Choose `Array` or `ArraySeq` if ...

  it is really needed for performance reasons and you **know** the size at allocation:
  `val xs = Array.fill(initSize)(initValue)`

7.1.7 Some quirks with Array

- **Reference equality** (not content equality):

```
scala> Vector(1,2,3) == Vector(1,2,3) // content equality
val res0: Boolean = true

scala> Array(1,2,3) == Array(1,2,3) // reference equality
val res1: Boolean = false  // aaargh!!
```

Note: The method == between two `ArraySeq` gives **content equality**.

- Special syntax for allocation **without** explicit initialization:
  `val xs = new Array[String](1000)  // 1000 null references`
- Does not work as well with generic types:

```
scala> def box[T](x: T) = Vector[T](x)  // works fine

scala> def abox[T](x: T) = Array[T](x)
  error: No ClassTag available for T
```

7.1.8 Immutable or mutable?

- **Immutable**: Cannot change element references, but efficient at creating a copy which is (partially) changed **Vector** or **List**
- **Mutable**: can change element references
  - Cannot **change size** after allocation:
    Array or **ArraySeq**: index and update anywhere
  - Can also change size after allocation:
    ArrayBuffer or **ListBuffer**
- **Often immutable sequence collections work great**, but if you **after performance measurement** find a bottleneck you can switch from **Vector** to e.g. **ArrayBuffer**.

7.1.9 What is a sequence algorithm?

- An algorithm is a stepwise description of the solution to a problem.
- A **sequence algorithm** is an algorithm where **elements in a sequence** constitute an important part of the **problem description** or **solution**.
- Example problem: sort a sequence of people by their age.
- **Seven** common programming problems that are solved with a sequence algorithm:
  - **Copying** all elements in a sequence to a **new** sequence
  - **Updating** the sequence: removing, adding, changing **individual** elements
  - **Transforming**: applying a **function** to **all** elements
  - **Filtering**: selecting certain elements that satisfy a **condition**
  - **Searching** for an element that satisfies a **search criterion**
  - **Sorting** according to some **order**
  - **Registering**: categorize or **count elements** with certain properties

---

7.1.10 Using ready-made sequence collection methods

- Often you can implement sequence algorithms by calling one or more **ready-made** methods.
- These ready-made methods are **optimized and well-tested** and are preferable if possible.
- Study the **quickref** to see what can be done with ready collections.
- It is **educational** to "**reinvent the wheel**" and implement some basic sequence algorithms **yourself** for better understanding, even though they already exist in Scala's collection library.
- Deepening: An overview of the collections in Scala's standard library: `https://docs.scala-lang.org/overviews/collections-2.13/introduction.html`
- (There will be more about implementation of collections and algorithms in the advanced course pfk.)

7.1.11 Some useful collection methods when implementing sequence algorithms

`xs.map(f)`	transformation, equivalent to `for x <- xs yield f(x)`
`xs.map(x => x)`	copying, equivalent to `for x <- xs yield x`
`xs.filter(p)`	filtering, include x if p(x)
`xs.filterNot(p)`	filtering, include x if !p(x)
`xs.distinct`	filtering, remove duplicates
`xs.take(n)`	new sequence with the first n elements, rest skipped
`xs.drop(n)`	new sequence with the first n elements skipped
`xs.takeWhile(p)`	filter, include from the start as long as p(x)
`xs.dropWhile(p)`	filter, skip from the start as long as p(x)
`xs.find(p)`	search from the front for the first element x where p(x) is true
`xs.indexOf(x)`	search from the front for index of element equal to x
`xs.lastIndexOf(x)`	search from the back for index of element equal to x
`xs.sorted`	sort with built-in (implicit given) order
`xs.sorted.reverse`	sort in reverse order
`xs.sortBy(f)`	sort according to f(x)
`xs.sortWith(lt)`	sort according to "less than" function `lt: (A, A) => Boolean`
`xs.count(p)`	count the number of elements where p(x) is true

Learn more handy methods in quickref

7.1.12 Updated sequence with the powerful method `patch`

The method `patch` can be used as: `xs.patch(fromPos, ys, nbrReplaced)`
to create a **new** sequence where **one** or **multiple** elements in xs are...

- replaced
- removed
- inserted

.. with new elements from `ys`

```
scala> val xs = Vector(1,2,3)

scala> xs.patch(2, Vector(-1), 1)     // replaced one elem
res0: scala.collection.immutable.Vector[Int] = Vector(1, 2, -1)

scala> xs.patch(1, Vector(42), 0)     // inserted one elem
res11: scala.collection.immutable.Vector[Int] = Vector(1, 42, 2, 3)

scala> xs.patch(0, Vector(), 2)       // removed two elems
res2: scala.collection.immutable.Vector[Int] = Vector(3)
```

7.1.13 Using for-expressions for filtering with guard

In a for-expression you can have a **guard** in the form of a boolean expression after the keyword `if`. Then the expression after `yield` will only be executed if the guard is true.

Syntax is as follows: (parentheses are not needed around the guard expression)

```
for x <- xs if expression1 yield expression2
```

Example:

```
scala> val odd = for x <- 1 to 6 if x % 2 == 1 yield x
```

odd becomes `Vector(1, 3, 5)`

7.1.14 Using the collection method `filter` for filtering

All collections in `scala.collection` have the method `filter`. It takes a predicate parameter `p: T => Boolean` and returns a new collection with the elements for which the predicate is true.

```
xs.filter(p)
```

Example: Suppose `xs` is `(1 to 6).toVector`

```
xs.filter(_ % 2 == 1)
```

The expression result is `Vector(1, 3, 5)`, which corresponds to:

```
for x <- xs if x % 2 == 1 yield x
```

In fact, the Scala compiler rewrites for-expressions with guards to calls of the method `filter` before code generation occurs.

7.1.15 Common sequence problems as function headers

Input and output for some common sequence problems:

```
def copy(xs: Vector[Int]): Vector[Int] = ???

def filter(xs: Vector[Int], p: Int => Boolean): Vector[Int] = ???

def findIndices(xs: Vector[Int], p: Int => Boolean): Vector[Int] = ???

def sort(xs: Vector[Int]): Vector[Int] = ???

def freq(xs: Vector[Int]): Vector[(Int, Int)] = ???  // (integer, frequency)
```

Exercise: How to implement these with `for`-expressions or ready collection methods?
Tips: For `sort&freq` see `sorted`, `distinct`, `count` in quickref

7.1.16 Implementation of sequence problems with `for`-expressions or ready collection methods

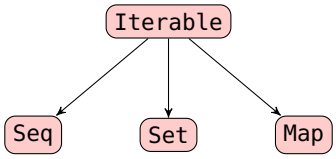
```
def copy(xs: Vector[Int]): Vector[Int] = for x <- xs yield x

def filter(xs: Vector[Int], p: Int => Boolean): Vector[Int] =
  for x <- xs if p(x) yield x

def findIndices(xs: Vector[Int], p: Int => Boolean): Vector[Int] =
  (for i <- xs.indices if p(xs(i)) yield i).toVector

def sort(xs: Vector[Int]): Vector[Int] = xs.sorted // more on sorting later

def freq(xs: Vector[Int]): Vector[(Int, Int)] = // more on registering soon
  for x <- xs.distinct yield x -> xs.count(_ == x)
```

Exercise: How to implement these with `map` and `filter` or other ready collection methods?

7.1.17 Implementation of sequence problems with `map`, `filter`

```
def copy(xs: Vector[Int]): Vector[Int] = xs.map(x => x)

def filter(xs: Vector[Int], p: Int => Boolean): Vector[Int] = xs.filter(p)

def findIndices(xs: Vector[Int], p: Int => Boolean): Vector[Int] =
  xs.indices.filter(i => p(xs(i))).toVector

def sort(xs: Vector[Int]): Vector[Int] = xs.sorted // more on sorting later

def freq(xs: Vector[Int]): Vector[(Int, Int)] = // more on registering soon
  xs.distinct.map(x => x -> xs.count(_ == x))
```

7.1.18 Hierarchy of collection types in `scala.collection` v2.13



`Iterable` has methods implemented with:

```
def foreach[U](f: Elem => U): Unit
def iterator: Iterator[A]
```

`Seq`: ordered in sequence
`Set`: unique elements
`Map`: pairs of (key, value)

The collection **`Vector`** is a `Seq` which is an `Iterable`.
Concrete collections are divided into these packages:

`scala.collection.immutable` where several are **automatically** imported
`scala.collection.mutable` which **must be imported** explicitly
(exception: primitive `scala.Array`)

7.1.19 Keep it open: use `Seq`

The type **`collection.immutable.Seq`** is a supertype of all sequence collections in `collection.immutable`. Example: copying a sequence:

- Copying a **specific** integer sequence:

```
def copyIntVector(xs: Vector[Int]): Vector[Int] = for x <- xs yield x
```

- Copying that works for all immutable integer sequences:

```
def copyIntSeq(xs: Seq[Int]): Seq[Int] = for x <- xs yield x
```

```
scala> val xs = Vector(1,2,3)
xs: Vector[Int] = Vector(1, 2, 3)

scala> val ys = copyIntVector(xs)
ys: Vector[Int] = Vector(1, 2, 3)

scala> val zs = copyIntSeq(xs)
val zs: Seq[Int] = Vector(1, 2, 3)
```

7.1.20 Implementation with generic functions

By generalizing function headers our solutions become usable for **all** sequences of type `Seq[T]`, where the unbound **type parameter** `T` at call time can be bound to any type. (More about type parameters later.)

```
def copy[T](xs: Seq[T]): Seq[T] = xs.map(x => x)

def filter[T](xs: Seq[T], p: T => Boolean): Seq[T] = xs.filter(p)
```

```
def findIndices[T](xs: Seq[T], p: T => Boolean): Seq[Int] =
  xs.indices.filter(i => p(xs(i))).toVector

def sort[T: Ordering](xs: Seq[T]): Seq[T] = xs.sorted // more on Ordering later

def freq[T](xs: Seq[T]): Seq[(T, Int)] =
  xs.distinct.map(x => x -> xs.count(_ == x))
```

The standard library's methods try to arrange so that the same concrete type in comes out, but sometimes another suitable concrete collection is chosen, e.g. an `Array` can become an `ArrayBuffer`.

7.1.21 Deepening: Using Java collections in Scala with `CollectionConverters`

Using `import scala.jdk.CollectionConverters.*`
gives convenient **interoperability** with Java and its standard library,
especially the methods **`asJava`** and **`asScala`**:

```
scala> import scala.jdk.CollectionConverters.*

scala> Vector(1,2,3).asJava
res0: java.util.List[Int] = [1, 2, 3]

scala> val xs = new java.util.ArrayList[String]()
xs: java.util.ArrayList[String] = []

scala> xs.add("hello")
res1: Boolean = true

scala> xs.asScala
res2: scala.collection.mutable.Buffer[String] = Buffer(hello)
```

Read more here:
https://docs.scala-lang.org/overviews/collections-2.13/conversions-between-java-and-scala-collections.html

---

7.1.22 Deepening: Creating generic Array

- In JVM bytecode it is unfortunately **not** possible to create a primitive generic array.
- Machine code must instead create an array of the most general reference type `Object` and then **type-test and typecast at runtime**.
  See for example the Java implementation of `ArrayList`:
  http://developer.classpath.org/doc/java/util/ArrayList-source.html

- But it is ***possible*** to create a generic array in Scala (but not in Java). This requires a `reflect.ClassTag` which provides type information at runtime for arrays.

```
scala> def fill[T](n: Int, x: T): Array[T] = Array.fill(n)(x)
-- Error:
1 |def fill[T](n: Int, x: T): Array[T] = Array.fill(n)(x)
  |                                                     ^
  |  No ClassTag available for T

scala> def fill[T: reflect.ClassTag](n: Int, x: T): Array[T] = Array.fill(n)(x)

scala> fill(42, "hello")
res2: Array[String] = Array(hello, hello, hello, hello, hello, hello, hello, hell
```

- The compiler then creates machine code that automatically performs the type conversions.

7.1.23 Repeated parameters become a sequence

With an asterisk after the parameter type, the number of arguments can vary:

```
def sumSizes(xs: String*): Int = xs.map(_.length).sum
```

```
scala> sumSizes("Zaphod")
res0: Int = 6

scala> sumSizes("Zaphod","Beeblebrox")
res1: Int = 16

scala> sumSizes("Zaphod","Beeblebrox","Ford","Prefect")
res3: Int = 27

scala> sumSizes()
res4: Int = 0
```

Repeated parameters (eng. *repeated parameters*) become a sequence of type `Seq` and more specifically an `ArraySeq`

7.1.24 Sequence collection as argument to repeated parameters

```
def sumSizes(xs: String*): Int = xs.map(_.size).sum

val veg = Vector("gurka","tomat")
```

If you *already have* a sequence collection, you can apply it to a function that has repeated parameters using an asterisk $*$
It should be written directly **after** the sequence collection, which you want the compiler

to interpret as a sequence of arguments, like this:

```
scala> sumSizes(veg*)
res5: Int = 10
```

7.1.25 Enumerations have an order

An enumeration of suits in a deck of cards with **enum**:

```
enum Suit:
  case Spade, Heart, Club, Diamond
```

Important enum methods to handle the elements' **order**:
`ordinal fromOrdinal values valueOf`

```
scala> Suit.Spade.ordinal      // from element to integer
val res0: Int = 0

scala> Suit.Club.ordinal
val res1: Int = 2

scala> Suit.fromOrdinal(3)    // from integer to element
val res2: Suit = Diamond

scala> Suit.values            // all elements in order
val res3: Array[Suit] = Array(Spade, Heart, Club, Diamond)

scala> Suit.valueOf("Spade")  // from string to element
val res4: Suit = Spade
```

7.1.26 Enumerations can have parameters and members

An **enum** can have parameters. Use **val** for external visibility:

```
enum Color(val consoleColor: String):
  case Black extends Color(Console.BLUE) // Blue color is visible on black background
  case Red   extends Color(Console.RED)
```

Inside the **enum** body you can have members, e.g. methods:

```
enum Suit(val color: Color):
  def show(isConsoleColor: Boolean = true): String =
    if isConsoleColor then color.consoleColor + toString + Console.RESET
    else toString

  case Spade   extends Suit(Color.Black)
  case Heart   extends Suit(Color.Red)
  case Club    extends Suit(Color.Black)
  case Diamond extends Suit(Color.Red)
```

```
scala> println(Suit.Club.show(isConsoleColor = false))
Club
```

7.1.27 Enum can correspond to full-fledged case classes

If you want to do pattern matching on enum values, parameters on the alternatives are needed for them to correspond to case classes:

```
enum Veg:
  def taste: String
  case Tomato(taste: String)
  case Banana(taste: String)
```

The above is automatically expanded by the compiler to the equivalent of this:

```
sealed trait Veg:
  def taste: String
object Veg:
  case class Tomato(taste: String) extends Veg
  case class Banana(taste: String) extends Veg
```

7.1.28 Enum and pattern matching

With parameters on each case and an abstract member for each attribute...

```
enum Veg:
  def taste: String
  case Tomato(taste: String)
  case Banana(taste: String)
```

...the automatic expansion to case classes makes this work nicely:

```
scala> val v = Veg.Tomato("nice")
val v: Veg = Tomato(nice)             // note the type: Veg

scala> v.taste  // works because Veg has a taste
val res0: String = nice

scala> val dontLikeBananas = v match:
           case Veg.Tomato(t) => t
           case Veg.Banana(_) => "always bad!"
```

The abstract member `def taste: String` is needed so that the attribute is visible through references of the less specific type `Veg`.
(More about abstract members later this week in inheritance.)

7.1.29 Advantages of `enum` compared to enumerations with integers

Why not just do this?

```
val (spade, heart, club, diamond) = (0, 1, 2, 3)
```

All elements have the same specific type according to the `enum` declaration:

```
scala> Suit.Heart              // all elements are of type Suit
val res5: Suit = Heart
```

- This is safer compared to just using integer values: the compiler can help distinguish between elements of different types and report errors if you use the wrong type by mistake.
- Invalid values cannot be represented (compared to all possible integers where only some are relevant).

Practice **enum** in this week's exercise `sequences` and lab `shuffle`.

7.1.30 Registration

- **Registration** involves algorithms for categorizing or counting the number of occurrences of elements with certain specific properties.
- Example:
  Outcome frequency when rolling a die 1000 times:

outcome		count
1	→	178
2	→	187
3	→	167
4	→	148
5	→	155
6	→	165

7.1.31 Registration of Dice Rolls in `Array`

We let position 0 represent the number of ones, position 1 represent the number of twos, etc. **Exercise**: implement ???

```
scala> def rollDice(): Int = scala.util.Random.nextInt(6) + 1

scala> val reg = new Array[Int](6)
reg: Array[Int] = Array(0, 0, 0, 0, 0, 0)

scala> for k <- 1 to 1000 do
         val i = ??? //roll dice, calculate correct index
         ??? //register roll in reg at correct position

scala> for i <- 1 to 6 do println(s"$i: ${reg(i - 1)}")
1: 178
2: 187
3: 167
4: 148
5: 155
6: 165
```

7.1.32 Registration of Dice Rolls in `Array`

Solution:

```
scala> def rollDice() = scala.util.Random.nextInt(6) + 1

scala> val reg = new Array[Int](6)
reg: Array[Int] = Array(0, 0, 0, 0, 0, 0)

scala> for k <- 1 to 1000 do
         val i = rollDice() - 1
         reg(i) = reg(i) + 1    // or: reg(i) += 1

scala> for i <- 1 to 6 do println(s"$i: ${reg(i - 1)}")
1: 178
2: 187
3: 167
4: 148
5: 155
6: 165
```

7.1.33 Create solutions to sequence problems from scratch

- Normally, one uses ready-made collection methods
- There is often a ready-made method that does what you want
- Otherwise, you can often do what you want by combining several ready-made collection methods

- We will now for educational purposes implement some own variants of updating from scratch.

For problems of the type KUTFSSR, it is part of the course to be able to 1) solve these with ready-made collection methods, and 2) implement own solutions using sequence, alternative, repetition, abstraction (**SARA**).

7.1.34 Create new sequence collection or change in place?

Two different principles in sequence algorithm construction:

- Create a **new sequence** without changing the input sequence
- Change **in place** (eng. *in-place*) in a **mutable** sequence

Choose between creating a new sequence or changing in place?

- Often it is **easiest to create a new collection** and copy over the elements after any changes while looping.
- If you have very large collections, you may need to change in place to save time/memory.

7.1.35 Algorithm: SEQ-COPY

Pseudocode for the algorithm SEQ-COPY which copies a sequence, here an Array of integers:

Input : Integer array xs
Output : A new integer array which is a copy of xs.

$result \leftarrow$ a new array with space for $xs.length$ elements
$i \leftarrow 0$
while $i < xs.length$ **do**
  $result(i) \leftarrow xs(i)$
  $i \leftarrow i + 1$
end
$result$

7.1.36 Implementation of SEQ-COPY with `while`

```
object seqCopy:

  def arrayCopy(xs: Array[Int]): Array[Int] =
    val result = new Array[Int](xs.length)
    var i = 0
    while i < xs.length do
      result(i) = xs(i)
      i += 1
    result

  def test: String =
    val xs = Array(1,2,3,4,42)
    val ys = arrayCopy(xs)
    if xs sameElements ys then "OK!" else "ERROR!"

  def main(args: Array[String]): Unit = println(test)
```

`xs.sameElements(ys)` is needed because == on an `Array` gives reference equality.

7.1.37 Type alias to abstract type names

With the help of the keyword **`type`** one can declare a **type alias** to give an **alternative** name to a certain type. Example:

```
scala> type Pt = (Int, Int)            // type alias
scala> type Pts = Vector[Pt]           // nested type alias

scala> def distToOrigo(pt: Pt): Double = math.hypot(pt._1, pt._2)

scala> val xs: Pts = Vector((1,1), (2,2), (3,4))
val xs: Pts = Vector((1,1), (2,2), (3,4))
```

```

scala> xs.head
val res0: Pt = (1,1)

scala> xs.map(distToOrigo)
val res1: Vector[Double] = Vector(1.4142135623730951, 2.8284271247461903, 5.0)
```

Type alias can be useful when:

- you have a long and complicated type and want to use a shorter name,
- you want to easily swap implementation later
  (e.g. if you want to use a case class instead of a tuple).

7.1.38 Example: SEQ-INSERT/REMOVE-COPY

Now we shall "reinvent the wheel" and as practice implement **insertion** and **removal** to a **new** sequence without use of sequence collection methods (except `length` and `apply`):

```
object PointSeqUtils:
  type Pt = (Int, Int)  // a type alias to make the code more concise

  def primitiveInsertCopy(pts: Array[Pt], pos: Int, pt: Pt): Array[Pt] = ???

  def primitiveRemoveCopy(pts: Array[Pt], pos: Int): Array[Pt] = ???
```

7.1.39 Pseudo-code for SEQ-INSERT-COPY

Input :`pts: Array[Pt], pt: Pt, pos: Int`

Output: A copy of pts but where pt is inserted at position pos

---

$result \leftarrow$ a new `Array[Pt]` with space for $pts.length + 1$ elements
for $i \leftarrow 0$ **to** $pos - 1$ **do**
  $result(i) \leftarrow pts(i)$
end
$result(pos) \leftarrow pt$
for $i \leftarrow pos + 1$ **to** $xs.length$ **do**
  $result(i) \leftarrow pts(i-1)$
end
$result$

Exercise: Write pseudo-code for SEQ-REMOVE-COPY

7.1.40 Insertion/removal in a copy of primitive Array

```
object PointSeqUtils:
  type Pt = (Int, Int)  // a type alias to make the code more concise

  def primitiveInsertCopy(pts: Array[Pt], pos: Int, pt: Pt): Array[Pt] =
    val result = new Array[Pt](pts.length + 1)  // initialized with null
    for i <- 0 until pos do result(i) = pts(i)
    result(pos) = pt
    for i <- pos + 1 to pts.length do result(i) = pts(i - 1)
    result

  def primitiveRemoveCopy(pts: Array[Pt], pos: Int): Array[Pt] =
    if pts.length > 0 then
      val result = new Array[Pt](pts.length - 1) // initialized with null
      for i <- 0 until pos do result(i) = pts(i)
      for i <- pos + 1 until pts.length do result(i - 1) = pts(i)
      result
    else Array.empty

  // ovan metoder implementerade med hjälp av den kraftfulla metoden patch:

  def insertCopy(pts: Array[Pt], pos: Int, pt: Pt) = pts.patch(pos, Array(pt), 0)

  def removeCopy(pts: Array[Pt], pos: Int) = pts.patch(pos, Array.empty[Pt], 1)
```

One can make **very easy mistakes** on boundaries/special cases: +-1, to/until, empty sequence etc.

7.1.41 Example: PolygonWindow

- A polygon can be represented as a sequence of points, where each point is an integer pair.
- `PolygonWindow` below is a window that can draw a polygon.

```
class PolygonWindow(width: Int, height: Int):
  val w = new introprog.PixelWindow(width, height, title = "PolygonWindow")

  def draw(pts: Seq[(Int, Int)]): Unit =
    if pts.size > 0 then
      for i <- 1 until pts.size do
        w.line(pts(i - 1)._1, pts(i - 1)._2, pts(i)._1, pts(i)._2)
      val last = pts.length - 1
      w.line(pts(last)._1, pts(last)._2, pts(0)._1, pts(0)._2)
```

```
object PolygonTest:
  val star =  Array((100,180), (150,100), (180,180), (90,130), (200, 130))
  val pw = new PolygonWindow(400,400)
  def main(args: Array[String]): Unit = pw.draw(star.toSeq)
```

7.1.42 Implementing Polygon

- A polygon can be represented as a sequence of points.
- We want to be able to add points, as well as remove points.
- A polygon can be implemented in many different ways:
  - **Mutable**
    - With the points in a `Array`
    - With the points in a `ArrayBuffer`
    - With the points in a `ListBuffer`
    - With the points in an `Vector`
    - With the points in a `List`
  - **Immutable**
    - As a case class with an immutable `Vector` that returns a new object upon update. We can allow the data structure to be **public** since everything is immutable.
    - As a "regular" class with some suitable **private** data structure where we **do not** allow mutation after initialization and return a new object upon update.

Choice of implementation **depends on** context & usage!

7.1.43 Example: PolygonArray, in-place modification

```
class PolygonArray(val maxSize: Int):
  type Pt = (Int, Int)
  private val points = new Array[Pt](maxSize)  // initialized with null
  private var n = 0
  def size = n

  def draw(w: PolygonWindow): Unit = w.draw(points.take(n).toSeq)

  def append(pts: Pt*): Unit =
    for i <- pts.indices do points(n + i) = pts(i)
    n += pts.length

  def insert(pos: Int, pt: Pt): Unit = // exercise: change pt to varargs pts
    for i <- n until pos by -1 do points(i) = points(i - 1)
    points(pos) = pt
    n += 1

  def remove(pos: Int): Unit = // exercise: change pos to fromPos, replaced
    for i <- pos until n do points(i) = points(i + 1)
    n -= 1

  override def toString = points.mkString("PolygonArray(",",",")")
```

- Initially, `points` is filled with nulls.
- The variable `n` keeps track of how many are actually used.

7.1.44 Example: PolygonVector, variable reference to immutable data structure

```
class PolygonVector:
  type Pt = (Int, Int)
  private var points = Vector.empty[Pt]  // note var declaration to allow mutation
  def size = points.size

  def draw(w: PolygonWindow): Unit = w.draw(points.take(size))

  def append(pts: Pt*): Unit =
    points ++= pts.toVector

  def insert(pos: Int, pt: Pt): Unit = // exercise: change pt to varargs pts
    points = points.patch(pos, Vector(pt), 0)

  def remove(pos: Int): Unit = // exercise: change pos to fromPos, replaced
    points = points.patch(pos, Vector(), 1)

  override def toString = points.mkString("PrimitivePolygon(",",",")")
```

7.1.45 Example: Polygon as immutable case class

```
object Polygon:
  type Pt = (Int, Int)
  type Pts = Vector[Pt]
  def apply(pts: Pt*) = new Polygon(pts.toVector)

case class Polygon(points: Polygon.Pts):
  import Polygon.Pt

  def size = points.size // for convenience but not really necessary (why?)

  def append(pts: Pt*): Polygon = copy(points ++ pts.toVector)

  def insert(pos: Int, pts: Pt*): Polygon = copy(points.patch(pos, pts, 0))

  def remove(pos: Int, replaced: Int = 1): Polygon =
    copy(points.patch(pos, Seq(), replaced))

  override def toString = points.mkString("Polygon(", "," ,")")
```

7.1.46 Sorting and comparing strings lexicographically

Character standard UTF-8: All uppercase letters are "less" than all lowercase letters:

```
scala> Array("hej","Hej","gurka").sorted
```

```
res0: Array[String] = Array(Hej, gurka, hej)
```

- Suppose we want to solve this problem "from scratch":
  to sort a sequence of strings
- Follow-up questions:
  - What does it mean for two strings to be "equal"?
  - What does it mean for one string to be "less" than another?
- To sort a sequence of strings we need to solve these subproblems:
  - **to compare strings**
  - **searching in sequences**
  - **SWAP** (if in-place sorting on a mutable sequence)

Here we use string comparison, searching and sorting to illustrate typical **imperative algorithms**. **Normally** one uses **built-in solutions** to these problems!

7.1.47 Comparing strings: equality

Suppose we cannot do `s1 == s2` but only compare strings character by character, e.g. like this: `s1(i) == s2(i)`. Also suppose we only have access to the methods `length` and `apply` on strings, as well as **`while`** and primitive type variables. **Solve the problem of *determining if two strings are equal.***

- Input: two strings
- Output: **`true`** if equal, otherwise **`false`**

1. Figure out your solution idea
2. Formulate the algorithm in pseudocode
3. Implement the algorithm in Scala:
   `def isEqual(s1: String, s2: String): Boolean = ???`

7.1.48 Algorithm example: string equality, pseudocode

```
def isEqual(s1: String, s2: String): Boolean =
  if (/* equal lengths */) then
    var foundDiff = false
    var i = /* first index */
    while !foundDiff && /* i within index bounds */ do
      if /* character at position i are different */ then foundDiff = true
      else i = /* next index */
    end while
    !foundDiff
  else false
end isEqual
```

This is a variant of so-called **linear search** where we search from the start of a sequence until we find what we are looking for (here we look for differing characters).

What does the implementation in executable Scala look like?

7.1.49 Algorithm example: string equality, implementation

```
def isEqual(s1: String, s2: String): Boolean =
  if s1.length == s2.length then
    var foundDiff = false
    var i = 0
    while !foundDiff && i < s1.length do
      if s1(i) != s2(i) then foundDiff = true
      else i += 1
    end while
    !foundDiff
  else false
end isEqual
```

7.1.50 Comparing strings: "less than"

By `s1 < s2` we mean that string s1 should be sorted before string s2 according to how the individual characters are ordered with the expression `s1(i) < s2(i)`.
Also suppose we only have access to the methods `length` and `apply` on strings, as well as **`while`** and primitive type variables, and `math.min`
Solve the problem of *determining if one string is "less" than another.*

- Input: two strings, s1, s2
- Output: **`true`** if s1 should be sorted before s2 otherwise **`false`**

1. Figure out your solution idea
2. Formulate the algorithm in pseudocode
3. Implement the algorithm in Scala:
   `def isLessThan(s1: String, s2: String): Boolean = ???`

7.1.51 Comparing strings: "less than"

Pseudocode:

```
def isLessThan(s1: String, s2: String): Boolean =
  val minLength = /* minimum of the lengths of s1 and s2 */

  def firstDiff(s1: String, s2: String): Int =
    /* index of first difference (if they start equal: minLength) */

  val diffIndex = firstDiff(s1, s2)
  if diffIndex == minLength then /* s1 is shorter than s2 */
```

```
  else /* character s1(diffIndex) is less than character s2(diffIndex) */
```

7.1.52 Comparing strings: "less than"

```
def isLessThan(s1: String, s2: String): Boolean =
  val minLength = math.min(s1.length, s2.length)

  def firstDiff(s1: String, s2: String): Int =
    var foundDiff = false
    var i = 0
    while !foundDiff && i < minLength do
      if (s1(i) != s2(i)) foundDiff = true
      else i += 1
    end while
    i
  end firstDiff

  val diffIndex = firstDiff(s1, s2)
  if diffIndex == minLength then s1.length < s2.length
  else s1(diffIndex) < s2(diffIndex)
end isLessThan
```

7.1.53 Searching

- **Searching** appears in many forms:
  in a data structure, whatever it may be, one often wants to be able to
  find an element with a certain property.
  Some ready linear searches in Scala's standard library:

```
scala> Vector("cucumber","tomato","broccoli").indexOf("tomato")
res0: Int = 1

scala> Vector("cucumber","tomato","broccoli").indexWhere(_.contains("o"))
res1: Int = 1

scala> Vector("cucumber","tomato","broccoli").find(_.contains("o"))
res2: Option[String] = Some(tomato)
```

- Searching for a certain index in a sequence:
  - Input: a sequence and a **search criterion**
  - Output: index of the first found element, otherwise -1
- Two typical variants of searching in a sequence:
  - Linear search: start from the beginning and search until a sought element is found

- Binary search: assume the sequence is sorted; start in the middle, choose the right half ...

7.1.54 Linear searching: find index for element x

Implement `indexOf`:

```
def indexOf(xs: Vector[Int], x: Int): Int = ???
```

Output: index `i` where `xs(i) == x`
If value is missing, return `-1`

```
def indexOf(xs: Vector[Int], x: Int): Int =
  var i = 0
  var found = false
  while !found && i < xs.length do
    if (xs(i) == x) found = true
    else i += 1
  if (found) i else -1
```

(If you are curious about binary search, see chapter 12: Optional Deepening.)

7.1.55 Sorting

Problem: We have an unsorted sequence of integers. We want to arrange this unsorted sequence into a sorted sequence from smallest to largest.

A *generalization* of the problem:

We have many elements of arbitrary type and an **ordering relation** that defines what we mean by one element being *less than* or *greater than or equal to* another element.

We want to solve the problem of arranging the elements so that for each element at position i the succeeding element at position $i+1$ is greater than or equal to the element at position i.

- Insertion sort **solution idea**: Take one element at a time from the unsorted list and **insert** it at the **correct position** in the sorted list and repeat until no unsorted elements remain.

7.1.56 Algorithmic complexity

- **Complexity** of an algorithm is often studied by analyzing how memory usage and runtime grow as the problem size increases

- Examples of different **time complexities**: time can, e.g., grow linearly, exponentially, or logarithmically with the number of elements
- Example **linear searching** (i.e., searching from the start until found):
  The number of repetition rounds is proportional to the number of elements and in the worst case we need to search till the end.
- More about complexity in the next course (pfk).

7.1.57 There are many different sorting algorithms

- Visualization of 15 different sorting algorithms in 6 min:
  https://www.youtube.com/watch?v=kPRA0W1kECg
- Different sorting algorithms have different time & memory complexity: best case, worst case, average case, nearly sorted, etc.
  https://en.wikipedia.org/wiki/Sorting_algorithm
- Different sorting algorithms are more or less suitable for parallelization on many cores.

7.1.58 Bogo sort

```
def bogoSort(xs: Vector[Int]) =
  var result = xs
  while result != result.sorted do
    result = scala.util.Random.shuffle(result)
  result
```

When does this finish?

Average number of comparisons for n elements: $n \cdot n!$

https://en.wikipedia.org/wiki/Bogosort

7.1.59 Sort to new vector with insertion sort: pseudo-code

It is probably easier to understand **insertion sort** if you sort to a new vector.
We will later see how to sort "in place" (eng. *in place*) in an array.

Input: an unsorted vector of integers
Output: a new, sorted vector of integers

```
def insertionSort(xs: Vector[Int]): Vector[Int] =
  val sorted = /* empty ArrayBuffer */
  for /* all elements in xs */ do
     /* linear search correct position in sorted */
```

```
    /* insert element at right position in sorted */
  end for
  sorted.toVector
```

7.1.60 Sort to new vector with insertion sort: implementation

```
def insertionSort(xs: Vector[Int]): Vector[Int] =
  val sorted = scala.collection.mutable.ArrayBuffer.empty[Int]
  for elem <- xs do
     // linear search correct position in sorted:
     var pos = 0
     while pos < sorted.length && sorted(pos) < elem do
       pos += 1
     end while
     // insert element at right position in sorted:
     sorted.insert(pos, elem)
  end for
  sorted.toVector
end insertionSort
```

7.1.61 Sort to new collection with arbitrary ordering predicate

```
def sortWith(xs: Vector[Int])(lt: (Int, Int) => Boolean ): Vector[Int] =
  val sorted = scala.collection.mutable.ArrayBuffer.empty[Int]
  for elem <- xs do  // insertion sort using lt as "less than"
     var pos = 0
     while pos < sorted.length && lt(sorted(pos), elem) do
       pos += 1
     end while
     sorted.insert(pos, elem)
  end for
  sorted.toVector
end sortWith
```

```
scala> val xs = Vector(1,2,1,2,12,42,1)

scala> sortWith(xs)(_ < _)
val res0: Vector[Int] = Vector(1, 1, 1, 2, 2, 12, 42)

scala> sortWith(xs)(_ > _)
val res1: Vector[Int] = Vector(42, 12, 2, 2, 1, 1, 1)
```

7.1.62 Insertion sort in place – pseudo-code

Input: an array of integers
Output: the same array, but now sorted

```
def insertionSortInPlace(xs: Array[Int]): Unit =
  for i <- 1 until xs.length do  //from SECOND to last
    var j = i
    while j > 0 && xs(j - 1) > xs(j) do
      /* swap places of xs(j) and xs(j - 1) */
      j -= 1;  // step backwards
```

See animation here: Insertion sort on Wikipedia
Go through all special cases and verify that this works!

7.1.63 Insertion sort in place – implementation

```
def insertionSortInPlaceSwap(xs: Array[Int]): Unit =
  def swap(i: Int, j: Int): Unit =
    val temp = xs(i)
    xs(i) = xs(j)
    xs(j) = temp
  end swap

  for i <- 1 until xs.length do  //from SECOND to last
    var j = i
    while j > 0 && xs(j - 1) > xs(j) do
      swap(j, j - 1)
      j -= 1;  // step backwards
    end while
  end for
end insertionSortInPlaceSwap
```

7.2 Exercise `Excercise07`

Goals

- ☐ Be able to read and write pseudocode for sequence algorithms and implement sequence algorithms according to pseudocode.
- ☐ Be able to implement sequence algorithms, both by copying to a new sequence and by in-place modification of an existing sequence.
- ☐ Be able to use built-in methods for updating, linear searching in, and sorting sequence collections.
- ☐ Be able to describe the difference in the use of mutable and immutable sequences, especially during updates.
- ☐ Understand how the sorting order is defined for strings.
- ☐ Be able to sort sequence collections containing objects of basic types using built-in and custom sorting orders with the methods `sorted`, `sortBy` and `sortWith`.
- ☐ Be able to implement linear searching according to different search criteria.
- ☐ Be able to describe the properties of the sequence collections `Vector`, `List`, `Array`, `ArrayBuffer`, and `ListBuffer`.
- ☐ Understand side effects of updating shared references to mutable elements.
- ☐ Be able to use functions with repeated parameters.
- ☐ Know how to implement functions with repeated parameters.
- ☐ Be able to implement integer counting in an integer array.

Preparations

- ☐ Study the concepts in chapter 7

7.2.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with description.*

Match each concept with the (simplified) description that best fits:

element	1	A	defines how elements of a type are arranged
collection	2	B	data structure with elements of same type
collection library	3	C	algorithm arranging elements in a specific order
sequence(collection)	4	D	algorithm finding elements according to a search criterion
sequence algorithm	5	E	how execution time grows with problem size
order	6	F	searching a sequence until element found
sorting	7	G	object in a data structure
searching	8	H	algorithm counting elements with certain properties
linear search	9	I	solution to problem using a sequence collection
counting	10	J	many ready-made collections with different properties
time complexity	11	K	how memory usage grows with problem size
memory complexity	12	L	zero or more elements of same type in a specific order

Task 2. *Different sequence collections.* Match each sequence collection with the (simplified) description that best fits:

`Vector`	1	A	mutable, fast indexing, can change size
`List`	2	B	immutable, quickly returns any modified collection
`Array`	3	C	immutable, quickly returns new collection modified at start
`ArrayBuffer`	4	D	primitive, mutable, fast indexing, fixed size
`ListBuffer`	5	E	mutable, fast to modify at start

Task 3. *Using sequence collections.* Assume the following variables are visible in the current namespace:

```
val xs: Vector[Int] = Vector(1, 2, 3)
val x: Int = 0
```

a) Match each expression on the left with the corresponding result on the right. If you are unsure of the result, consult the quick reference and test in REPL.
Tip: "colon on the collection side".

`x +: xs`	1	A	`true`
`xs +: x`	2	B	`Vector(2, 2, 3)`
`xs :+ x`	3	C	`1`
`xs ++ xs`	4	D	`error: value tail is not a member of Int`
`xs.indices`	5	E	`(0 until 3)`
`xs apply 0`	6	F	`Vector(1, 2, 3)`
`xs(3)`	7	G	`Vector(0, 1, 2, 3)`
`xs.length`	8	H	`false`
`xs.take(4)`	9	I	`java.lang.IndexOutOfBoundsException`
`xs.drop(2)`	10	J	`Vector(1, 2, 3, 0)`
`xs.updated(0, 2)`	11	K	`Vector(3)`
`xs.tail.head`	12	L	`error: value +: is not a member of Int`
`xs.head.tail`	13	M	`Vector(1, 2, 3, 1, 2, 3)`
`xs.isEmpty`	14	N	`2`
`xs.nonEmpty`	15	O	`3`

b) There are three cases where errors occur. Why? Are these compile-time errors or runtime errors?

Tip for continuation: Scala's standard library has many useful collections with uniform method sets. Learning the most important collection methods gives you a powerful toolbox. Read more here:

- quick reference (only allowed exam aid): https://fileadmin.cs.lth.se/pgk/quickref.pdf

- overview (by Prof. Martin Odersky, inventor of Scala, and others): https://docs.scala-lang.org/overviews/collections-2.13/introduction.html
- API documentation: https://www.scala-lang.org/api/current/scala/collection/

Task 4. *Copying sequences.* The class `Mutant` below can be used to create mutable instances with integers.[1]

```
class Mutant(var int: Int = 0)
```

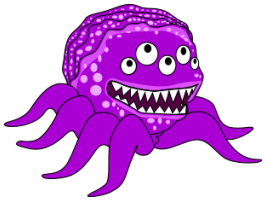


Figure 7.1: An instance of class Mutant where `int` might be 5.

Run the following in REPL after studying this: https://youtu.be/dpdOUEe9mm4

```
scala> val fem = new Mutant(5)
scala> val xs = Vector(fem, fem, fem)
scala> val ys = xs.toArray    // copies references to new Array
scala> val zs = xs.map(x => new Mutant(x.int)) // deep copies to new Vector
scala> xs(0).int = (new Mutant).int
```

a) Fill in the table below by writing the value of each expression to the right. Explain what happens. *Tip:* The method `eq` always compares references (not content).

`xs(0)`	
`ys(0).int`	
`zs(0).int`	
`xs(0) eq ys(0)`	
`xs(0) eq zs(0)`	
`(ys.toBuffer :+ new Mutant).apply(0).int`	

b) Implement the `deepCopy` function below using a **`while`** loop that performs *deep* copying, i.e., creates a new array with new, content-copied mutants.

```
def deepCopy(xs: Array[Mutant]): Array[Mutant] = ???
```

Use this algorithm:

[1] If the built-in base type Int, like `Mutant`, oddly could be used to create mutable instances, integer mathematics in Scala would dissolve into a frightening chaos.

Input : A mutant array xs
Output: A deep copy of xs

1 $result \leftarrow$ a new mutant array with space for as many elements as in xs
2 $i \leftarrow 0$
3 **while** *i less than the number of elements* **do**
4 create a copy of the element $xs(i)$ and place the copy in $result$ at position i
5 increment i by 1
6 **end**
7 $result$

c) Test your function and check that no tricky mutations via shared references are possible, as happened with `xs` and `ys` in the first subtask.

d) Is it common to do deep copying of all elements in immutable collections that only contain immutable elements "just to be safe"?

Tip for continuation: You can often solve basic subproblems using built-in collection methods from the standard library. For example, the copying in `deepCopy` in the previous task can easily be done using the collection method `map`.
But it's very good for your understanding if you can implement basic sequence algorithms yourself even though it is usually better to use ready-made, well-tested methods. In coming tasks you will implement your own versions of some sequence algorithms already existing in the standard library.

Task 5. *Updating sequences.* Declare these variables in REPL:

```
val xs = (1 to 4).toVector
val buf = xs.toBuffer
```

a) The expressions on the left are evaluated from top to bottom. Match with the correct result.

`{ buf(0) = -1; buf(0) }`	1	A	`error: value update is not a member`
`{ xs(0) = -1; xs(0) }`	2	B	`Vector(5, 2, 3, 4)`
`buf.update(1, 5)`	3	C	`ArrayBuffer(-1, 5, 3, 4, 5)`
`xs.updated(0, 5)`	4	D	`-1`
`{ buf += 5; buf }`	5	E	`Vector(1, -1, 5)`
`{ xs += 5; xs }`	6	F	`(): Unit`
`xs.patch(1,Vector(-1,5),3)`	7	G	`error: value += is not a member`
`xs`	8	H	`Vector(1, 2, 3, 4)`

Tip: Read about the methods in quick reference and test in REPL. Example:

```
scala> Vector(1,2,3,4).patch(from = 1, other = Vector(0,0), replaced = 3)
val res0: Vector[Int] = Vector(1, 0, 0)
```

b) Implement the `insert` function below using the sequence collection method `patch`.
Tip: Give argument `0` to parameter `replaced`.

```
/** Creates a copy of xs but with elem inserted at position pos. */
def insert(xs: Array[Int], elem: Int, pos: Int): Array[Int] = ???
```

c) Write pseudocode for an algorithm that implements `insert` using **`while`**.

d) Implement `insert` according to your pseudocode. Test in REPL and observe what happens if `pos` is negative? What happens if `pos` is just one step beyond the last position in `xs`? What happens if `pos` is several steps beyond the last position?

> *Tip for continuation:* It's not easy to get all edge cases right even in small algorithms such as `insert` above. Therefore, it is important to think carefully about your sequence algorithm with regard to different special cases. Use this checklist:
>
> 1. What happens if the sequence is empty?
> 2. Does it work for exactly one element?
> 3. Can indices become negative?
> 4. Can indices exceed length minus one?
> 5. Can there be an infinite loop, e.g., due to missing loop variable increment?
>
> Sometimes you want exceptions to be thrown on invalid input or other error conditions and `require` or `assert` are good to use then. In other cases you want the result to, for example, be an empty sequence if input is invalid. Such behaviors must be documented so that other users of your algorithms (or you yourself after forgetting the details) understand what happens in various cases.

Task 6. *Comparing strings in Scala.* In Scala, strings can be compared with the operators ==, !=, <, <=, >, >=, where equality/inequality is based on whether all characters in the string are equal or not, while greater/less is based on order in accordance with each character's Unicode value.[2]

a) What values do the following comparisons return?

```
scala> 'a' < 'b'
scala> "aaa" < "aaaa"
scala> "aaa" < "bbb"
scala> "AAA" < "aaa"
scala> "ÄÄÄ" < "ÖÖÖ"
scala> "ÅÅÅ" < "ÄÄÄ"
```

Unfortunately, the order of ÄÅÖ does not follow Swedish rules, but we will ignore that for simplicity; if interested in how to fix this, do task 20.

b) Which of the strings $s1$ and $s2$ comes first (i.e., is "less") if $s1$ is a prefix of $s2$ and $s2$ contains more characters than $s1$? ✎

Task 7. *Linear search according to different criteria.* Linear search means looking until you find what you are searching for in a sequence. This subproblem recurs often! Usually linear search starts from the beginning and continues until it finds an

[2] Advanced: All characters in a `java.lang.String` are represented according to the UTF-16 standard (https://en.wikipedia.org/wiki/UTF-16), meaning each Unicode code point is stored as either one or two 16-bit integers. String comparison in Scala and Java actually compares the 16-bit integers rather than characters. This difference does not matter when a string only contains characters that fit in one 16-bit integer, which practically includes almost all characters commonly used daily. The characters requiring two 16-bit integers are mostly rare Chinese characters, rare symbols, characters from extinct languages, and emoji. We will disregard such characters in this course.

element that satisfies the criterion. Depending on the contents of the sequence and the criterion, it may happen that you go through all elements without finding what you are looking for.

a) Linear search with built-in sequence collection methods.

```
val xs = ((1 to 5).reverse ++ (0 to 5)).toVector
```

Declare the above variable in REPL and match the expressions below with the correct values. Explain what happens.

`xs.indexOf(0)`	1	A	`Vector(1, 1)`
`xs.indexOf(6)`	2	B	`-1`
`xs.indexWhere(_ < 2)`	3	C	`true`
`xs.indexWhere(_ != 5)`	4	D	`Some(1)`
`xs.find(_ == 1)`	5	E	`Vector(1, 0, 1)`
`xs.find(_ == 6)`	6	F	`5`
`xs.contains(0)`	7	G	`Vector(4, 6)`
`xs.filter(_ == 1)`	8	H	`4`
`xs.filterNot(_ > 1)`	9	I	`1`
`xs.zipWithIndex.filter(_._1 == 1).map(_._2)`	10	J	`None`

b) Implement linear search in a string vector with a string predicate.

```
/** Returns first index where p is true. Returns -1 if not found. */
def indexOf(xs: Vector[String], p: String => Boolean): Int = ???
```

A string predicate `p: String => Boolean` is a function that takes a string as input and returns a boolean. Implement `indexOf` using a **`while`** loop. You can, for example, use a local boolean variable `found` to keep track of whether you found what is searched for according to the predicate.
When elements satisfying the predicate are missing, you must decide what should happen. The requirement on your implementation in this case is given by the documentation comment above.
Your function should behave as follows:

```
scala> val xs = Vector("hej", "on", "dough")
val xs: Vector[String] = Vector(hej, on, dough)

scala> indexOf(xs, _.contains('p'))
val res0: Int = 1

scala> indexOf(xs, _.contains('q'))
val res1: Int = -1

scala> indexOf(Vector(), _.contains('q'))
val res2: Int = -1

scala> indexOf(Vector("q"), _.length == 1)
val res3: Int = 0
```

Task 8. *Lab preparation: Implement integer counting in Array.* Counting means counting the number of occurrences of different values. Each time a new value occurs, we need to increment a frequency counter. A counter is needed for each value to be counted. We keep counting until all values are counted.

In this week's lab you will count the occurrence of different poker card combinations. In this exercise, as preparation for the lab, solve a similar counting problem: frequency analysis of many dice rolls. Dice counting needs six different counters. You can advantageously use a sequence collection with space for six integers. For example, position `0` holds the count of ones, position `1` holds the count of twos, etc.

a) Implement the algorithm below according to the pseudocode:

```
def registerDiceRoll(xs: Seq[Int]): Vector[Int] =
  val result = ??? /* Array with six zeros */
  xs.foreach{ x =>
    require(x >= 1 && x <= 6, "dice roll must be between 1 & 6")
    ??? /* count the occurrence of x */
  }
  result.toVector
```

b) Use the function `toss` below when testing your counting algorithm with a sequence collection containing at least 1000 dice rolls.

```
def toss(n: Int) = Vector.fill(n)(util.Random.nextInt(6) + 1)
```

Task 9. *Built-in methods for sorting.* There are several ways to order sequences by various criteria. For primitive types `Int`, `Double`, `String`, etc., built-in orderings exist so that the sequence collection method `sorted` works without extra arguments (if you are satisfied with the built-in order definition). Methods `sortBy` and `sortWith` also exist if you want to order a sequence with elements of some primitive type using your own ordering definition, or if you have your own classes in your sequence.

```
val xs = Vector(1, 2, 1, 3, -1)
val ys = Vector("abra", "ka", "dabra").map(_.reverse)
val zs = Vector('a', 'A', 'b', 'c').sorted

case class Person(firstName: String, surName: String)

val ps = Vector(Person("Kim", "Ung"), Person("kamrat", "Clementin"))
```

Declare the above in REPL and match the expressions below with the correct results. *Tip:* Uppercase letters sort before lowercase letters in the built-in ordering for the primitive types `String` and `Char`. Also, Swedish characters have a peculiar order.[3] Read about the sorting methods in the quick reference and try in REPL.

[3] Order originates from outdated character encoding standards: https://sv.wikipedia.org/wiki/ASCII

`'a' < 'A'`	1	A	`"ka"`
`"AÄÖö" < "AÅÖö"`	2	B	`1`
`xs.sorted.head`	3	C	`-1`
`xs.sorted.reverse.head`	4	D	`error: ...`
`ys.sorted.head`	5	E	`false`
`zs.indexOf('a')`	6	F	`0`
`ps.sorted.head.firstName.take(2)`	7	G	`3`
`ps.sortBy(_.firstName).apply(1).firstName.take(2)`	8	H	`true`
`xs.sortWith((x1,x2) => x1 > x2).indexOf(3)`	9	I	`"ak"`

We will later implement our own sorting algorithms as practice, but normally built-in sorting is used because it is efficient and well-tested. However, it is not uncommon to define custom orderings for your own classes, which we will study later in the course.

Task 10. *Built-in shuffle method.* In this week's lab you will implement your own shuffle algorithm and use it to shuffle a deck of cards. There is already a built-in method `shuffle` in the singleton object `Random` in package `scala.util`.

a) Look up documentation for `Random.shuffle` and study the method header. It says complicated things about `CanBuildFrom` etc. This clever detail, which we will not delve into in this course, is so that the method can return the appropriate type of collection. When you see such a method header you can assume the method works fine with several types of suitable collections in Scala's standard library.
Click on the `shuffle` documentation so you can see all text. What does the documentation say about the result? Is it in-place shuffling or copying to a new collection?

b) Try repeated shuffles of different types of sequences with different element types in REPL.

Task 11. *Repeated parameters.* You can declare a function that takes an argument sequence of arbitrary length, so-called *varargs*. The syntax consists of an asterisk $*$ after the type. The function is then said to have repeated parameters. In the function body you get access to the arguments in a sequence collection. The arguments are given arbitrarily many, separated by commas. Example:

```
/** Gives a vector of string lengths for arbitrary number of strings. */
def stringSizes(xs: String*): Vector[Int] = xs.map(_.size).toVector
```

a) Declare and use `stringSizes` in REPL. What happens if you call `stringSizes` with an empty argument list?

b) Sometimes you already have a sequence collection, e.g., `xs`, and want to pass each element as an argument to a varargs function. The syntax is `xs: _*` which makes the compiler convert the sequence collection into an argument sequence of the right type. Try this syntax by giving a `xs` of type `Vector[String]` as argument to `stringSizes`. Does it work even if `xs` is a sequence of length 0?

7.2.2 Extra Tasks; additional practice

Task 12. *Counting boolean values. Flipping a coin.*

a) Implement a function that counts many coin flips according to the below function header. Input is a sequence of boolean values where heads is coded as `true` and tails as `false`. For counting, use a local `Array[Int]`. The result must be a tuple where the count of heads is in the first place and count of tails in the second place.

```
def registerCoinFlips(xs: Seq[Boolean]): (Int, Int) = ???
```

b) Create a function `flips(n)` that generates a boolean `Vector` with n coin flips and use it to test your coin flip counting algorithm.

Task 13. *Copying and appending at the end.* Create the function `copyAppend` that implements the following algorithm, *after* fixing the **two bugs** below:

Input : Integer array xs and integer x
Output : A new integer array which is a copy of xs but with x appended at the end as an extra element.

$ys \leftarrow$ a new array with one more slot than xs
$i \leftarrow 0$
while $i \leq xs.length$ **do**
  $ys(i) \leftarrow xs(i)$
end
put x at the last position in ys
ys

Inspect your code according to the checklist in the previous tip box. Test your function for different cases: empty sequence, sequence with exactly one element, sequence with many elements.

Task 14. *Copy and reverse sequence.* Implement `seqReverseCopy` according to:

Input : Integer array xs
Output : A new integer array with elements of xs in reverse order.

$n \leftarrow$ number of elements in xs
$ys \leftarrow$ new integer array with space for n elements
$i \leftarrow 0$
while $i < n$ **do**
  $ys(n-i-1) \leftarrow xs(i)$
  $i \leftarrow i+1$
end
ys

a) Use a `while` loop as in the algorithm. Test your implementation in REPL and check it works for different cases.

b) Create a new implementation that uses a `for` loop starting from the end. Run it in REPL and check it works for different cases.

Task 15. *Copy all except one.* Implement copying an array *except* for one element at a specified position. First write pseudocode before implementing:

```
def removeCopy(xs: Array[Int], pos: Int): Array[Int]
```

Task 16. *In-place removal in array.* Sometimes you want to remove an element at a given position in an array without copying all elements to a new collection. One way is to move all subsequent elements one step towards lower indexes and fill the last position with a padding value, e.g. 0. Write pseudocode for such an algorithm first. Then implement the algorithm in a procedure with this signature:

```
def removeAndPad(xs: Array[Int], pos: Int, pad: Int = 0): Unit
```

Task 17. *Copying and inserting.*

a) Implement a function with this header according to the algorithm below:

```
def insertCopy(xs: Array[Int], x: Int, pos: Int): Array[Int]
```

Input : A sequence xs of type `Array[Int]` and integers x and pos
Output: A new sequence of type `Array[Int]` which is a copy of xs but with x inserted at position pos

$n \leftarrow$ number of elements in xs
$ys \leftarrow$ a new `Array[Int]` with space for $n+1$ elements
for $i \leftarrow 0$ **to** $pos-1$ **do**
  $ys(i) \leftarrow xs(i)$
end
$ys(pos) \leftarrow x$
for $i \leftarrow pos$ **to** $n-1$ **do**
  $ys(i+1) \leftarrow xs(i)$
end
ys

b) What must `pos` be for this to work with an empty array as input?

c) What happens if your function is called with a negative argument for `pos`?

d) What happens if your function is called with `pos` equal to `xs.size`?

e) What happens if your function is called with `pos` greater than `xs.size`?

Task 18. *In-place insertion in array.* One way to implement insertion in an array, without copying all elements to a new array with an extra slot, is to move all elements after `pos` one step to higher index, making room for the new element. In this solution the last element may "disappear" due to overwriting, since arrays cannot grow.
Write such an algorithm in pseudocode first and then implement it in a procedure with this header:

```
def insertDropLast(xs: Array[Int], x: Int, pos: Int): Unit
```

Task 19. *More built-in methods for linear search.*

a) Read in quick reference about the methods `lastIndexOf`, `indexOfSlice`, `segmentLength` and `maxBy` and describe what each can be used for.

b) Test the methods in REPL.

7.2.3 Advanced Tasks; challenges

Task 20. *Fix Swedish sorting order of ÄÅÖ.* Swedish letters come in a strange order for Swedes if special actions are not taken. Using class `java.text.Collator` you can get a `Comparator` for strings that follows local language rules for many languages on earth.

a) Verify that the sorting order is correct in REPL as below.

```
scala> val wrong = Vector("ö","å","ä","z").sorted
scala> val svColl = java.text.Collator.getInstance(new java.util.Locale("sv"))
scala> val svOrd = Ordering.comparatorToOrdering(svColl)
scala> val right = Vector("ö","å","ä","z").sorted(svOrd)
```

b) Use the above method to write a program that prints lines in a text file in correct Swedish sorting order. The program should be runnable with:
`scala sorted -sv textfil.txt`

c) Read more here:
stackoverflow.com/questions/24860138/sort-list-of-string-with-localization-in-scala

Task 21. *Fibonacci sequence with ListBuffer.* The collection `ListBuffer` is a mutable sequence that is fast and memory efficient when prepending. Investigate what happens here:

```
1 scala> val xs = scala.collection.mutable.ListBuffer.empty[Int]
2 scala> xs.prependAll(Vector(1, 1))
3 scala> while xs.head < 100 do {xs.prepend(xs.take(2).sum); println(xs)}
4 scala> xs.reverse.toList
```

The numbers produced on line 4 above are called Fibonacci numbers [4] and grow quickly.

a) Define and test the following function. It should internally use mutable `ListBuffer` but return a sequence of immutable `List`.

```
/** Produces a list of numbers in the Fibonacci sequence 1, 1, 2, 3, 5, 8 ...
  * where the largest number is less than max. */
def fib(max: Long): List[Long] = ???
```

b) How long should a Fibonacci sequence be so that the last element is as close as possible to `Int.MaxValue`?

c) Implement `fibBig` that uses `BigInt` instead of `Long` and lets your computer use its large memory while the planet warms a bit.

Task 22. *In-place reverse sequence.* Implement the following algorithm in the function `reverseChars` and test that it works for various cases in REPL.

Task 23. *Palindrome predicate.* A palindrome[5] is a word that remains unchanged when read backwards. Examples: kajak, dallassallad.
One way to implement a palindrome predicate is shown below:

```
def isPalindrome(s: String): Boolean = s == s.reverse
```

[4] sv.wikipedia.org/wiki/Fibonaccital
[5] https://sv.wikipedia.org/wiki/Palindrom

Input : An array xs of characters
Output : xs updated in-place, with characters in reverse order

```
n ← number of elements in xs
for i ← 0 to n/2 − 1 do
    temp ← xs(i)
    xs(i) ← xs(n − i − 1)
    xs(n − i − 1) ← temp
end
```

a) The above implementation may require going through all characters twice and need memory for double the number of characters. Why?

b) Create a palindrome test that traverses the elements at most once and does not need extra memory for a copy of the string. *Idea:* Compare pairs first and last, second and second last, etc.

Task 24. *More useful sequence collection methods.* Search on the web and read about these methods and test them in REPL:

- `xs.tabulate(n)(f)`
- `xs.forall(p)`
- `xs.exists(p)`
- `xs.count(p)`
- `xs.zipWithIndex`

Task 25. *Arrays don't behave, but `ArraySeqs` do!* Although `Array` is primitive, Scala's collection library has "magic" under the hood so arrays behave almost like "real" collections. Thus, you don't have to do special handling, like boxing, wrapper classes, or type casting, common for Java programmers.
However, limitations and anomalies still exist, for example regarding equality tests. If you want an array to behave like other collections, you can easily "wrap" it using `toSeq`, which on arrays returns an `ArraySeq`. This behaves as a normal immutable sequence without sacrificing the speed of a primitive array.

```
val as = Array(1,2,3)
val xs = as.toSeq
```

a) How does equality testing between two `ArraySeqs` work? What is the type of `xs` above? Is it possible to update a wrapped array?

b) What type of argument sequence do you get access to inside a function with repeated parameters?

c) Read here: `http://docs.scala-lang.org/overviews/collections/arrays.html` ★ and give an example of what else you cannot do with an array, besides content equality testing.

Task 26. *List or Vector?* Compare the time complexity between List and Vector when handling at the start and at the end, based on the following REPL session on your own machine. The below was run on an AMD Ryzen 7 5800X (16) @ 3.800GHz under Arch Linux 5.12.8-arch1-1 with Scala 3.0.1 and openjdk 11.0.11, but you should use what you have on your own computer. ★

How fast does the below run on your computer? When is List fastest and when is Vector fastest? How large are the differences in performance? [6]

```
> head -5 /proc/cpuinfo
processor    : 0
vendor_id    : AuthenticAMD
cpu family    : 25
model        : 33
model name    : AMD Ryzen 7 5800X 8-Core Processor

scala> def time(n: Int)(block: => Unit): Double =
     |   def now = System.nanoTime
     |   var timestamp = now
     |   var sum = 0L
     |   var i = 0
     |   while i < n do
     |     block
     |     sum = sum + (now - timestamp)
     |     timestamp = now
     |     i = i + 1
     |   val average = sum.toDouble / n
     |   println("Average time: " + average + " ns")
     |   average


// Exiting paste mode, now interpreting.

time: (n: Int)(block: => Unit)Double


scala> val n = 100000
scala> val l = List.fill(n)(math.random())
scala> val v = Vector.fill(n)(math.random())

scala> (for i <- 1 to 20 yield time(n){l.take(10)}).min
average time: 97.66952 ns
average time: 91.90033 ns
average time: 79.88311 ns
average time: 69.5963 ns
average time: 69.69892 ns
average time: 69.8033 ns
average time: 69.7705 ns
average time: 69.68491 ns
average time: 69.54222 ns
average time: 69.66051 ns
average time: 69.73661 ns
average time: 69.54112 ns
average time: 69.69141 ns
average time: 69.46341 ns
average time: 69.4098 ns
average time: 61.34162 ns
average time: 41.1333 ns
average time: 40.97051 ns
average time: 40.9075 ns
average time: 41.12321 ns
```

[6] You will learn more about such measurements in the LTH course "Evaluation of Software Systems" given at the end of year 1 for Computer Engineering students.

```
val res0: Double = 40.9075

scala> (for i <- 1 to 20 yield time(n){v.take(10)}).min
average time: 84.56978 ns
average time: 75.20167 ns
average time: 57.16529 ns
average time: 34.84469 ns
average time: 34.38478 ns
average time: 34.77709 ns
average time: 34.77179 ns
average time: 35.0506 ns
average time: 34.7967 ns
average time: 35.04258 ns
average time: 34.82559 ns
average time: 36.3673 ns
average time: 34.91029 ns
average time: 34.87239 ns
average time: 34.51958 ns
average time: 34.83949 ns
average time: 34.56169 ns
average time: 34.80719 ns
average time: 34.84459 ns
average time: 34.89468 ns
val res1: Double = 34.38478

scala> (for i <- 1 to 20 yield time(1000){l.takeRight(10)}).min
average time: 131365.106 ns
average time: 118632.787 ns
average time: 118440.066 ns
average time: 118687.567 ns
average time: 118428.487 ns
average time: 118871.686 ns
average time: 118964.797 ns
average time: 119030.236 ns
average time: 119262.534 ns
average time: 119228.344 ns
average time: 119226.494 ns
average time: 119310.933 ns
average time: 119352.854 ns
average time: 119121.913 ns
average time: 119133.664 ns
average time: 119015.193 ns
average time: 119276.674 ns
average time: 119224.882 ns
average time: 119301.771 ns
average time: 119444.401 ns
val res2: Double = 118428.487

scala> (for i <- 1 to 20 yield time(1000){v.takeRight(10)}).min
average time: 805.989 ns
average time: 365.219 ns
average time: 225.49 ns
average time: 125.92 ns
average time: 124.98 ns
average time: 130.689 ns
average time: 139.86 ns
average time: 128.29 ns
```

```
average time: 132.59 ns
average time: 125.729 ns
average time: 125.46 ns
average time: 130.59 ns
average time: 122.03 ns
average time: 121.9 ns
average time: 119.69 ns
average time: 120.48 ns
average time: 125.239 ns
average time: 126.09 ns
average time: 125.92 ns
average time: 125.91 ns
val res3: Double = 119.69
```

Why do different rounds in the for loop run at different speeds even though each round does the same thing?

★ **Task 27.** *Time complexity of various collections in Scala's standard library.*
Study differences in time complexity between different collections here:
docs.scala-lang.org/overviews/collections/performance-characteristics.html
Also read the criticism of simplifications in the above description here:
www.lihaoyi.com/post/ScalaVectoroperationsarentEffectivelyConstanttime.html
Read this thorough empirical review of performance in Scala's collection libraries:
www.lihaoyi.com/post/BenchmarkingScalaCollections.html

7.3 Lab: `shuffle`

Goals

- ☐ Be able to create and use sequence collections.
- ☐ Be able to implement the sequence algorithm SHUFFLE that modifies the contents of an array in place.
- ☐ Be able to record the number of occurrences of different values in a sequence.

Preparations

- ☐ Do exercise `Excercise07` in section 7.2
- ☐ Read through the entire lab and ensure that you understand how the SHUFFLE algorithm below works.
- ☐ Retrieve the given code via the course's GitHub location.

7.3.1 Background

This assignment is about card shuffling. Shuffling cards so that every possible permutation (order in which the cards lie) is equally probable is non-trivial; an unsystematic shuffle leads to a skewed distribution.

Given a good random number generator, it is possible to shuffle a deck by putting all cards in a pile and then repeatedly taking a random card from the pile and placing it on top of the deck until all cards are in the deck. The Fisher-Yates algorithm[7] (also called Knuth-shuffle) works this way. Here we refer to this algorithm as SHUFFLE. It is found in pseudocode below. Note especially that the upper limit for r includes i.

```
Input  : Array xs with n values to be shuffled "in place"
Output : xs updated in place with values rearranged in random order
for i ← (n − 1) to 0 do
    draw random number r such that 0 ≤ r ≤ i
    swap xs(i) and xs(r)
end
```

A deck has 52 cards, each with a different rank and suit. The card game poker involves drawing cards and getting certain combinations of cards, so-called "hands"[8]. These are ordered from better to worse; the player with the best hand wins. It is therefore interesting to know with what probability a certain hand appears when drawn from a shuffled deck.

The common poker hands are, in descending order of value, straight flush, four of a kind, full house, flush, straight, three of a kind, two pair, and pair. These are illustrated in section 7.3.5. There is also a hand called *royal (straight) flush* which denotes a straight flush with an ace as the highest card, but its probability is so low that in simulation it is unlikely to be encountered within a reasonable time.

During the lab, you will start by completing the unfinished class `Deck` shown in section 7.3.2, and found in the workspace on GitHub.

The lab instructions in section 7.3.3 give hints on how you should replace `???` in the given code skeletons with your solutions. With the help of the classes `Test` and `Test` you can test that your implementations work correctly.

[7] https://en.wikipedia.org/wiki/Fisher%E2%80%93Yates_shuffle#The_modern_algorithm
[8] https://sv.wikipedia.org/wiki/Pokerhand

```scala
package poker

case class Card(rank: Card.Rank, suit: Card.Suit):
  lazy val show = s"${rank.toString.head}${Card.suitChars(suit.ordinal)}"

object Card:
  enum Rank:
    case Ace, `2`, `3`, `4`, `5`, `6`, `7`, `8`, `9`, Ten, Jack, Queen, King

  enum Suit:
    case Spades, Hearts, Clubs, Diamonds

  val suitChars = "♠♡♣♢"
end Card
```

Figure 7.2: The fully implemented, immutable case class `Card`.

7.3.2 Given code

After your implementations of the methods `full` and `shuffle` work, you should use `Deck` in the singleton object `PokerProbability` to find out probabilities for different poker hands when dealing 5 cards from a well shuffled deck.
To assist you, below are code files, some with unfinished code that you shall complete. All code is in a package named `poker`.[9]

- `Card.scala` in fig. 7.2 on page 263 contains the fully implemented case class `Card` which represents a card and has a concise `toString` with rank and suit.
- `Deck.scala` in fig. 7.3 on page 264 contains the mutable class `Deck`, where you shall implement card shuffling in the method `shuffle`. The companion object has methods to create decks. You shall implement the method `full` that creates a complete deck of 52 cards ordered by rank and suit.
- `Hand.scala` in fig. 7.4 on page 265 contains an immutable case class `Hand` that represents a poker hand and has methods to determine which hand it is. The companion object contains factory methods that can create a new hand from individual cards or by drawing cards from a deck. You shall implement `tally`, which registers the number of cards of a certain rank in an array indexed by the zero-based `ordinal` integer for each card's `rank`.
- `PokerProbability.scala` in fig. 7.5 on page 266 has a main method that calculates poker probabilities, along with the helper method `register` which you shall implement.
- The singleton object `Test.scala`, which also exists in the week's given code, you shall use to test your implementation of `shuffle` with a deck containing only three cards. Repeated shuffles are performed, and the occurrences of each possible permutation are registered.

[9] You can browse the classes here:
https://github.com/lunduniversity/introprog/tree/master/workspace/w07_shuffle/

```scala
package poker

import scala.collection.immutable.ArraySeq

class Deck private (val initCards: ArraySeq[Card]):
  private var cards: Array[Card] = initCards.toArray

  def reset(): Unit = cards = initCards.toArray

  def apply(i: Int): Card = cards(i)

  def toSeq: ArraySeq[Card] = cards.to(ArraySeq)

  def show: String = cards.map(_.show).mkString(" ")

  def peek(n: Int): ArraySeq[Card] =
    cards.take(n).to(ArraySeq)

  def remove(n: Int): ArraySeq[Card] =
    val init = peek(n)
    cards = cards.drop(n)
    init
  end remove

  def prepend(moreCards: Card*): Unit =
    cards = (moreCards ++ cards).toArray

  /** Swaps cards at position a and b. */
  def swap(a: Int, b: Int): Unit = ???

  /** Randomly reorders the cards in this deck. */
  def shuffle(): Unit = ???

object Deck:
  def apply(cards: Seq[Card]): Deck = new Deck(cards.to(ArraySeq))

  /** Creates a new full Deck with 52 cards in rank and suit order. */
  def full(): Deck = ???
```

Figure 7.3: The unfinished class `Deck` with mutable deck of cards.

```scala
package poker

import scala.collection.immutable.ArraySeq

case class Hand(cards: ArraySeq[Card]):
  import Hand._

  /**
   * Yields a sequence of length 13, with positions 0-12 containing the
   * number of cards of that card's rank's ordinal number (zero based).
   */
  lazy val tally: ArraySeq[Int] = ???

  lazy val ranksSorted: ArraySeq[Int] = cards.map(_.rank.ordinal).sorted

  lazy val isFlush: Boolean = cards.length > 0 && cards.forall(_.suit == cards(0).suit)

  lazy val isStraight: Boolean =
    def isInSeq(xs: ArraySeq[Int]): Boolean =
      xs.length > 1 && (0 to xs.length - 2).forall(i => xs(i) == xs(i + 1) - 1)

    isInSeq(ranksSorted) ||  // special case with ace interpreted as 13:
      (ranksSorted(0) == 0) && isInSeq(ranksSorted.drop(1) :+ 13)

  lazy val isStraightFlush: Boolean = isStraight && isFlush
  lazy val isRoyalFlush:    Boolean = ???
  lazy val isFour:          Boolean = tally.contains(4)
  lazy val isFullHouse:     Boolean = tally.contains(3) && tally.contains(2)
  lazy val isThrees:        Boolean = tally.contains(3)
  lazy val isTwoPair:       Boolean = tally.count(_ == 2) == 2
  lazy val isOnePair:       Boolean = tally.contains(2)

  lazy val category: Category =
    if      /* isRoyalFlush    then Category.RoyalFlush
    else if */ isStraightFlush then Category.StraightFlush
    else if isFour          then Category.Fours
    else if isFullHouse     then Category.FullHouse
    else if isFlush         then Category.Flush
    else if isStraight      then Category.Straight
    else if isThrees        then Category.Threes
    else if isTwoPair       then Category.TwoPair
    else if isOnePair       then Category.OnePair
    else                         Category.HighCard

object Hand:
  def apply(cardSeq: Card*): Hand = new Hand(cardSeq.to(ArraySeq))
  def from(deck: Deck): Hand = Hand(deck.peek(5))
  def removeFrom(deck: Deck): Hand = Hand(deck.remove(5))

  enum Category:
    case
      RoyalFlush, StraightFlush, Fours, FullHouse, Flush,
      Straight, Threes, TwoPair, OnePair, HighCard
end Hand
```

Figure 7.4: The unfinished, immutable class `Hand` that represents a poker hand.

```scala
package poker

import scala.collection.immutable.ArraySeq

object PokerProbability:

  /**
   * For a given number of iterations, shuffles a deck, draws a hand and
   * returns a sequence with the frequency of each hand category.
   * Prints a dot every dotStep iteration
   */
  def register(n: Long, deck: Deck, dotStep: Long = 1e6.toLong): ArraySeq[Int] =
    ???
  end register

  @main def simulate: Unit =
    val defaultIter = 5
    val in = scala.io.StdIn.readLine(s"number of million iterations ($defaultIter): ")
    val n = (in.toIntOption.getOrElse(defaultIter) * 1e6).toLong
    val deck = Deck.full()
    val t0 = System.currentTimeMillis()
    val frequencies = register(n, deck)
    for c <- Hand.Category.values do
        val name = c.toString
        val percentages = frequencies(c.ordinal).toDouble / n * 100
        println(f"$name%16s $percentages%10.6f%%")
    end for
    val secs = (System.currentTimeMillis() - t0)/1000.0
    println:
      f"\n*** Total execution time: $secs%3.2f seconds"
```

Figure 7.5: The unfinished singleton object `PokerProbability` which calculates probabilities for different poker hands.

7.3.3 Mandatory tasks

Task 1. Implement the SHUFFLE algorithm.

a) Create your own implementation of the method `shuffle` in the class `Deck`. Follow the given algorithm in section 7.3.1 carefully. You can use `cards.length` to get the deck length, but you are welcome to use `cards.indices.reverse` instead. Implement and use the method `swap`.

b) Run `testShuffle` in `Test` which checks that the shuffle is evenly distributed by shuffling a deck of three cards and counting how often each possible permutation occurs. You should get a printout with six (3!) percentages that should be nearly equal.

Task 2. Create a complete, ordered deck.

a) Implement the method `full` which creates a 52-card standard deck ordered by suit and rank. Use the `Range` values in the companion object `Card`.

b) Run `testCreate` in `Test` and verify that you get cards of all four suits, and both aces and kings.

Task 3. Complete and test `Hand`.

a) Implement `tally` which should give an indexable sequence with 13 places where each index represents the count of the respective rank. Ace counts as lowest.

b) Test the class `Hand` using `Test`.

Task 4. Determine probabilities for different poker hands.

a) Implement the method `register` in `PokerProbability`. Use `from` and `category` in `Hand` to create and categorize a hand from a deck. Store the frequencies in a local array which you, when completed, convert to a sequence with `to(ArraySeq)`.

b) Run the main program in `PokerProbability`, preferably with fifty million iterations depending on your patience and your computer's speed. You should get approximately these probabilities[10]:

hand	*probability*
Straight flush (excl. Royal flush)	0.00139%
Four of a kind	0.02401%
Full house	0.1441%
Flush (excl. Royal and Straight flush)	0.1965%
Straight	0.3925%

Task 5. Also simulate the probability of Royal flush (a straight flush with ace as highest card). This requires on the order of 10^8 iterations for an accuracy of 2 significant digits. This can take many minutes on a reasonably fast computer, so it might be time for a break during the simulation...

[10] https://en.wikipedia.org/wiki/Poker_probability

Task 6. Discuss with the supervisor during the review the advantages of using specific enumerated values with `enum` instead of general integers and strings.

Task 7. Discuss with the supervisor during the review the pros and cons of using `lazy val` instead of `val` or `def` in an immutable class.

7.3.4 Optional extra tasks

Task 8. Implement an interactive card game, e.g. a simple poker variant. Start with something very simple, for example highest-card-wins, and build on with what you find fun.
You can for example create a method `def compareTo(other: Hand): Comparison` in the case class `Hand` which returns `Comparison.Worse` if `other` is worse, `Comparison.Equal` if the hands are equally good, and `Comparison.Better` if `other` is better. Step by step you can make it possible to compare more and more hands according to the special rules that apply for when different hands are considered better or equal. Read about the rules here: `https://en.wikipedia.org/wiki/List_of_poker_hands`

7.3.5 Pictures with examples of different poker hands

Figures 7.6 – 7.14 show pictures of different poker hands.

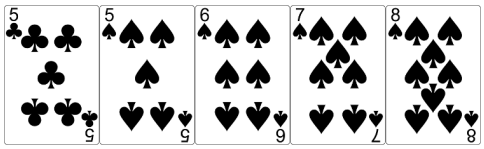


Figure 7.6: Pair: two cards have the same rank.

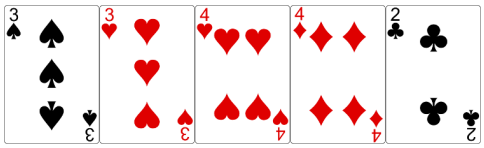


Figure 7.7: Two pair: the hand has two *different* pairs.

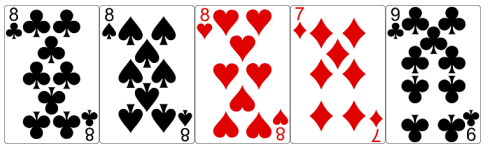


Figure 7.8: Three of a kind: three cards have the same rank.

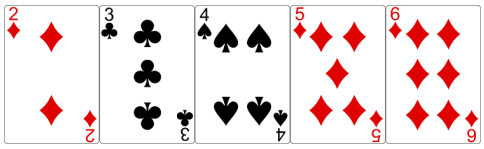


Figure 7.9: Straight: the card ranks form a sequence, ace can be either 1 or 14.

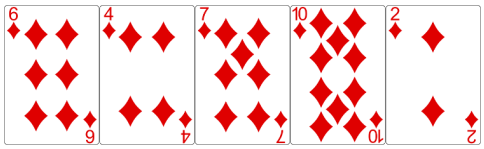


Figure 7.10: Flush: all cards have the same suit.

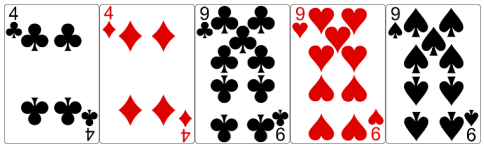


Figure 7.11: Full house: both three of a kind and a pair.

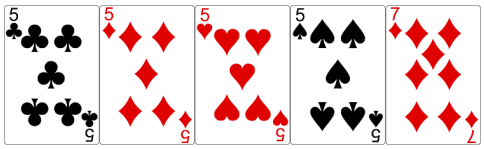


Figure 7.12: Four of a kind: four cards have the same rank.

Figure 7.13: Straight flush: both straight and flush. Special case: If the straight flush's highest card is an ace, it is called *royal flush* or *royal straight flush*.

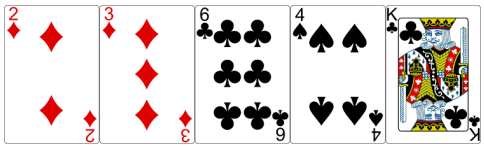


Figure 7.14: High card: no pattern is found.

Part III

Solutions

Chapter L

Solutions to the exercises

L.1 Solution `Excercise01`

L.1.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match concepts with description.*

literal	1	$\rightsquigarrow$	D	represents a specific data value
string	2	$\rightsquigarrow$	G	a sequence of characters
statement	3	$\rightsquigarrow$	F	a line of code that does something; can be separated with a semicolon
expression	4	$\rightsquigarrow$	H	combines values and functions into a new value
function	5	$\rightsquigarrow$	K	computes a return value when called
procedure	6	$\rightsquigarrow$	J	performs a side effect when called; return value is empty
runtime error	7	$\rightsquigarrow$	N	may occur while the program runs
compile error	8	$\rightsquigarrow$	M	may occur before execution starts
abstract	9	$\rightsquigarrow$	A	introduce new concepts that simplify coding
compile	10	$\rightsquigarrow$	C	translate code into executable form
type	11	$\rightsquigarrow$	I	describes what the data can be used for
for-loop	12	$\rightsquigarrow$	O	useful when the number of repetitions is known beforehand
while-loop	13	$\rightsquigarrow$	P	useful when the number of repetitions is unknown beforehand
assignment	14	$\rightsquigarrow$	L	to change the value of a variable
floating-point	15	$\rightsquigarrow$	E	decimal number with limited precision
boolean	16	$\rightsquigarrow$	B	either true or false

Solution Task 2. *Output in Scala REPL.*

a) For example:

```
scala> println("hello lasser")
```

b) If the right parenthesis is missing you can continue typing on the next line. This is indicated by a vertical bar at the start of each new line:

```
scala> println("hello lasser"
     | + "!"
     | )
hello lasser!
```

Solution Task 3. *String concatenation.*

a)

```
scala> "cucumber" + "tomato"
res1: String = cucumbertomato
```

value: `"cucumbertomato"`, type: `String`

b)

```
scala> res1 * 42
```

```
res2: String = cucumbertomatocucumbertomatocucumbertomatocucumbertomatocucumber
```

Solution Task 4. *When is the error detected?*

a) Type: `String`, value: `"hejhejhej"`

b) Runtime error:

```
scala> "hej" * Int.MaxValue
java.lang.OutOfMemoryError: Java heap space
```

c) Compile-time error: (indicated by text `<console> ... error:`)

```
scala> "hej" * true
<console>:12: error: type mismatch;
 found   : Boolean(true)
 required: Int
       "hej" * true
```

A type error means the compiler cannot make the types match in e.g. a function call. In Scala we find out about type errors already at compilation, whereas in other languages (e.g. Javascript) such errors are discovered during execution, possibly leading to hard-to-find bugs that may first manifest much later.

Solution Task 5. *Literals and types.*

a)

`1`	1	$\rightsquigarrow$	E	`Int`
`1L`	2	$\rightsquigarrow$	G	`Long`
`1.0`	3	$\rightsquigarrow$	J	`Double`
`1D`	4	$\rightsquigarrow$	F	`Double`
`1F`	5	$\rightsquigarrow$	H	`Float`
`'1'`	6	$\rightsquigarrow$	I	`Char`
`"1"`	7	$\rightsquigarrow$	A	`String`
`true`	8	$\rightsquigarrow$	C	`Boolean`
`false`	9	$\rightsquigarrow$	B	`Boolean`
`()`	10	$\rightsquigarrow$	D	`Unit`

b) The value goes beyond the limit of what fits in a 32-bit integer and ”wraps around” to the smallest possible integer `Int.MinValue` because that is how binary arithmetic with limited bits works in the CPU.

```
scala> Int.MaxValue + 1
res3: Int = -2147483648

scala> Int.MinValue
res4: Int = -2147483648
```

c) Both are integers but `Long` can represent larger numbers than `Int`.

d) Both are floating-point types but `Double` has double precision and can represent larger numbers with more decimal digits.

Solution Task 6. *Mathematical functions. Using documentation.*

a) Computing $2^{64} - 1$ using `math.pow` as below yields approximately $1.8 \cdot 10^{19}$

```
scala> math.pow(2, 64) - 1
res0: Double = 1.8446744073709552E19
```

b) Yes, the return type annotation `: Double` can be omitted.

- Why can the return type be omitted?
  Because the compiler's type inference can deduce the return type.
- Why might one want to omit it?
  It is shorter to write without it.
- Reasons to specify the return type:
  - With explicit return type you get help from the compiler, already at compile time, to check that the expression to the right of the equals sign is of the expected type.
  - By explicitly specifying the return type, readers who only see the method signature (and not the implementation) can clearly see what is returned.

c) About 500 km.

```
scala> circumference(12750 / 2) / 80
res0: Double = 500.6913291658733
```

Solution Task 7. *Variables and assignment. Mutable and immutable variables.*

a)

After line 1:	a: Int 13		
After line 2:	a: Int 13	b: Int 14	
After line 3:	a: Int 13	b: Int 14	c: Double 54.0
After line 4:	a: Int 13	b: Int 14	c: Double 54.0
After line 5:	a: Int 0	b: Int 14	c: Double 54.0
After line 6:	a: Int 0	b: Int 14	c: Double 55.0

b) Immutable variables are declared with the keyword **val**. It is not possible to assign a new value to an immutable variable; attempting to do so results in a compile-time error that says **error: reassignment to val**. Compile-time errors can be recognized by the text **error:**, as shown below:

```
scala> b = 0
<console>:12: error: reassignment to val
       b = 0
         ^
```

Solution Task 8. *Random numbers with* `math.random()`.

a) From documentation:

```
/** Returns a Double value with a positive sign,
 *  greater than or equal to 0.0 and less than 1.0.
 */
def random(): Double
```

Documentation comments, which start with `/**` and end with `*/`, give a description of how the function works. After the doc comment comes the function header, which here tells us the function is named `random` and always returns a `Double`. (The tool `scaladoc` can use documentation comments to automatically generate websites with special documentation pages and search functions.)

b)

```
scala> def roll: Int = (math.random() * 6 + 1).toInt

scala> roll
res0: Int = 4

scala> roll
res1: Int = 1
```

Solution Task 9. *Repetition with* **for**, `foreach` *and* **while**.

a)

```
for i <- 1 to 100 do print(s"$roll, ")
```

b)

```
(1 to 100).foreach(i => print(s"$roll, "))
```

c)

```
var i = 1
while i <= 100 do { print(s"$roll, "); i = i + 1 }
```

```
var i = 1
while i <= 100 do
    print(s"$roll, ")
    i += 1
```

Solution Task 10. *Alternatives with* **`if`** *statement and* **`if`** *expression.*

a)

```
for i <- 1 to 100 do
  if roll == 6 then print("CONGRATULATIONS! ") else print(":(")
```

or

```
for (i <- 1 to 100) if (roll == 6) print("CONGRATULATIONS! ") else print(":(")
```

b)

```
var i = 1
var n = 0
while i <= 100 do
  if roll == 6 then n = n + 1
  i = i + 1
println("The number of sixes: " + n)
```

c)

```
def isAdult(age: Int) = age >= 18
```

Solution Task 11. *Sequence, statement and block.*

a) The statements create this output:

```
true!hej
true!hej
true!hej
true!hej
```

b)

```
scala> def p = { print("hej"); print("true"); println("!")}
scala> p;p;p;p
```

c)

- Curly braces are used to group multiple statements. Curly braces are needed if you want to define a function consisting of more than one statement. Since Scala 3 you can alternatively use indentation to define a multi-line function with multiple statements.

- Semicolons separate multiple statements. Semicolons are only needed if you want to write multiple statements on the same line.

Solution Task 12. *Integer division.*

`4 / 42`	1	⇝	F	`0: Int`
`42.0 / 2`	2	⇝	C	`21.0: Double`
`42 / 4`	3	⇝	B	`10: Int`
`42 % 4`	4	⇝	G	`2: Int`
`4 % 42`	5	⇝	A	`4: Int`
`40 % 4 == 0`	6	⇝	D	`true : Boolean`
`42 % 4 == 0`	7	⇝	E	`false: Boolean`

Solution Task 13. *Boolean values.*

a) `true`
b) `false`
c) `true`
d) `true`
e) `false`
f) `true`
g) `true`
h) `true`
i) Exception thrown: `java.lang.ArithmeticException: / by zero`
j) `false`

Solution Task 14. *Boolean variables.*

2: Nothing is printed.

4: `watch out!!!`

Solution Task 15. *Turtle graphics with Kojo.*

a) Starting your Kojo program with `erase` causes execution to start in the same initial state: a blank canvas where the turtle points upward, the pen is down, and the pen color is red. That makes reasoning about what the program does from start to finish easier compared to if execution depends on results of previous executions.

b)

```
erase

front; left
front; left
front; left
front; left
```

c)

```
erase

front; left
front; right
```

```
front; left
front; right

front; left
front; right

front; left
```

L.1.2 Extra Tasks; additional practice

Solution Task 16. *Value and type.*

`1.0 + 18`	1	⇝	H	`19.0: Double`
`(41 + 1).toDouble`	2	⇝	K	`42.0: Double`
`1.042e42 + 1`	3	⇝	A	`1.042E42: Double`
`12E6.toLong`	4	⇝	I	`12000000: Long`
`32.toChar.toString`	5	⇝	E	`" ": String`
`'A'.toInt`	6	⇝	B	`65: Int`
`0.toInt`	7	⇝	F	`0: Int`
`'0'.toInt`	8	⇝	D	`48: Int`
`'9'.toInt`	9	⇝	L	`57: Int`
`'A' + '0'`	10	⇝	C	`113: Int`
`('A' + '0').toChar`	11	⇝	J	`'q': Char`
`"*!%#".charAt(0)`	12	⇝	G	`'*': Char`

Solution Task 17. *Statements and expressions.*

a) An expression can be evaluated and yields a useful value. A statement *does* something (e.g. prints something), but does not yield a useful value.

b) `println()`

c)
`valueMissing` contains Unit
Prints Unit
Prints `"()"`
Prints `"()"`
Prints first 'hej' from the inner call then `()` from the outer call

d) `Unit`

e) `Unit`

Solution Task 18. *Procedure with parameter.*

a)

```
var highscore = 0
```

b)

```
def updateHighscore(points: Int): Unit =
  if points > highscore then
    highscore = points
    println("REKORD!")
  else println("DON'T GIVE UP!")
```

c)

```
def updateHighscore(points: Int): String =
```

```
  if points > highscore then
    highscore = points
    "REKORD!"
  else "DON'T GIVE UP!"
```

Solution Task 19. *Floating point arithmetic.*

a)

```
scala> Double.MinPositiveValue
res0: Double = 4.9E-324
```

b)

```
scala> Double.MaxValue + Double.MinPositiveValue == Double.MaxValue
res2: Boolean = true
```

Solution Task 20. ***if*** *statements.*

1. Output: **false**
2. Output: **true**
3. Nothing is printed, the function is declared but not executed.
4. Output: `1:kop 2:munt 3:kop 4:kop 5:munt` or similar depending on the random values from `math.random()`.

Solution Task 21. ***if*** *expressions.* Note the type `Any` on the last two expressions.

```
scala> if vegetable == "tomato" then "good" else "not good"
res0: String = not good

scala> if fruit == "banana" then "good" else "not good"
res1: String = good

scala> if true then vegetable else 42
res2: Any = cucumber

scala> if false then vegetable else 42
res3: Any = 42
```

Solution Task 22. *Modulo operator* `%` *and Boolean values.*

a)

```
scala> def isEven(n: Int): Boolean = n % 2 == 0

scala> isEven(42)
res0: Boolean = true

scala> isEven(43)
res1: Boolean = false
```

b)

```
scala> def isOdd(n: Int): Boolean = !isEven(n)

scala> isOdd(42)
res2: Boolean = false

scala> isOdd(43)
res3: Boolean = true
```

Solution Task 23. *Differences between* `var`, `val`, `def`.

a)

```
  scala> var x = 30
  x: Int = 30

  scala> x + 1
  res6: Int = 31

  scala> x = x + 1
  x: Int = 31

  scala> x == x + 1
  res7: Boolean = false

  scala> val y = 20
  y: Int = 20

  scala> y = y + 1
  <console>:12: error: reassignment to val
         y = y + 1
           ^

  scala> var z = { println("hej z!"); math.random() }
  hej z!
  z: Double = 0.3381365875903367

  scala> def w = { println("hej w!"); math.random() }
  w: Double

  scala> z
  res8: Double = 0.3381365875903367

  scala> z
  res9: Double = 0.3381365875903367

  scala> z = z + 1
  z: Double = 1.3381365875903368

  scala> w
  hej w!
  res10: Double = 0.06420209879434557

  scala> w
  hej w!
  res11: Double = 0.5777951341051852
```

```

scala> w = w + 1
<console>:12: error: value w_= is not a member of object
       w = w + 1
```

b)

- `var name = expression` is used to declare a mutable variable. The name can be reassigned to new values using assignment statements.
- `val name = expression` is used to declare an immutable variable which cannot be changed after initialization. Attempting to assign it a new value results in a compile-time error.
- `def name = expression` is used to declare a function whose expression is evaluated every time it is called.

Solution Task 24. *Difference between* ***`if`*** *and* ***`while`***.

- Line 3: If lucky (50% chance) you get a single win printed once.
- Line 4: If lucky you get many wins in a row. The probability of n wins in a row is $(\frac{1}{2})^n$.

L.1.3 Advanced Tasks; challenges

Solution Task 25. *Logic and De Morgan's Laws.*

a) `point > 1000`

b) `point > 100`

c) `point <= highscore`

d) `point <= 0 || point >= highscore`

e) `point >= 0 && point <= highscore`

f) `clear`

g) `!clear`

Solution Task 26. *The string interpolator* `s`.

a)

```
The name 'Kim Finkodare' has 12 letters.
```

b)

```
println(s"$f has ${f.size} letters.")
println(s"$e has ${e.size} letters.")
```

Solution Task 27. *Assignment operators.*

After line1:	a:	Int	40			
After line2:	a:	Int	40	b:	Int	80
After line3:	a:	Int	50	b:	Int	80
After line4:	a:	Int	50	b:	Int	70
After line5:	a:	Int	100	b:	Int	70
After line6:	a:	Int	100	b:	Int	35

Solution Task 28. *Big numbers.*

a) `BigInt` can be used instead of `Int` for very large integers. There is also `Long` which has double the range of `Int`, while `BigInt` can have an arbitrarily large number of digits (until memory runs out) and thus represent enormously large numbers. `BigDecimal` can be used instead of `Double` for very large decimal numbers.

b)

```
scala> BigInt(2).pow(64)
res0: scala.math.BigInt = 18446744073709551616
```

c) Calculations are much slower and they are somewhat trickier to use.

Solution Task 29. *Precedence rules*

a) `77:  Int`

b) `13: Int`

c) `-13: Int`

Solution Task 30. *Documentation of packages in Java and Scala.*

a) Scala: `Pi`, Java: `PI`

b) You can search for and filter all occurrences of a particular string.

c) Computes the hypotenuse (Pythagorean theorem) without risking rounding errors in intermediate calculations.

Solution Task 31. *Accuracy and exceptions in arithmetic expressions.*

a) `-2147483648` which corresponds to `Int.MinValue`.

b) An exception is thrown: `java.lang.ArithmeticException: / by zero`

c) `1.0000000000000001E8` (as expected)

d) Rounded to `1E9` (floating-point arithmetic accuracy issue: a big float plus a (too) small float can give the same number; the small number 'disappears').

e) `45.00000000000001` (floating-point arithmetic accuracy issue: mathematically should be exactly 45.)

f) `Infinity` (same as `Double.PositiveInfinity`, representing positive infinity).

g) `2147483647` which is `Int.MaxValue`.

h) `NaN` meaning 'Not a Number'.

i) `NaN` meaning 'Not a Number'.

j) An exception is thrown: `java.lang.Exception: PANG!!!`

Solution Task 32. *Modulo arithmetic with negative numbers.* In Scala the result has the same sign as the dividend.

```
scala> 1 % 2
res0: Int = 1

scala> -1 % 2
res1: Int = -1

scala> -1 % -2
res2: Int = -1

scala> 1 % -2
res3: Int = 1
```

Solution Task 33. *Literal identifiers.*

a) The variable is named 'literal val', backticks allow names otherwise illegal, e.g. containing spaces or keywords.

b) Backticks in Scala allow all kinds of characters in names. Example use: Java has a method `java.lang.Thread.yield` but in Scala `yield` is a keyword; to work around this in Scala you can write `java.lang.Thread.`yield``

Solution Task 34. *`java.lang.Integer`, hexadecimal literals, BigDecimal.*

a)

```
scala> import Integer.{toBinaryString => toBin, toHexString => toHex}

scala> for i <- Seq(33, 42, 64) do println(s"$i \t ${toBin(i)} \t ${toHex(i)}")
33   100001    21
42   101010    2a
64   1000000   40
```

b) The hexadecimal integer $10c$ can be written as literal `0x10c` in Scala, Java and many other languages: [1]

```
scala> 0x10c
res0: Int = 268
```

c) [2]

```
scala> val c = 299792458
c: Int = 299792458

scala> BigDecimal(0x10).pow(c)
res68: scala.math.BigDecimal = 2.1248929632279066130609861108876 72E+360986089
```

Solution Task 35. *String formatting.*

```
val str = f"The giant cucumber is $g%1.3f meter lang"
```

(If you think `$g%1.3f` looks cryptic, take comfort that you now carry on a venerable heritage from the ancient language C and the legendary function `printf` to coming generations of initiated code wizards.)

Solution Task 36. *Multiplication warning.*

a) The second multiplication overflows the maximum allowed value of an `Int`. In the third multiplication an exception `java.lang.ArithmeticException: integer overflow` is thrown instead.

```
scala> Math.multiplyExact(1, 2)
res70: Int = 2

scala> Int.MaxValue * 2
res71: Int = -2

scala> Math.multiplyExact(Int.MaxValue, 2)
java.lang.ArithmeticException: integer overflow
```

[1] https://en.wikipedia.org/wiki/0x10c
[2] https://c418.bandcamp.com/album/0x10c

```
  at java.lang.Math.multiplyExact(Math.java:867)
  ... 42 elided
```

b) Used when you want to be 100

Solution Task 37. *Extra operators for exact multiplication.*

a)

```
scala> Int.MaxValue *! 1
res0: Int = 2147483647

scala> Int.MaxValue *! 2
java.lang.ArithmeticException: integer overflow
  at java.lang.Math.multiplyExact(Math.java:867)
  at IntExtra.$times$bang(<console>:16)
  ... 32 elided
```

b)

```
extension (i: Int)
  def *!(j: Int) = Math.multiplyExact(i,j)
  def +!(j: Int) = Math.addExact(i,j)
  def -!(j: Int) = Math.subtractExact(i,j)
```

c) It quickly becomes very cryptic with names consisting of multiple special characters. If you *really* want such operators it is *very* appropriate to also provide clear-text variants:

```
extension (i: Int)
  def mulExact(j: Int) = Math.multiplyExact(i,j)
  def *!(j: Int) = i mulExact j

  def addExact(j: Int) = Math.addExact(i,j)
  def +!(j: Int) = i addExact j

  def subExact(j: Int) = Math.subtractExact(i,j)
  def -!(j: Int) = i subExact j
```

L.2 Solution Excercise02

L.2.1 Basic Tasks

Solution Task 1. *Match concepts with descriptions.*

compile	1	⇝	C	machine code is generated from one or more source files
script	2	⇝	G	single code file, no main program needed
object	3	⇝	M	groups variables and functions
@main	4	⇝	E	where execution of the compiled program starts
program argument	5	⇝	L	can be passed via the args parameter to main
data structure	6	⇝	J	many elements in a whole; element-wise access
collection	7	⇝	K	data structure with elements of the same type
sequence collection	8	⇝	F	data structure with elements in a certain order
Array	9	⇝	D	a mutable, indexable sequence collection
Vector	10	⇝	B	an immutable, indexable sequence collection
Range	11	⇝	N	a collection representing a range of integers
yield	12	⇝	A	used in for-expressions to create a new collection
algorithm	13	⇝	H	step-by-step description of a solution to a problem
implementation	14	⇝	I	a specific realization of an algorithm

Solution Task 2. *Using the terminal.*

a)

```
> mkdir hello
> cd hello
> pwd
```

b)

```
> cd ..
> ls
```

Solution Task 3. *Create and run a Scala script.*

a)

```
Sum of the first 1000 numbers is: 500500
```

b) The compilation error is: `')' expected, but eof found`

c) The file should look like this:

```
val n = args(0).toInt
val summa = (1 to n).sum
println(s"Sum of the first $n numbers is: $summa")
```

The output is:

```
Sum of the first 5001 numbers is: 12507501
```

d) The runtime error is:

```
java.lang.ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0
```

Since the array `args` is empty when no program arguments are given, the index 0 does not exist.

Solution Task 4. *Scala application with `@main`.*

a) The compiler created 5 files in subdirectories under `.scala-build` named:

```
'hello$package.class'  'hello$package$.class'  'hello$package.tasty'
run.class   run.tasty
```

b) You get the error message if you remove the last curly brace. `eof` in the error message means end-of-file. This error message is common for unmatched parentheses, but the compiler often has difficulty giving a good error message if one of the parentheses is missing and it may point to an incorrect line for where it expects the missing character.

c) `Syntax Error: Expected a toplevel definition`. Without curly braces, indentation determines which parts of the code belong together. If you remove the indentation on the last line with the print statement, the compiler interprets this as being *outside* the main function and you get an error message because it is not allowed to have statements by themselves at top level. (However, it is allowed to have standalone statements in a script with `.sc` at the end of the filename.)

d) The annotation `@main` tells the compiler that the function is a main program and can serve as an entry point for execution.

Under the hood, the compiler creates an object with the same name as your main program. In that object, the compiler generates a method called `main` which takes a string array as a parameter and returns `Unit`. A compiled program must have at least one object with exactly one such main method because the JVM execution environment expects it and calls such a main method with a string array containing any program arguments when execution starts.

An alternative to `@main` is to define a so-called *primitive* main method in a single object. (This was necessary in old Scala 2, before the simpler `@main` annotation arrived in Scala 3.)

```
object Hello:
  def main(args: Array[String]): Unit =
    val message = "Hello world!"
    println(message)
```

Solution Task 5. *Create and use collections.*

`val xs = Vector(2)`	1	⇝	I	new reference to a sequence of length 1
`val ys = Array.fill(9)(0)`	2	⇝	C	new reference to a mutable sequence
`Vector.fill(9)(' ')`	3	⇝	J	new immutable sequence with blank characters
`xs(0)`	4	⇝	E	shortcut for `apply(0)`
`xs.apply(0)`	5	⇝	F	indexing, returns the first element
`xs :+ 0`	6	⇝	A	new collection with a zero appended at the end
`0 +: xs`	7	⇝	H	new collection with a zero prepended at the beginnin
`ys.mkString`	8	⇝	K	new string with all elements concatenated
`ys.mkString(",")`	9	⇝	G	new string with commas between elements
`xs.map(_.toString)`	10	⇝	D	new collection, elements converted to strings
`xs.map(_.toInt)`	11	⇝	B	new collection, elements converted to integers

Solution Task 6. *Compare `Array` and `Vector`.*

a)

Vector	1	⇝	B	immutable
Array	2	⇝	A	mutable

b)

Vector	1	⇝	B	creates variants with more/other elements quickly from existing ones
Array	2	⇝	A	slow when changing size (requires copying everything)

c)

Vector	1	⇝	A	`xs == ys` is **`true`** if all elements are equal
Array	2	⇝	B	unlike other Scala collections, == does not check content equality

Solution Task 7. *Calculate sum, min and max in `args`.*

```
@main def sumMinMax(args: Int*): Unit =
  println(s"${args.sum} ${args.min} ${args.max}")
```

```
> scala run sum-min-max.scala -- hej
Illegal command line: java.lang.NumberFormatException: For input string: "hej"
```

Solution Task 8. *Algorithm: SWAP.*

a) The pseudocode may look like this:

```
Declare integer variable temp.
Copy value from x to temp.
```

```
Copy value from y to x.
Copy value from temp to y.
```

b)
You need to declare a temporary variable where you can save one of the values so that it is not overwritten in the first assignment.

```
val temp = x
x = y
y = temp
```

Solution Task 9. *Indexing and assignment in Array with SWAP.*

```
@main def swapFirstLastArg(args: String*): Unit =
  val xs = args.toArray
  if xs.length > 1 then
    val temp = xs(0)
    xs(0) = xs(xs.length - 1)
    xs(xs.length - 1) = temp
  println(xs.mkString(" "))
```

Solution Task 10. ***for*** *expressions and* `map` *expressions.*

`for x <- xs yield x * 2`	1	$\rightsquigarrow$ A	`Vector(2, 4, 6)`
`for i <- xs.indices yield i`	2	$\rightsquigarrow$ E	`Vector(0, 1, 2)`
`xs.map(x => x + 1)`	3	$\rightsquigarrow$ D	`Vector(2, 3, 4)`
`for i <- 0 to 1 yield xs(i)`	4	$\rightsquigarrow$ B	`Vector(1, 2)`
`(1 to 3).map(i => i)`	5	$\rightsquigarrow$ C	`Vector(1, 2, 3)`
`(1 until 3).map(i => xs(i))`	6	$\rightsquigarrow$ F	`Vector(2, 3)`

Solution Task 11. *Algorithm: SUMBUG*

a) Bug: Since `i` is never incremented, the program gets stuck in an infinite loop. Fix: Add a statement at the end of the while-block that increases the value of i by 1. Bug: Since sum is only increased by 1 each time, the result will be the sum of n ones, not the first n integers. Fix: Change so that the sum increases by `i` each iteration, instead of 1. For -1, the result is 0. Explanation: i starts at 1 and is thus never less than n which is -1. The while block executes zero times, and after sum receives its initial value it never changes.

b) The sum is 39502716.
One possible implementation:

```
@main def sumn(n: Int): Unit =
  var sum = 0
  var i = 1
  while i <= n do
    sum = sum + i
    i = i + 1
```

```
  println(sum)
```

L.2.2 Extra Tasks; additional practice

Solution Task 12. *Algorithm: MAXBUG*

a) Bug: `i` is never incremented. The program gets stuck in an infinite loop. Fix: Add a statement that increments i at the end of the while loop.

b) Here is one implementation:

```
@main def maxn(args: String*): Unit =
  var max = Int.MinValue
  val n = args.length
  var i = 0
  while i < n do
    val x = args(i).toInt
    if x > max then
      max = x
    i += 1
  println(max)
```

c) Change the line initializing max to `var max = args(0).toInt`

d) To avoid `java.lang.IndexOutOfBoundsException: 0` you need a check that ensures nothing is done if the collection `args` is empty:

```
@main def maxn(args: String*): Unit =
  if args.size > 0 then
    var max = args(0).toInt
    val n = args.size
    var i = 0
    while i < n do
      val x = args(i).toInt
      if x > max then
        max = x
      i += 1
    println(max)
  else
    println("Empty")
```

Solution Task 13. *Algorithm MIN-INDEX.*

a) An unnecessary comparison occurs, but does not affect the result.

b)

```
def indexOfMin(xs: Array[Int]): Int =
  var minPos = 0
  var i = 1
  while i < xs.size do
    if xs(i) < xs(minPos) then
      minPos = i
    i += 1
  if xs.size > 0 then minPos else -1
```

Solution Task 14. *The data structure Range.*

a) value: `Range(1,2,3,4,5,6,7,8,9)`
type: `scala.collection.immutable.Range`

b) value: `Range(1,2,3,4,5,6,7,8,9,10)`
type: `scala.collection.immutable.Range`

c) value: `Range(0,5,10,15,20,25,30,35,40,45)`
type: `scala.collection.immutable.Range`

d) value: `10`, type: `Int`

e) value: `Range(0,5,10,15,20,25,30,35,40,45,50)`
type: `scala.collection.immutable.Range`

f) value: `11`, type: `Int`

g) value: `Range(0,1,2,3,4,5,6,7,8,9)`
type: `scala.collection.immutable.Range`

h) value: `Range(0,1,2,3,4,5,6,7,8,9)`
type: `scala.collection.immutable.Range`

i) value: `Range(0,1,2,3,4,5,6,7,8,9)`
type: `scala.collection.immutable.Range`

j) value: `Range(0,1,2,3,4,5,6,7,8,9,10)`
type: `scala.collection.immutable.Range.Inclusive`

k) value: `Range(0,1,2,3,4,5,6,7,8,9,10)`
type: `scala.collection.immutable.Range.Inclusive`

l) value: `Range(0,5,10,15,20,25,30,35,40,45)`
type: `scala.collection.immutable.Range`

m) value: `Range(0,5,10,15,20,25,30,35,40,45,50)`
type: `scala.collection.immutable.Range`

n) value: `11`, type: `Int`

o) value: `500500`, type: `Int`

L.2.3 Advanced Tasks; challenges

Solution Task 15. *Rock-Paper-Scissors game.* One (easy-to-understand?) solution that tries all combinations:

```
def winner(user: Int, computer: Int): String =
  if      choices(user) == "Rock" && choices(computer) == "Paper" then "Computer"
  else if choices(user) == "Rock" && choices(computer) == "Scissors"  then "You"
  else if choices(user) == "Paper" && choices(computer) == "Rock" then "You"
  else if choices(user) == "Paper" && choices(computer) == "Scissors"  then "Computer"
  else if choices(user) == "Scissors"  && choices(computer) == "Rock" then "Computer"
  else if choices(user) == "Scissors"  && choices(computer) == "Paper" then "You"
  else "None"
```

A trickier solution (and harder to understand?) using modulo arithmetic:

```
def winner(user: Int, computer: Int): String =
  val result = (user - computer + 3) % 3
  if user == computer then "None"
  else if result == 1 then "You"
  else "Computer"
```

Modulo arithmetic requires the elements in `choices` to be in *loses-to* order, that is Rock, Paper, Scissors. Adding 3 is to avoid negative numbers which behave differently in modulo arithmetic.

Solution Task 16. *Compare execution time for resizing between `Array` and `Vector`.*

a) With a computer that has an `i7-4790K CPU @ 4.00GHz` this was the result:

```
scala> def time(block: => Unit): Double =
     |   val t = System.nanoTime
     |   block
     |   (System.nanoTime - t)/1e6  // gives milliseconds
def time(block: => Unit): Double

scala> val as = Array.fill(1e6.toInt)(0)
val as: Array[Int] = Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
large output truncated, print value to show all

scala> val vs = Vector.fill(1e6.toInt)(0)
val vs: Vector[Int] = Vector(0, 0, 0, 0, 0, ...
large output truncated, print value to show all

scala> val ast = (for i <- 1 to 10 yield time(as :+ 0)).sum / 10.0
val ast: Double = 1.8719819999999998

scala> val vst = (for i <- 1 to 10 yield time(vs :+ 0)).sum / 10.0
val vst: Double = 0.006485099999999999

scala> ast / vst
val res3: Double = 288.6589258453995
```

b) `Vector` is two orders of magnitude faster in this example. The reason is that every size change of an `Array` requires allocation and element-wise copying of a completely

new `Array`, whereas the immutable `Vector` can reuse the entire data structure with already allocated elements when new elements are added.

Solution Task 17. *Memory usage for `Range`.*

a) The variable `interval` refers to objects taking up 12 bytes.

b) The variable `sequence` refers to objects taking up about 4 billion bytes.

Solution Task 18. *Examine the generated byte-code.*

a) The function `plusxy` looks like this:

```
public int plusxy(int, int);
  descriptor: (II)I
  flags: (0x0001) ACC_PUBLIC
  Code:
    stack=2, locals=3, args_size=3
       0: iload_1
       1: iload_2
       2: iadd
       3: ireturn
    LineNumberTable:
      line 2: 0
    LocalVariableTable:
      Start  Length  Slot  Name   Signature
          0       4     0  this   Lplusxy$package$;
          0       4     1     x   I
          0       4     2     y   I
```

The instruction `iadd` performs the actual addition.

b) There is one more parameter in the bytecode. The instruction `iadd` now occurs twice. The `iadd` instruction adds exactly two integers at a time.

```
public int plusxyz(int, int, int);
  descriptor: (III)I
  flags: (0x0001) ACC_PUBLIC
  Code:
    stack=2, locals=4, args_size=4
       0: iload_1
       1: iload_2
       2: iadd
       3: iload_3
       4: iadd
       5: ireturn
    LineNumberTable:
      line 2: 0
    LocalVariableTable:
      Start  Length  Slot  Name   Signature
          0       6     0  this   Lplusxyz$package$;
          0       6     1     x   I
          0       6     2     y   I
          0       6     3     z   I
```

c) The prefix `i` in the instruction name `iadd` stands for "integer" and indicates that integer addition is intended.

L.3 Solution `Excercise03`

Solution Task 1. *Match concepts with description.*

function header	1	⇝	C	has parameter list and optionally a return type
function body	2	⇝	N	code executed on function call
parameter list	3	⇝	A	describes names and types of parameters
block	4	⇝	O	may have local names; last line gives value
named arguments	5	⇝	H	arguments can be passed in any order
default argument	6	⇝	L	arguments can be omitted
value call	7	⇝	B	argument evaluated before call
name call	8	⇝	D	delayed evaluation of argument
map	9	⇝	F	applies a function to each element of a collection
pure function	10	⇝	I	always returns same result for same arguments
predicate	11	⇝	G	a function returning a boolean value
random seed	12	⇝	M	produces a repeatable pseudo-random sequence
anonymous function	13	⇝	E	function without a name, also called lambda
recursive function	14	⇝	K	a function that calls itself
stack trace	15	⇝	J	list of calls at runtime error

Solution Task 2. *Define and call functions.*

a)

```
def increase(x: Int = 1): Int = x + 1
```

b) `5`

c)

```
def reduce(x: Int = 1): Int = x - 1
```

d) `1`

e)

- *Short, simplified explanation:* The parameter in the function header is a local name for input that can be used in the function body, while the argument is the actual value of the parameter that is passed in the call.

- *Longer, more precise explanation:* A **parameter** is a declaration of an immutable variable in a function header whose name is locally available in the function body. At a call, the parameter name is *bound* to a specific argument. An **argument** is an expression that is applied to a function at call time. Normally the argument is evaluated before the call happens, but if the parameter type is preceded by `=>` the evaluation of the argument is delayed and instead happens *each time* the parameter name occurs in the function body.

Solution Task 3. *Implement function in different ways.*

a)

```
def sumFirst(n: Int): Int = ???
```

b)

```
def sumFirst(n: Int): Int = (1 to n).sum
```

```
scala> sumFirst(-1)
val res0: Int = 0
```

c)

```
def sumFirst(n: Int): Int =
  var result = 0
  var i = 1
  while i <= n do
    result += i
    i += 1
  end while
  result
end sumFirst
```

```
scala> sumFirst(-1)
val res1: Int = 0
```

Solution Task 4. *The text game AliensOnEarth.*

a) `"penguin"` is the best option with probability $\frac{1}{2} + \frac{1}{2} \cdot \frac{1}{3} = \frac{2}{3}$

b)

`options.indices`	1	$\rightsquigarrow$	B	integer sequence with all indices in a sequence
`"1X2".toLowercase`	2	$\rightsquigarrow$	A	convert string to lowercase
`Random.nextInt(n)`	3	$\rightsquigarrow$	C	random number in range `0 until n`
`try { } catch { }`	4	$\rightsquigarrow$	E	catch exceptions to prevent crash
`""" ... """`	5	$\rightsquigarrow$	F	string spanning multiple lines
`s.stripMargin`	6	$\rightsquigarrow$	D	removes margin up to vertical bar
`e.printStackTrace`	7	$\rightsquigarrow$	G	prints exception information

Solution Task 5. *Pure functions.*

- The functions `inc`, `addY` and `isPalindrome` are pure. Note that the variable `y` is initialized to `0` and cannot be changed because it is declared with the keyword `val`.

Solution Task 6. *Apply function to each element in a collection. Function as argument.*

`for i <- 1 to 3 yield increment(i)`	1	⇝ E	`Vector(2, 3, 4)`
`Vector(2, 3, 4).map(i => increment(i))`	2	⇝ A	`xs`
`xs.map(increment)`	3	⇝ B	`Vector(4, 5, 6)`
`xs.map(increment).map(increment)`	4	⇝ D	`Vector(5, 6, 7)`
`xs.foreach(increment)`	5	⇝ C	`()`

Solution Task 7. *Anonymous functions.*

`(0 to 2).map(i => i + 1)`	1	⇝ B	`(2 to 4).map(i => i - 1)`
`(1 to 3).map(_ + 1)`	2	⇝ D	`Vector(2, 3, 4)`
`(2 to 4).map(math.pow(2, _))`	3	⇝ E	`Vector(4.0, 8.0, 16.0)`
`(3 to 5).map(math.pow(_, 2))`	4	⇝ A	`Vector(9.0, 16.0, 25.0)`
`(4 to 6).map(_.toDouble).map(_ / 2)`	5	⇝ C	`Vector(2.0, 2.5, 3.0)`

Solution Task 8. *Create your own control structure using call-by-name parameter.*

a)

```
def repeat(n: Int)(block: => Unit): Unit =
  var i = 0
  while i < n do
    block
    i += 1
  end while
```

b)

```
repeat(100):
  val diceRoll = (math.random() * 6 + 1).toInt
  print(s"\$diceRoll ")
```

c) If the parameter `block` were not declared with call-by-name, the argument would be evaluated once before the call and then the same result used at every iteration. With call-by-name, the block can contain code that e.g. updates a variable which we want to happen every iteration. Call-by-name is similar to the argument code being "pasted" into each place in the function body where the parameter name occurs.

Solution Task 9. *Learn to read a stack trace.*A stack trace contains the following information:

1. an error message
2. names of all functions called at the time of the runtime error, according to all activation records on the call stack
3. current namespace for each function, that is package/singleton object

4. source file name for each function
5. line number in each function
6. the first function shown is the one where the error occurred
7. possibly the error occurs in standard library functions and then your own function is earlier in the call chain

Example of a stack trace:

```
> cat fel.scala
@main def run =
  println("Hej Scala!" + Vector().head)
> scala run fel.scala
Compiling project (Scala 3.3.0, JVM)
Compiled project (Scala 3.3.0, JVM)
Exception in thread "main" java.util.NoSuchElementException: empty.head
  at scala.collection.immutable.Vector.head(Vector.scala:279)
  at fel$package$.run(fel.scala:2)
  at run.main(fel.scala:1)
>
```

L.3.1 Extra Tasks; additional practice

Solution Task 10. *Function with multiple parameters.*

a)

```
def sum(x: Int, y: Int): Int = x + y

def diff(x: Int, y: Int): Int = x - y
```

b) It is `-100` because `0 - 100 == -100`

c) It is 15 because the nested call corresponds to
`diff(100, 42 + 43) == (100 - 85)`

d) It is 185 because the nested call corresponds to
`sum(42 + 43, 100 - 0) == (85 + 100)`

e) It is 256 because `Byte.MaxValue == 127` and `Byte.MinValue == -128` and `sum(127 + 128, 1)`

Solution Task 11. *Average value.*

```
def avg(x: Int, y: Int): Double = (x + y) / 2.0
```

Solution Task 12. *Function calls with named arguments.*

a)

```
Namn: Triangelsson, Stina
Namn: Oval, Viktor
```

b)

- The caller can choose the order themselves.

- The code becomes easier to understand if parameter names are self-describing.
- Helps to prevent bugs caused by swapped parameters.

Solution Task 13. *Function as a genuine value.*

a)

`multipleCalls(1, greet)`	1	⇝ D	`f2("Hello!")`
`multipleCalls(3, greet)`	2	⇝ B	`multipleCalls(3, f1)`
`multipleCalls(2, f1)`	3	⇝ A	`f2("Hello!\nHello!")`
`multipleCalls(1, f3)`	4	⇝ C	`f3()`

b) `f1` and `f3` have type `() => Unit` and `f2` has type `String => Unit`.

c) No. `f1` and `f2` have two different function types.

d) Yes, that works fine.

e) No. When the function has no parameter, the compiler needs more information to be sure that you want a function value.

f) Yes! Now with type information in place, the compiler knows what you want to do.

Solution Task 14. *Discarded result values and return type `Unit`.*

a) Procedures return the empty value and `println` is a procedure. When the empty value is printed, you see `()`.

b) Procedures return the empty value. If you explicitly specify return type `Unit`, you have a better chance that the compiler can warn you when calculations are discarded. A warning does not stop execution, but is a way for the compiler to give you tips about things that might need fixing in your code.

c) In Scala variable declaration, just like an assignment statement, is not an expression and has no value.

d) The code becomes easier to read and the compiler gets better possibility to help with warnings if result values risk being discarded.

Solution Task 15. *Call-by-name.*
The block is an expression that has the value `(): Unit`. Evaluation of the block happens where the name `b` appears in the procedure body, which is twice.

```
scala> doThisTwice { println("goededag") }
goededag
goededag
```

L.3.2 Advanced Tasks; challenges

Solution Task 16. *Mutability of parameters.*

a) No, in Scala the parameter is immutable and it will be a compile error if you try to assign it a new value in the function body.

b) c) Yes, it is perfectly fine in both Java and Python to change the value of the parameter in the function body by assignment, but the code risks to become confusing. https://stackoverflow.com/questions/2970984

Solution Task 17. *Call-by-value and call-by-name.*

a) Each call of `snark` causes a print and a delay before returning 42.
`42 + 42 == 84` which becomes the expression value.

```
scala> snark + snark
snark snark val res1: Int = 84
```

b) The expression `snark` is evaluated directly at the call and the parameter `x` is bound to the value 42 and in the function body $42+42$ is computed. The print happens only once.

```
callByValue(snark)
snark val res2: Int = 84
```

c) The evaluation of the expression `snark` is delayed until each occurrence of parameter `x` in the function body. The print happens twice.

```
callByName(snark)
snark snark val res3: Int = 84
```

d) Evaluation of the expression `zzz` is delayed until each occurrence of parameter `x` in the function body. The print happens once because **`val`** variables are assigned their value once at the delayed initialization.

```
callByName(zzz)
snark val res4: Int = 84
```

Solution Task 18. *Create your own control structure for iteration with loop variable.*

a)

```
def repeat(n: Int)(p: Int => Unit): Unit =
  var i = 0
  while i < n do
    p(i)
    i += 1
  end while
end repeat
```

b)

```
repeat(100){ i =>
  print("i ")
```

```
  println(math.random())
}
```

You can use fewer braces with the help of colon:

```
repeat(100): i =>
  print("i ")
  println(math.random())
```

Solution Task 19. *Split parameter list and curried functions.*

a)

```
1 scala> def add2(a: Int)(b: Int) = a + b
2 def add2(a: Int)(b: Int): Int
3
4 scala> add2(1)(1)
5 val res0: Int = 2
```

b)

- Line 3:

```
doremi doremi doremi
```

- Line 5:

```
lalalalalalala
```

Solution Task 20. *Recursion.*

a) `countdown` prints out `x` and does a recursive call with `x - 1` as argument, but only if the base condition `x > 0` is true. The result is a finite repetition. `finalCountdown` calls itself recursively but lacks a base case that can stop the recursion, which generates an infinite repetition. At -128 there is *overflow* since the bits are not enough for larger negative numbers and the counting starts over at 127. (If you reduce the delay to `Thread.sleep(1)` it becomes quite fast *stack overflow*)

b) Since `1/x` was *after* the recursive call in the previous subtask, we never reached this (potentially disastrous) calculation, but just put many activation frames on the stack at each call. If we place `1/x` *before* the recursive call, we reach this expression immediately and an exception is thrown due to division by zero.

c) The last line leads to many more recursive calls, as the base condition and recursion are constructed. Feel free to insert a `println` statement before the recursive call and examine exactly what happens.

Solution Task 21. *Investigate tail recursion by throwing exceptions.* `countdown` is tail-recursive because the recursive call is *last* and can be optimized to a `while`-loop by the compiler. It is fine to run all the way until it explodes, even with 10000 calls, and the error message only contains one call to `countdown`.
`countdown2` is not tail-recursive because it has an expression `after` the recursive call. In the error printout you can see all recursive calls to `countdown2` before the base case occurs. At `countdown2(10000)` the base case is not fulfilled before a `StackOverflowError` occurs.

Solution Task 22. *`@tailrec` annotation.* The first time `countNoTailrec(100000L)` is called it causes `StackOverflowError`. With annotation `@tailrec` we get a compilation error because the compiler cannot optimize a non-tail-recursive function. If the function is rewritten, the compiler can optimize it so that the recursion is replaced with a **`while`**-loop and we can run as long as we want without stack overflow. And it runs very fast!!

L.4 Solution `Excercise04`

L.4.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match concepts with descriptions.*

module	1	⇝	C	code unit with abstractions that can be reused
singleton object	2	⇝	B	module that can have state; exists in a single instance
package	3	⇝	D	module creating namespace; machine code gets own folder
import	4	⇝	F	makes name locally available without full path
export	5	⇝	P	makes name visible outside as a member of this object
lazy initialization	6	⇝	G	allocation occurs only when name is referenced
member	7	⇝	E	belongs to an object; accessed via dot notation if visible
attribute	8	⇝	H	variable representing (part of) an object's state
method	9	⇝	A	function that is a member of an object
private	10	⇝	K	modifies visibility of an object member
overloading	11	⇝	J	methods with same name but different parameter types
name shadowing	12	⇝	L	local name hides same name in enclosing block
namespace	13	⇝	I	environment where all names are unique
uniform access	14	⇝	M	changing between def and val does not affect usage
dot notation	15	⇝	O	used to access non-private parts
type alias	16	⇝	N	alternative name for type, often improves readability

Solution Task 2. *Nested singleton objects, import, visibility and dot notation.*

a)

```
object Underworld:
  var x = 0
  var y = 1

  object Mole:
    var x = Underworld.x + 10
    var y = Underworld.y + 9

  object Worm:
    private var x = Mole.x
    var y = Mole.y + 190
    def isMoleFood: Boolean = x == Mole.x && y == Mole.y
```

b)

```
scala> :load Underworld.scala
scala> import Underworld.*
scala> Worm.isMoleFood
val res0: Boolean = false
scala> Worm.y = Mole.y
scala> Worm.isMoleFood
```

```
val res1: Boolean = true
```

c)

```
scala> import Mole.*
scala> import Worm.*
scala> x = -1
scala> Mole.x
val res2: Int = -1

scala> Worm.x
1 |Worm.x
  |^^^^^^^^
  |variable x cannot be accessed as a member of Underwold.Worm.type from module

scala> Underworld.x
val res3: Int = 0
```

Explanation: When importing all visible members in Worm, those that are not private will shadow other members with the same name. It is Mole's `x` variable that is assigned `-1` because Worm's `x` is private and not visible externally. Underworld's members are shadowed by Worm's `y` and Mole's `x`, but you can still access them using dot notation.

Solution Task 3. *Export.*

a) Similarity: Both **`import`** and **`export`** control visibility. Difference: **`import`** controls local visibility *inside* an object while **`export`** controls visibility *outside* an object.

b) You can conveniently gather members from other objects and make them visible from your own object using **`export`**.

```
object MyObject:
  export java.awt.Color.*  // all colors become members of MyObject
  export math.{atan2, Pi}  // atan2 and Pi become members of MyObject
```

```
scala> object MyObject:
     |   export java.awt.Color.*
     |   export math.{atan2, Pi}
     |

scala> MyObject.RED
val res0: java.awt.Color = java.awt.Color[r=255,g=0,b=0]

scala> MyObject.atan2(3,3) / MyObject.Pi
val res1: Double = 0.25
```

Solution Task 4. *Tuples.*

a) `depth` has type `Double`.

b) `secret` has type `(String, (Int, Int, Double))`.

c)

```
object Underworld3D:
  private val secret = ("the ascent to the uberworld", (3, 4, 0.0))

  object Mole:
```

```
    var pos = (5, 3, math.random() * 10 + 1)

    def depth: Double  = pos._3

  object Worm:
    private var pos = (0, 0, 10.0)

    def isMoleFood: Boolean = pos == Mole.pos

    def isRightUnderTheRise: Boolean =
      pos._1 == secret._2._1 && pos._2 == secret._2._2
```

d) Zero-tuple.

Solution Task 5. *Lazy initialization.*

a) `"nu!"` is printed only the first time `z` is used.

```
scala> z
nu!
val res19: Array[Int] = Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0)

scala> z
val res20: Array[Int] = Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
```

b) The array allocation happens the first time `z` is used (and not at declaration).

```
scala> lazy val z = { println("nu!"); Array.fill(1e9.toInt)(0)}
val z: Array[Int] = <lazy>

scala> z
nu!
java.lang.OutOfMemoryError: Java heap space
```

c) The print `"nu!"` occurs only when singleton object `zzz` is used for the first time. We should move the initialization of `b` before `a` or make `a` a **`lazy val`**.

d)

```
scala> import test.*
import test.*

scala> zzz.a      // "nu!" is printed when we use zzz
nu!               // this did *not* happen when we imported test
val res0: Int = 42

scala> buggig.a   // a is 0 because b was not initialized
val res1: Int = 0

scala> funkar.a   // with lazy val we avoid the problem
val res2: Int = 42


scala> zzz.a     // second time initialization already done, no "nu!"
val res3: Int = 42
```

e) `lazy val a = expression` means the initialization expression is evaluated *once*, but the evaluation is delayed until the name `a` is actually used. Whereas `def b = expression` means the function body expression is evaluated *each time* the name `b` is (possibly) used.

Solution Task 6. *Extension methods.*

a)

```
scala> extension (i: Int) def inc = i + 1
```

b)

```
scala> extension (i: Int) def dec = i - 1
```

c)

```
extension (i: Int)
  def inc = math.incrementExact(i)
  def dec = math.decrementExact(i)
```

d) Using `math.incrementExact` and `math.decrementExact` throws an exception if we cross the boundary:

```
scala> math.incrementExact(Int.MaxValue)
java.lang.ArithmeticException: integer overflow
  at java.base/java.lang.Math.incrementExact(Math.java:1023)
  at scala.math.incrementExact(package.scala:418)
  ... 34 elided
```

Solution Task 7. *Extension methods.*

a) According to the documentation, the `PixelWindow` class has these parameters:

- `width : Int` specifies the window width, default `800`
- `height: Int` specifies the window height, default `640`
- `title : String` specifies the window title, default `"PixelWindow"`
- `background: Color` specifies background color, default `java.awt.Color.black`
- `foreground: Color` specifies foreground color, default `java.awt.Color.green`

You can create new window instances like this:

```
val w1 = new introprog.PixelWindow()
val w2 = new introprog.PixelWindow(100, 200, "My nice new window")
```

b) You can also download the latest `introprog` like this:

```
curl -o introprog_3-1.4.0.jar -sLO https://fileadmin.cs.lth.se/introprog.jar
```

c)

```
> scala repl --jar introprog_3-1.4.0.jar
scala> val w = new introprog.PixelWindow(400,300,"HEJ")
scala> w.line(100, 100, 200, 100)
scala> w.line(200, 100, 200, 200)
scala> w.line(200, 200, 100, 200)
scala> w.line(100, 200, 100, 100)
```

d)

```
package hello

object Main:
  val w = new introprog.PixelWindow(400, 300, "HEJ")

  var color = java.awt.Color.red

  def square(p: (Int, Int))(side: Int): Unit =
    if side > 0 then
      // side == 1 gives a square that is a single pixel
      val d = side - 1

      w.line(p._1,     p._2,     p._1 + d, p._2,     color)
      w.line(p._1 + d, p._2,     p._1 + d, p._2 + d, color)
      w.line(p._1 + d, p._2 + d, p._1,     p._2 + d, color)
      w.line(p._1,     p._2 + d, p._1,     p._2,     color)

  def main(args: Array[String]): Unit =
    println("Draw square:")
    square(300,100)(50)
```

e)

```
> scala run hello-window.scala --jar introprog_3-1.4.0.jar --main-class hello.M
```

f)

```
> scala run hello-window.scala --dep se.lth.cs::introprog:1.4.0 --main-class he
```

g)

```
//> using scala 3.3
//> using dep se.lth.cs::introprog:1.4.0
```

Solution Task 8. *Color*.

a)

```
object Color:
  import java.awt.{Color as JColor}

  val mole   = new JColor( 51,  51,   0)
  val soil   = new JColor(153, 102,  51)
  val tunnel = new JColor(204, 153, 102)
```

b)

```
package hello

object Color:
  import java.awt.{Color as JColor}
```

```
  val mole   = new JColor( 51, 51,    0)
  val soil   = new JColor(153, 102, 51)
  val tunnel = new JColor(204, 153, 102)


object Main:
  val w = new introprog.PixelWindow(width = 400, height = 300, title = "HEJ")

  type Pt = (Int, Int)

  var color = java.awt.Color.red

  def straight(p:  Pt)(d: Int) = w.line(p._1, p._2, p._1 + d - 1, p._2, color)

  def fill(p: Pt)(s: Int) = for i <- 0 until s do straight((p._1, p._2 + i))(s)

  def square(p: (Int, Int))(side: Int): Unit =
    if (side > 0) then
      val d = side - 1  // side == 1 should give a square that is one pixel big
      w.line(p._1,     p._2,     p._1 + d, p._2,     color)
      w.line(p._1 + d, p._2,     p._1 + d, p._2 + d, color)
      w.line(p._1 + d, p._2 + d, p._1,     p._2 + d, color)
      w.line(p._1,     p._2 + d, p._1,     p._2,     color)

  def main(args: Array[String]): Unit =
    import Color.*
    color = soil
    fill(100,100)(75)
    color = tunnel
    fill(100,100)(50)
    color = mole
    fill(150,150)(25)
```

c) Tuple parentheses can be omitted in calls to `straight` and `fill` when a single tuple argument is the only parameter in its parameter list.

Solution Task 9. *Events.*

a) The immutable integer variable `KeyPressed` in `introprog.PixelWindow.Event` has the value 1.

b) The code line below handles the key press case:

```
case PixelWindow.Event.KeyPressed => println(s"lastKey == \$w.lastKey")
```

c) When the up arrow key on the keyboard is pressed, `w.lastKey` gets the string value `"Up"`. The test program prints the following when the up arrow key is pressed and released:

```
lastEventType: 1 => KeyPressed
lastKey == Up
lastEventType: 2 => KeyReleased
lastKey == Up
```

d) A loop that lets the user draw lines with the mouse:

```
var start = (0,0)
while w.lastEventType != PixelWindow.Event.WindowClosed do
  w.awaitEvent(10)  // wait for next event max 10 milliseconds
  w.lastEventType match {
    case PixelWindow.Event.MousePressed  =>
      start = w.lastMousePos

    case PixelWindow.Event.MouseReleased =>
      w.line(start._1, start._2, w.lastMousePos._1, w.lastMousePos._2)

    case PixelWindow.Event.WindowClosed  =>
       println("Goodbye!");
    case _ =>
  }
  PixelWindow.delay(100) // wait for 0.1 seconds
```

L.4.2 Extra Tasks; additional practice

Solution Task 10. *Functions are objects with an `apply` method.* Yes, you can write:

```
scala> plus(42, 43)
```

The compiler inserts `.apply` for you.

Solution Task 11. *Create modules using singleton objects.*

a)

```
scala> "pearIceCream".split('I')
val res0: Array[String] = Array(pear,ceCream )
```

b)

```
scala> Test()
--- FREQUENCY ANALYSIS OF:
Five ants are more than four elephants. Eat cucumber.
# letters : 43
# words   : 9
# sentences: 2

--- FREQUENCY ANALYSIS OF:
Galaxies in my breams. Tomato is good. Pear split.
# letters : 39
# words   : 9
# sentences: 3

--- FREQUENCY ANALYSIS OF:
Five ants are more than four elephants. Eat cucumber. Galaxies in my breams. To
# letters : 82
# words   : 18
# sentences: 5
```

c) The object `statistics` has mutable state in the variable `history`. The state changes when calling `printFreq`.

d)

```
object count:
  extension (s: String)
    def nbrOfLetters:Int = s.count(_.isLetter)
    def nbrOfWords:Int = split.words(s).size
    def nbrOfSentences: Int = split.sentences(s).size
```

Solution Task 12. *Tuples as parameters.*

```
def distxy(x1: Int, y1: Int, x2: Int, y2: Int): Double =
  hypot(x1 - x2, y1 - y2)

def distpt(p1: (Int, Int), p2: (Int, Int)): Double =
  hypot(p1._1 - p2._1, p1._2 - p2._2)

def distp(p1: (Int, Int))(p2: (Int, Int)): Double =
```

```
  hypot(p1._1 - p2._1, p1._2 - p2._2)
```

Solution Task 13. *Tuples as function results.*

```
def statistics(xs: Vector[Double]): (Int, Double, (Double, Double)) =
  (xs.size, xs.sum / xs.size, (xs.min, xs.max))
```

```
scala> statistics(Vector(0, 2.5, 5))
val res10: (Int, Double, (Double, Double)) = (3,2.5,(0.0,5.0))
```

Solution Task 14. *Create modules using packages.*

a)

```
> code paket.scala
> scala paket.scala
> find . -type d         # linux command that lists all subdirectories
./.scala-build/project_103be31561-3d0d386400/classes
./.scala-build/project_103be31561-3d0d386400/classes/main
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p2
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p2/
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p1
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p1/
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p1/
```

b)

```
> scala run paket.scala --main-class gurka.tomat.banan.Main
Hej paket p1.p11!
Hej paket p1.p12!
Hej paket p2.p21!
```

c) Yes, in Scala 3 packages are allowed to have variables and functions at top level. https://stackoverflow.com/a/56566166

L.4.3 Advanced Tasks; challenges

Solution Task 15. *How to do without* `do while` *in Scala 3?*

a) It gives a compilation error:

```
> scala repl --scala 3
Welcome to Scala 3.1.3 (17.0.3, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> var i = 0
var i: Int = 0

scala> do i += 1 while (i < 10)
-- [E103] Syntax Error: ----------------------------------------
1 |do i += 1 while (i < 10)
```

```
  |^^
  |Illegal start of statement
```

b)

```
> scala repl --scala 3
Welcome to Scala 3.1.3 (17.0.3, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> var i = 0
var i: Int = 0

scala> while
     |   i += 1
     |   i < 10
     | do ()

scala> i
val res0: Int = 10
```

Solution Task 16. *Postfix operators for increment and decrement.*

```scala
extension (i: Int)
  def ++ = i + 1
  def -- = i - 1
```

Solution Task 17. *Use ready package: Color chooser.*

a) The chosen color is returned after the user clicks **OK**

```
scala> introprog.Dialog.selectColor()
val res1: java.awt.Color = java.awt.Color[r=0,g=204,b=0]
```

b) The default color red is returned after the user clicks **Cancel**

c) Color chooser reverts to default color.

Solution Task 18. *Use ready package: user dialogs.*

a)

```
scala> introprog.Dialog.show("Game over!")
```

b) The function `input` returns a string which is the empty string `""` if the user clicks **Cancel**

```
scala> val name = introprog.Dialog.input("What is your name?")
name: String = Oddput Superkodare
```

c) The function `select` returns a string with the text on the button that the user clicked.

```
scala> introprog.Dialog.select("Wat kies jij?",Vector("Steen","Schaar","Papier"
val res4: String = Papier
```

Solution Task 19. *Create your own `jar` file.*

a)

`jar -create -verbose -file <name of created jar file> <name of what to package>`

b)

```
package hello

object Main:
  def main(args: Array[String]): Unit = println("Hello package!")
```

```
scala compile hello.scala --destination .
```

c)

```
> jar -c -v -f my.jar hello
> ls
> scala repl --jar my.jar
scala> hello.Main.main(Array())
Hello package!
```

d)

```
> scala run --jar my.jar  --main-class hello.Main
```

Solution Task 20. *How large is JDK8?* The JDK8 platform includes 4240 precompiled classes, organized in 217 different packages. See Stackoverflow: http://stackoverflow.com/questions/3112882

L.5 Solution Excercise05

L.5.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match concepts with description.*

class	1	⇝	H	template for creating multiple instances of the same type
instance	2	⇝	I	copy of an object with its own state memory
constructor	3	⇝	M	creates instance, allocates space for state memory
class parameter	4	⇝	K	binds to arguments given at construction
reference equality	5	⇝	B	instances are considered different even if states are equal
structural equality	6	⇝	J	instances considered equal if they have the same state
case class	7	⇝	F	no need to write new; automatic structural equality
getter	8	⇝	L	indirect access to attribute value
setter	9	⇝	A	indirect assignment to attribute value
companion object	10	⇝	D	sees private members of class with same name
factory method	11	⇝	E	helper function for indirect construction
`null`	12	⇝	G	a value that does not refer to any instance
`new`	13	⇝	C	keyword for direct instance creation of a class

Solution Task 2. *Class and instance.*

a)

`Singelpunkt.x`	1	⇝	B	`1`
`Punkt.x`	2	⇝	G	`value is not a member of object`
`val p  = new Singelpunkt`	3	⇝	C	`Not found: type`
`val p1 = new Punkt`	4	⇝	D	`p1: Punkt = Punkt@27a1a53c`
`val p2 = Punkt()`	5	⇝	F	`p2: Punkt = Punkt@51ab04bd`
`{ p1.x = 1; p2.x }`	6	⇝	E	`3`
`(new Punkt).y`	7	⇝	H	`2`
`{ val p: Punkt = null; p.x }`	8	⇝	A	`java.lang.NullPointerException`

b)

error	*type*	*explanation*
`value is not a member of object`	compile-time error	there is no instance named `Punkt`. *The error message refers to the fact that the class's auto-generated constructor proxy lacks a variable named* `x`.
`Not found: type`	compile-time error	there is no class named `Singelpunkt`
`NullPointerException`	runtime error	you cannot reference attributes in an instance that does not exist

Solution Task 3. *Class parameters.*

a)

`val p1 = Point(1, 2)`	1	$\rightsquigarrow$ C	`p1: Point = Point@30ef773e`
`val p2 = Point()`	2	$\rightsquigarrow$ A	`missing argument for parameter`
`val p2 = Point(3, 4)`	3	$\rightsquigarrow$ E	`p2: Point = Point@218cf600`
`p2.x - p1.x`	4	$\rightsquigarrow$ B	`2`
`Point(0, 1).y`	5	$\rightsquigarrow$ F	`1`
`Point(0, 1, 2)`	6	$\rightsquigarrow$ D	`too many arguments for constructor`

b)

error	*type*	*explanation*
`missing argument for parameter`	compile-time error	you must provide arguments at construction of class `Point`
`too many arguments for constructor`	compile-time error	the number of arguments does not correspond to the number of class parameters

Solution Task 4. *Immutable class with default arguments.*

a)

`val p1 = Point3D()`	1	⇝ C	`p1: Point3D = Point3D@2eb37eee`
`val p2 = Point3D(y = 1)`	2	⇝ F	`p2: Point3D = Point3D@65a9e8d7`
`Point3D(z = 2).z`	3	⇝ E	`value cannot be accessed`
`p2.y = 0`	4	⇝ B	`Reassignment to val`
`p2.y == 0`	5	⇝ A	`false`
`p1.x == Point3D().x`	6	⇝ D	`true`

b) The problem is that as the class `Point3D` is declared it is not possible to read off the `z` coordinate after an instance is constructed. It would be better if the `z` attribute were also a `val`.

Solution Task 5. *Case class, `this`, equality, `toString` and companion object.*

a)

`val p1 = Pt(1, 2)`	1	⇝ E	`Pt(1,2)`
`val p2 = Pt(y = 3)`	2	⇝ C	`Pt(0,3)`
`val p3 = MutablePt(5, 6)`	3	⇝ A	`MPt(5,6)`
`val p4 = Mutable()`	4	⇝ D	`Not found`
`p2.moved(dx = 1) == Pt(1, 3)`	5	⇝ F	`true`
`p3.move(dy = 1) == MutablePt(5, 7)`	6	⇝ B	`false`

b) The compiler infers `MutablePt` because that is the type of the self-reference `this`.

```
scala> :type new MutablePt().move()
MutablePt
```

c) Instantiation with universal apply methods is sugar that makes code easier to read and write. This is possible thanks to the fact that at compilation automatically a constructor proxy is created that instantiates the object with the keyword `new`. A constructor proxy is a companion object with an apply method.
One case where `new` must explicitly be used is when implementing your own apply method in a companion object. If `new` is not used inside the apply method, the same method will be called recursively instead of creating a new instance. See the following example:

```
class Point3D(val x: Int, val y: Int, val z: Int)

object Point3D:
  var secretNumber = 42
  def apply(x: Int, y: Int, z: Int): Point3D =
    if secretNumber == 42 then
      Point3D(x, y, z) // The code will get stuck in an infinite loop.

    else new Point3D(x, y, z) // Works because 'new' is used.
```

d) A method that reads (parts of) an object's (private) state without modifying it is called a *getter*.

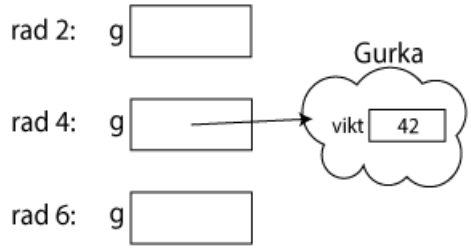
Solution Task 6. *Implement parts of the classes `Pos`, `KeyControl`, `Mole` and `BlockWindow` needed during the lab `blockbattle1`.* This task is lab preparation. Evaluate your solutions through your own tests in REPL.

a) It is not possible to call `Pos.moved(0,1)`. The reason is that `moved` does not exist in the companion object `Pos`, hence the error message "value `moved` is not a member of object `Pos`". To call a method defined inside a class you must call it through a (reference to an) instance of the class.

L.5.2 Extra Tasks; additional practice

Solution Task 7. *Instantiation with `new` and the value `null`.*

a) Lines 3 and 7 both give the error message `java.lang.NullPointerException`, because of attempts to reference members using a `null` reference, which does not point to any object.



b)

Solution Task 8. *Create a point class that can handle polar coordinates and a class that represents a polygon using these points.*

a)

```
package graphics

case class Point(x: Double, y: Double):
  val r: Double = math.hypot(x, y)
  val theta: Double = math.atan2(y, x)
  def +(p: Point): Point = Point(x + p.x, y + p.y)

object Point:
  def polar(r: Double, theta: Double): Point =
    Point(r * math.cos(theta), r * math.sin(theta))
```

b)

```
package graphics

case class Polygon(points: Vector[Point]):
  val nbrOfCorners: Int = points.length

object Polygon:
  def regular(nbrOfCorners: Int, radius: Double, midPoint: Point): Polygon =
    val points = new Array[Point](nbrOfCorners)
    for i <- 0 until nbrOfCorners do
```

```
      val theta = i * (2 * math.Pi) / nbrOfCorners
      points(i) = Point.polar(radius, theta) + midPoint
    end for
    Polygon(points.toVector)
```

c) A perfect circle cannot be created, but it is possible to get so close that it is impossible to see the corners. Try e.g. with 50 corners as below.

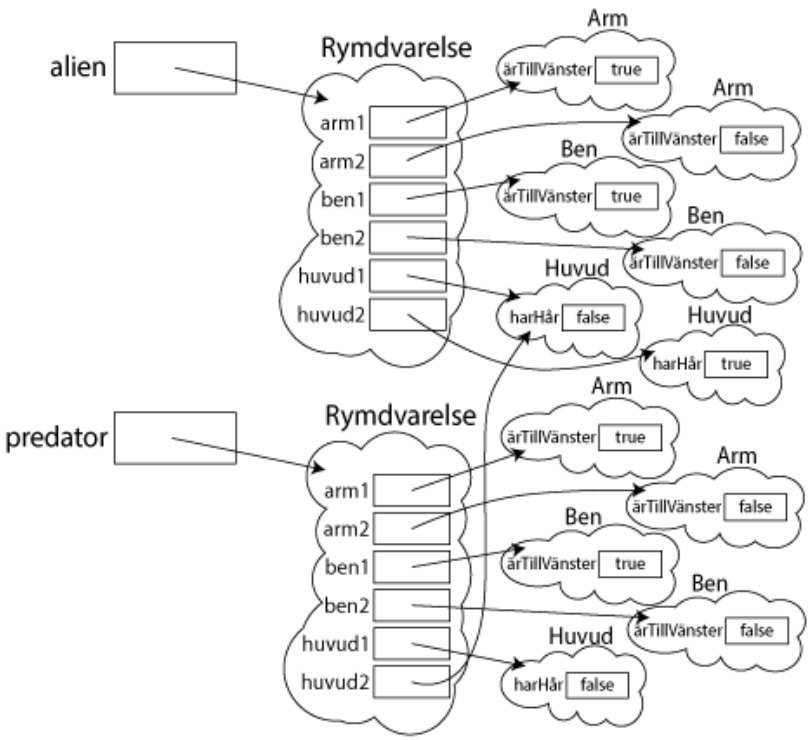
```
scala> import graphics.*
scala> val circle = Polygon.regular(50,70,Point(-25,-25))
scala> val window = PolygonWindow()
scala> window.draw(circle)
```

An irregular polygon can also be created. Use the constructor of `Polygon` directly. For example as below.

```
scala> val irregular =
         Polygon(Vector(Point(1,8), Point(33,14), Point(99,87), Point(42,56)))
scala> window.draw(irregular)
```

Solution Task 9. *Classes, instances and garbage.*

a) We create two aliens, `alien` and `predator`, each with two arms and two legs, and each with two heads (one bald and one hairy). Afterwards, neither `alien` nor `predator` is bald since both have one head with hair. Then the reference to `predator`'s hairy head is made to refer to `alien`'s bald head. Now the predator is totally bald and shares head with the alien.



b) Since there is no longer any reference pointing to that object, the garbage collector will take care of it and eventually the place in memory will be overwritten when the

space is needed. Objects that have no reference to them cannot be accessed.

Solution Task 10. *Case class. Immutable square.*

a)

```
case class Square(val x: Int = 0, val y: Int = 0, val side: Int = 1):
  val area: Int = side * side

  def moved(dx: Int, dy: Int): Square = Square(x + dx, y + dy, side)

  def isEqualSizeAs(that: Square): Boolean = this.side == that.side

  def scale(factor: Double): Square =
    Square(x, y, (side * factor).round.toInt)

object Square:
  val unit: Square = Square()
```

b)

```
scala> val (s1, s2) = (Square(), Square(1, 10, 1))
val s1: Square = Square(0,0,1)
val s2: Square = Square(1,10,1)

scala> val s3 = s1 moved (1,-5)
val s3: Square = Square(1,-5,1)

scala> s1 isEqualSizeAs s3       // same size
val res0: Boolean = true

scala> s2 isEqualSizeAs s1       // same size
val res1: Boolean = true

scala> s1 isEqualSizeAs Square.unit   // s1 has side 1
val res2: Boolean = true

scala> s2.scale(math.Pi) isEqualSizeAs s2  // different size
val res3: Boolean = false

scala> s2.scale(math.Pi) == s2.scale(math.Pi) // same content
val res4: Boolean = true

scala> s2.scale(math.Pi) eq s2.scale(math.Pi)  // different objects
val res5: Boolean = false

scala> Square.unit eq Square.unit   // same object
val res6: Boolean = true
```

L.5.3 Advanced Tasks; challenges

Solution Task 11. *Content equality between different types.*

```
scala> 42 == "Fourtytwo"
1 |42 == "Fourtytwo"
  |^^^^^^^^^^^^^^^^^
  |Values of types Int and String cannot be compared with == or !=

scala> Cucumber(50) == Car("Sedan")
val res0: Boolean = false
```

The second expression is problematic because it will always result in **false**, since the classes `Cucumber` and `Car` are two incomparable types that should not be compared in terms of content equality. This reduces type safety which increases the risk of subtle bugs where wrong types are compared.
Equality comparisons done between primitive types are type-checked by the compiler and therefore can produce compilation errors if two different types, such as `Int` and `String`, are compared. However, this usually does not apply to user-defined types, which means that an equality comparison between different user-defined types will always result in **false**.
It is possible, however, to achieve the same type checks for user-defined types as for primitive types by importing `scala.language.strictEquality`.

```
import scala.language.strictEquality
class Cucumber(val weight: Int)

class Car(val type: String)
```

```
scala> Cucumber(50) == Car("Sedan")
1 |Cucumber(50) == Car("Sedan")
  |^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |Values of types Cucumber and Car cannot be compared with == or !=
```

Solution Task 12. *Attribute representation. Private constructor. Factory method.*

a) It will be a compilation error, because the constructor is private.

```
scala> class Point private (val x: Int, val y: Int)
     | object Point:
     |   def apply(x: Int = 0, y: Int = 0): Point = new Point(x, y)
     |   val origo = apply()
     |
// defined class Point
// defined object Point

scala> new Point(0, 0)
1 |new Point(0, 0)
  |    ^^^^^
  |constructor Point cannot be accessed as a member of Point from module class
```

b)

- By having a private constructor and only allowing indirect instantiation via the factory method it is easy to change attribute representation in the future without existing code needing to change.
- The access rules for companion objects are such that companions see each other's private parts.

c)

```
class Point private (private val p: (Int, Int)):
  def x: Int = p._1
  def y: Int = p._2

object Point:
  def apply(x: Int = 0, y: Int = 0): Point = new Point(x, y)
  val origo = apply()
```

Solution Task 13. *Visibility of class parameters and constructor,* ***`private`****`[this]`*.

a) `Cucumber5` is broken. Since the weight in `Cucumber5` is private to the instance and not the class, an instance cannot access another instance's weight.

```
11 |  def buddyWeight = buddy.weight
   |                    ^^^^^^^^^^^
   |value weight cannot be accessed as a member of (Cucumber5.this.buddy : Cucu
```

b)

```
scala> new Cucumber1(42).weight
1 |new Cucumber1(42).weight
  |^^^^^^^^^^^^^^^^^^^
  |value weight cannot be accessed as a member of Cucumber1 from module class

scala> new Cucumber2(42).weight
val res0: Int = 42

scala> new Cucumber3(42).weight
1 |new Cucumber3(42).weight
  |^^^^^^^^^^^^^^^^^^^
  |value weight cannot be accessed as a member of Cucumber3 from module class

scala> val noneCucumber: Cucumber4 = null
val noneCucumber: Cucumber4 = null

scala> new Cucumber4(42, noneCucumber).buddyWeight
java.lang.NullPointerException: Cannot invoke "rs$line$1$Cucumber4.weight()" be
  at rs$line$1$Cucumber4.buddyWeight(rs$line$1:8)
  ... 38 elided

scala> new Cucumber4(42, new Cucumber4(84, null)).buddyWeight
val res2: Int = 84

scala> new Cucumber6(42)
1 |new Cucumber6(42)
  |     ^^^^^^
  |constructor Cucumber6 cannot be accessed as a member of Cucumber6 from modul
```

```

scala> new Cucumber7(-42)
1 |new Cucumber7(-42)
  |    ^^^^^^
  |constructor Cucumber7 cannot be accessed as a member of Cucumber7 from modul

scala> Cucumber7(-42)
java.lang.IllegalArgumentException: requirement failed: negative weight: -42

scala> val g = Cucumber7(42)
val g: Cucumber7 = Cucumber7@51fd1c7c

scala> g.weight
val res4: Int = 42

scala> g.weight = -1

scala> g.weight
val res5: Int = -1
```

Solution Task 14.

a)
Line 1:

```
  java.lang.IllegalArgumentException: requirement failed: negative weight: -42
```

`Cucumber8.apply` requires that `weight >= 0` otherwise `require` throws an exception.
Line 5:

```
  java.lang.IllegalArgumentException: requirement failed: negative weight: -1
```

The setter `weight_=` requires `weight >= 0` otherwise `require` throws an exception.
Line 7:

```
  java.lang.IllegalArgumentException: requirement failed: negative weight: -958
```

Since `42 - 1000` is less than zero `require` throws an exception.

b) You can impose your own specific requirements on what values are allowed so you have greater control that nothing unexpected happens.

Solution Task 15. *Object with mutable state.*

a)

```
class Frog private (initX: Int = 0, initY: Int = 0):
  private var _x: Int = initX
  private var _y: Int = initY
  private var _distanceJumped: Double = 0

  def x: Int = _x
  def y: Int = _y

  def jump(dx: Int, dy: Int): Unit =
    _x += dx
    _y += dy
```

```
    _distanceJumped += math.hypot(dx, dy)


  def randomJump: Unit =
    def rnd = util.Random.nextInt(10) + 1
    jump(rnd, rnd)

  def distanceToStart: Double = math.hypot(x,y)
  def distanceJumped: Double = _distanceJumped
  def distanceTo(f: Frog): Double = math.hypot(x - f.x, y - f.y)

object Frog:
  def spawn(): Frog = Frog()
```

b) Example test program:

```
object FrogTest:
  def test(): Unit =
    val f1 = Frog.spawn()
    assert(f1.x == 0 && f1.y == 0, "Test of spawn, reqt 1 & 4 failed.")

    f1.jump(4, 3)
    assert(f1.x == 4 && f1.y == 3, "Test of jump, reqt 1 & 4 failed.")

    f1.jump(4, 3)
    assert(f1.distanceJumped == 10, "Test of jump, reqt 2 failed.")

    f1.jump(-4, -3)
    assert(f1.distanceToStart == 5, "Test of jump, reqt 3 failed.")

    for x <- 1 to 10000 do
      val f2 = Frog.spawn()
      f2.randomJump
      assert(f2.x > 0 && f2.x <= 10 && f2.y > 0 && f2.y <= 10,
        "Test of randomJump, reqt 5 failed.")

    println("Test Ok!")
```

c) A method that is an indirect reading of attribute values is called a getter.

d)

```
class Frog private (initX: Int = 0, initY: Int = 0):
  private var _x: Int = initX
  private var _y: Int = initY
  private var _distanceJumped: Double = 0

  def jump(dx: Int, dy: Int): Unit =
    _x += dx
    _y += dy
    _distanceJumped += math.hypot(dx, dy)
```

```
  def x: Int = _x
  def x_=(newX: Int): Unit = // Setter for x
    _distanceJumped += math.abs(x - newX)
    _x = newX

  def y: Int = _y
  def y_=(newY: Int): Unit = // Setter for y
    _distanceJumped += math.abs(y - newY)
    _y = newY


  def randomJump: Unit =
    def rnd = util.Random.nextInt(10) + 1
    jump(rnd, rnd)

  def distanceToStart: Double = math.hypot(x,y)
  def distanceJumped: Double = _distanceJumped
  def distanceTo(f: Frog): Double = math.hypot(x - f.x, y - f.y)

object Frog:
  def spawn(): Frog = Frog()
```

e)

```
object FrogSimulation:
  def isAnyCollision(frogs: Vector[Frog]): Boolean =
    var found = false
    frogs.indices.foreach(i =>  // generate all pairs (i,j)
      for j <- i + 1 until frogs.size do
        if !found then
          found = frogs(i).distanceTo(frogs(j)) <= 0.5
    )
    found

  def jumpUntilCrash(n: Int = 100, initDist: Int = 8): (Int, Double) =
    val frogs = Vector.fill(n)(Frog.spawn())
    (0 until n).foreach(i => frogs(i).x = i * initDist)
    var count = 0
    while !isAnyCollision(frogs) do
      frogs(util.Random.nextInt(n)).randomJump
      count += 1
    (count, frogs.map(_.distanceJumped).sum)


  def run(nbrOfCrashTests: Int = 10) =
    for i <- 1 to nbrOfCrashTests do
      val (n, dist) = jumpUntilCrash()
      println(s"\nAntalet looprundor innan grodkrock: $n")
      println(s"Totalt avstånd hoppat av alla grodor: $dist")
```

Solution Task 16. *Object with mutable state.*

```
class Square private (val initX: Int, val initY: Int, val initSide: Int):
  private var nMoves = 0
  private var sumCost = 0.0

  private var _x = initX
  private var _y = initY

  private var _side = initSide

  private def addCost(): Unit =
    sumCost += math.hypot(x - initX, y - initY) * side

  def x: Int = _x
  def y: Int = _y

  def side = _side

  def scale(factor: Double): Unit = _side = (_side * factor).round.toInt

  def move(dx: Int, dy: Int): Unit =
    _x += dx; _y += dy
    nMoves += 1
    addCost()

  def moveTo(x: Int, y: Int): Unit =
    _x = x; _y = y
    nMoves += 1
    addCost()

  def cost: Double = sumCost

  def pay: Double = {val temp = sumCost; sumCost = 0; temp}

  override def toString: String =
    s"Square[($x, $y), side: $side, #moves: $nMoves times, cost: $sumCost]"

object Square:
  private var created = Vector[Square]()

  def apply(x: Int, y: Int, side: Int): Square =
    require(side >= 0, s"side must be positive: $side")
    val sq = (new Square(x, y, side))
    created :+= sq
    sq

  def apply(): Square = apply(0, 0, 1)

  def totalNumberOfMoves: Int = created.map(_.nMoves).sum

  def totalCost: Double = created.map(_.cost).sum
```

L.6 Solution Excercise06

L.6.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match on constant values.*

a) Scala's `match` expression compares stepwise the value with each `case` and then returns a value belonging to the corresponding `case`.

b)

```
scala.MatchError
```

Runtime error, occurs for certain input during execution.

Solution Task 2. *Guards in case branches.*
The guard introduced at `case 'c'` generates a random number between 0 and 1 and if the number is not greater than 0.5, there will be no match with `case 'c'` and the program tries further until the default case.
The guard condition must be met in order to match as usual.

Solution Task 3. *Pattern match on attributes in case classes.*
G100true. When changing the order: Gtrue100.
`match` tests if the companion object `Cucumber` is of type `Cucumber` with two parameter values. The specified parameters are assigned names, `weight` is given the name `w` and `isRotten` the name `rotten` and then printed. If these names are switched, they are printed in the reversed order.

Solution Task 4. *Match on case objects and the usefulness of `sealed`.*

a)

```
Cannot extend sealed trait Color in a different source file
```

The error message occurs because the REPL treats each input individually and therefore does not allow the subtype `Spades` to extend the supertype `Color` since it was sealed. More about this later in the course...

b) -

c) Assuming `import Deck._` has been written...

```
def paracolor(f: Color): Color = f match
  case Spades  => Clubs
  case Hearts => Diamonds
  case Diamonds   => Hearts
  case Clubs  => Spades
```

d)

```
<console>:17: warning: match may not be exhaustive.
It would fail on the following input: Diamonds
```

The warning appears already at compilation.

e)

```
scala.MatchError: Diamonds (of class Diamonds)
  at .paracolor(<console>:17)
```

This is a runtime error.

f) If a class is **sealed**, it means that if an element should be matched and is a subtype of this class, Scala gives a warning already at compile time if there is a risk of a `MatchError`, i.e., if the **match** expression is not exhaustive and there are cases not covered by a **case**.
A sealed supertype means that the programmer gets a warning at compile time if a case is not covered, and in that case which of the subtypes is missing, as well as other help from the compiler. This requires, however, that all the subtypes share the same file as the sealed class.

Solution Task 5. *Pattern match enumeration.*a)

```
def paracolor(f: Color): Color = f match
  case Color.Spades => Color.Clubs
  case Color.Hearts => Color.Diamonds
  case Color.Diamonds => Color.Hearts
  case Color.Clubs => Color.Spades
```

Similar to task 4c an **import** statement can also be written here to access members of `Color` without dot notation. However, it is not always advantageous to import members into the global namespace, since name clashes can occur. Suppose an example where we work on a program with a graphical user interface that has a color `Red` defined. Also suppose that we want to import yet another red color for the suits hearts and diamonds, also named `Red`. In this scenario, a name clash would occur since `Red` is already defined, so the import could not proceed.

b) When pattern matching, **sealed trait** used with **case** objects works in practice similarly to using **enum**. We saw in subtask 4d that the REPL warned already at compile time that the match is not exhaustive. This also applies when using **enum**.

Solution Task 6. *Meaning of lowercase and uppercase initial letters when matching.*

a) Both `str` and `whatever` match the input, regardless of what it is, because they start with a lowercase letter.
However, `str` has a guard that the string must start with `g`, which means only **val** `g = "cucumber"` matches this. However, **val** `x = "urka"` is caught by `whatever`, which lacks a guard.

b)

```
<console>:16: warning: patterns after a variable pattern cannot match (SLS 8.1.
```

and

```
<console>:17: warning: unreachable code due to variable pattern 'tomat' on line
```

Even though a class `tomato` exists, Scala treats the **match** case as a catch-all because it starts with a lowercase letter. This means any object not of type `Cucumber` will print *tomat* and the last case cannot be reached.

c)

```
case `tomato` => println("tomato")
```

Solution Task 7. *Match on the contents of a Vector.*

```
hguod
hguod
42
```

For each element in `xss`, a match is done that results in a string. What happens in each branch is explained below.

1. The first match branch is never activated because `xss` contains no empty vector.
2. The second branch fits `Vector("hej")`, and the variable `a` is bound to `"hej"`.
3. The third branch matches `Vector("on", "dough")` where the first value is ignored (underscore) and the second value is bound to `b`.
4. The fourth branch matches a sequence with three values where the middle value is `"x"`. The last branch is not triggered here but would match anything not caught earlier.

Solution Task 8. *Use `Option` and match on potentially missing values.*

a)

1. `var maybe` becomes an `Option` holding `Int` but with no value, called `None`.
2. Since `var maybe` has no value, its size is 0.
3. `var maybe` is assigned the value 42 which is stored in a `Some` indicating a value exists.
4. Since `var maybe` now contains a value, its size is 1.
5. Since `var maybe` contains a value, it is not empty.
6. Since `var maybe` contains a value, it is defined.
7. `def increaseIfExists` matches an `Option[Int]` with its different cases.
   If there is a value, i.e., `opt: Option[Int]` is a `Some`, it returns a `Some` with the original value plus 1.
   If there is no value, i.e., `opt: Option[Int]` is a `None`, it returns `None`.
8. -
9. -
10. -
11. `def increaseIfExists` is applied to `maybe` and returns a `Some` with the value in `maybe` plus 1, i.e., 43.
12. `def öka` takes an `Int` and returns this value plus 1.
13. `map` applies `def öka` to the sole element in `maybe`, 42. This returns a `Some` with value 43 assigned to `merKanske`.

b)

1. `val` `meningen` becomes a `Some` with value 42.
2. `val` `ejMeningen` becomes an `Option[Int]` without value, i.e., `None`.
3. `map(_ + 1)` is applied on `meningen` and increases the existing value by 1 to 43.
4. `map(_ + 1)` is applied on `ejMeningen` but since no value exists, it remains `None`.
5. `map(_ + 1)` is applied again on `ejMeningen` but now `orElse` method is included. If an `Option` has no value (`None`), the code in `orElse` executes, which here prints out *saknas* for the missing value.
6. The same call from the previous line is executed this time on `meningen` and since a value exists, only the first part that increases the value by 1 runs.

This method can be used instead of the `match`-version in the previous example due to its simpler form. An `Option` contains either a value or none so a longer `match` is unnecessary.

c)

1. A vector `xs` is created with every fifth number from 42 to 82.
2. An empty `Int` vector `e` is created.
3. `headOption` takes the first value of vector `xs` and returns it wrapped in an `Option`, `Some(42)`.
4. The first value in vector `xs` is wrapped in an `Option` and then extracted by `get`, 42.
5. Same as previous line but uses `getOrElse`, which returns 0 if the `Option` is `None`. Since `xs` has at least one value, this returns the same as before, 42.
6. Like previous line but instead of returning 0 if missing, returns an `Option[Int]` with 0 as value.
7. `headOption` tries to take the first element of empty vector `e` but since no value exists, returns `None`.
8.
```
java.util.NoSuchElementException: None.get
```

   Like preceding line returns `None`, but calling `get` on `None` causes a runtime error.
9. Like previous returns `None` from empty vector `e`, but `getOrElse` returns 0 instead.
10. Like previous uses `getOrElse` on the `None` and returns an `Option[Int]` holding 0.
11. A vector containing `xs` and three `e` vectors is created.
12. `map` applies `lastOption` on each sub-vector extracting last elements into a new vector. Since `e` is empty, returns `None` for those.
13. Same as previous but uses `flatten` to remove all `None` elements leaving only actual values.

14. `lift` method fetches element at index 0 in `xs` wrapped in an `Option` which is `Some(42)`.
15. `lift` tries to get element at index 1000 in `xs`, which doesn't exist, so returns `None`.
16. Same as previous but applied to vector `e`, and then `getOrElse` returns 0 since `None`.
17. `find` searches `xs` for first element > 50 and returns it in `Some(52)`.
18. `find` searches `xs` for first element < 42 but none exists, so returns `None`.
19. `find` on `e` searching for element > 42 and prints *HITTAT!* if found. Since `e` is empty, `None` is returned and `foreach` does not run.

d) Using `-1` as a return value for missing or error cases can cause runtime errors that are hard to detect. **`null`** can cause crashes if misused during execution. `Option` avoids such problems, and if `getOrElse` or similar is used, the program does not crash. Moreover, functions returning `Option` do not need as much documentation about what can be returned; instead, the code itself shows it explicitly.
Finally, `Option` is more type-safe than **`null`**. When you return an `Option`, the type of the contained value is specified, if any, which makes it easier to understand and limits what can be returned.

Solution Task 9. *Throw exceptions.*

a)

1. An `Exception` is thrown with message *PANG!*.
2. Various types of `Exception` are shown.
3. An exception of type `IllegalArgumentException` is thrown with message *wrong wrong wrong*.
4. An exception with message `stormWind!` is thrown and caught by the **`catch`** expression. A **`match`** inspects the exception and prints the message, and returns -1.

b) For example:
`OutOfMemoryError`, when the program runs out of memory.
`IndexOutOfBoundsException`, when accessing a vector position larger than available.
`NullPointerException`, when calling a method or similar on a null reference.

c) If try and catch branches have the same type, here `Int`, the compiler infers that type for the entire expression. If the catch branch returns a different type such as `String`, then the most precise inferred type for the whole expression is `Matchable`, a direct subtype of the most general type `Any`.

Solution Task 10. *Catch exceptions with `scala.util.Try`.*

a)

1. **`def`** `pang` is defined that throws an `Exception` with message *PANG!*.
2. Scala utilities `Try`, `Success` and `Failure` are imported.

3. `def` `pang` is called in `Try` which catches the exception and wraps it in a `Failure`.
4. The method `recover` matches the exception in the `Failure` from above with a `case` and turns that `Failure` into a `Success`, similar to catch.
5. The string *silent* runs in the previous test but since no exception is thrown it is wrapped in a `Success` and `recover` does nothing. It only handles exceptions.
6. `def` `maybePang` is defined that is equally likely to return the string *silent* or call `pang`.
7. `def` `maybeOk` is defined that tests `def` `maybePang` with `Try`.
8. A vector `xs` is filled with results of 100 runs of `maybeOk` consisting of `Success` and `Failure`.
9. The element at index 13 in vector `xs` is matched to either `Success` (prints ":)") or `Failure` (prints ":(" plus error message).
10. -
11. -
12. Method `isSuccess` tests if element at `xs(13)` is `Success`, returns `true` if so.
13. Method `isFailure` tests if element at `xs(13)` is `Failure`, returns `true` if so.
14. Method `count` counts how many elements in `xs` are `Failure`.
15. Method `find` searches `xs` for a `Failure` element and returns it wrapped in an `Option`.
16. `badOpt` is assigned first `Failure` found in `xs`.
17. `goodOpt` is assigned first `Success` found in `xs`.
18. Printing `badOpt` yields `Option[scala.util.Try[String]] = Some(Failure(java.lang.Exception: PAN`
19. Calling `get` on `badOpt` returns the `Failure` inside the `Option`.
20. Calling `get` again on the `Failure` extracts and throws the exception.
21. Using `getOrElse` on the `Failure` returns the alternative string "bomb disarmed!" since it is a `Failure`.
22. Using `getOrElse` on the `Success` returns the contained string "silent".
23. The same method is mapped over all elements of `xs` and printed.
24. `toOption` is mapped over `xs` turning `Success` into `Some` with the string "silent" and `Failure` into `None`.
25. `flatten` removes `None`s from the vector, leaving just the `Some` elements.
26. The `size` returns the number of `Success` elements counted.

b) `pang` has return type `Nothing`, a special Scala type which is subtype of all types but has no values.

c) `Nothing` is a subtype of every type in Scala's hierarchy, including `String`, which implies `String` includes both strings and `Nothing`, so the inferred type is `String`.

L.6.2 Advanced Tasks; challenges

Solution Task 11. *Use matching or dynamic binding?*

a)

```
package vegopoly

trait Vegetable:
  def weight: Int
  def isRotten: Boolean
  def isWorthEating: Boolean

case class Cucumber(weight: Int, isRotten: Boolean) extends Vegetable:
  val isWorthEating: Boolean = (!isRotten && weight > 100)

case class Tomato(weight: Int, isRotten: Boolean) extends Vegetable:
  val isWorthEating: Boolean = (!isRotten && weight > 50)

object Main:
  def randomWeight: Int = (math.random()*500 + 100).toInt

  def randomDecay: Boolean = math.random() > 0.8

  def randomCucumber: Cucumber = Cucumber(randomWeight, randomDecay)

  def randomTomato: Tomato = Tomato(randomWeight, randomDecay)

  def randomVegetable: Vegetable =
    if math.random() > 0.2 then randomCucumber else randomTomato

  def main(args: Array[String]): Unit =
    val harvest = Vector.fill(args(0).toInt)(randomVegetable)
    val edible = harvest.filter(_.isWorthEating)
    println("Number of vegetables harvested: " + harvest.size)
    println("Number of edible vegetables: " + edible.size)
```

b) Add the following `case class`:

```
case class Broccoli(weight: Int, isRotten: Boolean) extends Vegetable:
  val isEatable: Boolean = (!isRotten && weight > 80)
```

Also add the following in `object Main` before `def randomVegetable`:

```
def randomBroccoli: Broccoli = Broccoli(randomWeight, randomRotten)
```

Finally, modify `def randomVegetable` to:

```
def randomVegetable: Vegetable =     // choose e.g. this distribution:
  val rnd = math.random()
  if rnd > 0.5 then randomCucumber      // 50% chance cucumber
```

```
else if rnd > 0.2 then randomTomato // 30% chance tomato
else randomBroccoli             // 20% chance broccoli
```

c) The advantages of the **match** version, and pattern matching generally, are that it is easy to change how matching is done. This makes it easy to change the definition of edibility. However, if these change rarely but vegetables change often, the polymorphic alternative may be preferred since it is easier to implement and change when vegetables change.

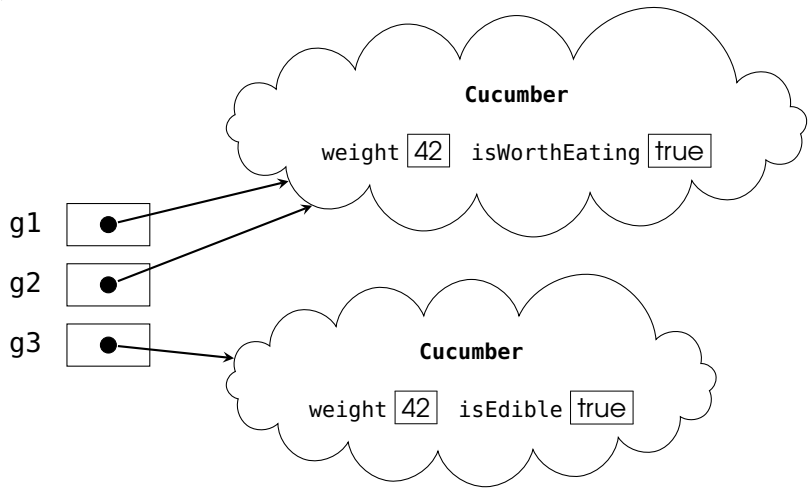
Solution Task 12. *Method* `equals`.

a)

1. A class `Cucumber` is created with parameters `weight` of type `Int` and `isEatable` of type `Boolean`.
2. `g1` is assigned an instance of `Cucumber` with `weight = 42` and `isEatable = true`.
3. `g2` is assigned the same `Cucumber` object as `g1`.
4. `g3` is assigned a new `Cucumber` instance with the same parameter values as `g1`.
5. The == (`equals`) method compares `g1` and `g2` and returns **true**.
6. == (`equals`) method compares `g1` and `g3` and returns **false**.
7. `def equals(x\$1: Any): Boolean`

As seen the parameter of `equals` is of type `Any`. Since the program does not know the class, it uses `Any.equals`. Unlike primitive types that compare content with `equals`, classes default to reference equality unless otherwise specified. `g1` and `g2` refer to the same object but `g3` points to a different one, so `g1 == g3` is false.

b)



c) -

d) In the first three lines the same actions happen as in subtask a. Now when comparing `Cucumber` objects, the `equals` method is overridden by the one defined specifically for `Cucumber`. Since both objects compared are of type `Cucumber`, the case `that: Cucumber` matches. It then compares the weights and edible booleans returning `Boolean` indicating equality, which in both cases is true.

e) In subtask a, `g1 == g3` was false despite content equality. After overriding, this expression returns **`true`**, demonstrating that == now uses content equality.

Solution Task 13. *Polynomial.*

a) TODO!!!

b) TODO!!!

Solution Task 14. *`Option` as a collection.*
Examples of methods that exist both for `Vector` and `Option`: `foreach`, `filter`, `fold` etc.
The method `contains` returns a `Boolean` indicating whether it holds a value or not.

Solution Task 15. *Catch exceptions with **`catch`** in Java and Scala.* TODO!!!

Solution Task 16. *Polynomial, continuation: reduction.*

Solution Task 17. *Type-safe content test with method ===.*

Solution Task 18. *Override `equals` with content equality also for non-final classes.*

Solution Task 19. *Override equals with inheritance.*

Solution Task 20. *Special matchings.* TODO!!!

Solution Task 21. *Extractors.* TODO!!!

Solution Task 22. *Polynomial, continuation: polynomial division.* TODO!!!

L.7 Solution `Excercise07`

L.7.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match concepts with description.*

element	1	$\rightsquigarrow$	G	object in a data structure
collection	2	$\rightsquigarrow$	B	data structure with elements of same type
collection library	3	$\rightsquigarrow$	J	many ready-made collections with different properties
sequence(collection)	4	$\rightsquigarrow$	L	zero or more elements of same type in a specific order
sequence algorithm	5	$\rightsquigarrow$	I	solution to problem using a sequence collection
order	6	$\rightsquigarrow$	A	defines how elements of a type are arranged
sorting	7	$\rightsquigarrow$	C	algorithm arranging elements in a specific order
searching	8	$\rightsquigarrow$	D	algorithm finding elements according to a search criterion
linear search	9	$\rightsquigarrow$	F	searching a sequence until element found
counting	10	$\rightsquigarrow$	H	algorithm counting elements with certain properties
time complexity	11	$\rightsquigarrow$	E	how execution time grows with problem size
memory complexity	12	$\rightsquigarrow$	K	how memory usage grows with problem size

Solution Task 2. *Different sequence collections.*

`Vector`	1	$\rightsquigarrow$	B	immutable, quickly returns any modified collection
`List`	2	$\rightsquigarrow$	C	immutable, quickly returns new collection modified at start
`Array`	3	$\rightsquigarrow$	D	primitive, mutable, fast indexing, fixed size
`ArrayBuffer`	4	$\rightsquigarrow$	A	mutable, fast indexing, can change size
`ListBuffer`	5	$\rightsquigarrow$	E	mutable, fast to modify at start

Solution Task 3. *Using sequence collections.*

a)

`x +: xs`	1	⇝	G	`Vector(0, 1, 2, 3)`
`xs +: x`	2	⇝	L	`error: value +: is not a member of Int`
`xs :+ x`	3	⇝	J	`Vector(1, 2, 3, 0)`
`xs ++ xs`	4	⇝	M	`Vector(1, 2, 3, 1, 2, 3)`
`xs.indices`	5	⇝	E	`(0 until 3)`
`xs apply 0`	6	⇝	C	`1`
`xs(3)`	7	⇝	I	`java.lang.IndexOutOfBoundsException`
`xs.length`	8	⇝	O	`3`
`xs.take(4)`	9	⇝	F	`Vector(1, 2, 3)`
`xs.drop(2)`	10	⇝	K	`Vector(3)`
`xs.updated(0, 2)`	11	⇝	B	`Vector(2, 2, 3)`
`xs.tail.head`	12	⇝	N	`2`
`xs.head.tail`	13	⇝	D	`error: value tail is not a member of Int`
`xs.isEmpty`	14	⇝	H	**`false`**
`xs.nonEmpty`	15	⇝	A	**`true`**

b)

error	*type*	*explanation*
`value +: is not a member of Int`	compile-time error	Operators ending with colon are right-associative. The method call `xs +: x` corresponds in dot notation to `x.+:(xs)` and there is no method named `+:` on integers.
`IndexOutOfBoundsException`	runtime error	There are only 3 elements and indexing starts at 0 in sequence collections.
`value tail is not a member of Int`	compile-time error	The method `head` returns the first element and integers do not have the sequence collection method `tail`.

Solution Task 4. *Copying sequences.*

a)

`xs(0)`	`rs$line5$Mutant@66d766b9` new instances get new hex codes
`ys(0).int`	`0` since `ys` contains the same instance as `xs`
`zs(0).int`	`5` since `!(xs(0) eq zs(0))`
`xs(0) eq ys(0)`	`true` because same instance
`xs(0) eq zs(0)`	`false` since different instances
`(ys.toBuffer :+ new Mutant).apply(0).int`	`0` since the non-deep-copied ArrayBuffer copy refers to the same instance at the first position as both `ys` and `xs` and `x(0).int` became zero in an assignment on line 5 in REPL run

Note that copying with `toArray`, `toVector`, `toBuffer`, etc. *is not deep*, i.e., only instance references are copied, not the instances themselves.

b)

```
def deepCopy(xs: Array[Mutant]): Array[Mutant] =
  val result = Array.ofDim[Mutant](xs.length) // filled with null references
  var i = 0
  while i < xs.length do
    result(i) = new Mutant(xs(i).int) // copy with same content at the same place
    i += 1
  result
```

It is also fine to create the result array with `new Array[Mutant](xs.length)`. You can also use `size` instead of `length`.

c)

```
scala> class Mutant(var int: Int = 0)
// defined class Mutant

scala> def deepCopy(xs: Array[Mutant]): Array[Mutant] =
     |   val result = Array.ofDim[Mutant](xs.length)
     |   var i = 0
     |   while i < xs.length do
     |     result(i) = new Mutant(xs(i).int)
     |     i += 1
     |   result

scala> val xs = Array.fill(3)(new Mutant)
xs: Array[Mutant] = Array(rs$line$2$Mutant@46a123e4, rs$line$2$Mutant@44bc2449,
rs$line2$Mutant@3c28e5b6)

scala> val ys = deepCopy(xs)
ys: Array[Mutant] = Array(rs$line$2$Mutant@14b8a751, rs$line2$Mutant@7345f97d,
rs$line$2$Mutant@554566a8)

scala> xs(0).int = 5

scala> ys(0).int
val res0: Int = 0
```

d) No, since elements cannot be mutated one can safely share references between collections. There is no possibility that changes affect multiple collections at the same time. However, deep copies of collections with mutable elements are commonly (though often time-consuming) made to ensure the original collection does not change.

Solution Task 5. *Updating sequences.*

a)

`{ buf(0) = -1; buf(0) }`	1	⇝ D	`-1`
`{ xs(0) = -1; xs(0) }`	2	⇝ A	`error: value update is not a member`
`buf.update(1, 5)`	3	⇝ F	`(): Unit`
`xs.updated(0, 5)`	4	⇝ B	`Vector(5, 2, 3, 4)`
`{ buf += 5; buf }`	5	⇝ C	`ArrayBuffer(-1, 5, 3, 4, 5)`
`{ xs += 5; xs }`	6	⇝ G	`error: value += is not a member`
`xs.patch(1,Vector(-1,5),3)`	7	⇝ E	`Vector(1, -1, 5)`
`xs`	8	⇝ H	`Vector(1, 2, 3, 4)`

b)

```
def insert(xs: Array[Int], elem: Int, pos: Int): Array[Int] =
  xs.patch(from = pos, other = Array(elem), replaced = 0)
```

c) Pseudocode below is written so that it compiles though it is incomplete.

```
def insert(xs: Array[Int], elem: Int, pos: Int): Array[Int] =
  val result = ??? /* new array with space for one more element than xs */
  var i = 0
  while(???){/* copy elements before pos and increment i */}
  if i < result.length then /* put elem in result at position i */
  while(???){/* copy the rest */}
  result
```

d)

```
def insert(xs: Array[Int], elem: Int, pos: Int): Array[Int] =
  val result = new Array[Int](xs.length + 1)
  var i = 0
  while i < pos && i < xs.length do  { result(i) = xs(i); i += 1}
  if i < result.length then { result(i) = elem; i += 1 }
  while i < result.length do { result(i) = xs(i - 1); i += 1}
  result
```

```
scala> insert(Array(1, 2), 0, pos = -1)
val res2: Array[Int] = Array(0, 1, 2)

scala> insert(Array(1, 2), 0, pos = 0)
val res3: Array[Int] = Array(0, 1, 2)

scala> insert(Array(1, 2), 0, pos = 1)
```

```
val res4: Array[Int] = Array(1, 0, 2)

scala> insert(Array(1, 2), 0, pos = 2)
val res5: Array[Int] = Array(1, 2, 0)

scala> insert(Array(1, 2), 0, pos = 42)
val res7: Array[Int] = Array(1, 2, 0)
```

Solution Task 6. *Comparing strings in Scala.*

a)

```
true
true
true
true
true
false
```

b) *s1* comes first.

Solution Task 7. *Linear search according to different criteria.*

a)

`xs.indexOf(0)`	1	⇝ F	`5`
`xs.indexOf(6)`	2	⇝ B	`-1`
`xs.indexWhere(_ < 2)`	3	⇝ H	`4`
`xs.indexWhere(_ != 5)`	4	⇝ I	`1`
`xs.find(_ == 1)`	5	⇝ D	`Some(1)`
`xs.find(_ == 6)`	6	⇝ J	`None`
`xs.contains(0)`	7	⇝ C	`true`
`xs.filter(_ == 1)`	8	⇝ A	`Vector(1, 1)`
`xs.filterNot(_ > 1)`	9	⇝ E	`Vector(1, 0, 1)`
`xs.zipWithIndex.filter(_._1 == 1).map(_._2)`	10	⇝ G	`Vector(4, 6)`

b) With a boolean variable `found`:

```
def indexOf(xs: Vector[String], p: String => Boolean): Int =
  var found = false
  var i = 0
  while i < xs.length && !found do
      found = p(xs(i))
      i += 1
  if found then i - 1 else -1
```

Or without `found`:

```
def indexOf(xs: Vector[String], p: String => Boolean): Int =
  var i = 0
  while i < xs.length && !p(xs(i)) do i += 1
```

```
  if i == xs.length then -1 else i
```

Or maybe you want to start searching from the end; the solution below is probably easier to understand (?) and definitely more concise, but it does *not* fulfill the requirement to return the index of the *first* occurrence as stated in the task. However, if the context allows returning *any* index for which the predicate holds, or if you want to search backwards, this is fine:

```
def indexOf(xs: Vector[String], p: String => Boolean): Int =
  var i = xs.length - 1
  while i >= 0 && !p(xs(i)) do i -= 1
  i
```

There are many other ways to solve this. When implementing algorithms, both in programming exams and in professional development, there are often many different solutions with various properties, advantages, and disadvantages. The important thing is that the solution works well according to requirements, the code is understandable, and the implementation is not so inefficient that users get tired waiting for the result...

Solution Task 8. *Lab preparation: Implement integer counting in Array.*

a)

```
def registerDiceRoll(xs: Seq[Int]): Vector[Int] =
  val result = Array.fill(6)(0)
  xs.foreach{ x =>
    require(x >= 1 && x <= 6, "dice roll must be between 1 & 6")
    result(x - 1) += 1
  }
  result.toVector
```

b)

```
scala> registerDiceRoll(toss(1000))
val res0: Vector[Int] = Vector(171, 163, 166, 152, 184, 164)

scala> registerDiceRoll(toss(1000))
val res1: Vector[Int] = Vector(163, 161, 158, 174, 161, 183)
```

Solution Task 9. *Built-in methods for sorting.*

`'a' < 'A'`	1	⇝	E	`false`
`"AÄÖö" < "AÅÖö"`	2	⇝	H	`true`
`xs.sorted.head`	3	⇝	C	`-1`
`xs.sorted.reverse.head`	4	⇝	G	`3`
`ys.sorted.head`	5	⇝	I	`"ak"`
`zs.indexOf('a')`	6	⇝	B	`1`
`ps.sorted.head.firstName.take(2)`	7	⇝	D	`error: ...`
`ps.sortBy(_.firstName).apply(1).firstName.take(2)`	8	⇝	A	`"ka"`
`xs.sortWith((x1,x2) => x1 > x2).indexOf(3)`	9	⇝	F	`0`

It will fail in the expression trying to sort a sequence of `Person` instances directly with `sorted`:

```
scala> ps.sorted
No implicit Ordering defined for Person.
```

It fails because the compiler cannot find any ordering definition for your own classes. Later in the course we will see how to create custom orderings if you want `sorted` to work on sequences of your own classes, but often `sortBy` and `sortWith` suffice.

Solution Task 10. *Built-in shuffle method.*

a) `Random.shuffle` returns a new shuffled sequence collection of the same type. The original collection's order is not affected.

b) Example usage of `random.shuffle`:

```
scala> import scala.util.Random

scala> val xs = Vector("Sten", "Sax", "Påse")
val xs: Vector[String] = Vector(Sten, Sax, Påse)

scala> (1 to 10).foreach(_ => println(Random.shuffle(xs).mkString(" ")))
Sax Påse Sten
Sten Påse Sax
Sten Sax Påse
Sten Sax Påse
Sten Påse Sax
Sten Påse Sax
Sax Sten Påse
Sten Påse Sax
Sax Påse Sten
Sax Påse Sten

scala> (1 to 5).map(_ => Random.shuffle(1 to 6))
val res1: IndexedSeq[IndexedSeq[Int]] =
  Vector(Vector(5, 2, 1, 4, 3, 6), Vector(6, 5, 4, 2, 1, 3),
  Vector(3, 1, 4, 6, 5, 2), Vector(3, 2, 6, 5, 1, 4),
  Vector(5, 3, 4, 6, 1, 2))

scala> (1 to 1000).map(_ => Random.shuffle(1 to 6).head).count(_ == 6)
val res2: Int = 168
```

Solution Task 11. *Repeated parameters.*

a)

```
scala> def stringSizes(xs: String*): Vector[Int] = xs.map(_.size).toVector
def stringSizes(xs: String*): Vector[Int]

scala> stringSizes("hej")
val res0: Vector[Int] = Vector(3)

scala> stringSizes("hej", "on", "dough", "")
val res1: Vector[Int] = Vector(3, 2, 3, 0)

scala> stringSizes()
val res2: Vector[Int] = Vector()
```

Calling with an empty argument list gives an empty integer sequence.

b)

```
scala> val xs = Vector("hej","on","dough", "")
val xs: Vector[String] = Vector(hej, on, dough, "")

scala> stringSizes(xs: _*)
val res0: Vector[Int] = Vector(3, 2, 5, 0)

scala> stringSizes(Vector(): _*)
val res1: Vector[Int] = Vector()
```

Yes, it works fine with an empty sequence.

L.7.2 Extra Tasks; additional practice

Solution Task 12. *Counting boolean values. Flipping a coin.*

a)

```
def registerCoinFlips(xs: Seq[Boolean]): (Int, Int) =
  val result = Array.fill(2)(0)
  xs.foreach(x => if (x) result(0) += 1 else result(1) += 1)
  (result(0), result(1))
```

b)

Solution Task 13. *Copying and appending at the end.*

```
def copyAppend(xs: Array[Int], x: Int): Array[Int] =
  val ys = new Array[Int](xs.length + 1)
  var i = 0
  while i < xs.length do
    ys(i) = xs(i)
    i += 1
  ys(xs.length) = x
  ys
```

The two bugs in the algorithm are (1) the condition should be strictly less than and (2) the loop variable increment is missing inside the loop.

Solution Task 14.

```
def seqReverseCopy(xs: Array[Int]): Array[Int] =
  val n = xs.length
  val ys = new Array[Int](n)
  var i = 0
  while i < n do
    ys(n - i - 1) = xs(i)
    i += 1
  ys
```

a)

```
def seqReverseCopy(xs: Array[Int]): Array[Int] =
  val n = xs.length
  val ys = new Array[Int](n)
  for i <- (n - 1) to 0 by -1 do
    ys(n - i - 1) = xs(i)
  ys
```

Solution Task 15. *Copy all except one.*

```
def removeCopy(xs: Array[Int], pos: Int): Array[Int] =
  val n = xs.size
  val ys = Array.fill(n - 1)(0)
```

Input : A sequence xs of type `Array[Int]` and pos
Output: A new sequence of type `Array[Int]` which is a copy of xs but with the element at position pos removed

$n \leftarrow$ number of elements xs
$ys \leftarrow$ a new `Array[Int]` with space for $n-1$ elements
for $i \leftarrow 0$ **to** $pos-1$ **do**
  $ys(i) \leftarrow xs(i)$
end
$ys(pos) \leftarrow x$
for $i \leftarrow pos+1$ **to** $n-1$ **do**
  $ys(i-1) \leftarrow xs(i)$
end
ys

```
  for i <- 0 until pos do
    ys(i) = xs(i)
  for i <- (pos + 1) until n do
    ys(i - 1) = xs(i)
  ys
```

Solution Task 16. *In-place removal in array.*

Input : A sequence xs of type `Array[Int]`, a position pos, and a padding value pad
Output: An updated sequence of xs where the element at position pos is removed and subsequent elements moved one step towards lower index with the last element assigned the value pad

$n \leftarrow$ number of elements in xs
for $i \leftarrow pos+1$ **to** $n-1$ **do**
  $xs(i-1) \leftarrow xs(i)$
end
$xs(n-1) \leftarrow pad$

```
def remove(xs: Array[Int], pos: Int, pad: Int = 0): Unit =
  val n = xs.size
  for i <- (pos + 1) until n do
    xs(i - 1) = xs(i)
  xs(n - 1) = pad
```

Solution Task 17. *Copying and inserting.*

a)

```
def insertCopy(xs: Array[Int], x: Int, pos: Int): Array[Int] =
  val n = xs.size
  val ys = Array.ofDim[Int](n + 1)
  for i <- 0 until pos do
    ys(i) = xs(i)
```

```
  ys(pos) = x
  for i <- pos until n do
    ys(i + 1) = xs(i)
  ys
```

b) `pos` must be `0`.

c)

```
java.lang.ArrayIndexOutOfBoundsException: -1
```

d) Element `x` is added to the end of the array, i.e., the returned array is larger than the input array.

e)

```
java.lang.ArrayIndexOutOfBoundsException: 5
```

You get `ArrayIndexOutOfBoundsException` since indexing is out of size bounds.

Solution Task 18. *In-place insertion in array.*

Input : A sequence xs of type `Array[Int]` and integers x and pos
Output : xs updated in-place, where element x is inserted at pos and subsequent elements are shifted one step, the last element lost

$n \leftarrow$ number of elements in xs
$ys \leftarrow$ a clone of xs
$xs(pos) \leftarrow x$
for $i \leftarrow pos + 1$ **to** $n - 1$ **do**
  $xs(i) \leftarrow ys(i - 1)$
end

```
def insertDropLast(xs: Array[Int], x: Int, pos: Int): Unit =
  val n = xs.size
  val ys = xs.clone
  xs(pos) = x
  for i <- pos + 1 until n do
    xs(i) = ys(i - 1)
```

Solution Task 19. *More built-in methods for linear search.*

a)

- `lastIndexOf` is good if you want to search backwards; without it you would otherwise have to use `xs.reverse.indexOf(e)`
- `indexOfSlice(ys)` searches for the index where a whole sequence `ys` begins, unlike `indexOf(e)` which only searches for a single element.
- `segmentLength(p, i)` returns the length of the longest consecutive sequence where all elements satisfy predicate `p`, starting from position `i`
- `xs.maxBy(f)` applies function `f` on all elements in `xs` and returns the element producing the maximum value; similarly `minBy(f)` gives the minimum according to `f`

b) –

L.7.3 Advanced Tasks; challenges

Solution Task 20. *Fix Swedish sorting order of ÄÅÖ.* TODO!!!

Solution Task 21. *Fibonacci sequence with ListBuffer.*

a)

```
def fib(max: Long): List[Long] =
  val xs = scala.collection.mutable.ListBuffer.empty[Long]
  xs.prependAll(Vector(1, 1))
  while xs.head < max do xs.prepend(xs.take(2).sum)
  xs.reverse.drop(1).toList
```

b)

```
scala> fib(Int.MaxValue).size
val res0: Int = 46
```

c)

```
def fibBig(max: BigInt): List[BigInt] =
  val xs = scala.collection.mutable.ListBuffer.empty[BigInt]
  xs.prependAll(Vector(BigInt(1), BigInt(1)))
  while xs.head < max do xs.prepend(xs.take(2).sum)
  xs.reverse.drop(1).toList
```

```
scala> fibBig(Long.MaxValue).size
val res0: Int = 92

scala> fibBig(BigInt(Long.MaxValue).pow(64)).size
val res1: Int = 5809

scala> fibBig(BigInt(Long.MaxValue).pow(128)).last
val res2: BigInt = 466572805528355449194553611102863153950720005186045547177525

scala> fibBig(BigInt(Long.MaxValue).pow(128)).last.toString.size
val res3: Int = 2428

scala> fibBig(BigInt(Long.MaxValue).pow(256)).last.toString.size
val res4: Int = 4856

scala> fibBig(BigInt(Long.MaxValue).pow(1024)).last.toString.size
java.lang.OutOfMemoryError: Java heap space
```

Solution Task 22. *In-place reverse sequence.*

```
def reverseChars(xs: Array[Char]): Unit =
  val n = xs.length
  for i <- 0 to (n/2 - 1) do
    val temp = xs(i)
    xs(i) = xs(n - i - 1)
    xs(n - i - 1) = temp
```

Solution Task 23. *Palindrome predicate.*

a) Reversing with `reverse` can require one traversal plus memory for the copy. The content comparison requires another traversal. (This is not a big problem since the world's longest palindrome is no longer than 19 letters and is an obscure Finnish word rarely uttered in daily speech. Which one?)

b)

```
def isPalindrome(s: String): Boolean =
  val n = s.length
  var foundDiff = false
  var i = 0
  while i < n/2 && !foundDiff do
    foundDiff = s(i) != s(n - i - 1)
    i += 1
  !foundDiff
```

Solution Task 24. *More useful sequence collection methods.*

```
scala> val xs = Vector.tabulate(10)(i => math.pow(2, i).toInt)
xs: Vector[Int] = Vector(1, 2, 4, 8, 16, 32, 64, 128, 256, 512)

scala> xs.forall(_ < 1024)
val res0: Boolean = true

scala> xs.exists(_ == 3)
val res1: Boolean = false

scala> xs.count(_ > 64)
val res2: Int = 3

scala> xs.zipWithIndex.take(5)
val res3: Vector[(Int, Int)] = Vector((1,0), (2,1), (4,2), (8,3), (16,4))
```

Solution Task 25. *Arrays don't behave, but `ArraySeqs` do!*

a) `xs` provides content equality and is of type `Seq[Int]` with underlying type `ArraySeq[Int]`. You cannot assign elements in an `ArraySeq`, as it is immutable. It does not update when the original array changes.

```
scala> val as1 = Array(1,2,3)
val as1: Array[Int] = Array(1, 2, 3)

scala> val as2 = Array(1,2,3)
val as2: Array[Int] = Array(1, 2, 3)


scala> val (xs1, xs2) = (as1.toSeq, as2.toSeq)
val xs1: Seq[Int] = ArraySeq(1, 2, 3)
val xs2: Seq[Int] = ArraySeq(1, 2, 3)

scala> as1 == as2
val res0: Boolean = false

scala> xs1 == xs2
```

```
val res1: Boolean = true

scala> as1(0) = 42

scala> xs1
val res2: Seq[Int] = ArraySeq(1, 2, 3)

scala> xs1(0) = 42
value update is not a member of Seq[Int]
```

b) With repeated parameters you get an `ArraySeq`.

```
scala> def f(xs: Int*) = xs
def f(xs: Int*): Seq[Int]

scala> println(f(1,2,3))
ArraySeq(1, 2, 3)
```

c) It is not possible to have a generic array as function result without attaching a context bound `ClassTag` to the type parameter so that the compiler can generate code for type conversion required at JVM runtime. See example here:
http://docs.scala-lang.org/overviews/collections/arrays.html

Solution Task 26. The sequence collection `List` is almost twice as fast when processed at the beginning but about 1000 times slower when processed at the end of a sequence with 100000 elements.
Different executions run at different speeds on the JVM, among other things due to optimizations that occur when the JVM warms up and the so-called Just-In-Time compilation does its mighty job. It sometimes suddenly runs significantly slower when the garbage collector is forced to do a time-consuming cleanup of the memory.

Solution Task 27. –